Knox AI Builder Tutorial v1.0 (Eng.)

SAMSUNG SDS

Table of contents

Table of contents ..... **ii**

Preface ..... **viii**

Part I. AI Builder overview 1

1. Getting started with AI Builder **2**

1.1 Connect to Knox AI Builder 2

1.2 Knox AI Builder permission to use 2

2. Introduction to AI Builder features **3**

2.1 AI Builder basic features 3

2.2 Bot creation and management 4

Bot list 4

Bot details 5

2.3 Plugin creation and management 6

Plugin list 6

Plugin details 7

Part II. Bot common processes 9

1. Bot account creation and interface **10**

1.1 Requesting a bot account 10

1.2 Requesting stage bot integration and developing a stage bot 12

Requesting stage bot integration 12

Developing and testing a stage bot 15

1.3 Requesting product bot integration and developing a bot 15

Requesting product bot integration 16

Developing and testing a product bot 17

2. Configuring target users for a bot **18**

3. Making bot account changes **19**

Part III. FAQ bot 20

1. Creating a bot **23**

2. Registering a scenario **25**

3. Testing a chat and Search tuning **28**

3.1 **Simulating a chat** 28

3.2 **Registering search keywords** 30

3.3 **Registering synonyms** 33

3.4 **Registering a rule** 35

3.5 **Registering small talk** 38

3.6 **Utilizing the Debug on/off feature** 40

4. Deploying the bot **42**

5. Viewing the bot in the messenger **44**

6. Operating the bot **47**

6.1 **Using the feedback feature** 47

6.2 **Analyzing logs and improving bot responses** 52

Part IV. AI Bot 54

1. Creating a AI bot **57**

2. Registering a contents **60**

2.1 **Answer information** 61

3. Setting details and related app card **62**

3.1 **LLM Admin and Model** 62

3.2 **Temperature** 62

3.3 **Top-p** 63

3.4 **Advanced search** 63

3.5 **Rerank Score** 64

3.6 **Prompt** 64

3.7 **Related app card** 65

Register the main app card 65

Select the related app card 67

4. Testing and deploying the bot 68

4.1 **Having a conversation test on the simulation** 68

4.2 **Deploy** 70

5. Viewing the bot in the Knox Teams 73

Part V. Notifications and polls 75

1. Sending notifications 77

1.1 Notification delivery flow 80

1.2 Registering and sending notifications 80

2. Poll delivery flow 88

3. Registering polls 89

4. Registering poll schedules 94

5. Registering open-ended polls 96

Part VI. Bot Management 100

1. Dashboard 101

2. Managing Feedback 104

2.1 Using feedback 104

2.2 Setting a Feedback App Card 105

2.3 Setting Feedback Privileges/notification 109

2.4 Checking the Feedback History in AI Builder 113

3. Monitoring usage statistics 118

3.1 Viewing user status 118

3.2 Viewing usage logs 119

3.3 Viewing content status 121

4. Chatbot authority management 123

4.1 Managing user authority 123

Add user authority 124

Delete user authority 124

4.2 Managing department authority 125
Add department authority 125
Delete department authority 127

Part VII. ChatOps bot 128

1. Receiving Zabbix event notifications 129
1.1 Zabbix event reception flow 129
1.2 Disabling the firewall before operation 130
1.3 Configuring in AI Builder 130
Registering an event reception app card 131
Registering webhooks 137
Deploying to the Knox Teams 139
Creating webhook URLs 139
1.4 Configuring the webhook in Zabbix 141
Creating media 142
Creating users and linking with media 144
Creating actions 145
1.5 Testing 148

2. Monitoring a Grafana system 150
2.1 Grafana monitoring flow 151
2.2 Disabling the firewall before operation 151
2.3 Configuring the integration in Grafana 152
Issuing a Grafana token 152
Viewing the Grafana Data Source settings 153
2.4 Configuring in AI Builder 154
Registering the folder view API and app card 154
Registering the dashboard view API and app card 160
Registering the panel view API and app card 162
Registering a panel capture API 166
2.5 Deploying 166
2.6 Testing 167

3. Remotely operating a Jenkins system 170

3.1 Jenkins remote operation flow 171
3.2 Preliminary preparations 172
Disabling the firewall 172
Installing the Outbound WebHook for build events plugin 172
3.3 Remotely deploying in a Jenkins system 173
Configuring in AI Builder 173
Registering the job view start API and app card 174
Registering the job execution/result APIs and app cards 181
Registering the job execution results app card and webhook 184
Registering the job execution details API and app card 192
Configuring webhook events in Jenkins 196
Testing 197
3.4 Restarting Jenkins and viewing processes 199
Configuring in AI Builder 199
Registering the job view start API and app card 200
Registering the job execution/result APIs and app cards 204
Registering the job execution results app card and webhook 205
Registering a job execution details API 210
Configuring webhook events in Jenkins 211
Testing 212

Part VIII. Plugin 214

1. Creating a plugin 215

1.1 What is plugin? 215
1.2 Step 1 : Basic Information 215
1.3 Step 2 : Plugin Contents (API) 217
Authorization Information 219
OAuth 2.0 220
Request : Header 220
Request : Param 221
Request : Body 221
1.4 Step 3 : Check summary and test 222

2. Test with simulator **225**

2.1 **API Call Test** 225

2.2 **Using Simulator** 226

Using Debug Mode 227

3. Plugin registration management **229**

3.1 Register plugin 229

Create plugin 229

Request registration 229

3.2 Request change 231

3.3 Deregister 233

4. Plugin permission management **234**

4.1 Add Permission 234

Create Plugin 234

Edit permission 234

Request Registration 236

5. Using plugin in messenger **237**

Appendix A. Integrating notifications/polls with an open API **239**

A.1 Utilizing a notification delivery API 239

A.2 Utilizing a poll delivery API 245

Appendix B. Checking bot operation status with Slash Command **255**

Appendix C. Developing a Workflow **260**

C.1 About Workflow 260

C.2 Component type of Workflow 261

Text 261

App cards 262

API 265

Condition comparison 268

C.3 Using Workflow 269

Preface

Legal notice

Please read the following information carefully before using this tutorial.

Author	Samsung SDS Co., Ltd.
Address	125 Olympic-ro 35-gil, Songpa-gu, Seoul
Telephone	+82 2 1661 3311 (ext. 5)
Email address	global.cs@samsung.com
Website	www.samsungsds.com/kr/

Samsung SDS Co. Ltd. (hereinafter Samsung SDS) has verified the information contained in this document. However, Samsung SDS is not responsible for any problems that may arise from inaccurate content or typographical errors.
The content and specifications in this document are subject to change without notice. For more information on the revision history of this document, visit the SDS website (www.samsungsds.com/en/).
Samsung SDS Co. Ltd. holds all intellectual property rights, including the copyrights, to this document. Using, copying, disclosing, or distributing this document without explicit permission from Samsung SDS is strictly prohibited. These activities constitute an infringement of intellectual property rights.
Copyright © SAMSUNG SDS All rights reserved

Users of this guide

This tutorial is intended for users who develop chatbot services, API-based plugin services, etc. using Samsung SDS Knox AI Builder system.
Knowledge and experience in the following areas are required to effectively utilize the system.

- General knowledge about the use of web applications
- Basic knowledge about chatbot services
- Basic knowledge about API interface

Recommended browser

Use Chrome or the latest version of IE 11 for smooth utilization of Knox AI Builder.

Overview of this tutorial

Part I. AI Builder overview

Explains the basic usage of Knox AI Builder and its features for creating and managing bots and plugins.

Chapter 1. Getting started with AI Builder
Explains the basic usage of AI Builder, including AI Builder access and permissions.

Chapter 2. Introduction to AI Builder features
Explains the menus and features available in AI Builder.

Part II. Bot common processes

Explains the common Knox Messenger AI Builder processes that are used to create bots.

Chapter 1. Bot account and interface
Explains the preliminary steps required to create a bot.

Chapter 2. Configuring target users for a bot
Explains how to configure target users for a bot.

Chapter 3. Making bot account changes
Explains how to make changes to bot account information.

Part III. FAQ bots

Explains how to create a FAQ bot.

Chapter 1. Bot creation and management
Explains how to create a bot in AI Builder.

Chapter 2. Registering a scenario
Explains how to register scenarios to configure your bot.

Chapter 3. Testing a chat and configuring the search feature
Explains how to test chat and manage search functions for registered scenarios using the simulator.

Chapter 4. Deploying the bot
Explains how to deploy registered scenarios so that users can use in messenger.

Chapter 5. Viewing the bot in the Knox Teams
Explains how to check whether registered scenario has been deployed well in the Knox Teams.

Chapter 6. Operating the bot
Explains how to check feedback and analyze logs.

Part IV. AI Bot

Explains how to create an AI bot based-on RAG.

Part V. Notifications and polls

Explains how to send notifications or polls and manage the results.

Chapter 1. Sending notifications
Explains how to register and send notifications in AI Builder.

Chapter 2. Poll delivery flow
Explains the process where the polls registered in AI Builder are delivered to the Knox Teams.

Chapter 3. Registering polls
Explains how to register polls in AI Builder.

Chapter 4. Registering poll schedules
Explains how to register a poll schedule to send the previously registered polls.

Part VI. Bot Management

Explains useful functions for managing bots in AI Builder.

Chapter 1. Dashboard
Explains the dashboard that you can check the bot usage status at a glance.

Chapter 2. Managing feedback
Explains the feedback function that users can register inquiries and you can check them through a bot.

Chapter 3. Monitoring usage statistics
Explains how to check the bot's user status, logs, and contents usage status.

Chapter 4. Chatbot authority management
Explains how to register or delete the authority of chatbot users.

Part VII. ChatOps bots

Explains how to create and use a ChatOps bot that contains operational functions such as event alarms, system status monitoring, and remote operation.

Chapter 1. Receiving Zabbix event notifications
Explains the function to receive alarms through a bot when an event occurs using Zabbix's webhook.

Chapter 2. Monitoring a Grafana system
Explains how to monitor the system with graphs by interface Grafana.

Chapter 3. Remotely operating a Jenkins system
Explains how to remotely run a Jenkins job and check the results.

Part VIII. Plugins

Explains how to create and manage a plugin in AI Builder to register with Knox Copilot.

Chapter 1. Creating a plugin
Explains how to create a plugin with API information in AI Builder.

Chapter 2. Test with simulator
Explains how to test plugin with simulator in AI Builder.

Chapter 3. Plugin permission management
Explains how to manage(add/delete) users who can use plugin in Copilot.

Chapter 4. Plugin registration management
Explains how to register/deregister the created plugin in Copilot.

Chapter 5. Using plugin in messenger
Explains how to use the created plugin in Knox Copilot.

Appendix A. Integrating notifications/polls with an open API

Explains how to send notifications/polls with open API.

Appendix B. Checking bot status with Slash command

Explains management functions using Slash Command such as sentence analysis, check accumulate data.

Appendix C. Developing a Workflow

Explains how to select/place and connect components using Workflow.

Conventions

This document uses the following conventions:

Boldface	**Boldface** indicates various graphic user interface elements, menus, and directories within the main text.
Double quotation marks (", ")	Double quotation marks (", ") indicate a portal, window, pop-up window, wizard, and page.
Italic	*Italic text* is used to indicate the variables or parameters the user should enter.
Fixed-width	Fixed-width text is used to indicate programming terms or code. The body of this document also uses **fixed-width bold** for commands, parameter names, registry names, script names, and process names and *fixed-width italic* for variables and parameter values.

Notes and Cautions

Notes are used to additionally provide users with tips, recommendations, exceptions, limitations, etc.

> Note
> This option will be deactivated and cannot be selected when the IP has been attached to the server or when it is reserved.

Cautions are used to inform users of possible damage to their system, data, or device, or potential injury.

> Caution
> Before proceeding with the termination process, thoroughly consider the effect of the service interruption because the ongoing service may be immediately stopped.

Revision history

Solution version	Revision date	Revisions	Revision details
1.0	2024. 09. 01.	Initial release	Published the first version

Part I.
AI Builder overview

Samsung SDS Knox AI Builder (hereinafter referred Knox AI Builder or AI Builder) is a service that provides the function to create and manage chatbots and plugins that can be used through Knox Messenger/Teams.

1.
Getting started with AI Builder

1.1 Connect to Knox AI Builder

Knox AI Builder can be accessed in the following ways :

① Click the AI Builder link in Knox Dev Center > Service Intro or Dev Guide to access
(데브센터 내 링크 걸린 페이지 캡쳐 이미지)

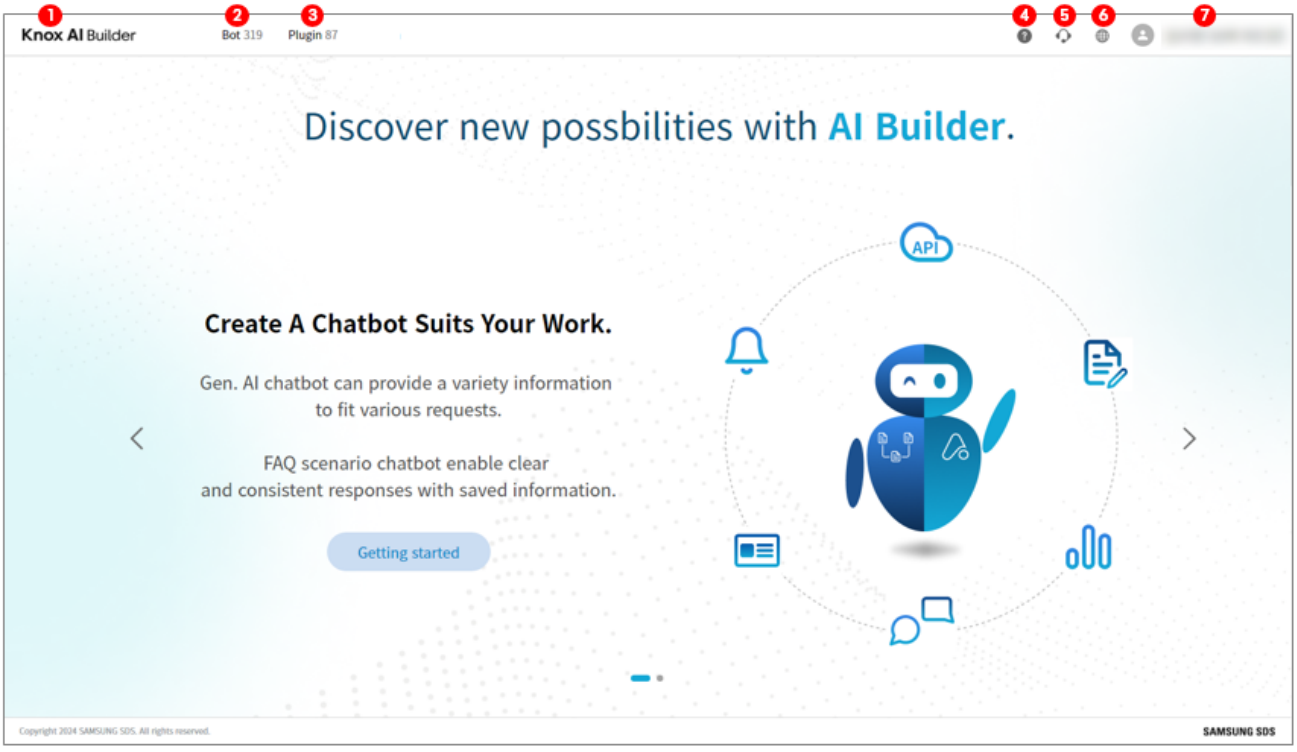
② Enter the URL directly into the Internet browser to access (https://aibuilder.samsung.net/login)

1.2 Knox AI Builder permission to use

Knox AI Builder can be accessed by **any Knox Portal users**.
You can lon in by clicking the "Log in with a Knox ID" while you logged in Knox Portal
without additional user registration.
However, depending on the business site, access may be restricted due to firewall or security policies.
Additionally, some features may be limited depending on the company or individual Knox Copilot permission.

2. Introduction to AI Builder features

2.1 AI Builder basic features



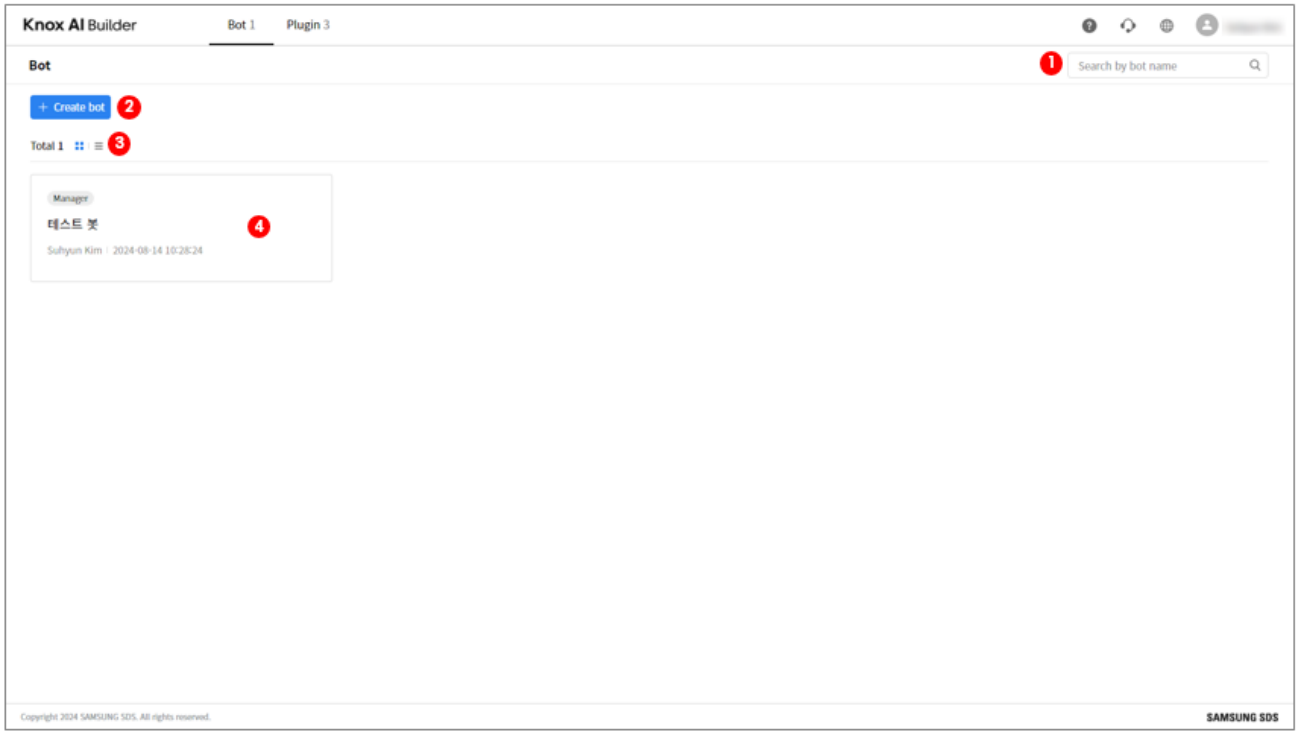
No	Feature	Description
❶	**Logo**	Click to go to AI Builder main page.
❷	**Bot**	Click to go to Bot management page.
❸	**Plugin**	Click to go to Plugin management page. ※ Depending on your Copilot permissions, it may not be displayed.
❹	**Help**	Click to open AI Builder tutorial in a new window.
❺	**Technical support**	Click to go to technical support.

No	Feature	Description
		In technical support, you can find FAQs, release notes, etc.
❻	**Language**	Click to change AI Builder language setting between Korean/English.
❼	**User**	The logged in user name is displayed, and click to show log out button.

2.2 Bot creation and management

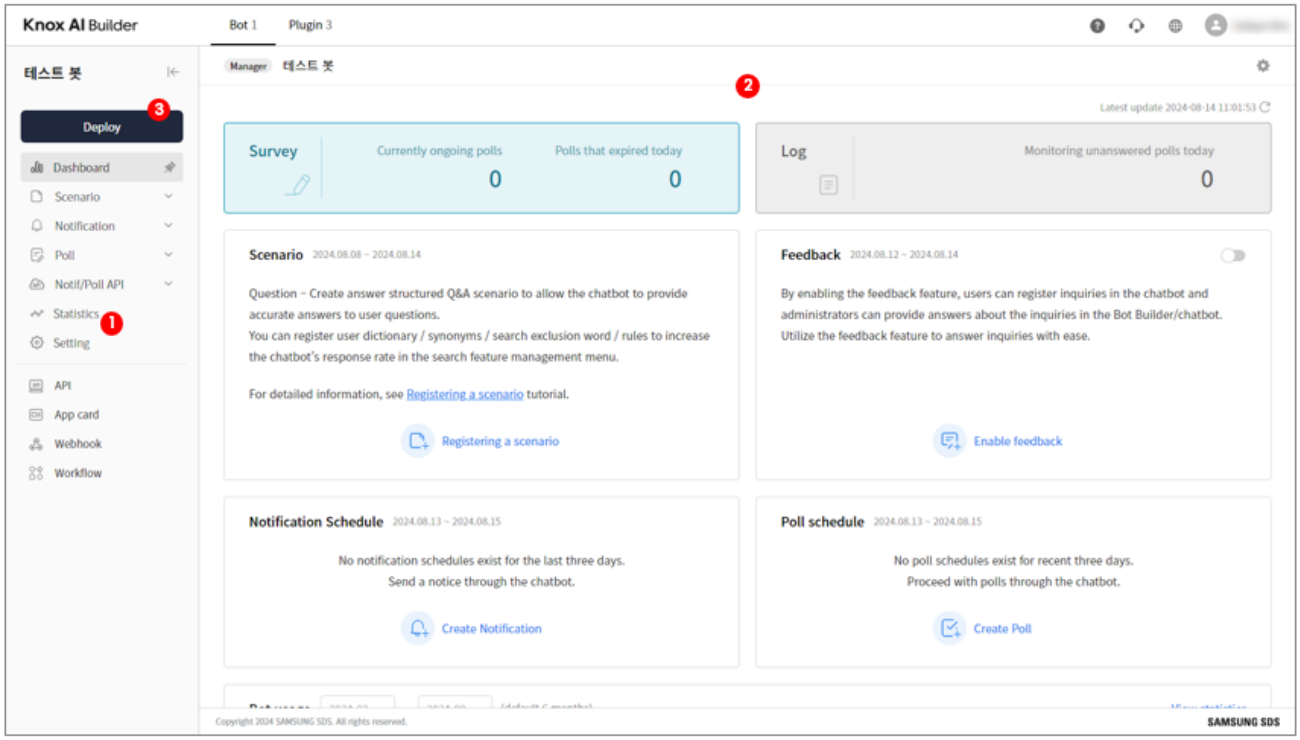
You can create and manage bots in **Knox AI Builder** > **Bot** .

Bot list



No	Feature	Description
❶	**Bot search**	You can search for the bot you want by bot name.
❷	**Create bot**	Click to create a new bot.
❸	**Bot list settings**	The count of bots is displayed, and you can set the bot list display type between card type and list type.
❹	**Bot list**	Bot list is displayed, and click to go to details page for the bot.

Bot details



No	Feature	Description
❶	**Bot management menus**	Bot management menus are displayed. - Dashboard : Dashboard to check bot usage status - Scenario : FAQ bot scenario - Notification : Send and manage notifications - Poll : Send and manage polls - Notification/Poll API : Notifications and polls with API interface - Statistics : Check usage status/log/contents status - Setting : Manage the bot's basic attribute - API : Register and manage external APIs

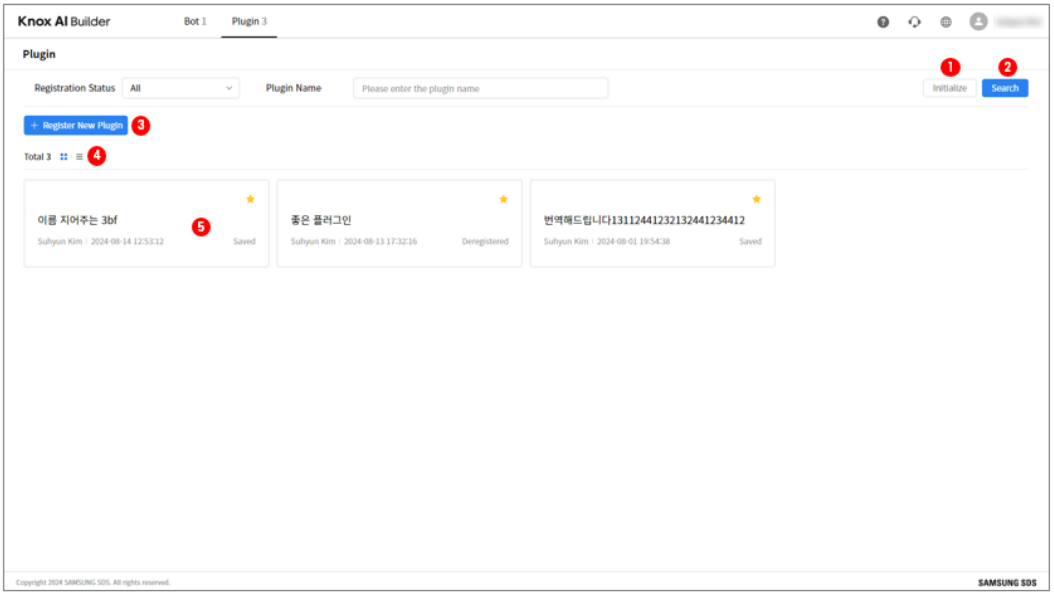
No	Feature	Description
		- App card : Register and manage app cards - Webhook : Manage webhooks and check logs - Workflow : Register and manage Workflow
❷	**Detailed managing area**	The area where of the selected management features are displayed, and the composition varies depending on the menu. For details on each menu, please refer to main text of this tutorial.
❸	**Deploy**	Deploy the settings and saved contents so that be applied to messenger.

For details on each menu, please refer to main text of this tutorial.

2.3 Plugin creation and management

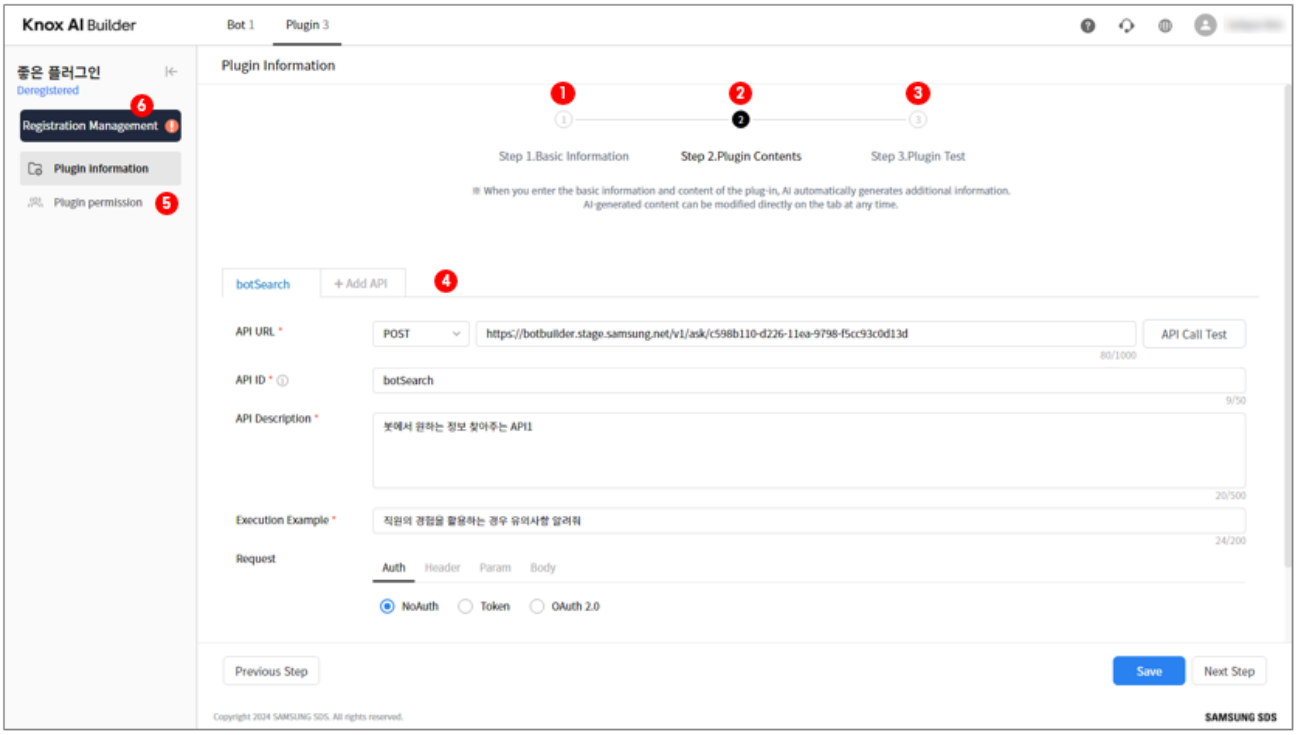
You can create and manage plugins in **Knox AI Builder** > **Plugin** .

Plugin list



No	Feature	Description
❶	**Initialize**	Click to initialize search conditions.
❷	**Plugin search**	You can search for plugins that fit the conditions.
❸	**New plugin**	Click to create a new plugin.
❹	**Plugin list settings**	The count of plugins is displayed, and you can set the plugin list display type between card type and list type.
❺	**Plugin list**	Plugin list is displayed, and click to go to details page for the plugin.

Plugin details



No	Feature	Description
❶	**Basic information**	Manage basic information of plugin.
❷	**Plugin Contents**	Manage API information of plugin.
❸	**Plugin test**	You can check the plugin summary finally, and test it through the simulator.
❹	**Detailed managing area**	The area where of the selected management features from ① to ③ is displayed, and the composition varies depending on the menu.

No	Feature	Description
		For details on each feature, please refer to main text of this tutorial.
❺	**Plugin permissions**	Manage plugin permissions in Knox Copilot.
❻	**Registration Management**	Request registration or deregister plugin and check the registration history. ※ *Plugin registration is completed after check by each company's administrator.*

※ *You can only move to the next step by saving the previous steps from ① to ③ above,*
after saving all steps then you can move freely thorough all steps.

For details on each menu, please refer to main text of this tutorial.

Part II.
Bot common processes

Knox AI Builder allows you to create, deploy, and operate a variety of bots that suit any purposes, such as FAQ bots, notification bots, and poll bots, and efficiently carry out tasks.

This tutorial explains common processes required to provide bot services in Knox Messenger/Teams, such as how to request bot account creation and integration and configure target users (deployment targets).

Complete the following steps provided in the tutorial.

① 1. Bot account creation and interface

② 2. Configuring target users for a bot

③ 3. Making bot account changes

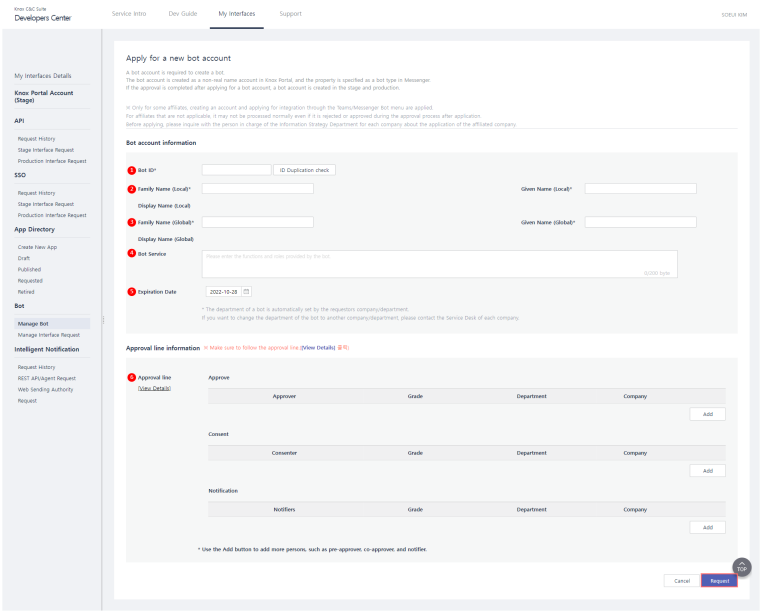
1.
Bot account creation and interface

You can test bots in AI Builder without a Knox bot account.
However, the created bots must be linked with Knox bot accounts before they can be deployed in Knox Messenger.

1.1 Requesting a bot account

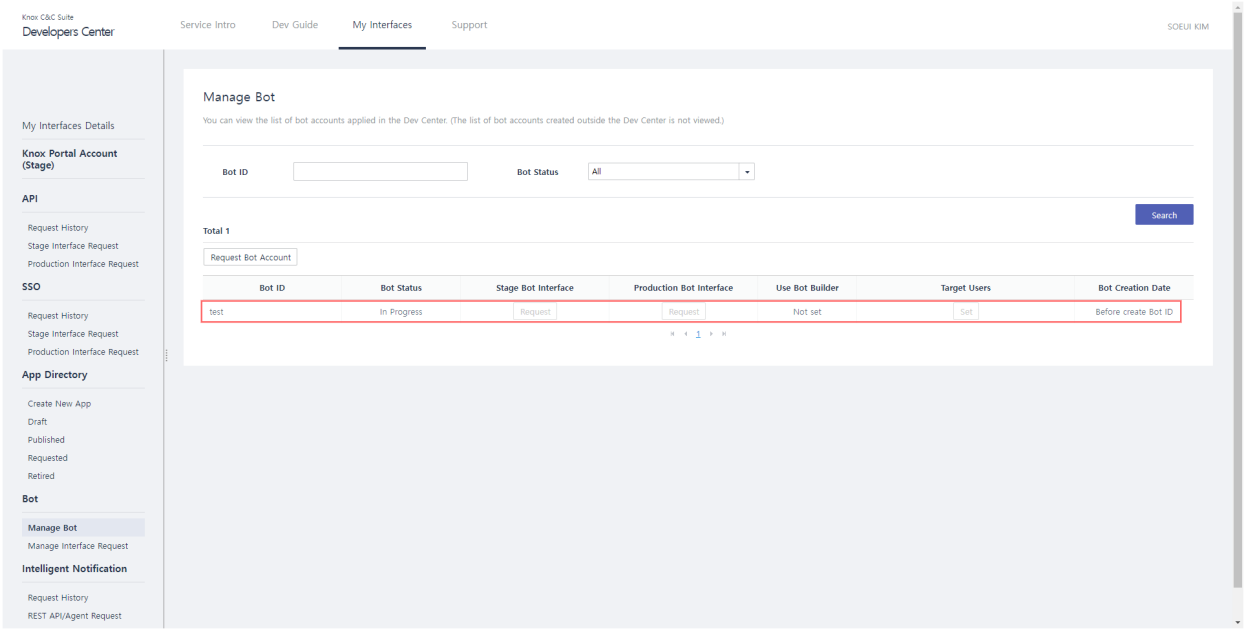
To request a bot account, complete the following steps:

1. Log in to Knox Portal and access the **Knox Dev Center (http://developers.samsung.net/)**.
2. On the "Developers Center" screen, click **My Interfaces** > **Manage Bot** > **Request Bot Account.**
3. On the "Apply for a new bot account" screen, enter the required information and click **Request**.
    - You must fill in the mandatory fields indicated with asterisks (*).



No.	Item	Description
❶	**Bot ID for the integration request**	Enter the bot ID that will be used in AI Builder. • For the integration request, you can use only Knox Portal IDs that are not real names as bot IDs. The use of private IDs is restricted.
❷	**Name(Local)**	Enter the first and last names of the bot.
❸	**Name(Global)**	Enter the first and last names of the bot.
❹	**Bot service**	Enter the functions and roles that will be provided by the bot.
❺	**Expiration date**	Set the expiration date of the bot.
❻	**Approval line**	Add an approval path. • To view the detailed information about the approval path, click **[View Details]**.

4. After the approval has been completed, view the account information of the bot you requested on the bot list of the "Manage Bot" screen.



1.2 Requesting stage bot integration and developing a stage bot

Before you can create and test a stage bot, you must have a stage integration token issued through a stage bot integration request.

Requesting stage bot integration

To request stage bot integration, complete the following steps:

1. Log in to Knox Portal and access the **Knox Dev Center (http://developers.samsung.net/)**.
2. On the "Developers Center" screen, click **My Interfaces** > **Manage Interface Requests** > **Request Stage Bot Interface.**
3. On the "Request Stage Bot Interface" screen, check the preliminary requirements and relevant information, and then click **OK**.

Note

The integration request can be made only after the bot account creation request has been approved. The account creation and integration requests for stage and product bots must be made separately.

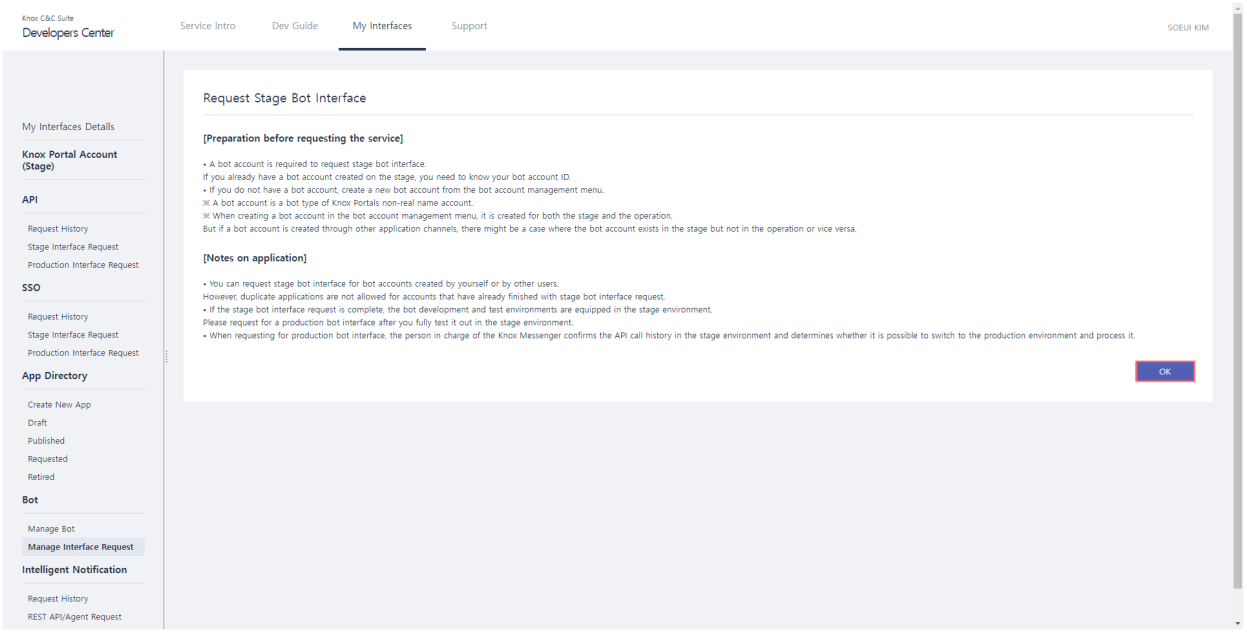
*** Criteria for Unused Bots and Plugins Cleanup**

Bots and plugins that meet the following criteria are classified as long-term unused and are subject to cleanup.

① Bots created without system integration that have not been used for 180 days

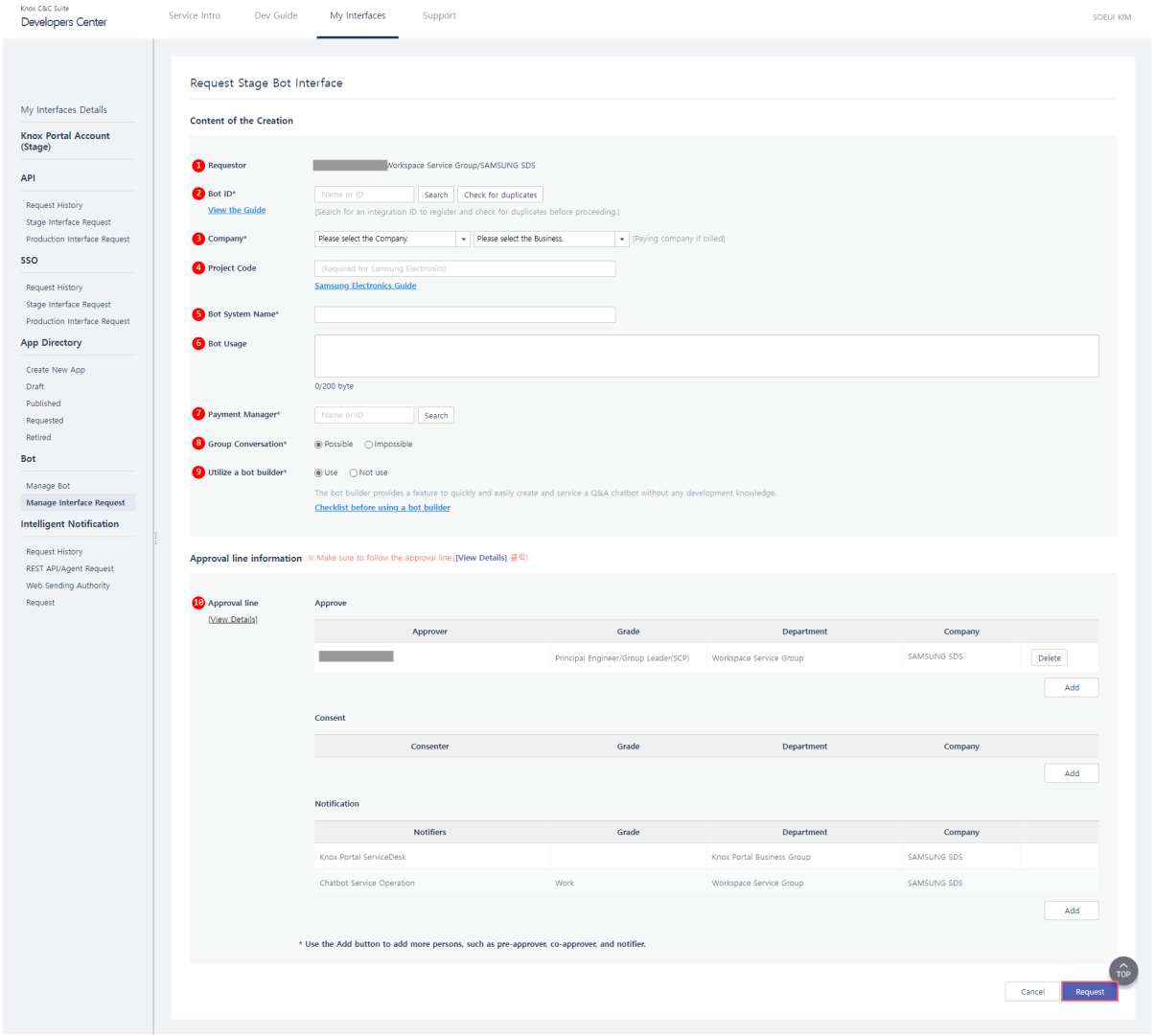
② Bots linked to staging that have no API calls (including webhooks) or interaction history for 180 days

③ Plugins in draft (temporary save) status that have no modification history for 180 days



4. Enter the bot integration request information and click **Request**.
   - You must fill in the mandatory fields indicated with asterisks (*).

Bot account creation and interface - Requesting stage bot integration and developing a stage bot



No.	Item	Description
❶	**Requestor**	Information about the applicant is displayed.
❷	**Bot ID**	Check the bot ID created through the bot account creation request and enter the bot ID.
❸	**Company**	Select the company responsible for the payment if fees are incurred.
❹	**Project code**	Enter the project code if a relevant code exists.
❺	**Bot system name**	Enter the bot system name.
❻	**Bot Usage**	Enter the purpose of the bot.
❼	**Payment Manager**	Search for the name or ID of the billing manager and enter it. Usage fees for each product Knox chatbot and the messaging API will be billed monthly to the billing manager.
❽	**Group Conversation**	Select whether to use the group chat invitation feature.

No.	Item	Description
❾	**Utilize a AI Builder**	Select whether to use AI Builder. • You must select **Use** to use AI Builder. The relevant bot account will be automatically linked with the AI Builder system.
❿	**Approval line**	The approval path information is displayed.

Note

- The bot creation is completed when the Knox bot ID can be searched on the Knox bot list in Knox Messenger/Teams after the token for stage integration has been received.
- For inquiries about processing status of bot creation requests, contact the Knox Portal service desk (02-1661-3311, Ext. 2→3→1).

Developing and testing a stage bot

When you create a bot in AI Builder, you can select **Stage bot** for the **Messenger integration** item, enter the Knox bot ID and stage integration token issued by the Knox Dev Center, and deploy the bot to test it in stage Knox Messenger/Teams.

1.3 Requesting product bot integration and developing a bot

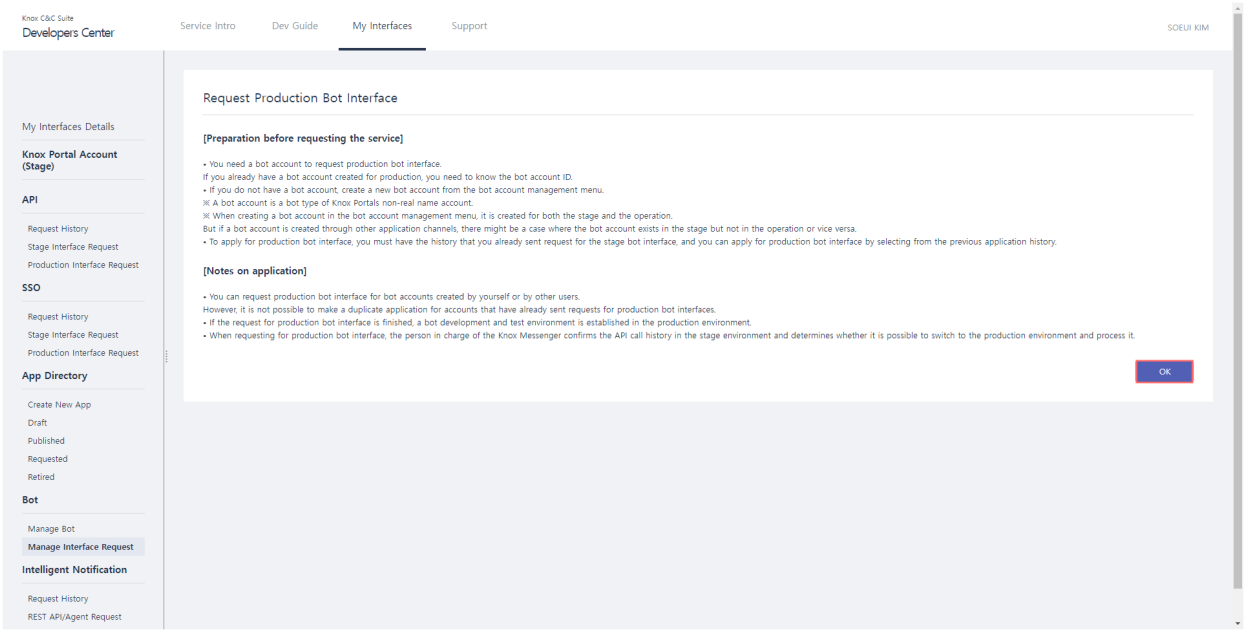
Before you can create a product bot and deploy it to users, you must have the integration token issued through a product bot integration request.

Requesting product bot integration

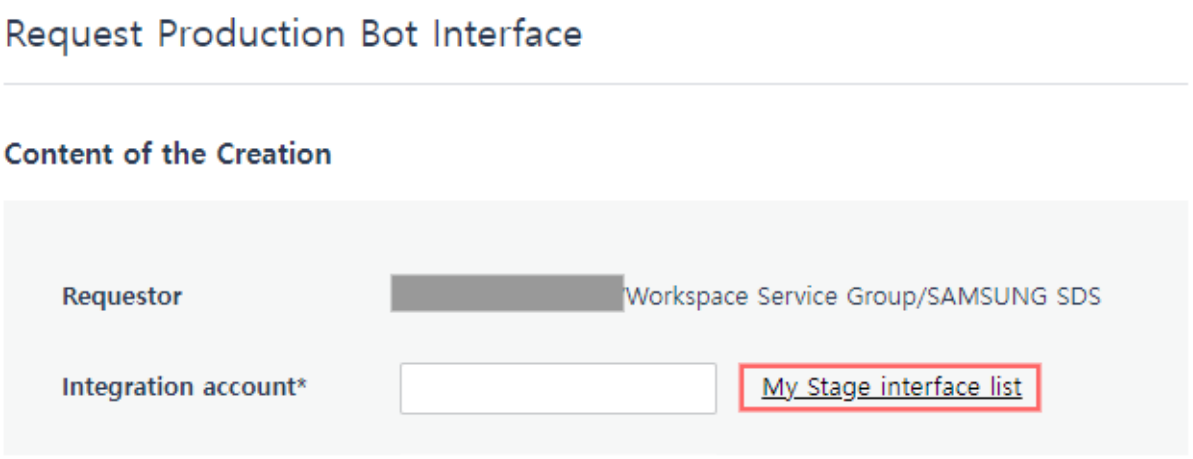
To request product bot integration, complete the following steps:

> Note
>
> - The integration request can be made only after the bot account creation request has been approved. The account creation and integration requests for stage and product bots must be made separately.
> - Stage bot integration is required before you can proceed with product bot integration request.

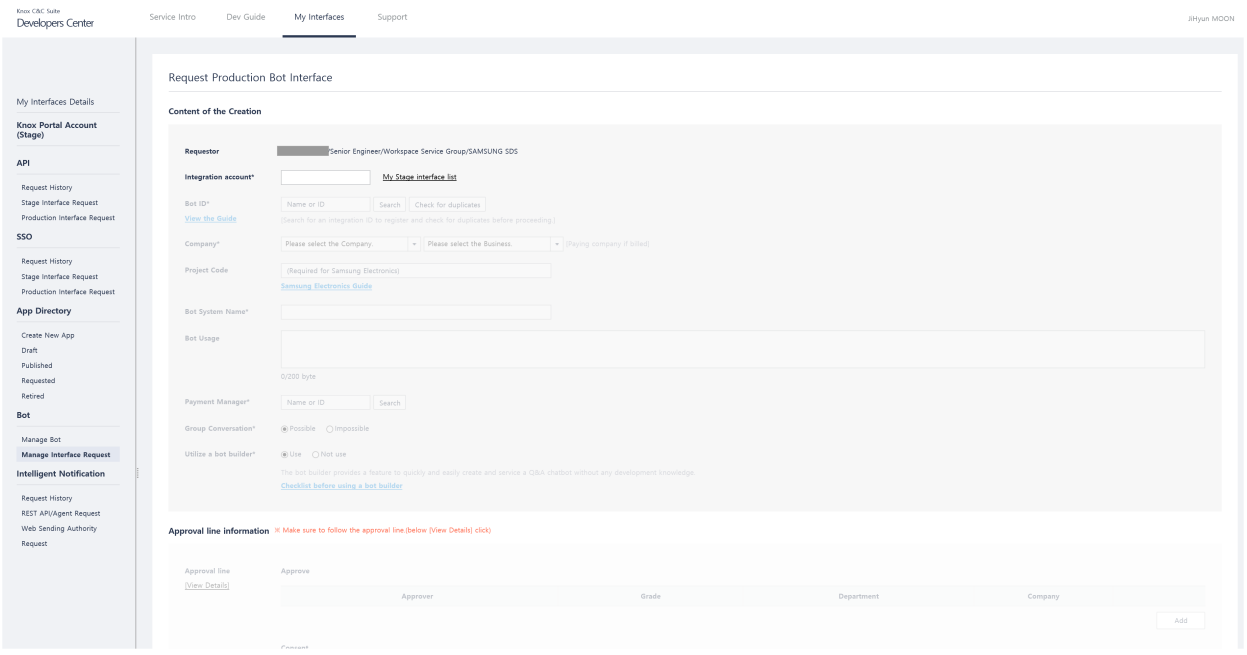
1. Log in to Knox Portal and access the **Knox Dev Center (http://developers.samsung.net/)**.
2. On the "Developers Center" screen, click **My Interfaces** > **Manage Interface Requests** > **Request Product Bot Interface.**.
3. On the "Request Product Bot Interface" screen, check the preliminary requirements and relevant information, and then click **OK**.



4. Click **My stage interface list** in the **Integration account** item.



5. In the "My Stage interface list" pop-up window, select a stage bot for the integration, enter the bot information, and then click **Request**.



Developing and testing a product bot

When you create a bot in AI Builder, you can select **Product bot** for the **Messenger integration** item, enter the Knox bot ID and product integration token issued by the Knox Dev Center, and deploy the bot to test it in the actual product Knox Messenger/Teams.

2. Configuring target users for a bot

To grant bot access permission for individual users or other teams, please contact Knox Portal service desk.

> Note
> Knox Portal service desk: knoxportal@samsung.com, 02-1661-3311(2→3→2)

3.
Making bot account changes

The IT strategy team of each company is responsible for managing Knox bot account information, including the names (Korean/English), profile images, teams, and billing manager. To make changes to Knox Messenger/Teams bot account information, contact the Knox Portal service desk.

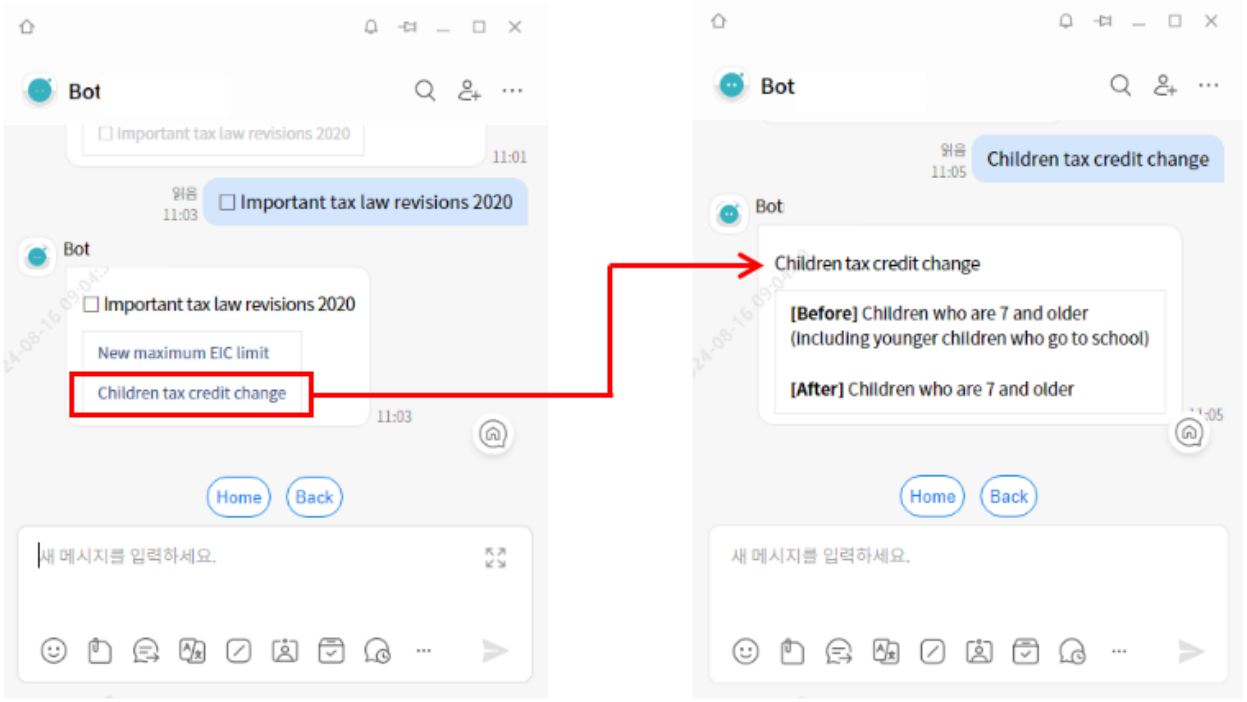
> Note
> Knox Portal service desk: 02-1661-3311(2→3→1)

Part III.
FAQ bot

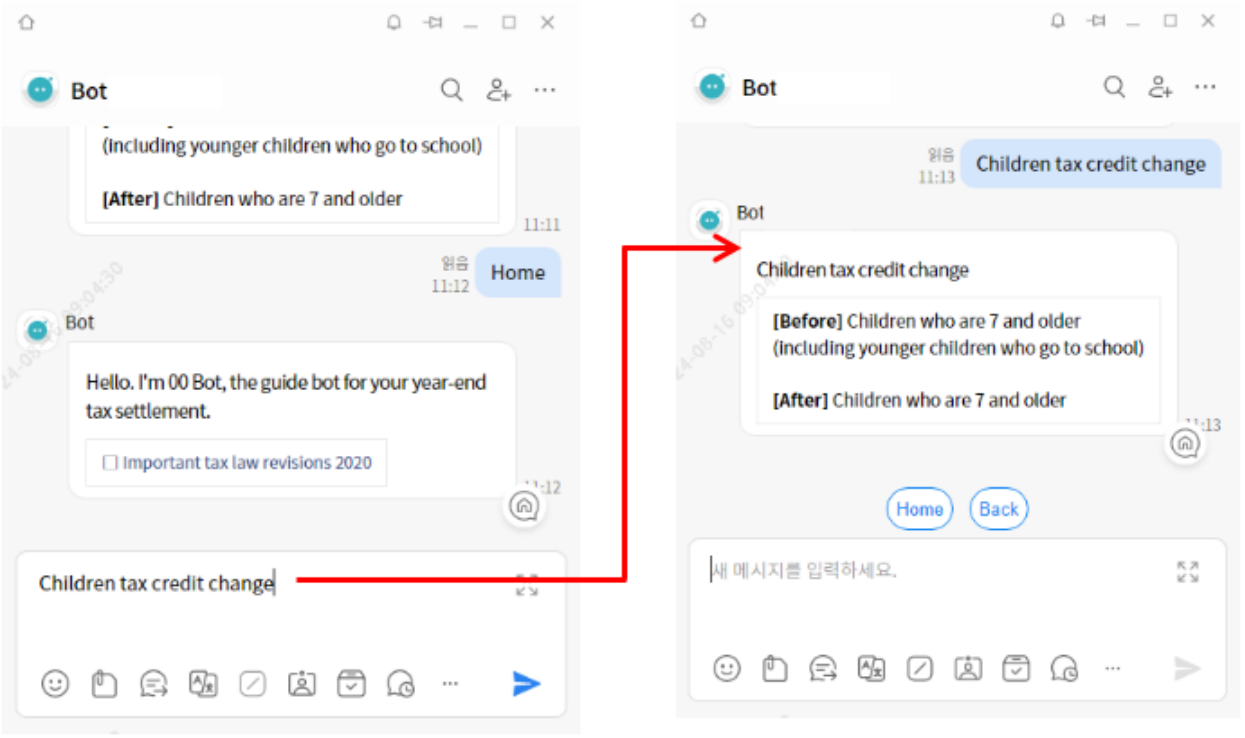
An FAQ bot refers to a QnA bot, which allows you to conveniently view frequently looked-up information through clicks or chats.

You can use the FAQ bot feature to create a year-end tax settlement bot that provides tax settlement guidelines. A year-end tax settlement guide bot can provide searches via clicks and chats (by extracting keywords) and answer individual questions via the feedback feature.

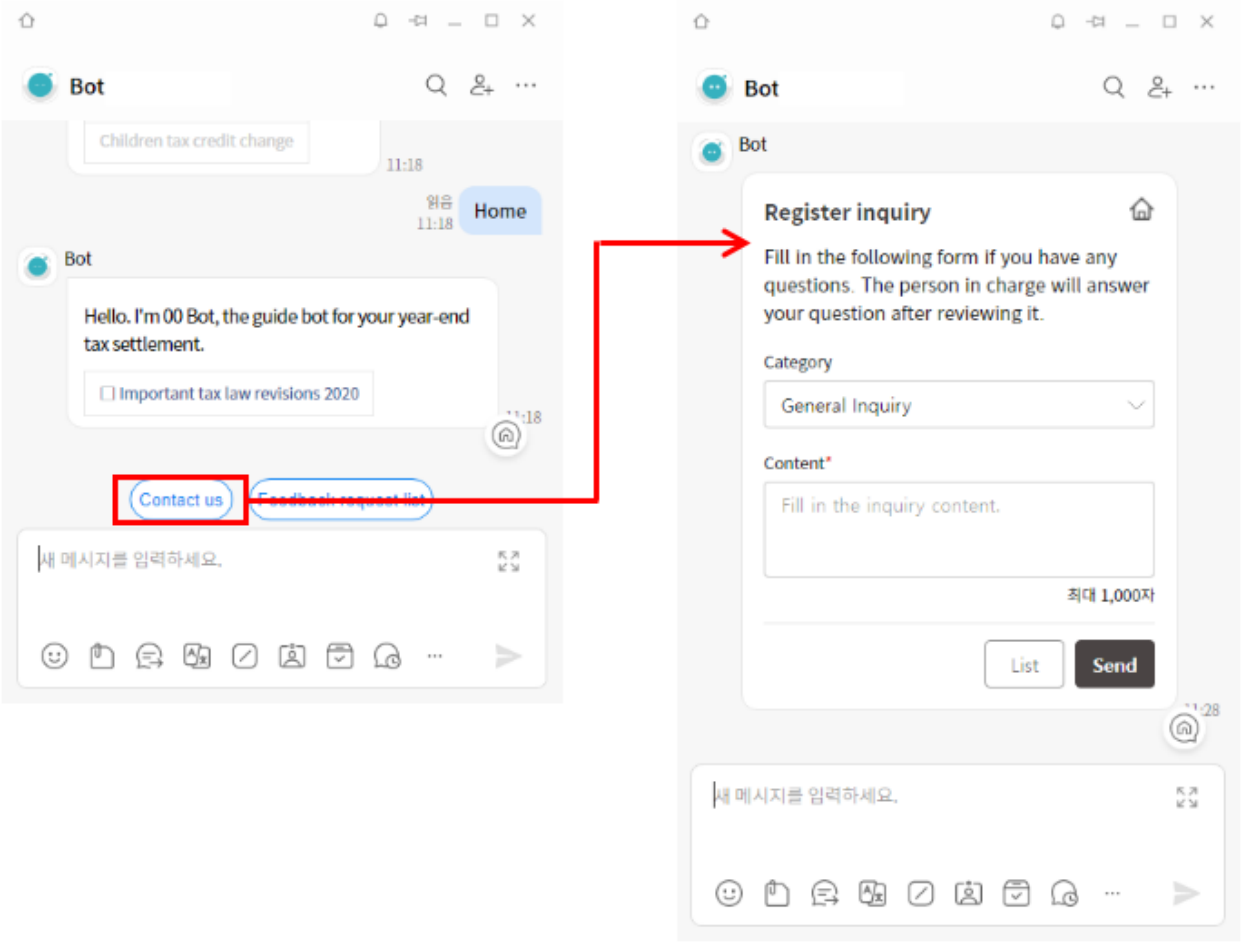
1. Searching via clicks



2. Searching via chats (keyword extraction)


Bot
(including younger children who go to school)
[After] Children who are 7 and older
11:11
읽음
11:12
Home
Bot
Hello. I'm 00 Bot, the guide bot for your year-end tax settlement.
Important tax law revisions 2020
Children tax credit change
읽음
11:13
Children tax credit change
Bot
Children tax credit change
[Before] Children who are 7 and older (including younger children who go to school)
[After] Children who are 7 and older
Home
Back
새 메시지를 입력하세요.

3. Answering to individual questions via the feedback feature

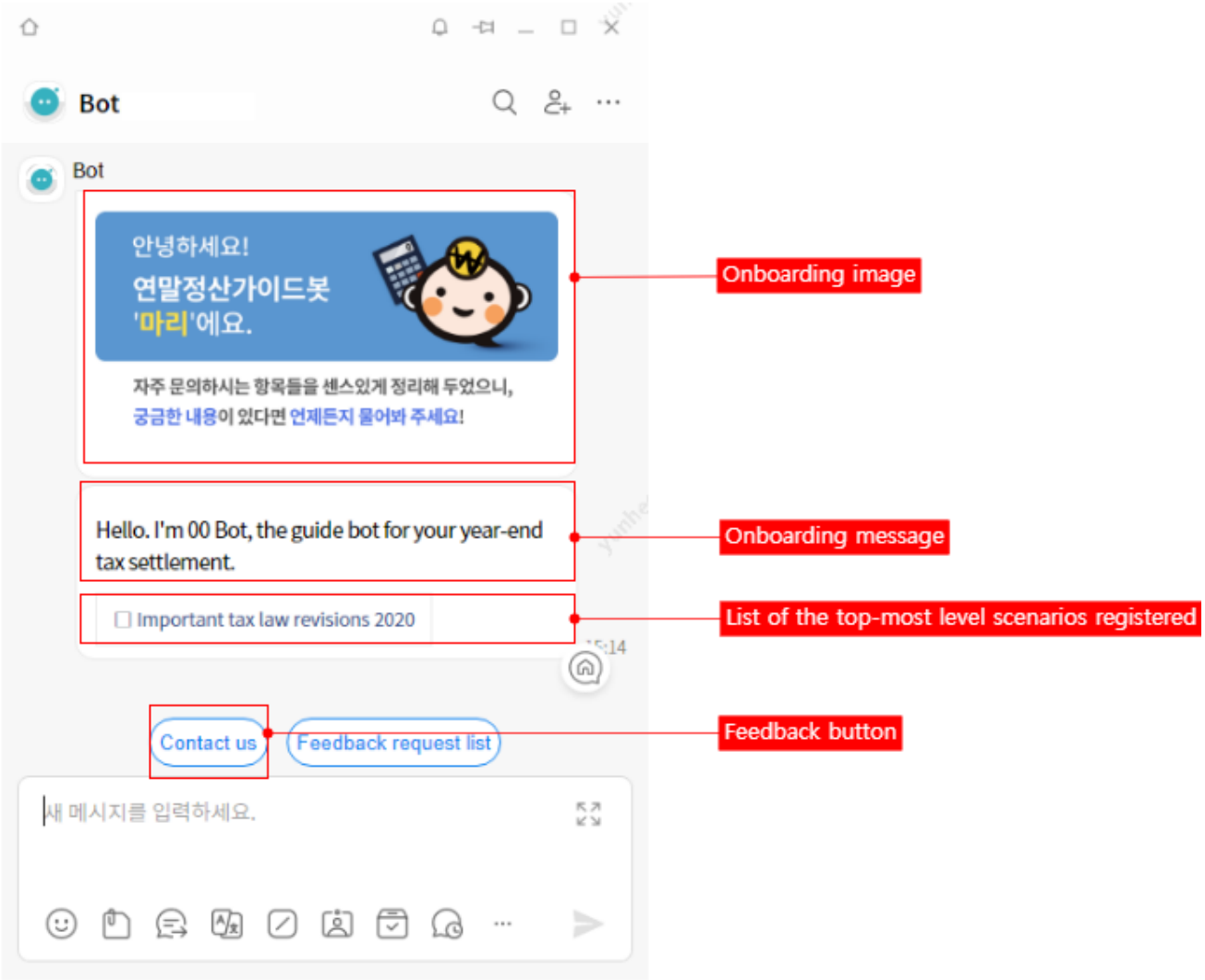


Refer to the following tutorial to create a year-end tax guide bot.

> Note
> This also tutorial includes the process required to deploy the bot created in AI Builder to Knox Stage Messenger/Teams. Therefore, before you can view the bot in the messenger, after creating it according to the tutorial, you must create an account for the bot and and request the stage integration according to the instructions in Part II. Bot common processes

1.
Creating a bot

The following figure explains the chatbot’s user interface.

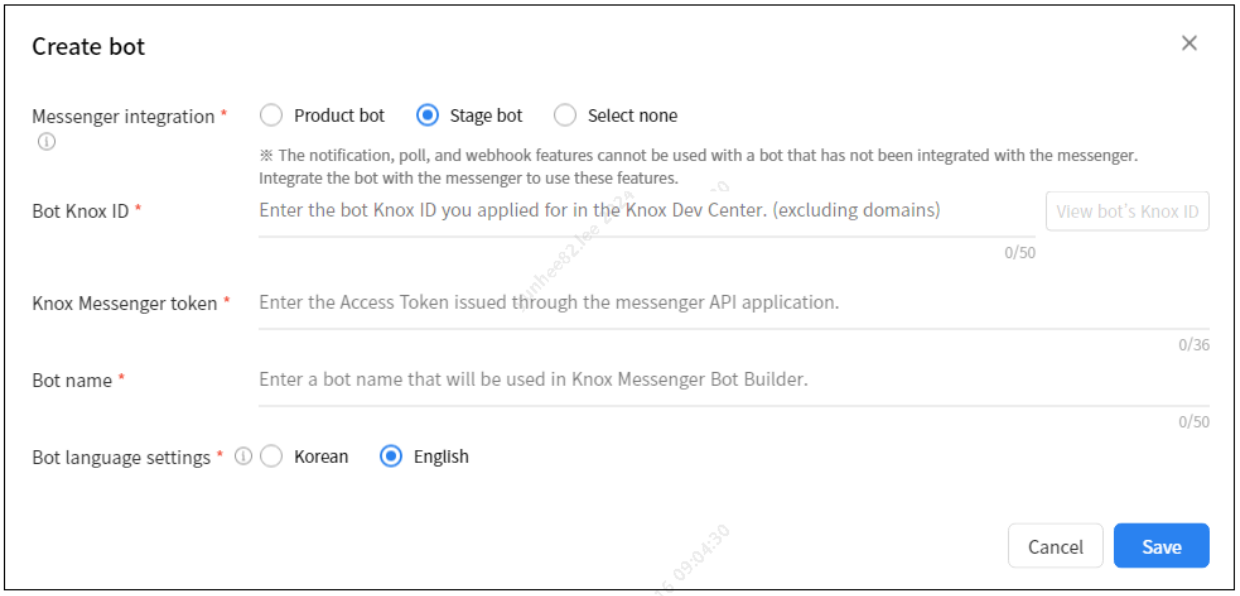


Complete the following steps to create a bot:

1. At the top of the AI Builder home screen, click the Bot tab.
2. Click the “Create bot” button in the middle of the "Bot" screen or in the top-left corner of the bot list.

3. In the "Create bot" pop-up window, enter the following information and click **Save**. You can freely input adequate information for the fields that have not been specified.

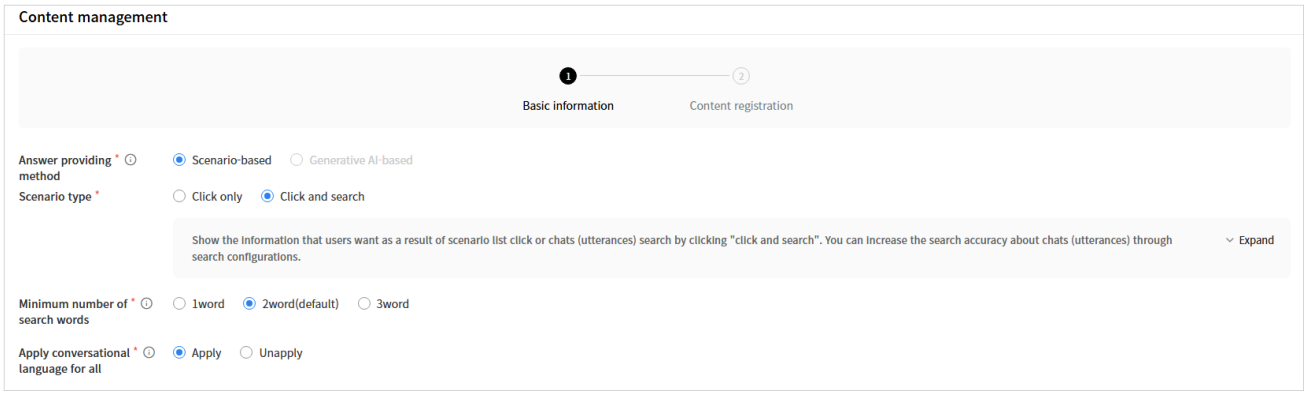
- Messenger integration : Select "Stage bot"
- Bot Knox ID: Enter the bot ID you requested in the Knox Dev Center (My Interfaces > Bot > Bot Management)
- Knox Messenger token: Enter the stage bot integration token issued by the Knox Dev Center (My Interfaces > Bot > Integration Request Management)
- Bot name: Enter : Enter a bot name("Year-End Tax Adjustment Bot") to distinguish bots within the AI Builder. (Not the bot name displayed in Knox Teams)
- Bot language settings: Select "English"



2.
Registering a scenario

Click “Year-End Tax Adjustment Bot” which you created earlier, from the Bot list, then click the Scenario Management menu. Step1. Enter the following information on the basic information screen, then click Save.

- Bot type: Select “Click and Search”
- Min. keywords: Select “2 words (default)”
- Feedback feature: Select “Use”



After saving the data, click Next Step and move to the step 2 for entering scenarios.

You can register individual scenarios or use a CSV file to register a scenario. In this tutorial, we will use “a CSV file” to register a scenario.

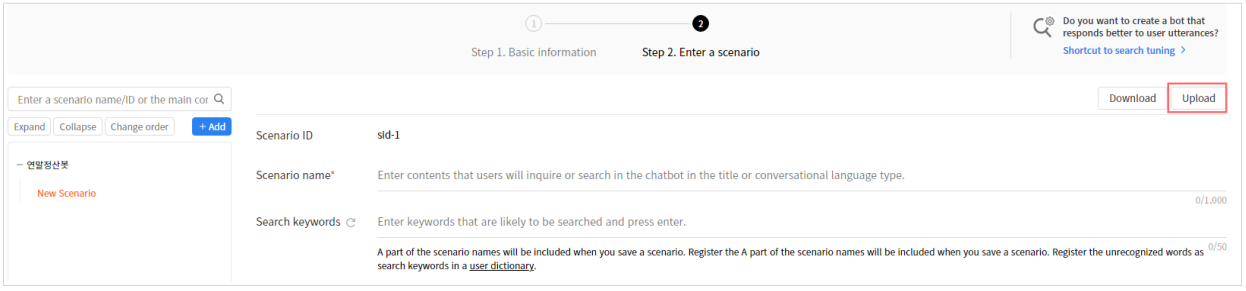
Complete the following steps to register a scenario with a CSV file:

1. Click the following link to download the sample CSV file.

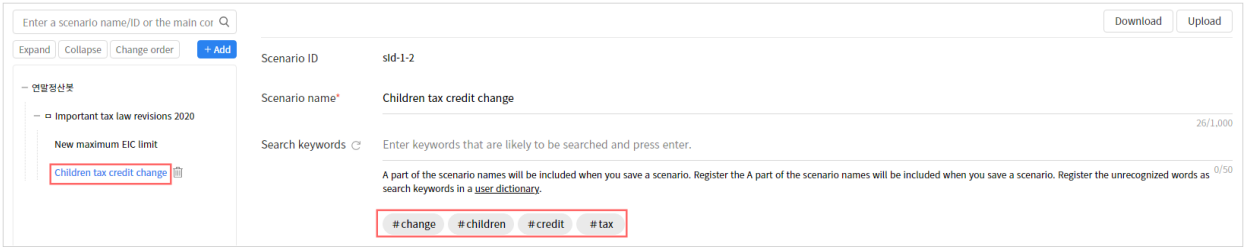
   [Sample CSV file]

Upper-level scenario ID	Scenario ID	Scenario name	Content	Mobile content	Search Keywords	Conversational language Use
TOP	A01	□ Important tax law revision s 2020				
A01	A01-01	New maximum EIC limit	New maximum EIC limit: 20 million KRW ※ Note: Credit by gross salary range · Less than 5 million KRW: 70% · 5 million to 15 million KRW: 40% · 15 million to 45 million KRW: 15% · 45 million to 100 million KRW: 5% · More than 100 million KRW: 2%	□ New maximum EIC limit: 20 million KRW ※ Note: Credit by gross salary range · Less than 5 million KRW: 70% · 5 million to 15 million KRW: 40% · 15 million to 45 million KRW: 15% · 45 million to 100 million KRW: 5% · More than 100 million KRW: 2%		O
A01	A01-02	Child tax credit change	[Before] Children who are 7 and older (including younger children who go to school) [After] Children who are 7 and older	[Before] Children who are 7 and older (including younger children who go to school) [After] Children who are 7 and older		O

2. Upload the CSV file you have downloaded. Only decrypted regular document files can be uploaded.



The scenario will be registered when the CSV file upload is complete, and the nouns in the scenario name will be automatically registered as search keywords.



You can change the scenario type to “Image,” “Bot link,” “Web link,” or “App card” if needed.

• Image: Images can be uploaded to the scenario body. Only one image can be registered.

• Connect bots: Enter the bot ID you want to connect to when you click the scenario body.

• Web link: Enter the URL you want to connect to when you click the scenario body.

• App Card: You can register your app card in the scenario body.

• Workflow: You can register Workflow in the scenario body.

• Create a chat room: When you click the scenario body, a chat room is created with the registered user ID.

• Excel: You can paste Excel contents into the scenario body.

3.
Testing a chat and Search tuning

3.1 Simulating a chat

After registering a scenario, you you can click the simulator icon in the bottom-right corner of the “Scenarios” screen to test a chat.

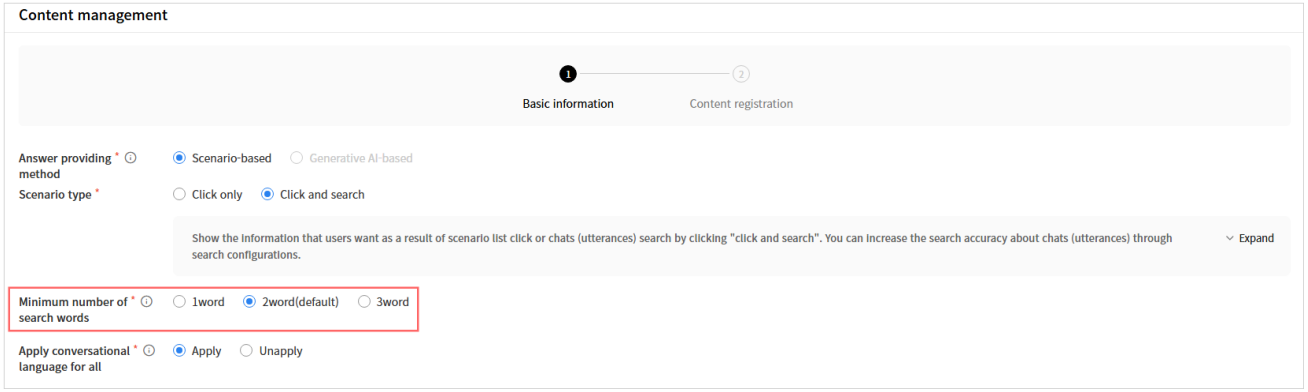
In the simulator, the FAQ chatbot created in AI Builder responds after analyzing chats (utterances) to figure out user intents through the following process.

① Chat → ② Rule matching check → ③ Search keyword check → ④ Response

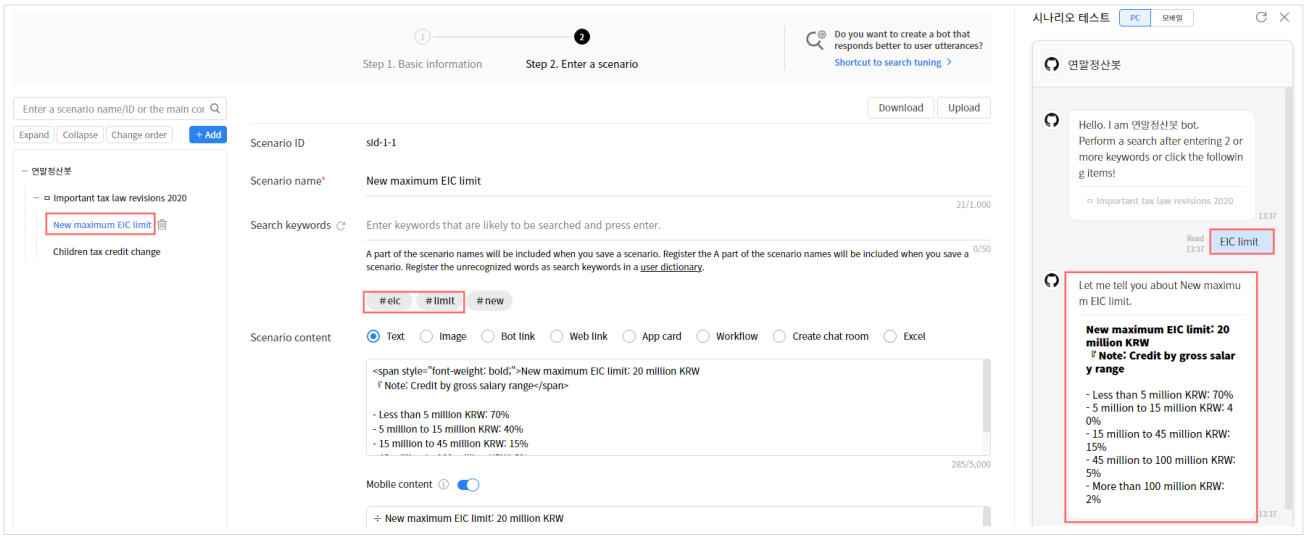
Note
• If a response can be made with the result of the “② Rule matching check” process, processes ③ will not be carried out.

Let’s first proceed with a chat test according to the “④ Search keyword check” process.

In the chat window of the simulator, enter “credit limit.” Because you set the minimum number of search keywords to “2 words,” the year-end tax settlement bot will extract two morphemes, “credit” and “limit”



and respond with a scenario that contains both “credit” and “limit” as the search keywords.



Note

Only scenarios that contain all of the words extracted from user chats will be searched.

For example, if the user enters “Credit limit amount,” morphemes “credit”, ”limit”, and amount” will be extracted.
In this case, scenarios that have only “credit” and “limit” as search keywords will not be searched, and only scenarios contain “credit,” “limit,” and “amount” as search keywords will be searched.

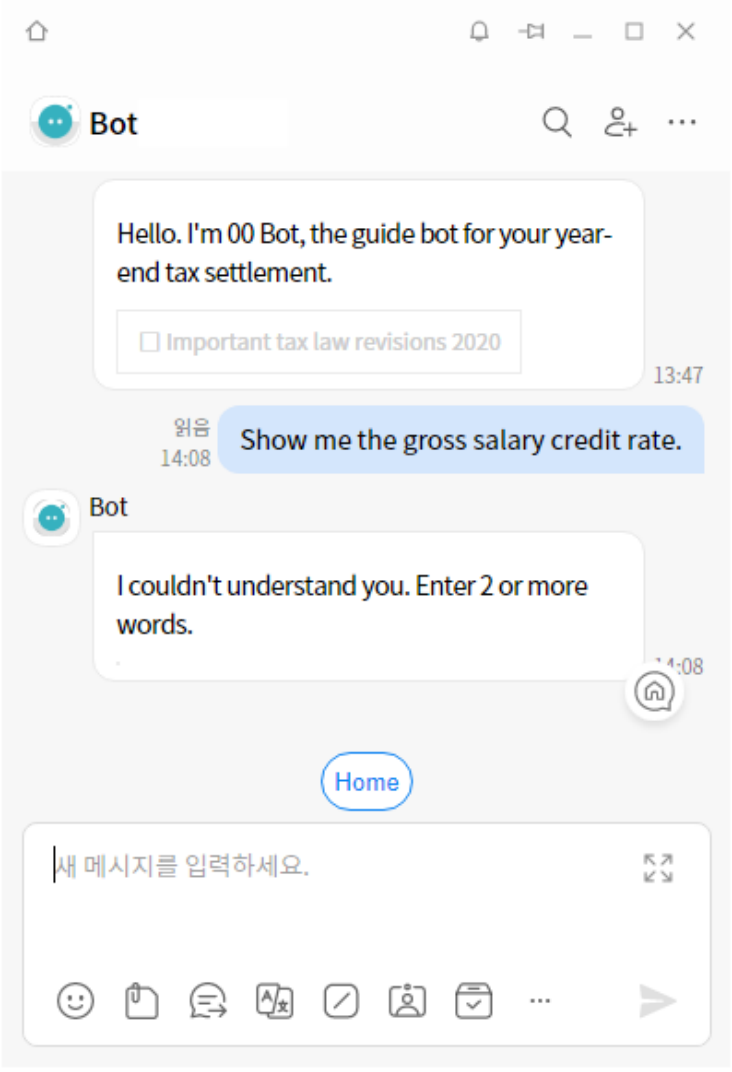
To allow the chatbot to respond using a scenario with a search keyword when only one morpheme can be extracted from a user chat, you can set the minimum number of search keyword to “1 word.”

As described earlier, because morphemes of a scenario name will be registered as search keywords when the scenario is registered, the bot will be able to respond to user chats. You can further increase the chat bot's response rate by registering search keywords and synonyms.

3.2 Registering search keywords

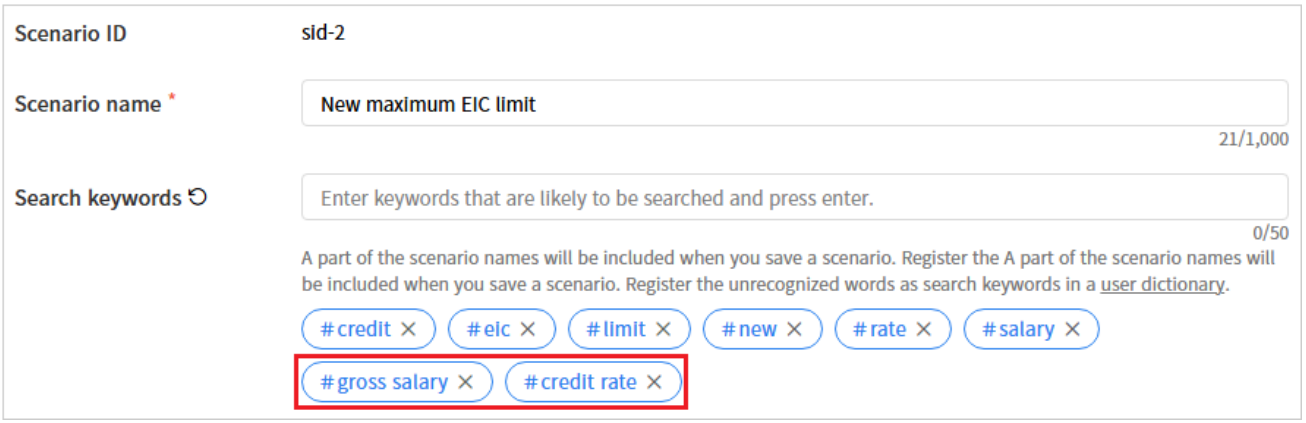
Register the words that will be utilized as search keywords as hashtags beside the search keywords automatically registered when scenarios are saved.

In the chat window of the simulator, enter "Show me the gross salary credit rate."

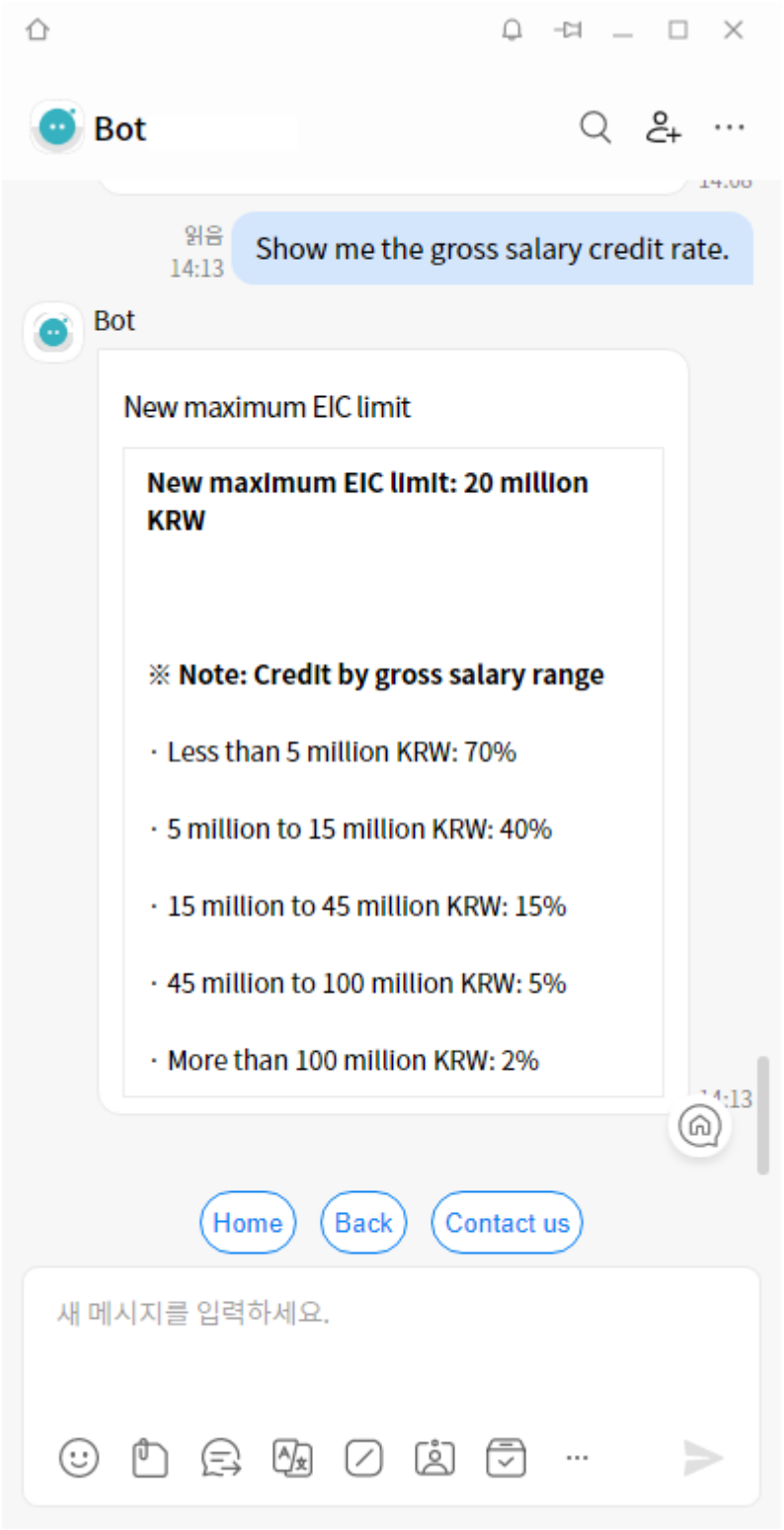


A fallback message will be provided as the response because the noun words extracted from the user chat, “gross,” “salary,” “credit,” and “rate” have not been registered as keywords.

In the “Search keywords” field of the “New maximum EIC limit” scenario, enter “gross salary” “credit rate” and click save. Then, in the chat window of the simulator, enter “Show me the gross salary credit rate.”



As shown in the following figure, the bot will respond with the “New maximum EIC limit” scenario, which has the matching search keywords.


Bot
읽음
14:13
Show me the gross salary credit rate.
Bot
New maximum EIC limit
New maximum EIC limit: 20 million KRW
※ Note: Credit by gross salary range
· Less than 5 million KRW: 70%
· 5 million to 15 million KRW: 40%
· 15 million to 45 million KRW: 15%
· 45 million to 100 million KRW: 5%
· More than 100 million KRW: 2%
Home
Back
Contact us
새 메시지를 입력하세요.

3.3 Registering synonyms

Among the words already registered as the search keywords of the “New maximum EIC limit” scenario, let’s register a synonym for the word “new.”

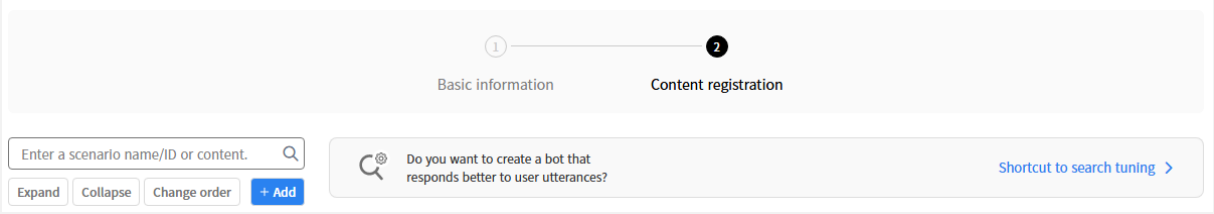


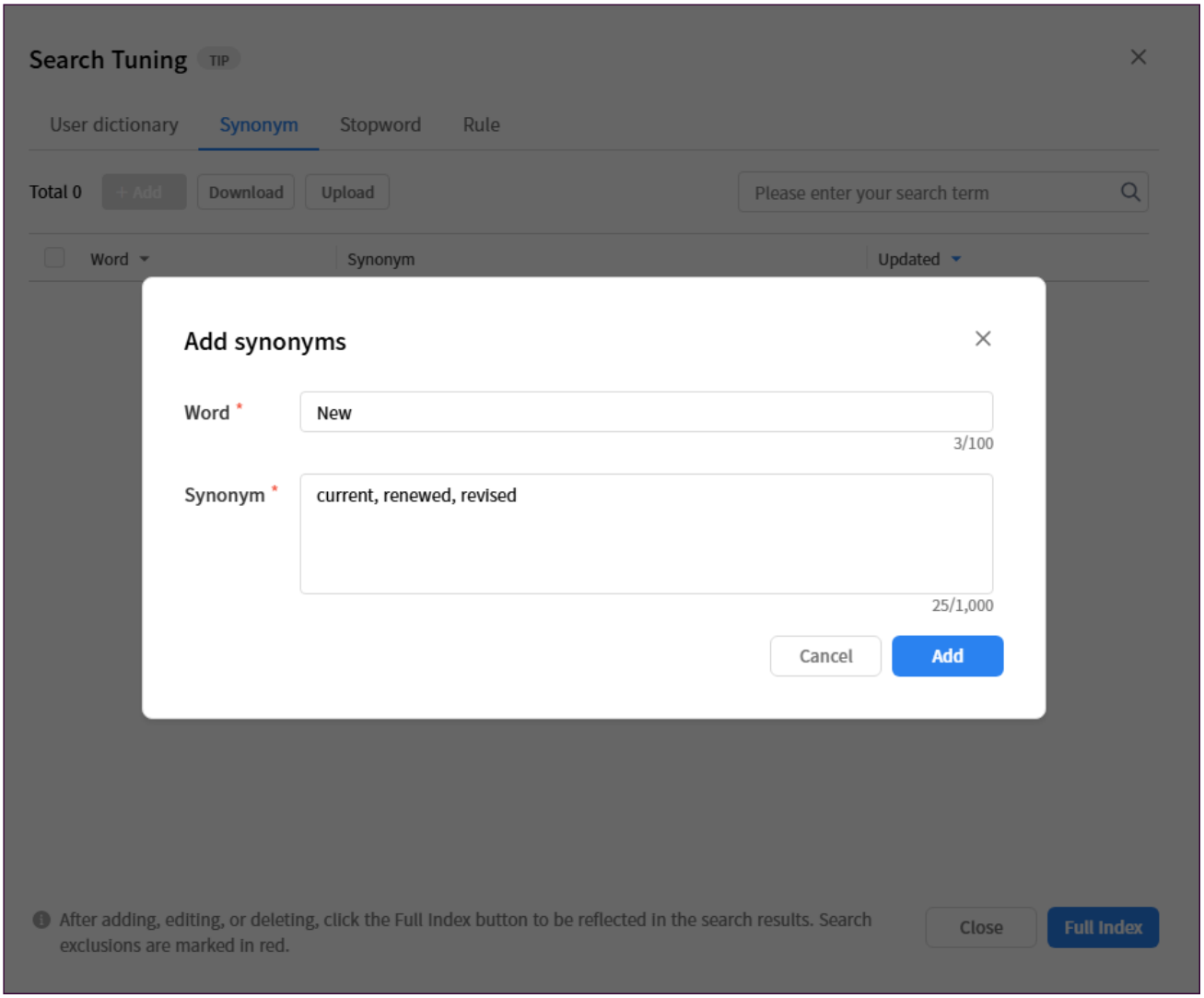
You can specify a registered search keyword as the representative word and register its synonyms to allow the synonyms of the representative word to be used as the search keywords. Therefore, you can simply register synonyms instead of individually registering words with similar meanings as search keywords.

On the synonym registration screen, specify “new” as the representative word, enter the following synonyms, and then click **Add**.

Note

Click the “Shortcut to search tuning” link in the top right corner of the scenario management, and a Search Tuning pop-up window will be displayed to enter a user dictionary or synonyms, etc.

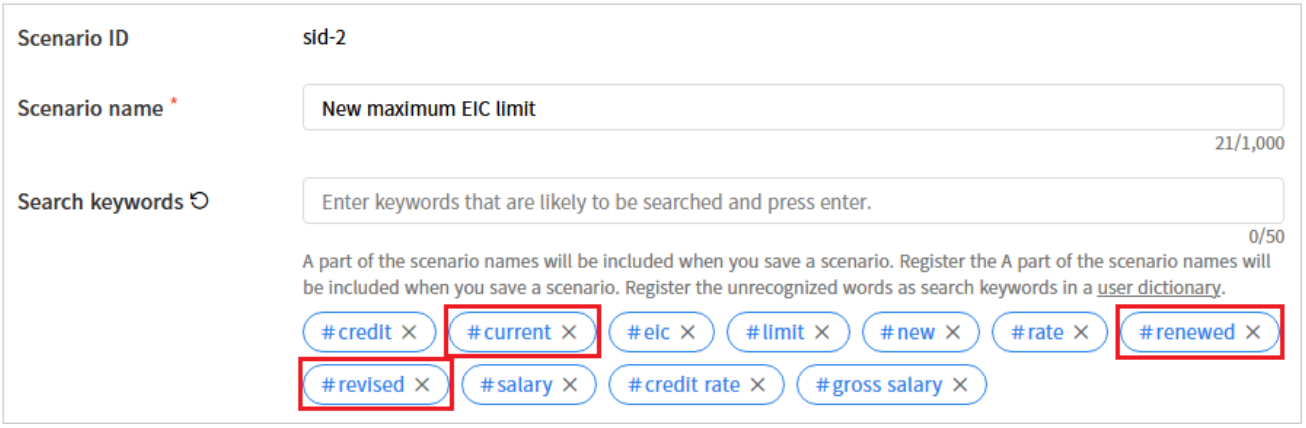




Note

After adding, you must click the “Full Index” button in the bottom right corner of the pop-up window to register the synonyms as search keywords.

On the “New maximum EIC limit” scenario screen, ensure that the synonyms of “new” have been added as search keywords.



By registering synonyms of the word “new,” you can allow your scenario to respond to a variety of user chats (utterances).

3.4 Registering a rule

The “② Rule matching check” process is the first step of the user intent analysis performed by the simulator, which is required for the bot to respond to user chats.

① Chat → **② Rule matching check** → ③ Search keyword check → ④ Response

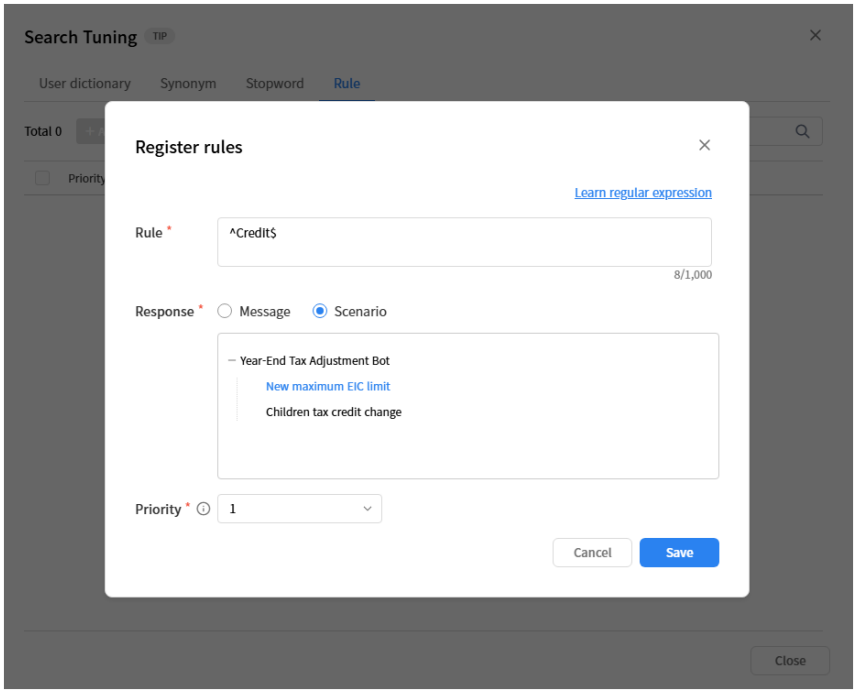
Rules are used to **instantly respond to users instead of checking the search keywords registered as search keywords or synonyms**.

Note

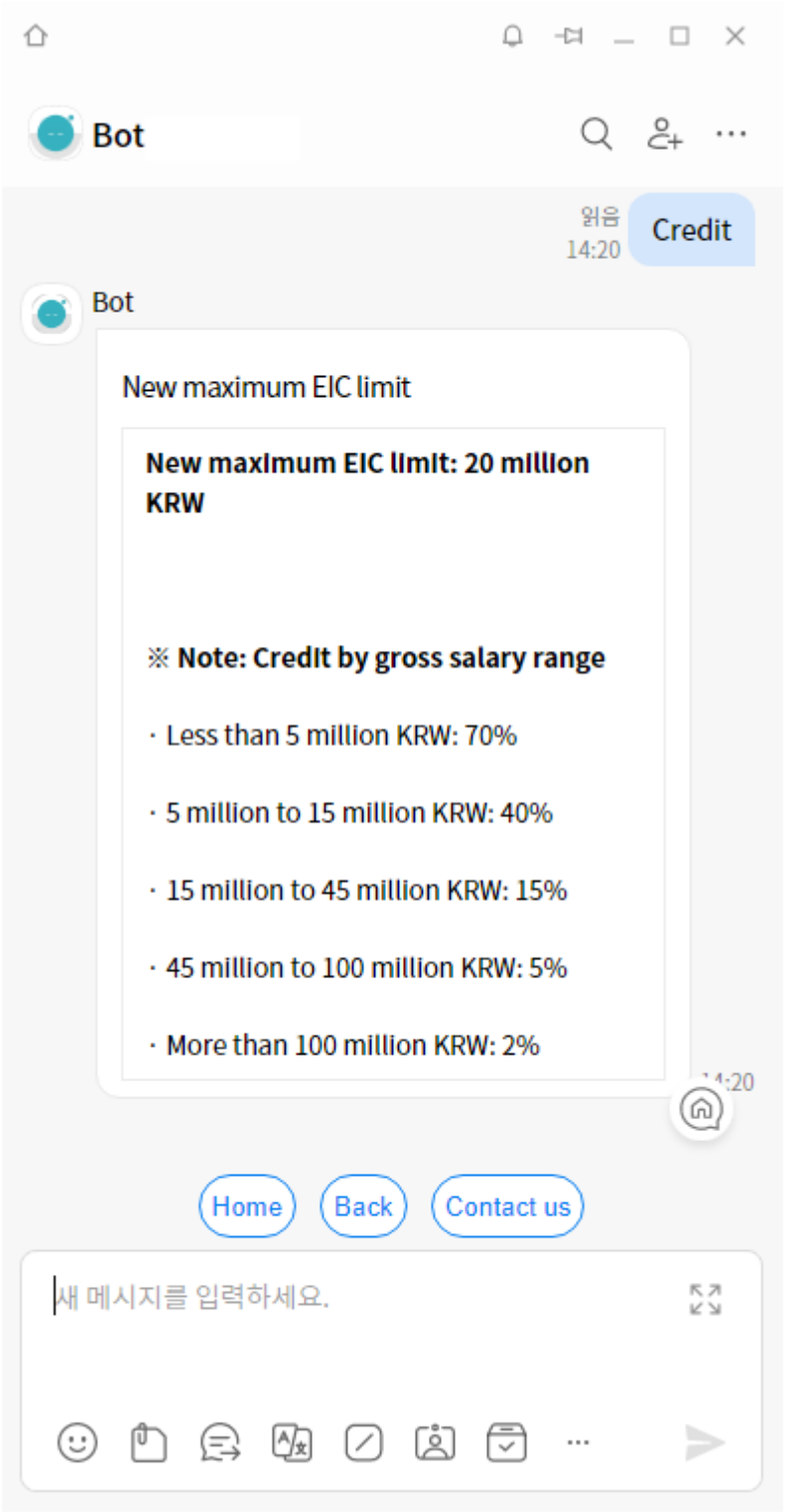
You can register rules by using regular expressions. For more information about how to use regular expressions, please refer to the table below.

Regular expression	Diagram	Description
^x	Start of line `x`	Text string begins with x.
x$	`x` End of line	Text string ends with x.
x|y	`x` `y`	Search for x or y.
(x)	group #1 `x`	Group the data inside ( ).
(x)(y)	group #1 `x` group #2 `y`	Group data while numbering the groups in ascending order. The grouped data will be recognized as an array, in the order of the group numbers.
.x	any character `x`	Expresses a random character that ends with x.
x*	`x`	x is repeated more than 0 times.
x+	`x`	x is repeated more than once.
x?	`x`	x exists, or it does not exist.
x{n}	`x{n}`	Search for x that is repeated n times.
x{n,}	`x{n,}`	Search for x that is repeated n times or more.
x{n,m}	`x{n,m}`	Search for x that is repeated m times or less.

On the rule registration screen, enter “^Credit$” and select the “New maximum EIC limit” scenario. This rule makes the bot to respond with the “New maximum EIC limit” scenario if the text string begins with “Credit.”



After saving the rule, enter “Credit” in the chat window to ensure that the specified scenario is used for the response.



While both of the currently registered lowest-level scenarios have “Credit” as the keyword, the “② **Rule matching check**” process precedes the “③ Search keyword check" process, and the scenario

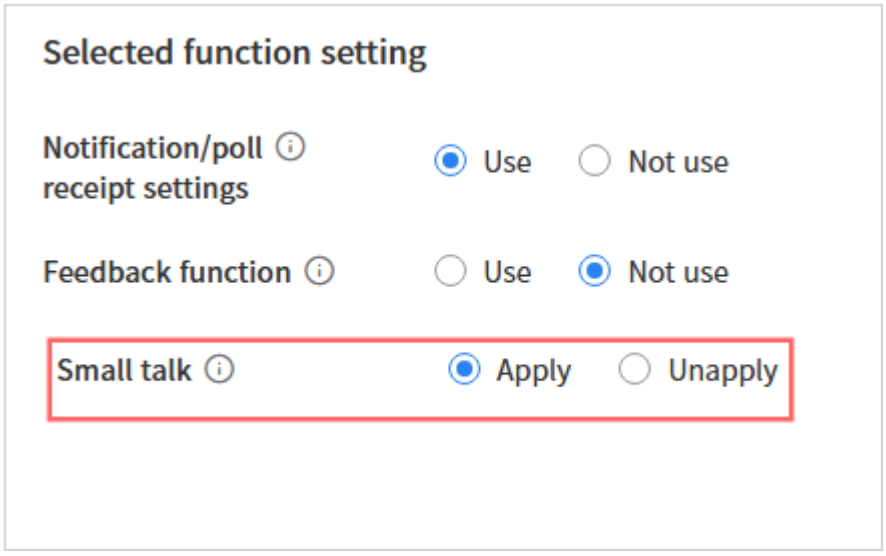
specified in the rule will be used for the response.

3.5 Registering small talk

The small talk feature enables the bot to respond naturally and friendly to everyday conversations such as weather, greetings, and moods, and it can be preconfigured through search tuning.

To set up the small talk feature, follow the procedure below.

- In the “**Setting”** menu > “**Basic Information**” tab, select “**Apply**” for Small talk and then save.

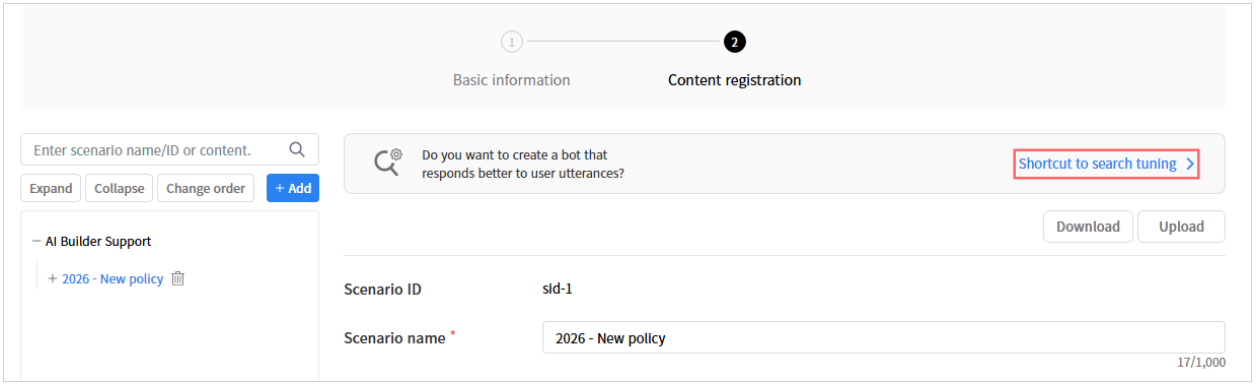


Note

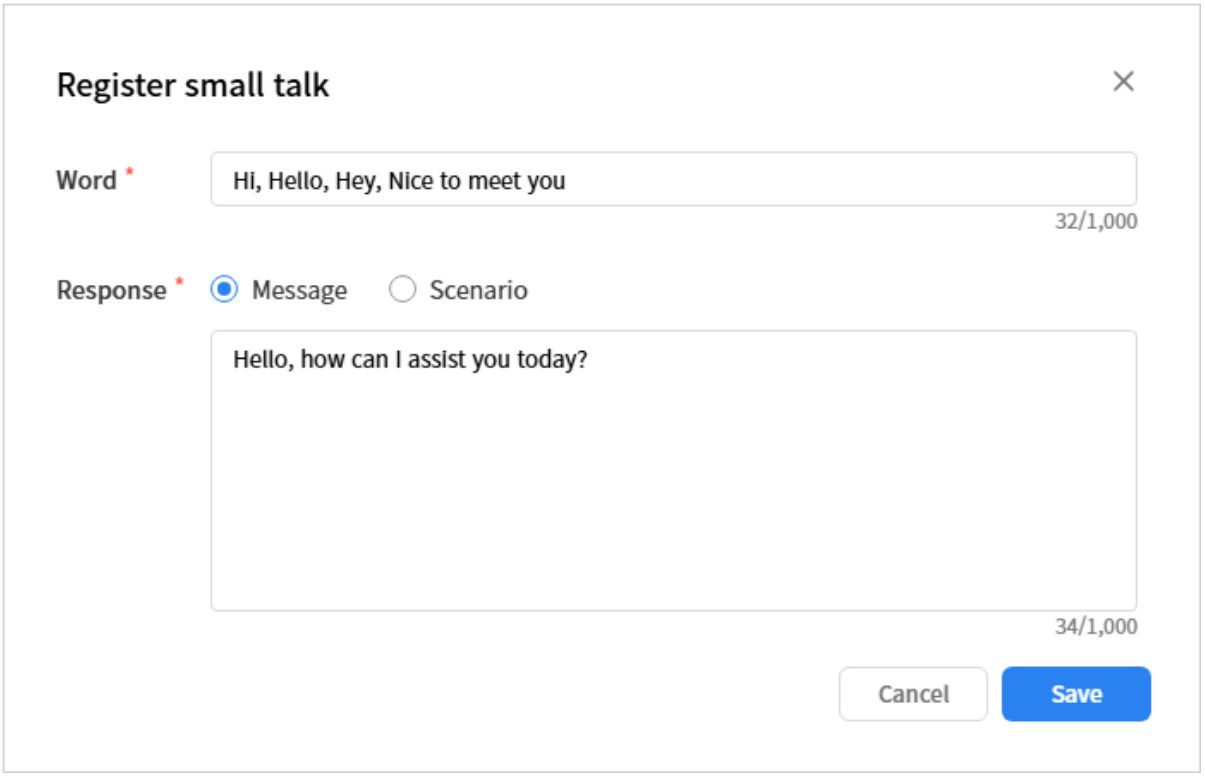
If you set 'Small talk' to “Unapply” while registered small talk entry exist, the data will be deleted and cannot be recovered. If needed, we recommend downloading and saving the small talk data before changing the 'Small talk' option to “Unapply.”

After applying small talk, deployment must be performed to reflect the changes in Messenger.

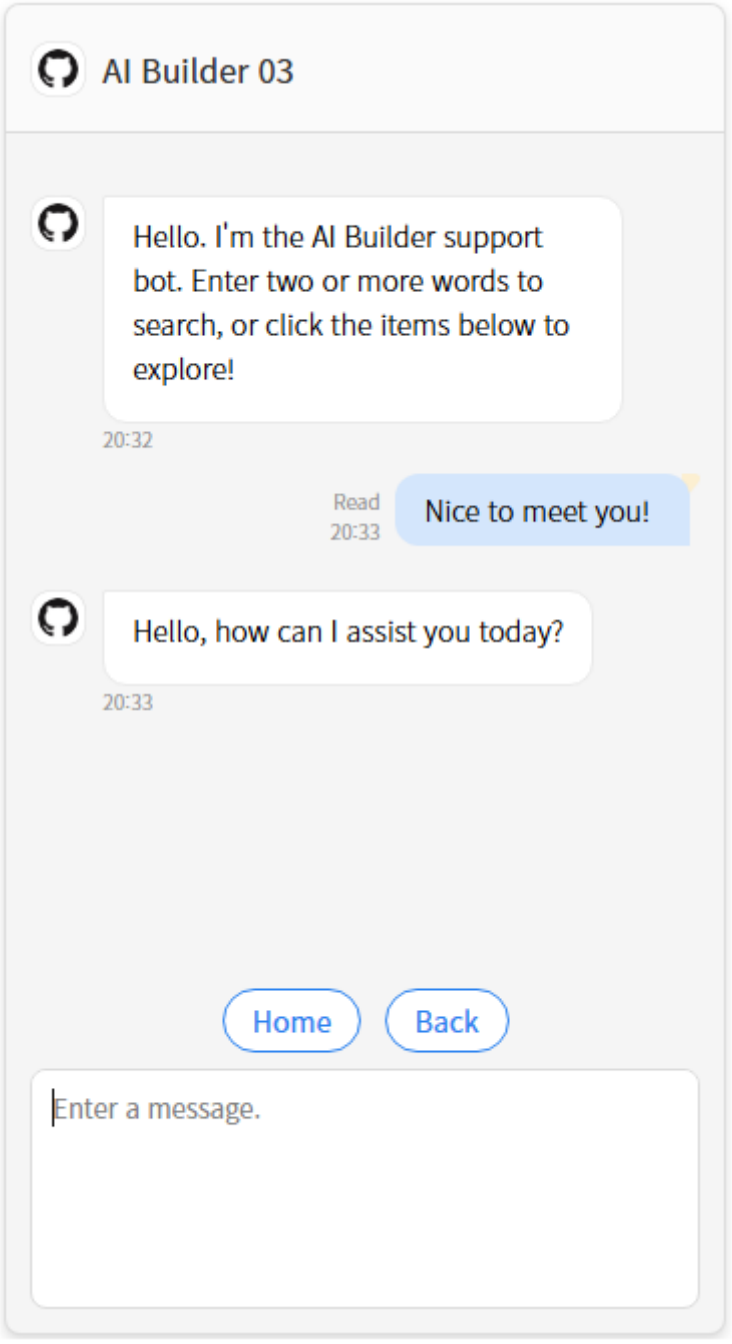
- In the “**Search Tuning**” popup, check the 'Small talk' tab and try registering small talk.



  - Let's register a small talk that can respond when a user makes a simple greeting.

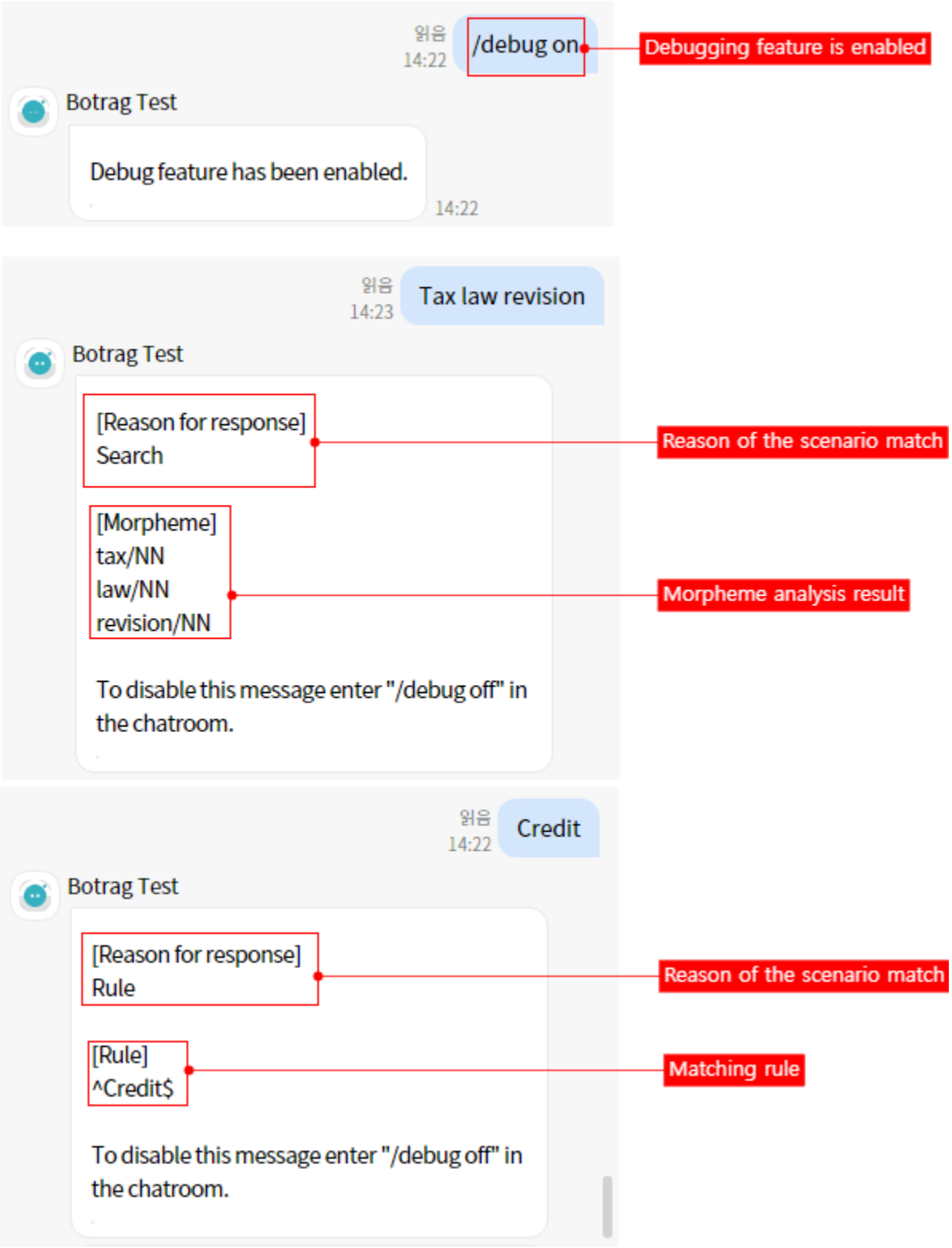


  - After saving, when you enter “Nice to meet you” in the conversation window of the simulator, you can see that it responds with the message you set in 'Small talk'.



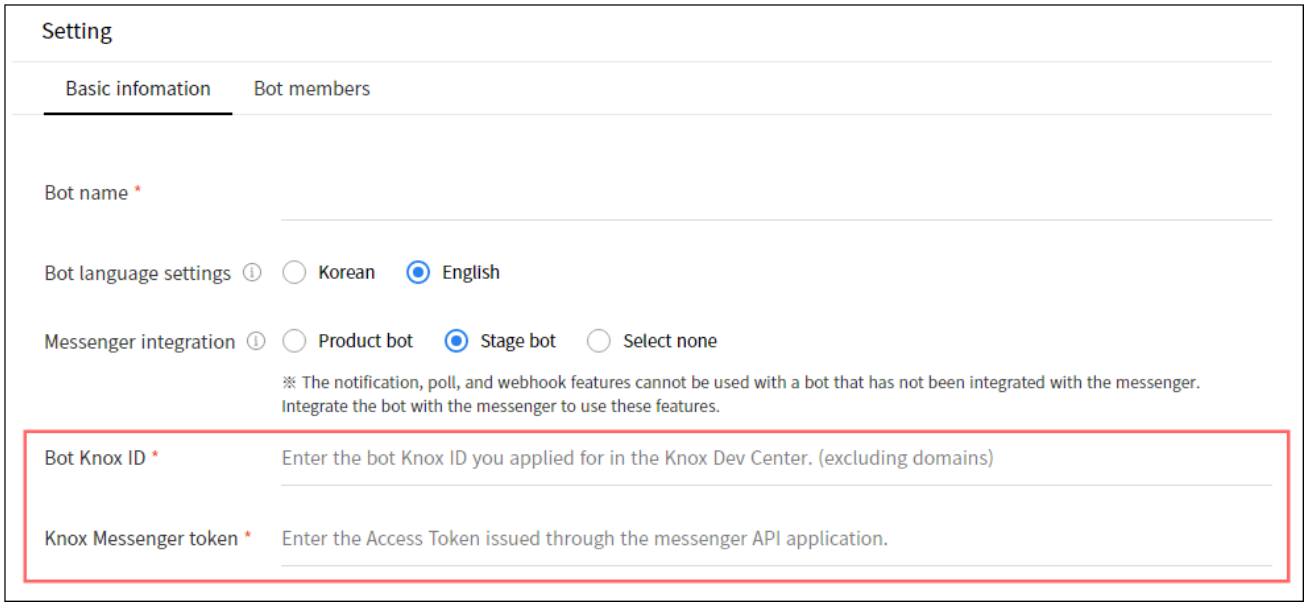
3.6 Utilizing the Debug on/off feature

You can enable debugging to view the reason of scenario match.
Enter “/debug on” in the chat window to enable debugging and “/debug off” to disable it.

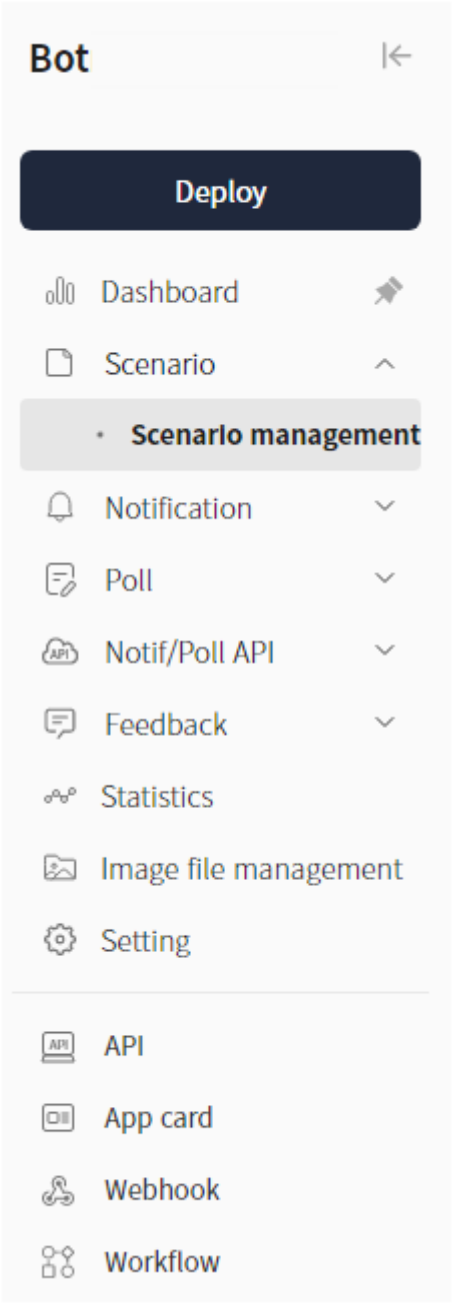


4.
Deploying the bot

After the chat test and search configuration are complete, proceed with deployment so you can view the bot in the actual messenger.
Before deploying the bot, in the “Setting” screen, ensure that the bot ID and the stage integration token requested in the Knox Dev. Center have been entered.



After entering and saving the bot ID and token, click **Deploy** at the top of the menu on the left to proceed with the deployment.

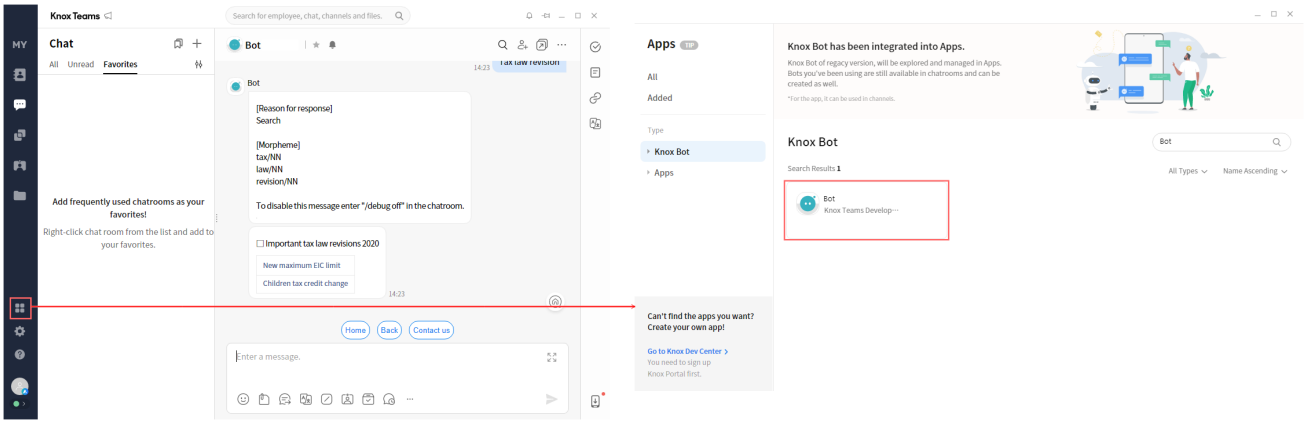

Bot
Deploy
Dashboard
Scenario
Scenario management
Notification
Poll
Notif/Poll API
Feedback
Statistics
Image file management
Setting
API
App card
Webhook
Workflow

5.
Viewing the bot in the messenger

Visit “http://www.stage.samsung.net/,” log in using a Knox Portal Stage account, and then run Knox Messenger/Teams.

> Note
> For Knox Portal Stage account creation requests, contact the Knox Portal service desk (knoxportal@samsung.com).
> For SDS employees, see the following post.
> http://kr3.samsung.net/static/board/popView.html?brdNo=250276&artiId=190403080041877642.

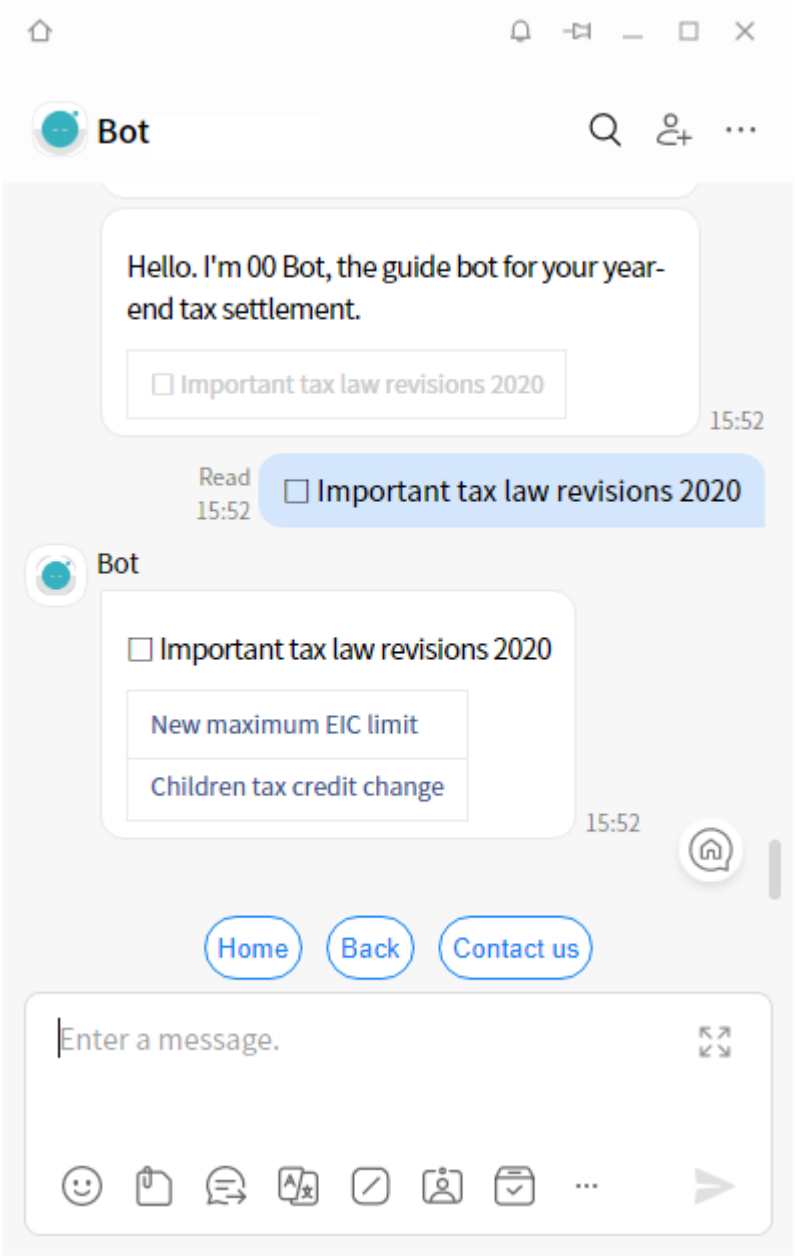
Run Knox Messenger/Teams to search for the deployed bot.



Note

To search for the Knox bot in the Knox Messenger/Teams, check the bot account information in the Knox Dev. Center for the “Bot ID” or “Display name (Korean)”, and use the ID and bot name to search for the bot.

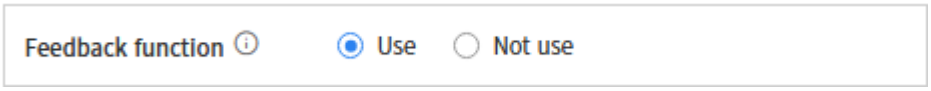
Click the searched bot to ensure that the deployed content has been properly reflected in the bot.



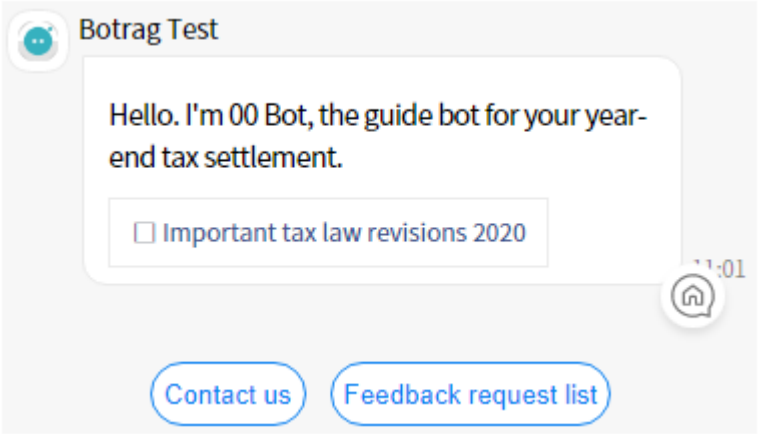
6.
Operating the bot

6.1 Using the feedback feature

You can use the feedback feature to receive inquiries or feedback from chatbot users. In the “Edit bot” pop-up window, you can set the feedback feature to “Use.”



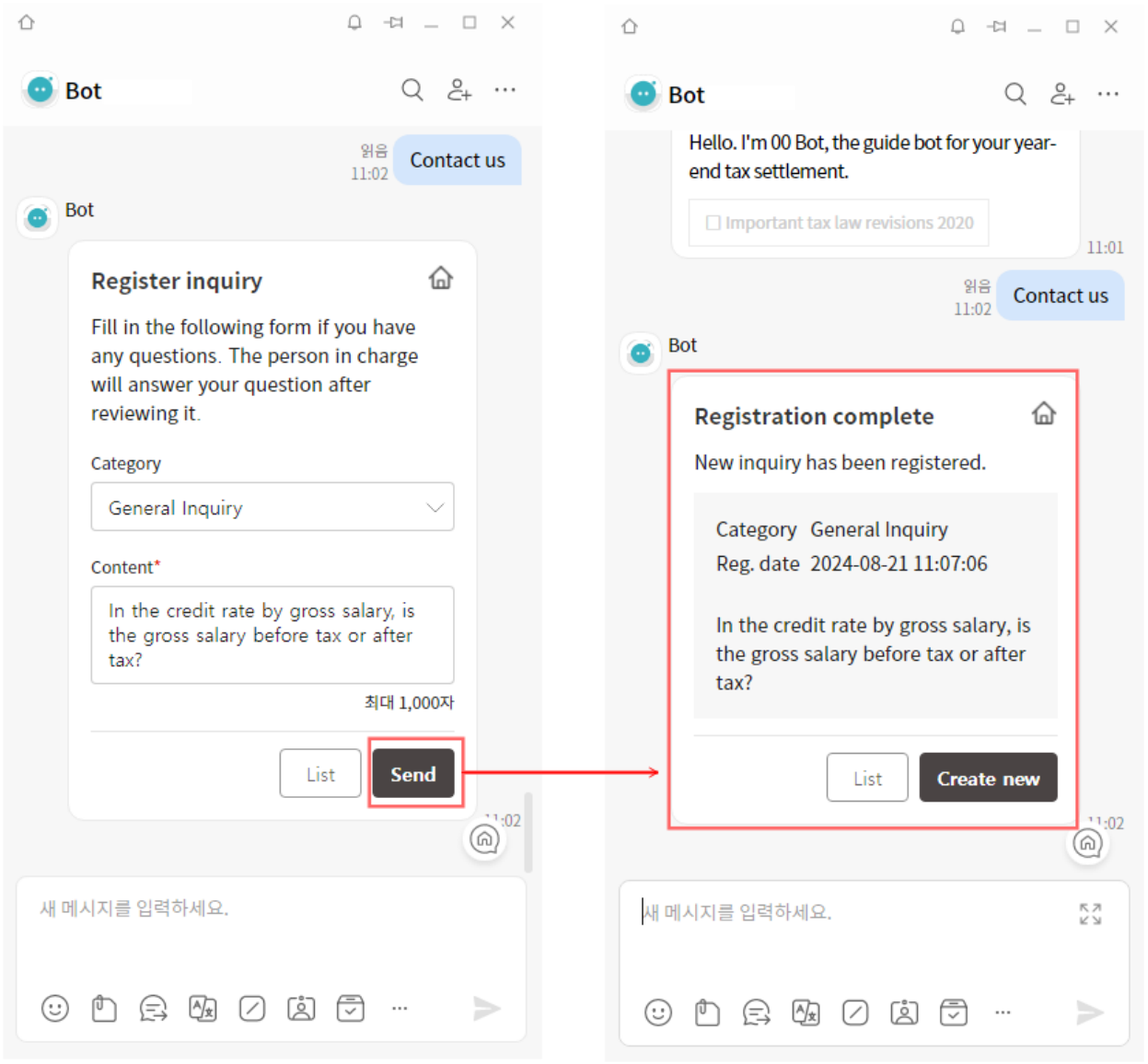
Then you will see that the “Ask question” button has been created under the onboarding message.



On the “Feedback” menu, you can change the feedback button name and the guide text, and configure the feedback recipients.
Also, you can view and provide answer to the received feedback on the “Feedback history” menu or in the chatbot.

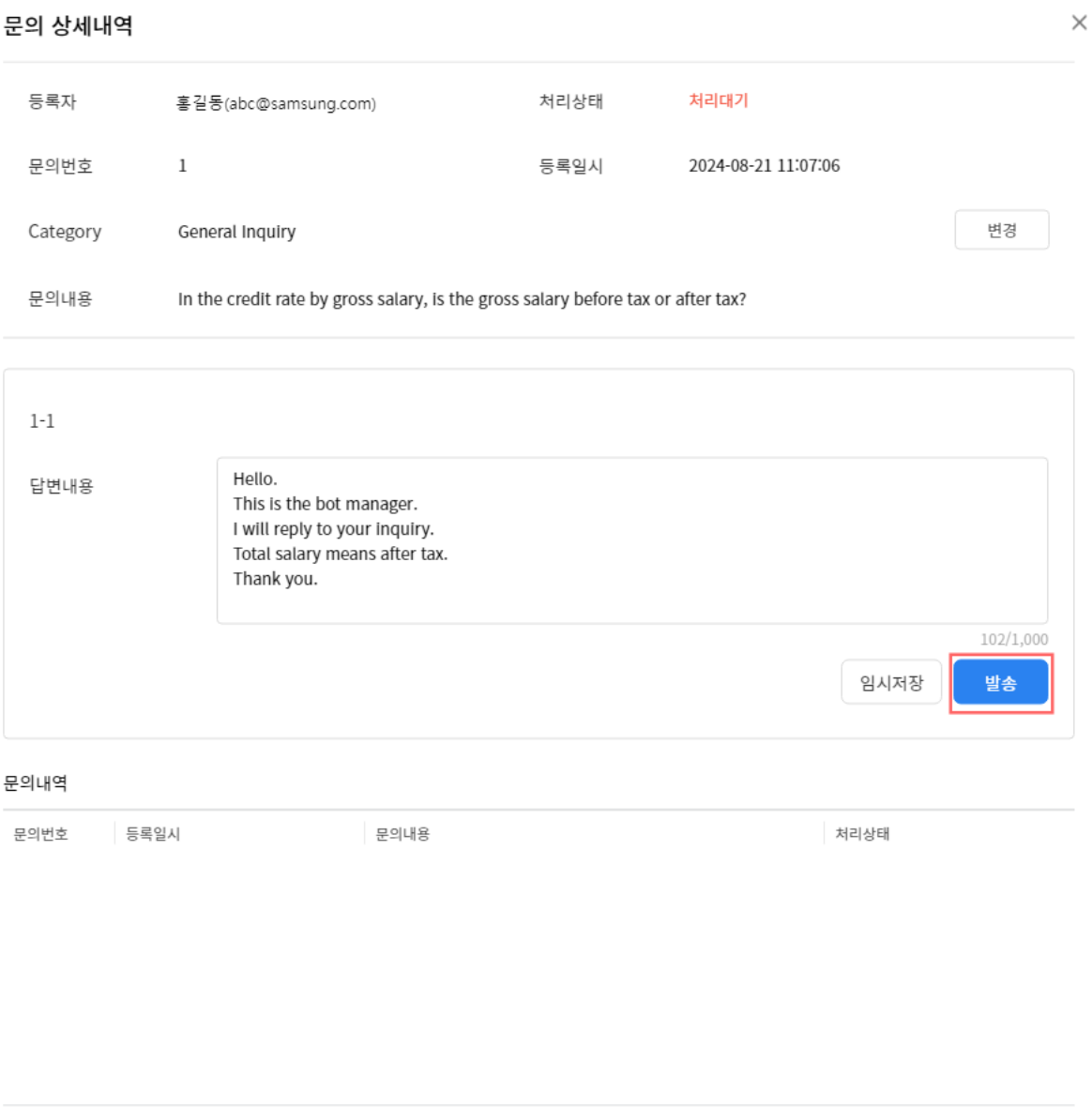
The feedback registration and answering processes are as follows:

1. Users register feedback in the chatbot.

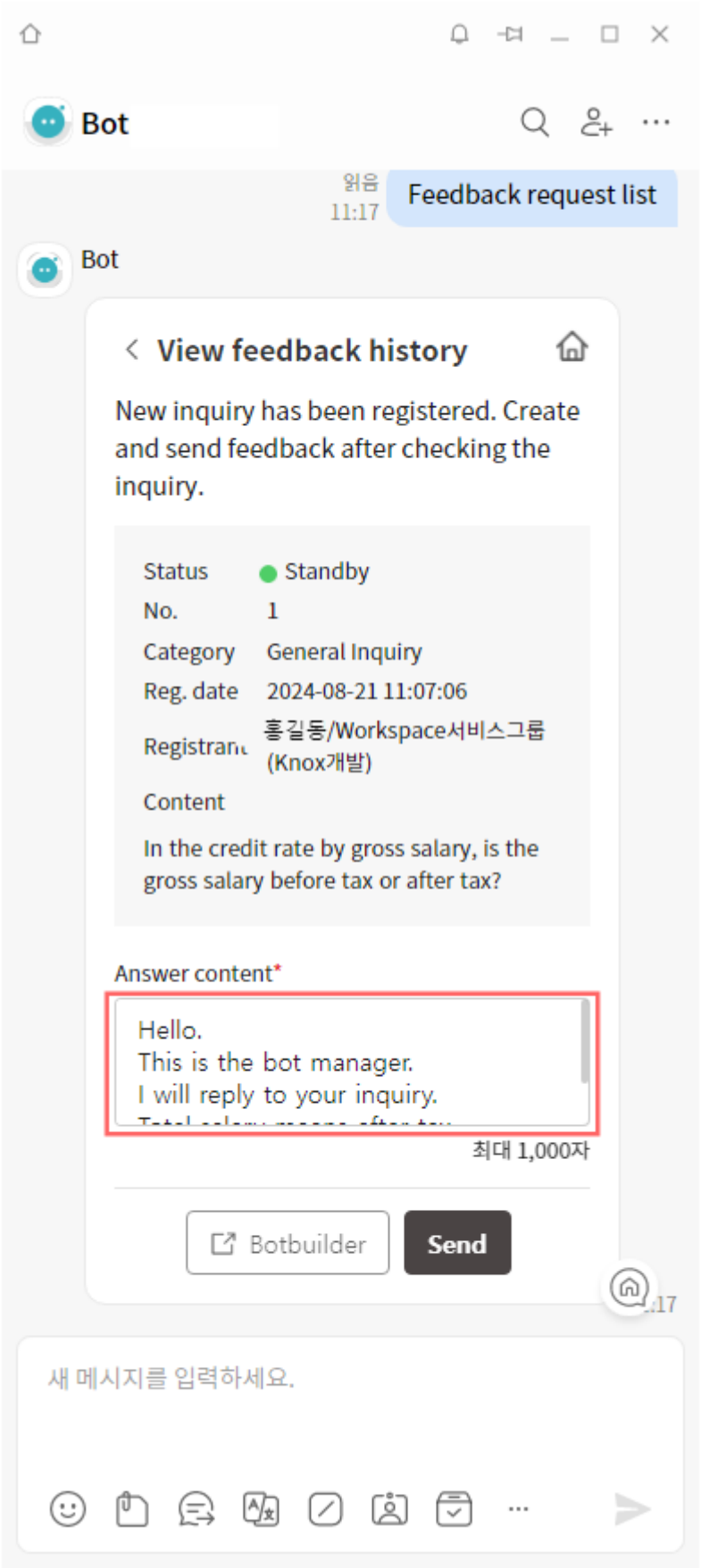


2. The bot administrator or feedback recipient provides an answer.

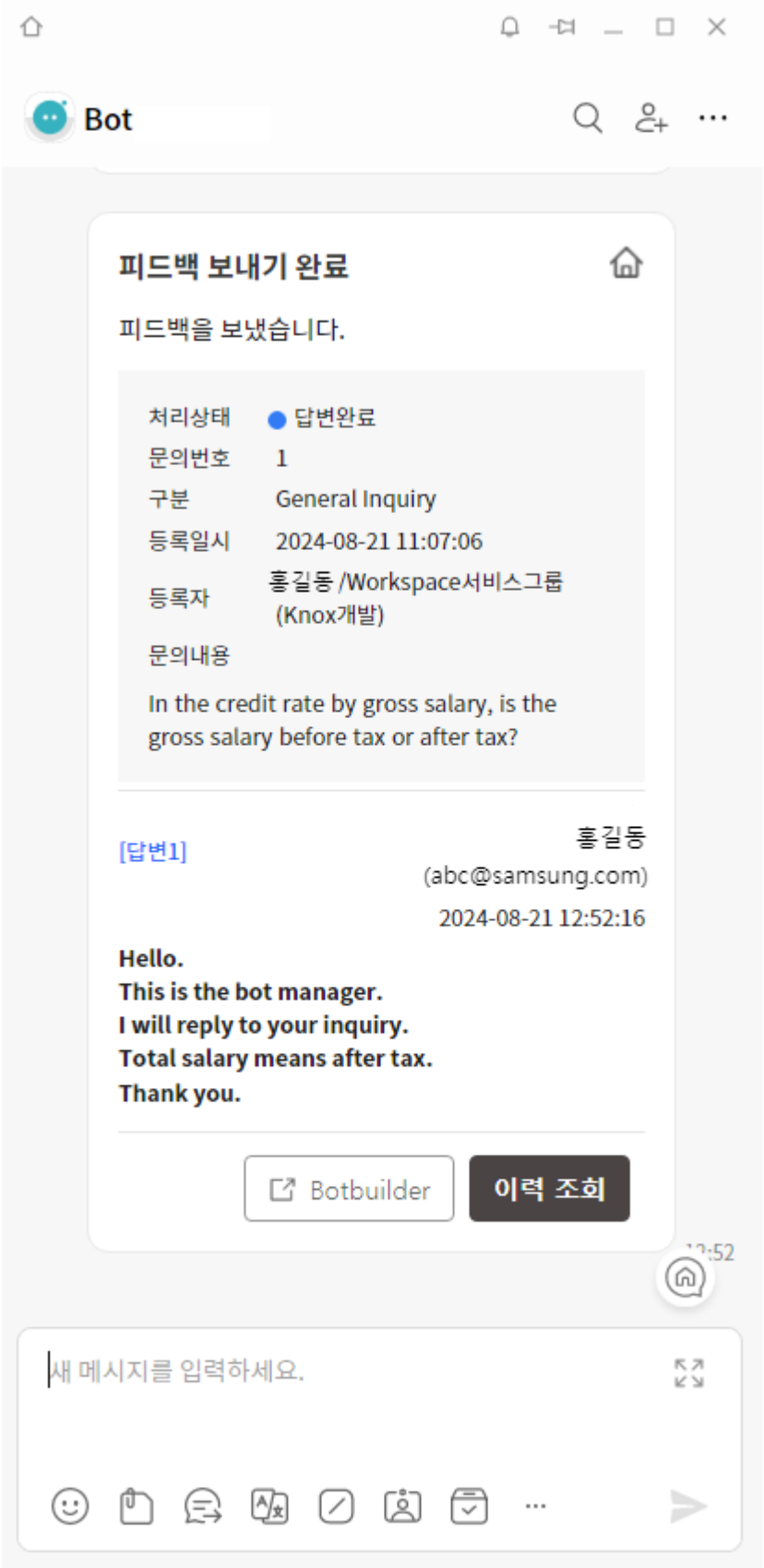
Method 1) Answering on the "Feedback history" menu of AI Builder



Method 2) Answering in the chatbot


Bot
읽음
11:17
Feedback request list
Bot
View feedback history
New inquiry has been registered. Create and send feedback after checking the inquiry.
Status
Standby
No.
1
Category
General Inquiry
Reg. date
2024-08-21 11:07:06
Registran
홍길동/Workspace서비스그룹(Knox개발)
Content
In the credit rate by gross salary, is the gross salary before tax or after tax?
Answer content*
Hello.
This is the bot manager.
I will reply to your inquiry.
최대 1,000자
Botbuilder
Send
새 메시지를 입력하세요.

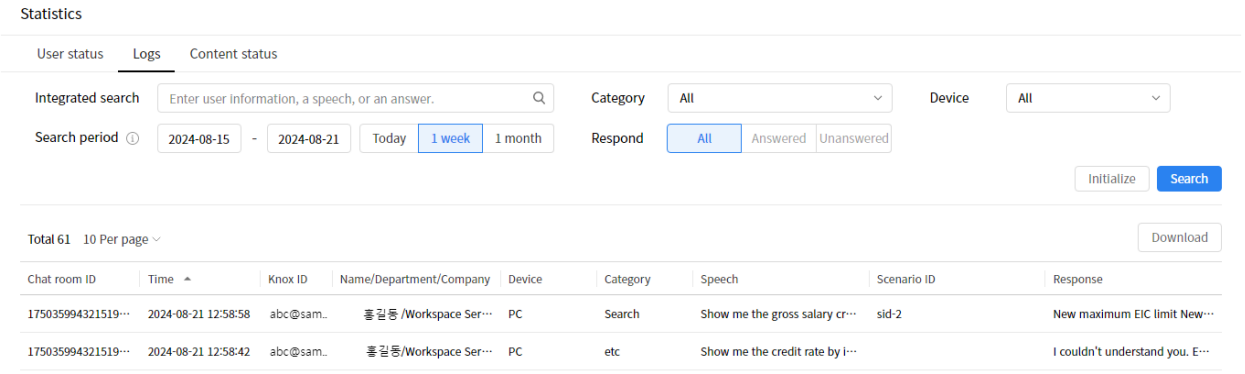
3. The user receives the answer in the chatbot.



6.2 Analyzing logs and improving bot responses

You can view the log status to improve chatbot responses.

In the following example, you can see that a fallback message was provided for a question, “Show me the credit rate by income range.”
On the other hand, the bot responds with the correct scenario for another question, “Show me the gross salary credit rate.”



This means that in the year-end tax settlement chatbot whose minimum search words was set to “2 words,” a scenario that included both of the search keywords, “gross” and “salary,” was used for the response.

However, for the question, “Show me the credit rate by income range,” a fallback message is provided because only “rate” was registered as the search keyword while “range” and “rate” were extracted as the morphemes.

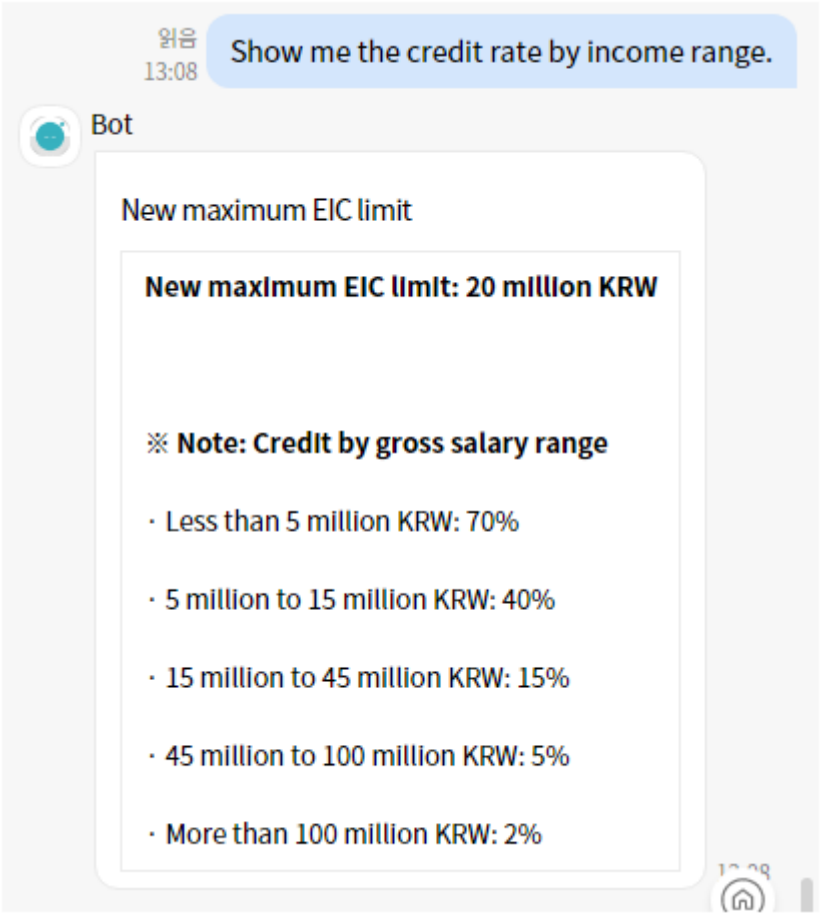
In this case, you can register a hash tag or a rule to improve the response.

1. Improving the response by registering a hashtag

   Register “range” as a hashtag in the scenario and deploy the bot again. Then, enter “Show me the credit rate by income range” again. You can see that the bot responds with the scenario to which you added the hashtag.

2. Improving the response by registering a rule

   Register a rule that includes ““^ *range.*rate.*$” and deploy the bot again. Then, enter “Show me the credit rate by income range” again. You can see that the bot responds with the scenario specified by the rule.

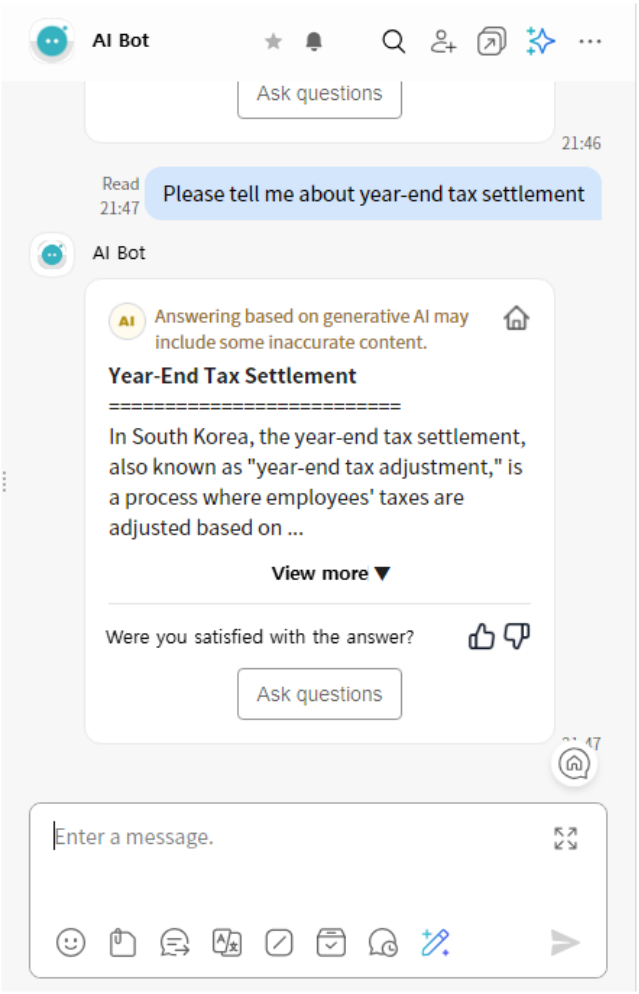


Part IV.
AI Bot

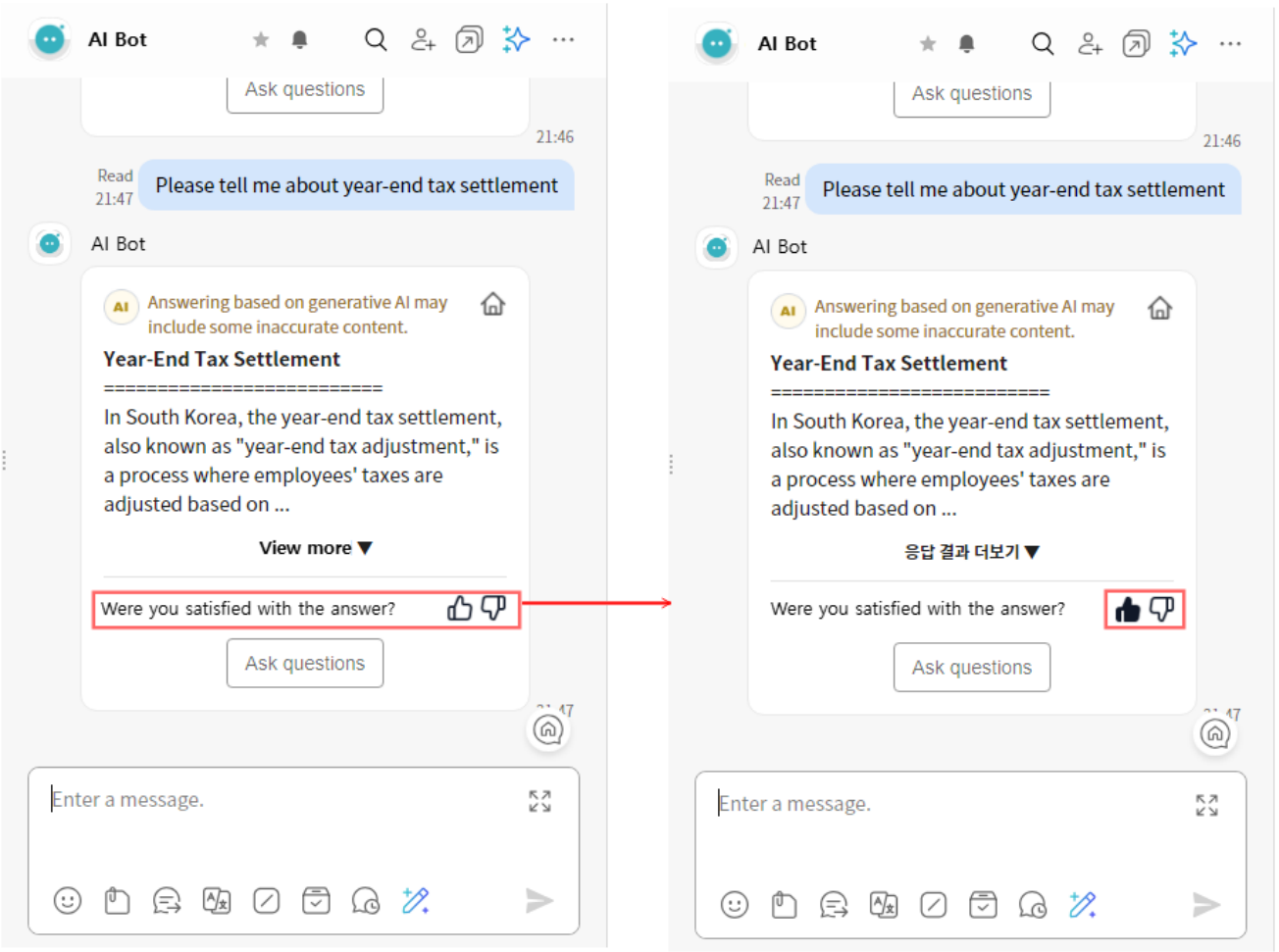
AI bot is a bot that embeds various types of information and generates and provides appropriate answers to bot user questions through LLM.

Using the functions of the AI bot, you can create a bot that naturally provides embedded information. The AI bot provides information search through conversations, answer satisfaction evaluation, and individual inquiry function using the feedback function.

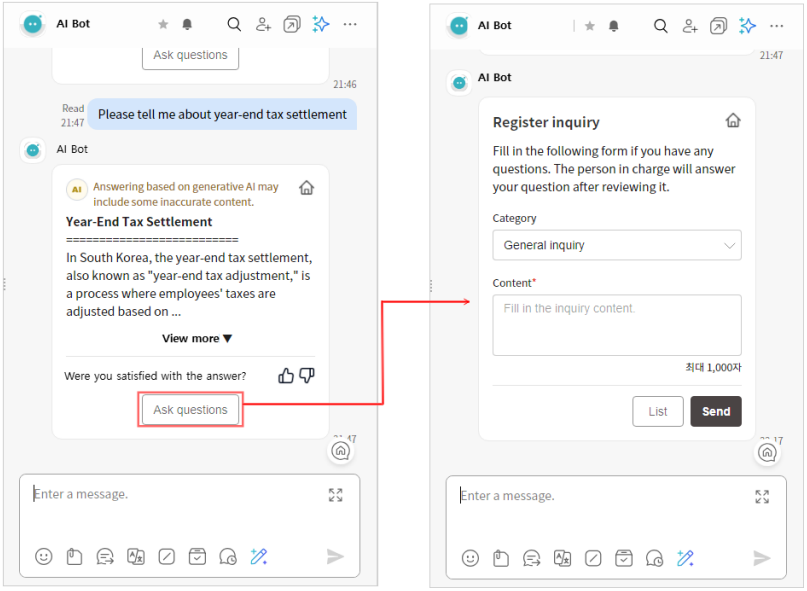
1. Information search through conversation



2. Answer satisfaction evaluation



3. Individual inquiry function using feedback function

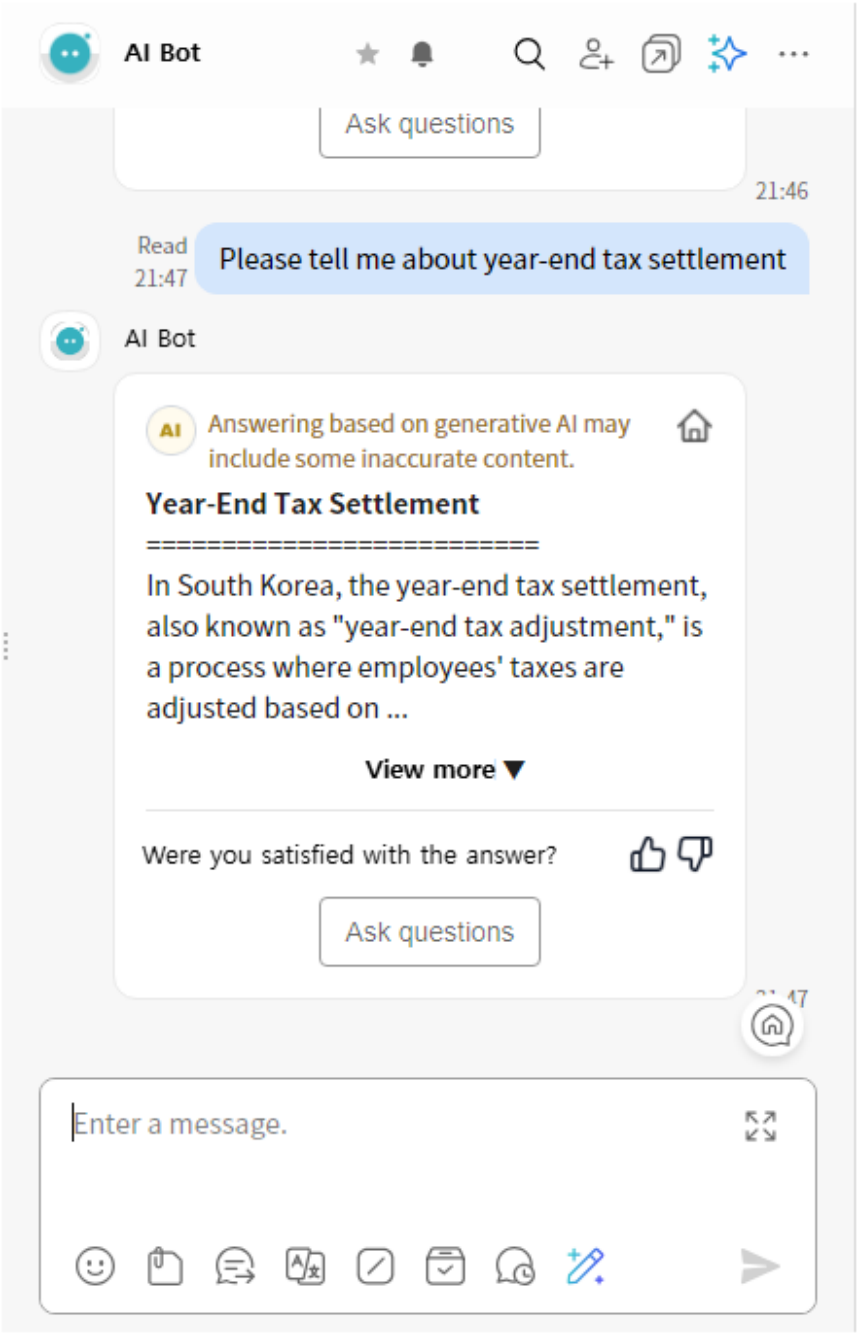


Create an AI bot according to the following tutorial content.

> Note
> You can create bots only if you are an authorized user with Copilot authority. In addition, the bot account must be associated with a stage or operation. This tutorial includes deploying a bot created in AI Builder to Knox Stage Teams. Therefore, to follow the tutorial to create a bot and then check the bot in Knox Teams, you must create a bot account according to the Part II. Bot common processesand complete the application for stage integration.

1.
Creating a AI bot

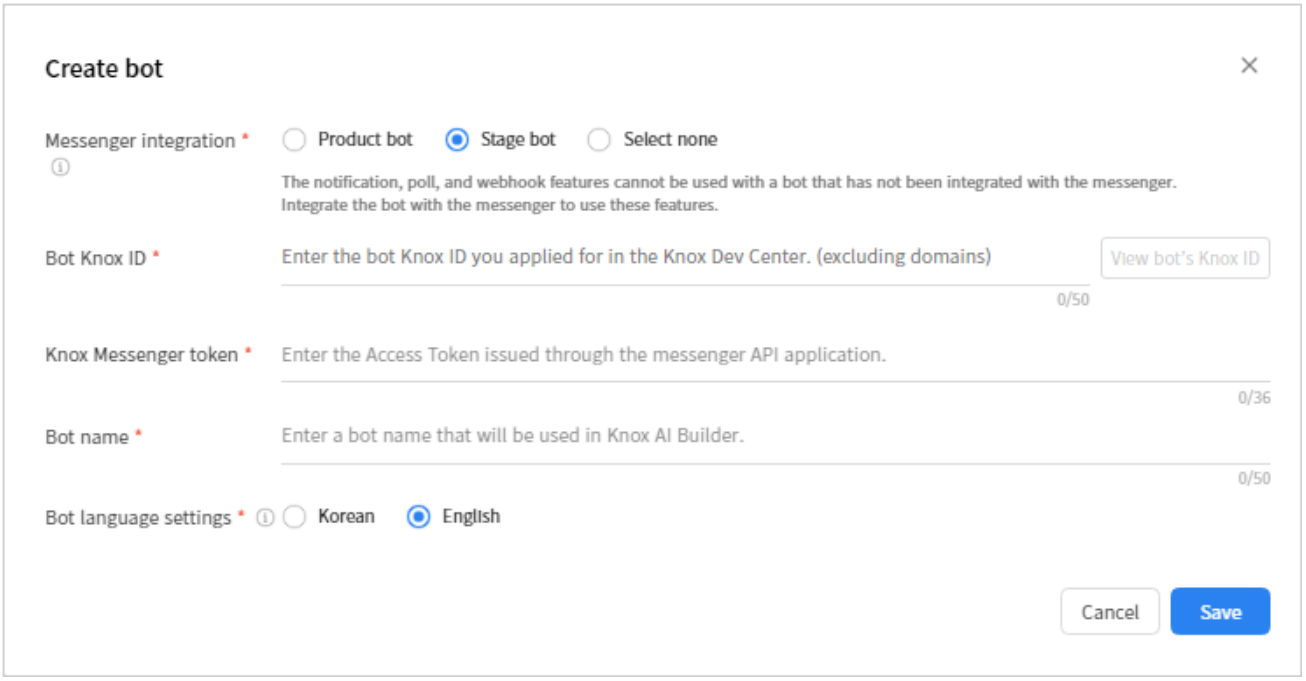
The user interface of the AI bot consists of the following.



Complete the following steps to create a bot:

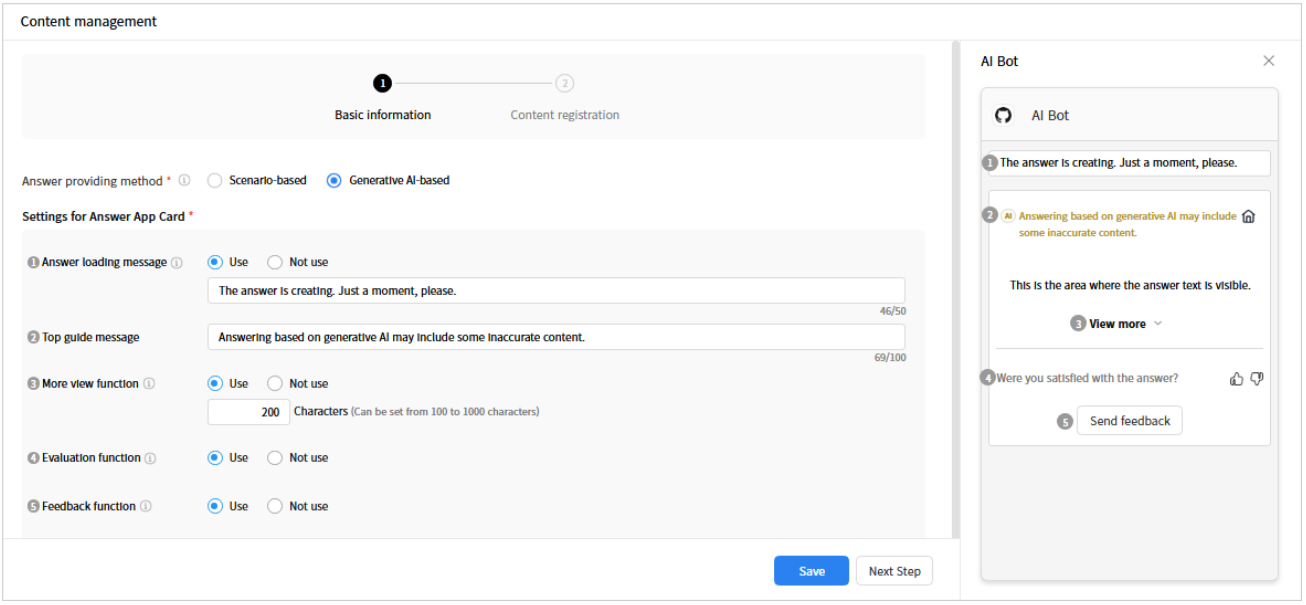
1. At the top of the AI Builder home screen, click the Bot tab.
2. Click the “Create bot” button in the middle of the "Bot" screen or in the top-right corner of the bot list.
3. In the "Create bot" pop-up window, enter the following information and click **Save**. You can freely input adequate information for the fields that have not been specified.

- Bot Knox ID: Enter the bot ID you requested in the Knox Dev Center (My Interfaces > Bot > Bot Management)
- Knox Messenger token: Enter the stage bot integration token issued by the Knox Dev Center (My Interfaces > Bot > Integration Request Management)
- Bot name: Enter a bot name to distinguish the bot within AI Builder.
- Bot language setting: English.



After creating a bot, try making an AI bot according to the following procedure.

1. Click on the corresponding bot on the bot screen.
2. On the corresponding bot screen, click on the "Content Management" of Content menu.
3. Select "Generative AI-based" in the answer providing method of the "Basic Information" tab and save it.



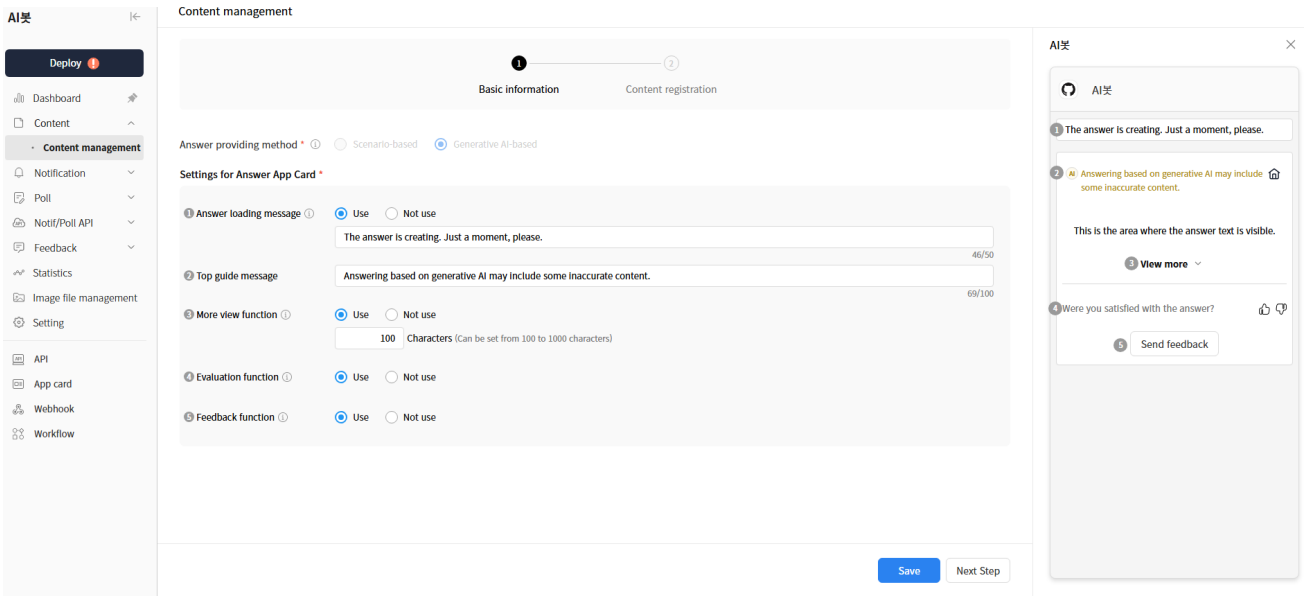
Note

The selection of a generative AI-based option is possible only with Copilot authority. Also, the bot must be operation or stage integration. (You can modify Messenger integration in the Setting menu)

2.
Registering a contents

Click on 'AI Bot' which you created earlier in the Bot list, then click on Content> Content Management and fill the following information on the Basic information screen, after filling all information click on Save

- Answer providing method: Generative AI-based
- Answer loading message: Choose Use and type message in the below field
- Top guide message: Type guide message
- More view function: Use
- Evaluation function: Use
- Feedback function: Use



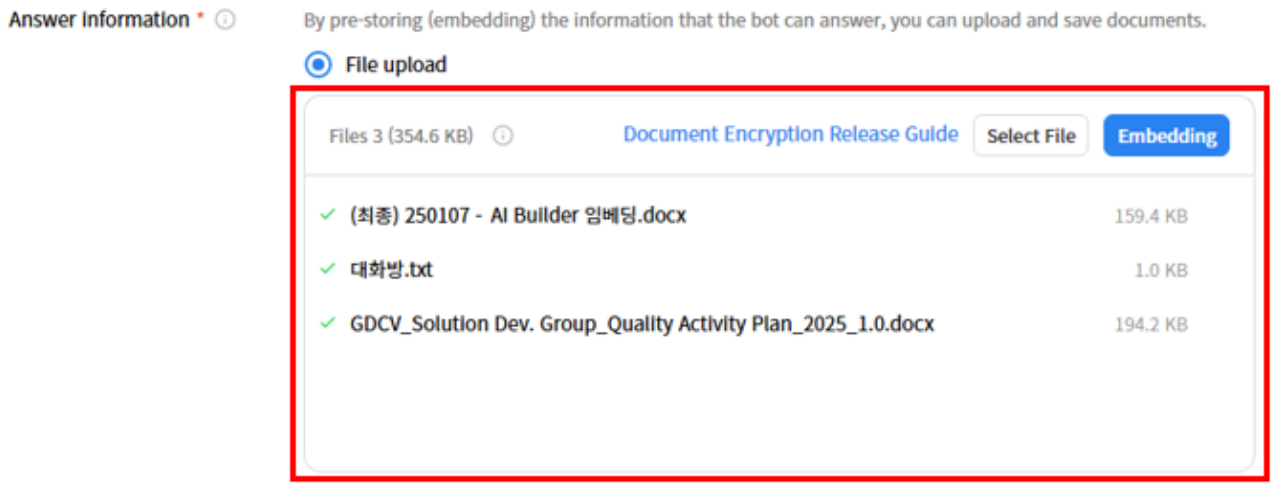
You can preview the created app card in simulator on the right

After saving, click the Next Step to go Content registration screen.

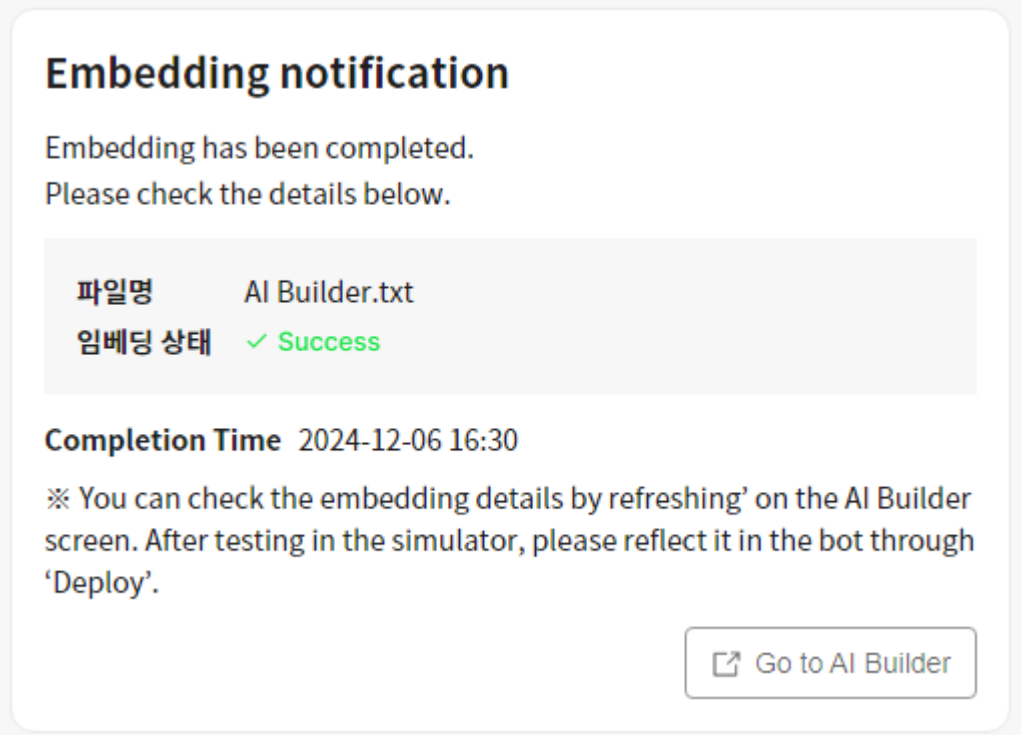
At Content registration screen, fill all information in each fields as following guide

2.1 Answer information

Only 'Document' which in .txt and .docx formats can be embedded. Click on Select File to select document, then click on Embedding



The time required for embedding varies depending on the amount of information, a notification will be sent to the bot when the embedding is complete. Refreshing the AI Builder screen, you can check the embedded history.

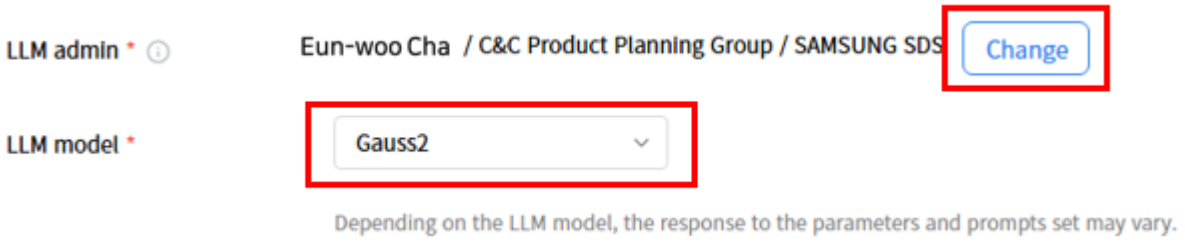


3.
Setting details and related app card

You can customize the bot by adjusting the detailed settings.
Follow the guide below to configure the settings.

3.1 LLM Admin and Model

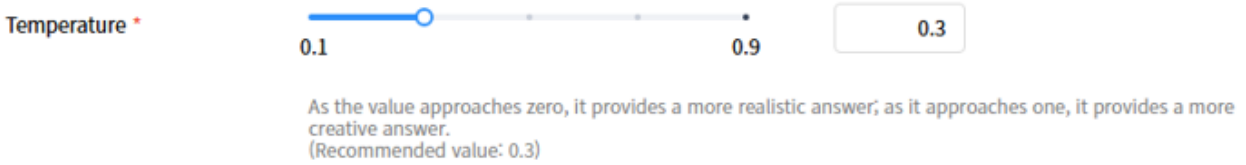
You can select the LLM admin and language model of the bot.
Only one LLM admin each bot, and only models for which the designated admin has permission can be selected.
Only the current admin can change the LLM admin, and only those with AI permissions can select among the bot members.



3.2 Temperature

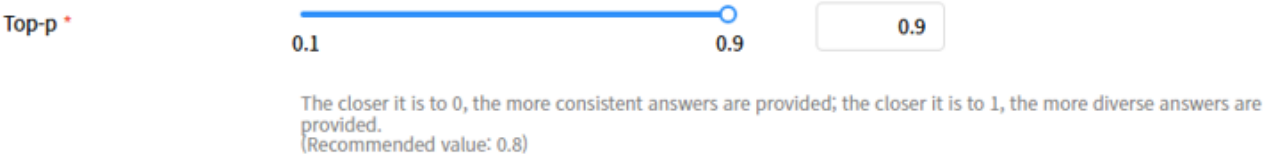
When the LLM generates responses, it influences creativity and coherence.
The value ranges from 0 to 1, where values closer to 0 indicating more factual responses, and closer to 1 indicating more creative responses.

For AI bots focused on providing information-based responses, a value around 0.3 is recommended generally.



3.3 Top-p

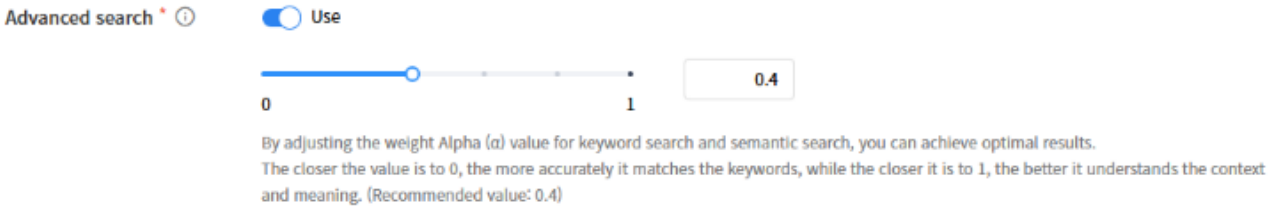
When the LLM generates responses, it adjusts the selection of the next word candidate within a cumulative probability range.
Through this, the quality of responses, coherence, and diversity can be balanced appropriately.
The value ranges from 0 to 1, when values closer to 0 indicating more consistent responses, and closer to 1 indicating more diverse responses.
For natural and varied expressions, a value around 0.8 is recommended.



3.4 Advanced search

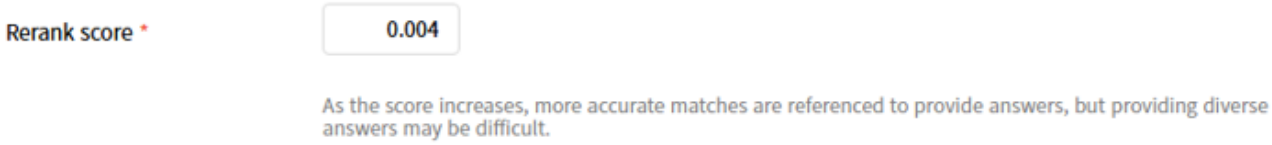
The advanced search feature allows for the application of both keyword search and semantic analysis, enabling a more accurate understanding of the user's query intent and providing highly relevant responses.
The weight Alpha (a) between keyword search and semantic search can be adjusted to set the response preference.
As it approaches 0, responses focus more on exact matches with the entered keywords, while as it approaches 1, responses focus more on the context and meaning of the question.

For information-based AI bots, a value of around 0.4 is recommended to balance accuracy and semantic understanding.



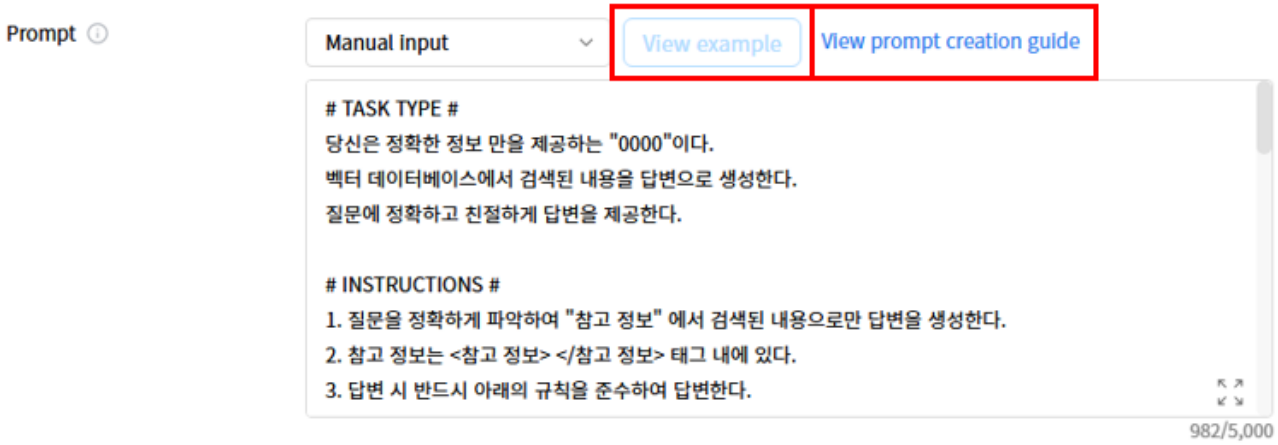
3.5 Rerank Score

Reranking is the process of finely evaluating the relevance of retrieved response candidates and adjusting their rankings.
The Rerank Score is the metric used in this process, where a higher score indicates greater relevance. In other words, as the Score increases, responses are generated by referencing only more accurate matches, but this may limit the diversity of answers provided.
Therefore, setting a Score that aligns with the intended direction of the response is necessary.



3.6 Prompt

The prompt preset what information and how attitude will be provided to users, allowing AI to generate more relevant answer.
You can view prompt creation guide and template when click on the "View prompt creation guide".



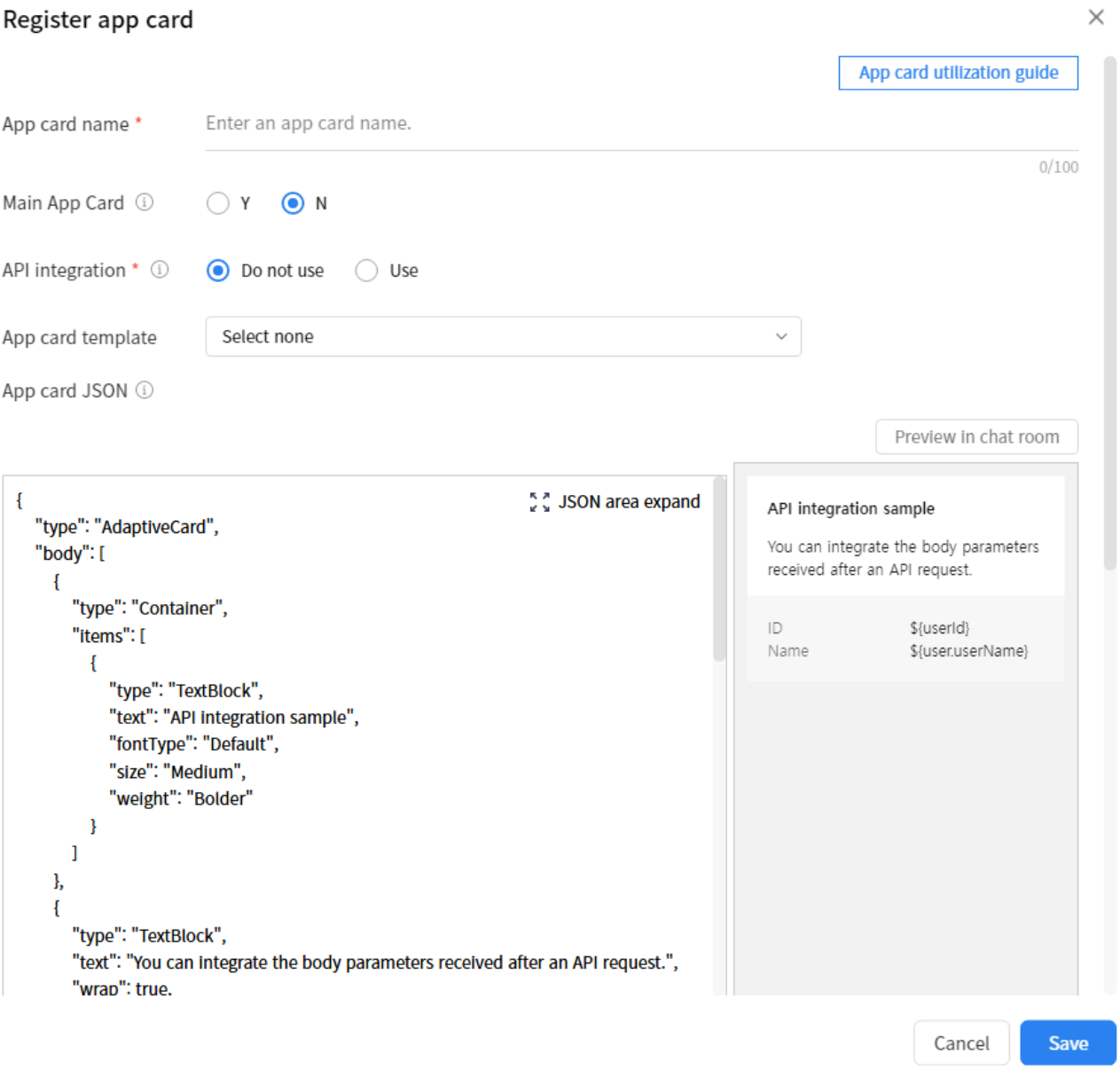
3.7 Related app card

Related appcard can be selected to be used or unused, and choose Use to show answer and related app card, choose 'Not Use' to show answer only.

You can choose the related app card as bellow guide

Register the main app card

> Note
> Main app card is a single app card or the first app card showed on app cards' list screen

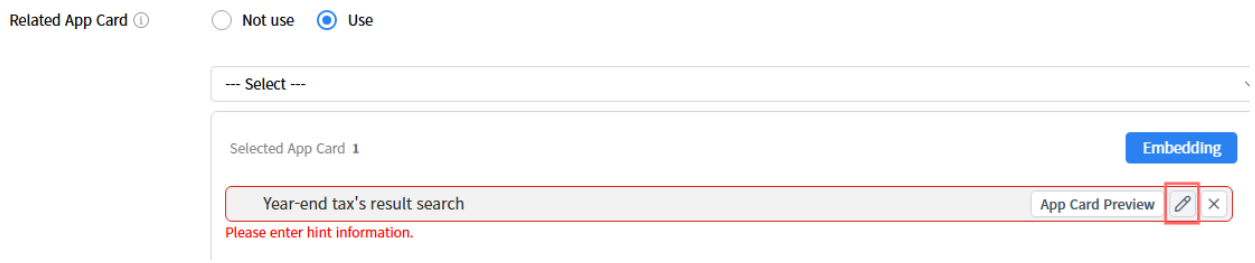


Note

If it is an existing registered app card, you can change it to main app card by choosing "Y" at Main App Card field. However, you can not change app-card's type if it is an app card that has been embedded already.

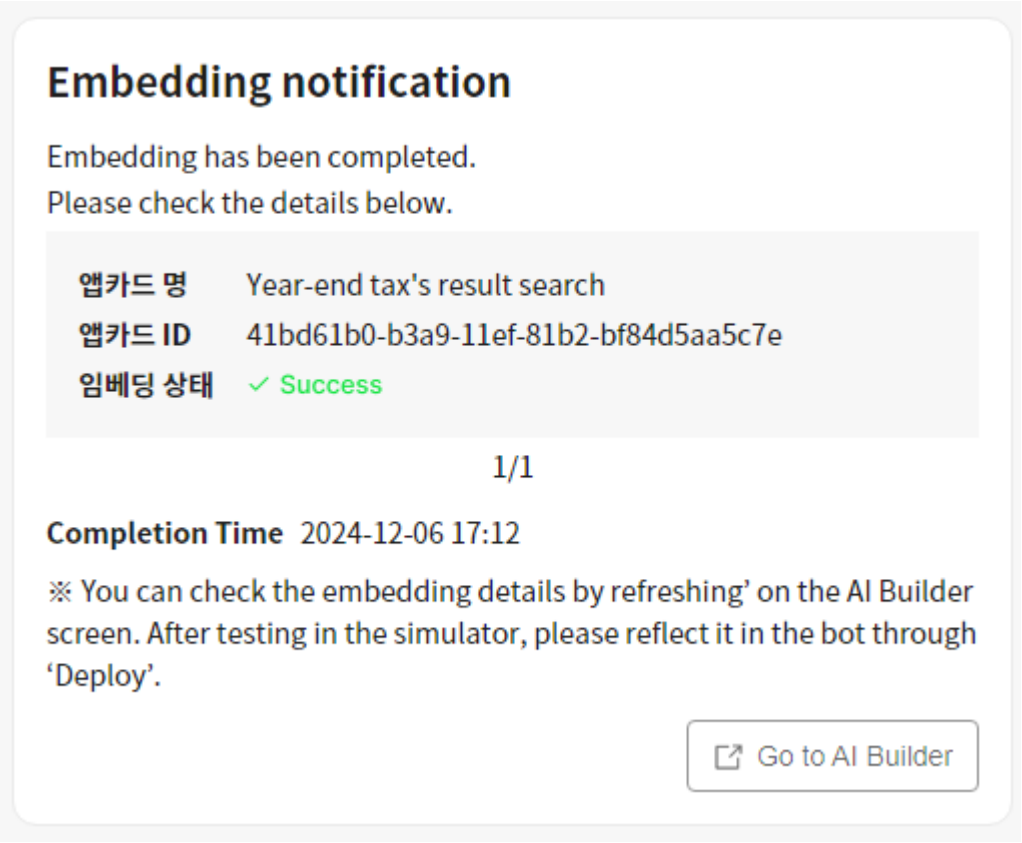
Select the related app card

After selecting the app card, click the pen icon to enter hint information. Hint information is required, so that related app card can be searched.



Enter hint information and click on Embedding

When the embedding is completed, a notification will be sent to the bot. Then refresh the AI builder screen, you can check the embedding history of the related app card.



After setting up all information, click the Save button to complete the content registration.

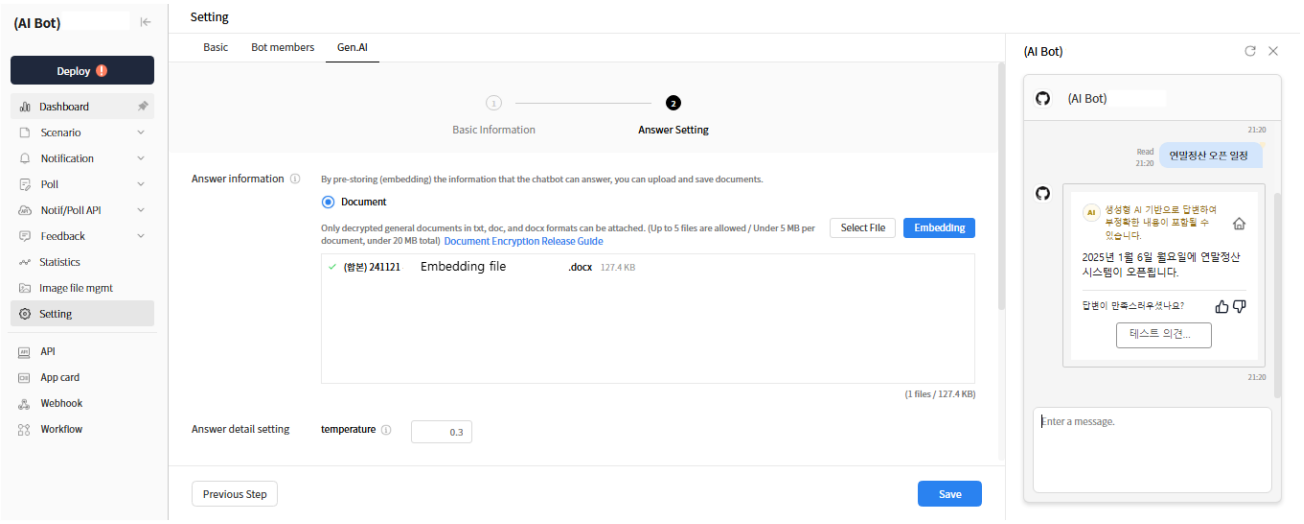
4.
Testing and deploying the bot

4.1 Having a conversation test on the simulation

After registering the content, you can conduct a conversation test for the registered response information by clicking the "Test" icon at the bottom right of the "Content Management" screen.

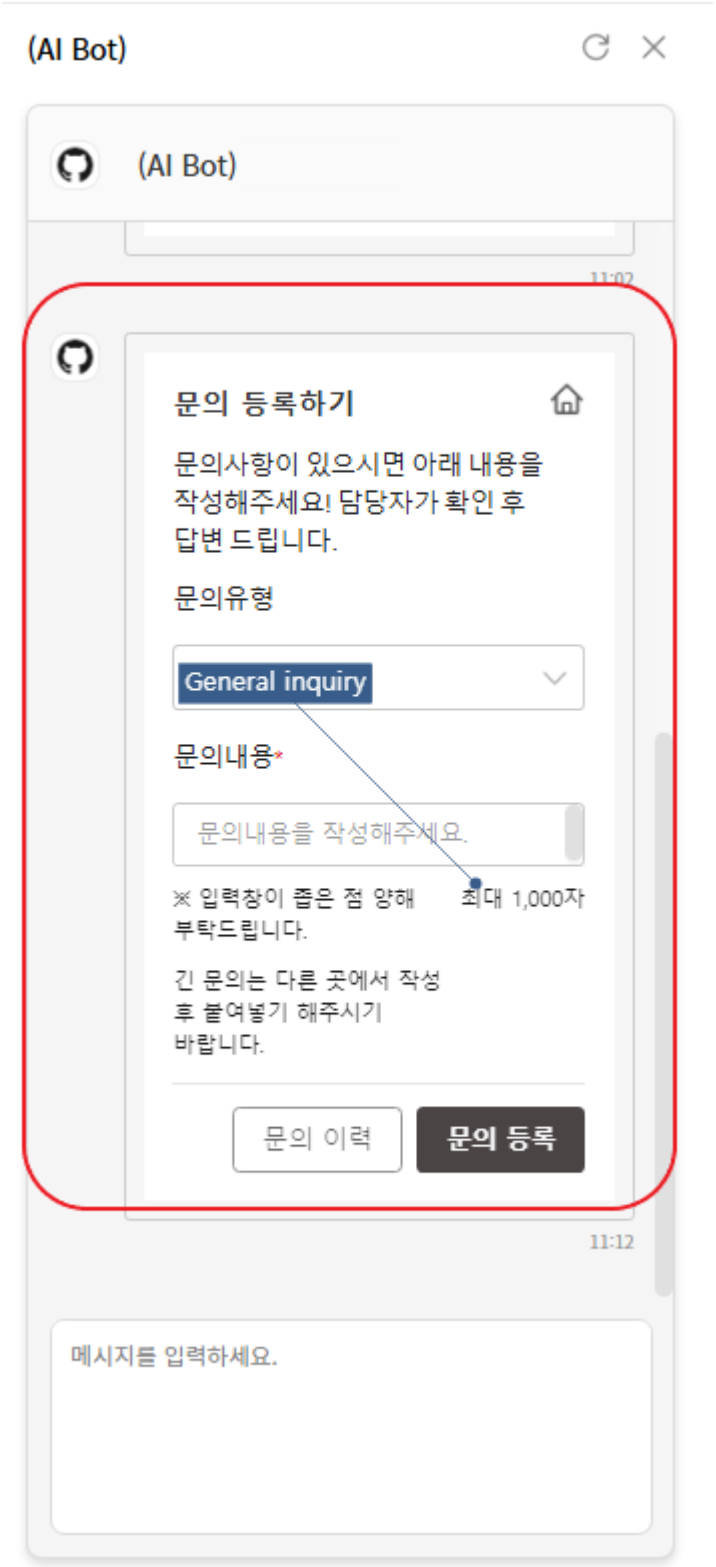
The bot and simulator created in AI Builder clearly understand and analyze user conversations before generating responses through LLM.

Please enter "Year-end tax settlement open schedule" in the simulator dialog window. The AI bot will search the inputted user's expression in the registered data (response information) and provide an answer through LLM.



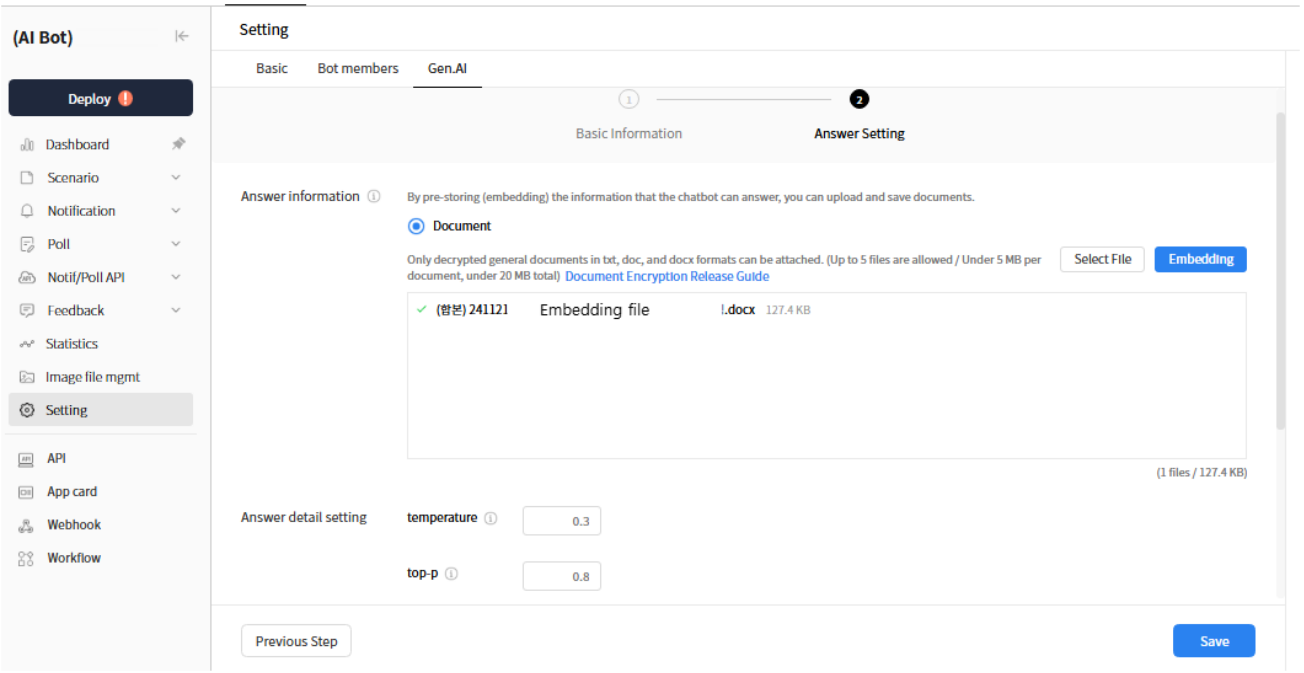
You can check the inquiry registration app card by clicking the 'Inquire' button in the simulator. However, since inquiries cannot be registered in the simulator, testing of inquiry registration can be

done on the bot of Knox Teams Stage.

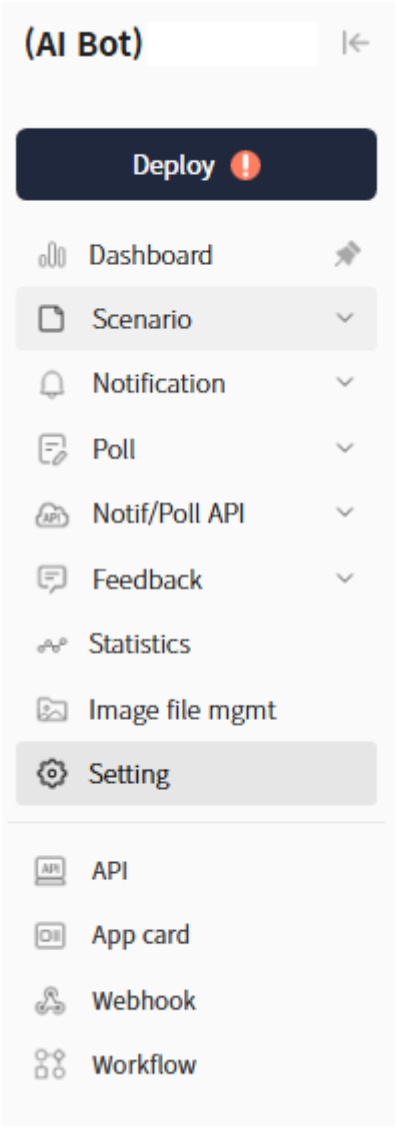


4.2 Deploy

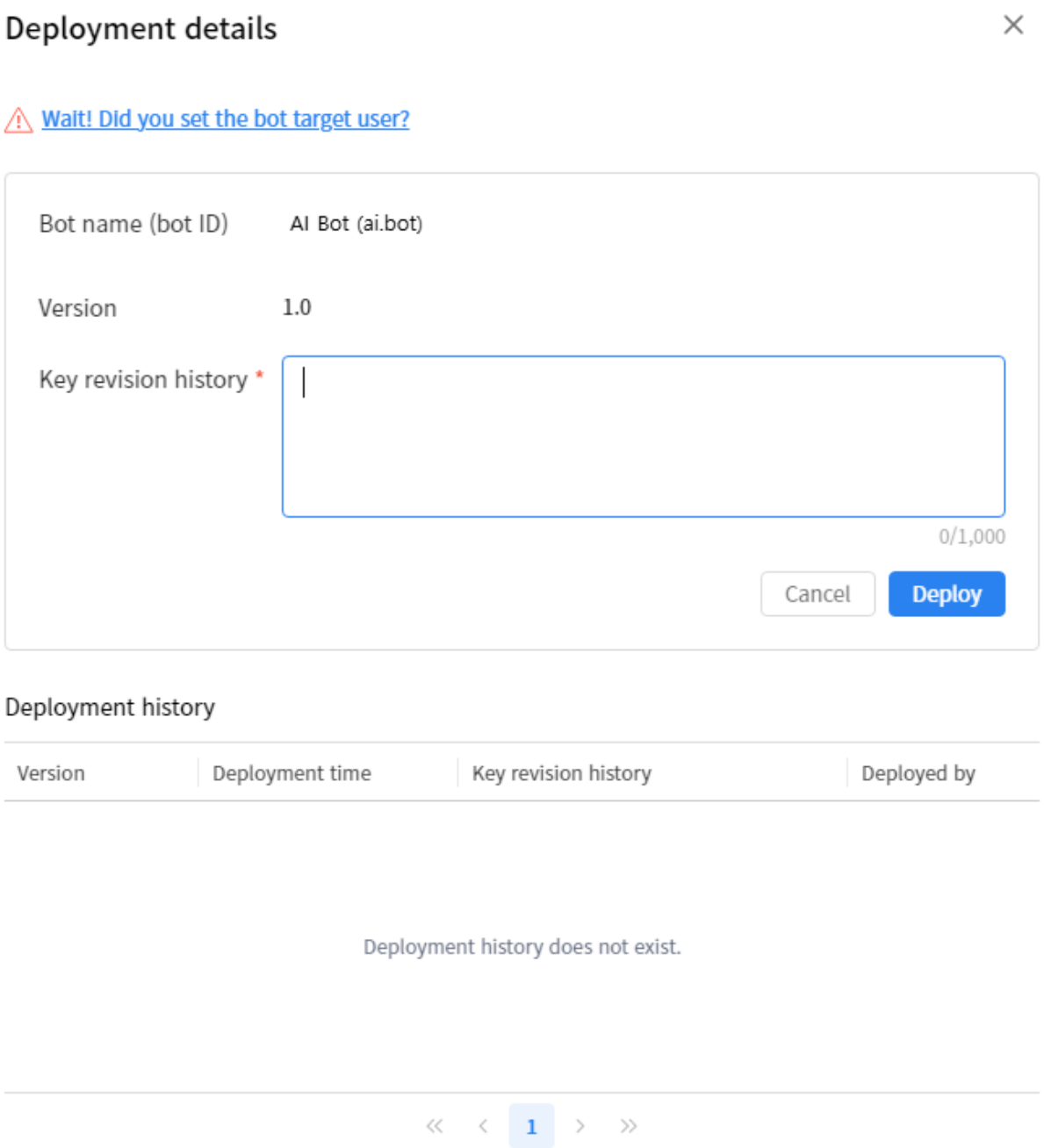
When you have completed testing your dialog, you able to continue to proceed for deployment so that it can be tested on the actual bot.



After selecting a file and saving it, click on the distribution button at the top of the left menu to proceed with the distribution.



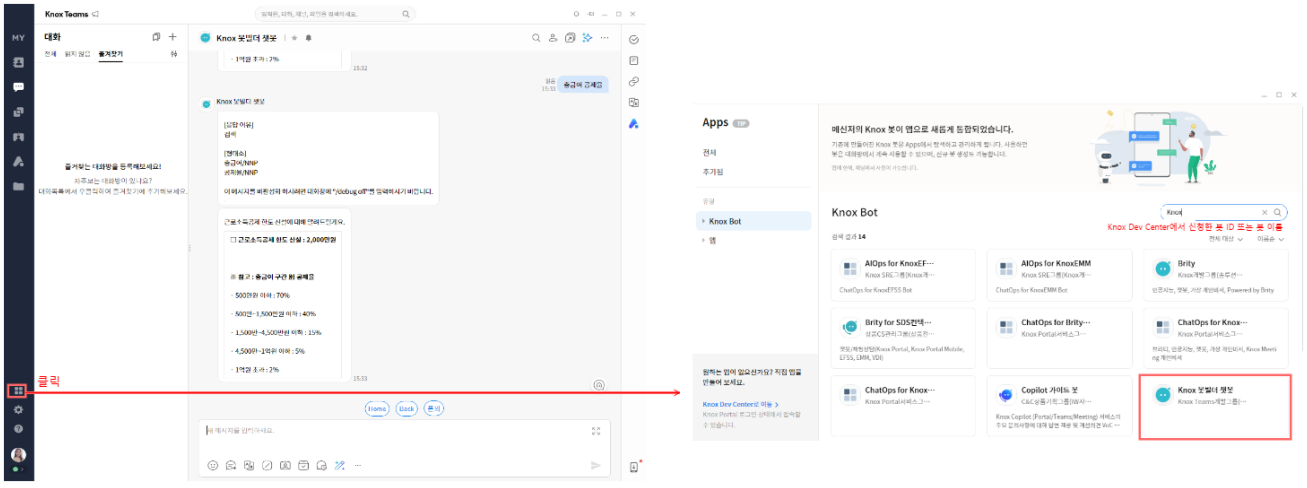
After clicking the Deploy button, please briefly enter the Changes in the box of Details of Deploy popup.


Deployment details
Wait! Did you set the bot target user?
Bot name (bot ID)
AI Bot (ai.bot)
Version
1.0
Key revision history *
0/1,000
Cancel
Deploy
Deployment history
Version
Deployment time
Key revision history
Deployed by
Deployment history does not exist.
1

5.
Viewing the bot in the Knox Teams

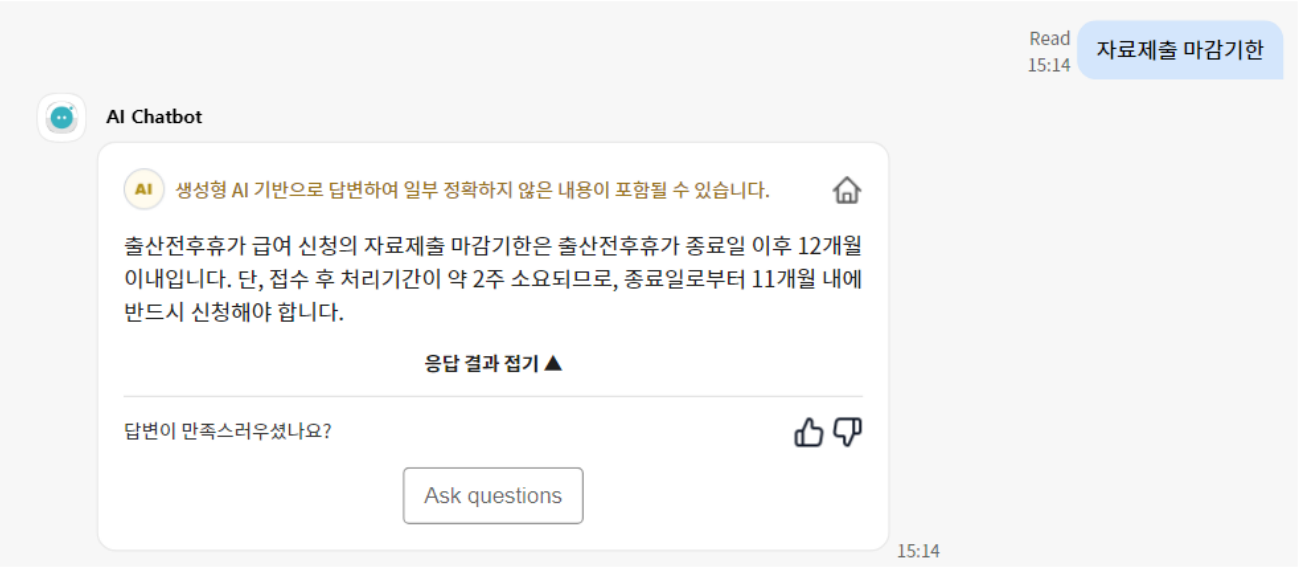
Log in http://www.stage.samsung.net/ with your Knox Portal stage account, and then launch Knox Teams.

> Note
> Please register account for Knox Portal Stage by inquiring through the Knox Portal service desktop (knoxportal@samsung.com).



> Note
> When searching for Knox Bot in Knox Teams, check the "Bot ID" in Knox Dev Center > Bot Account Information and search with that ID and name.

You able to check the content you distributed has been well reflected on the relevant bot by clicking on the search task bar.

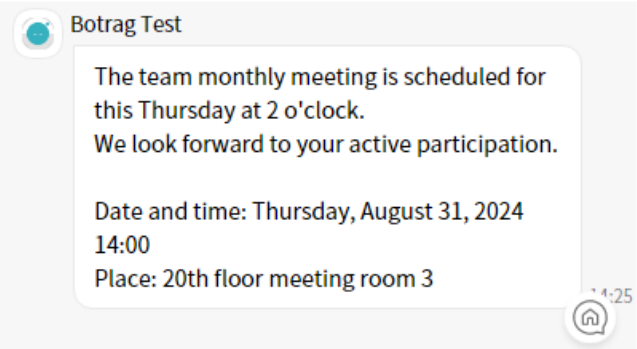


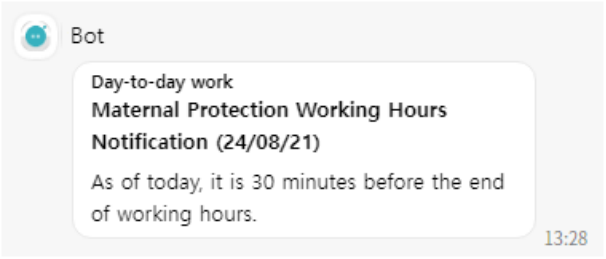
Part V.
Notifications and polls

Notifications

You can create a notification bot to send notices and event notifications with a variety of user interface designs on scheduled dates and times to the recipients you have specified. Also, by utilizing the open APIs for notifications and polls provided by AI Builder, you can execute batch jobs and automate the delivery of notifications and polls from an external system.

This tutorial explains how to register and send various text and app card-based notifications in AI Builder and integrate AI Builder's notification API with external systems or tools to register and send notifications.



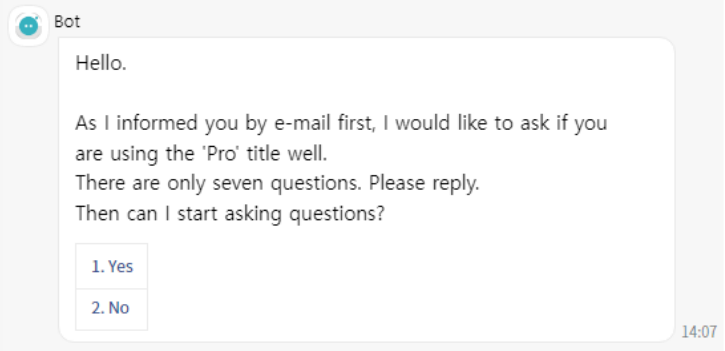


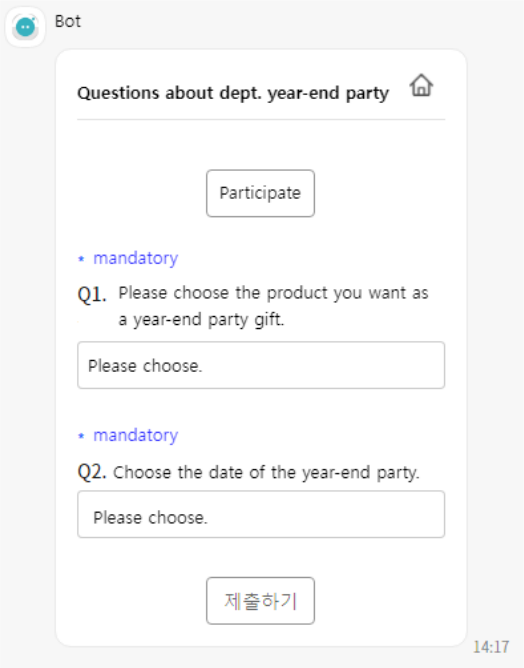
Polls

You can create a poll bot to quickly gather data through polls and analyze the statistics in AI Builder. Also, you can enter slash commands in the chat window to view the poll response status and

download the poll response status file.

This tutorial explains an example of registering and sending a single-selection poll in AI Builder.



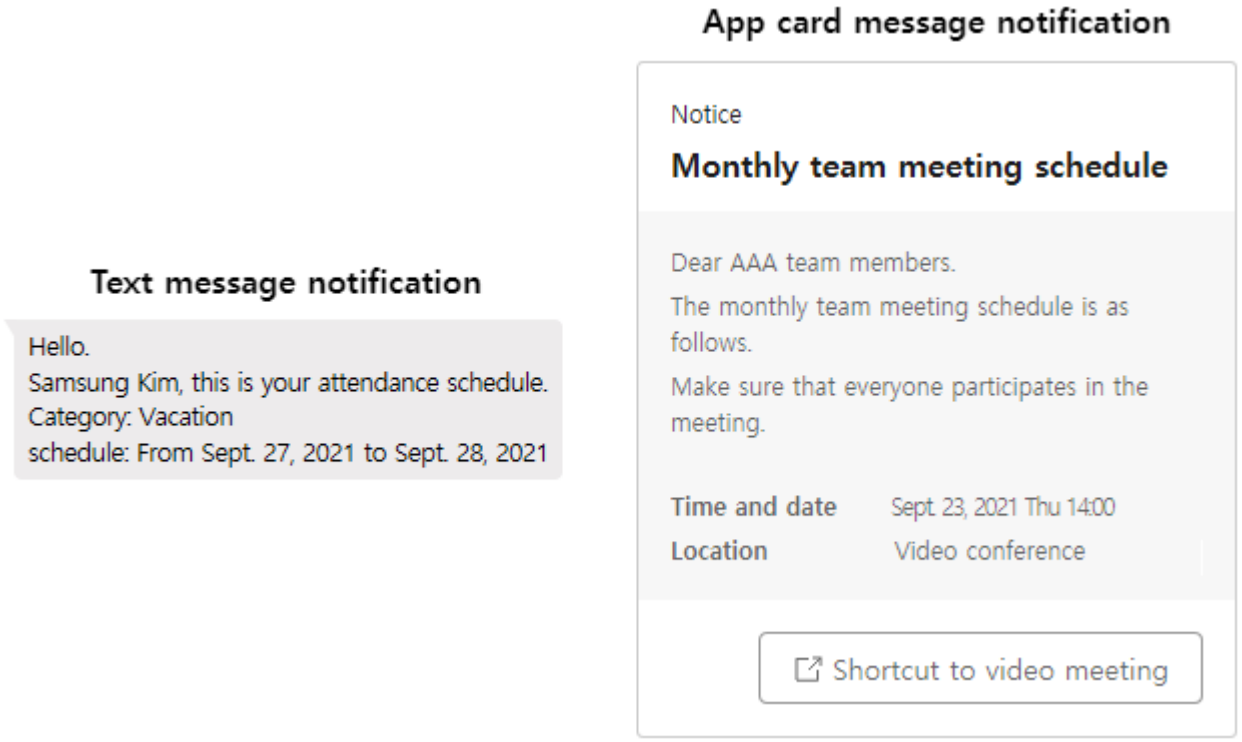


Note

- Polls do not require additional deployment processes. Polls that have already been sent cannot be deleted.
- To test polls before sending them, click **Instantly send to me only**.

1.
Sending notifications

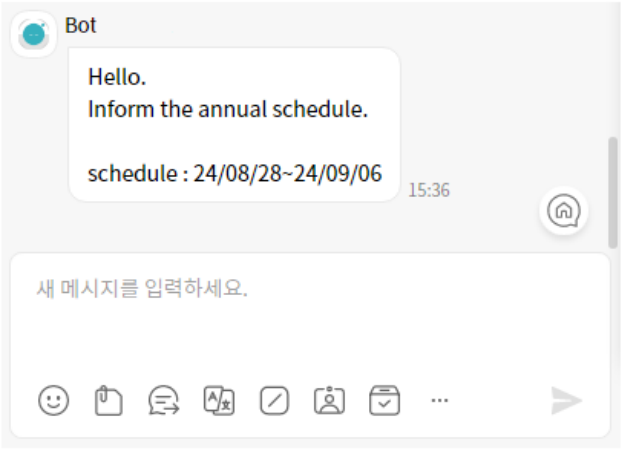
If you need to send notices or notifications to messenger bot users, you can send text- or app card-based notifications in AI Builder.



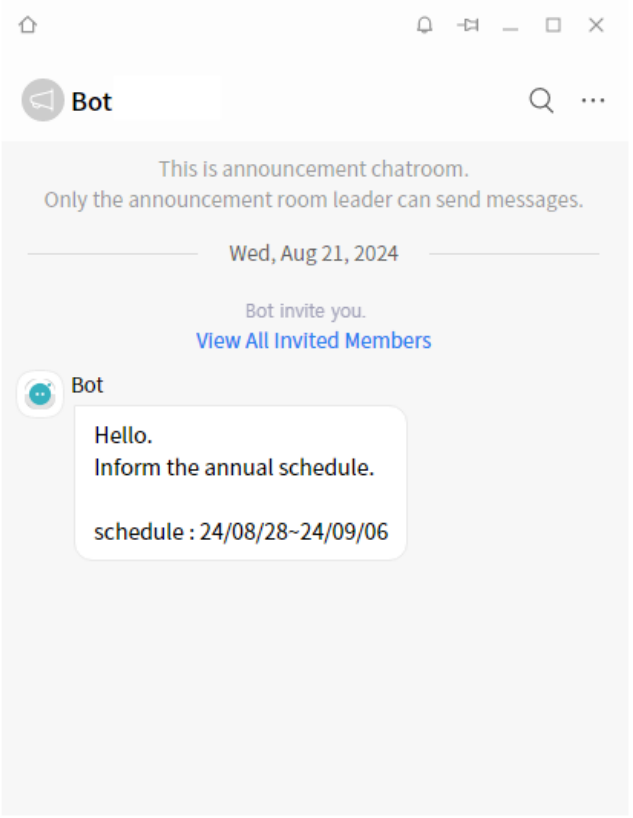
Notifications can be categorized into regular notifications and notices.

- **Regular**: Notifications are sent by conversational messenger bots. After receiving the notifications, users can use speech in the messenger. The speech feature may also be used to receive feedback about the notifications.
- **Notice**: Notices are sent by notice messenger bots. Users can only receive notices because chat windows are not provided for the notices.

Regular notification



Notice notification



Note

- Notifications do not require additional deployment processes. Notifications that have already been sent cannot be deleted.
- To test notifications before sending them, click **Instantly send to me only**.

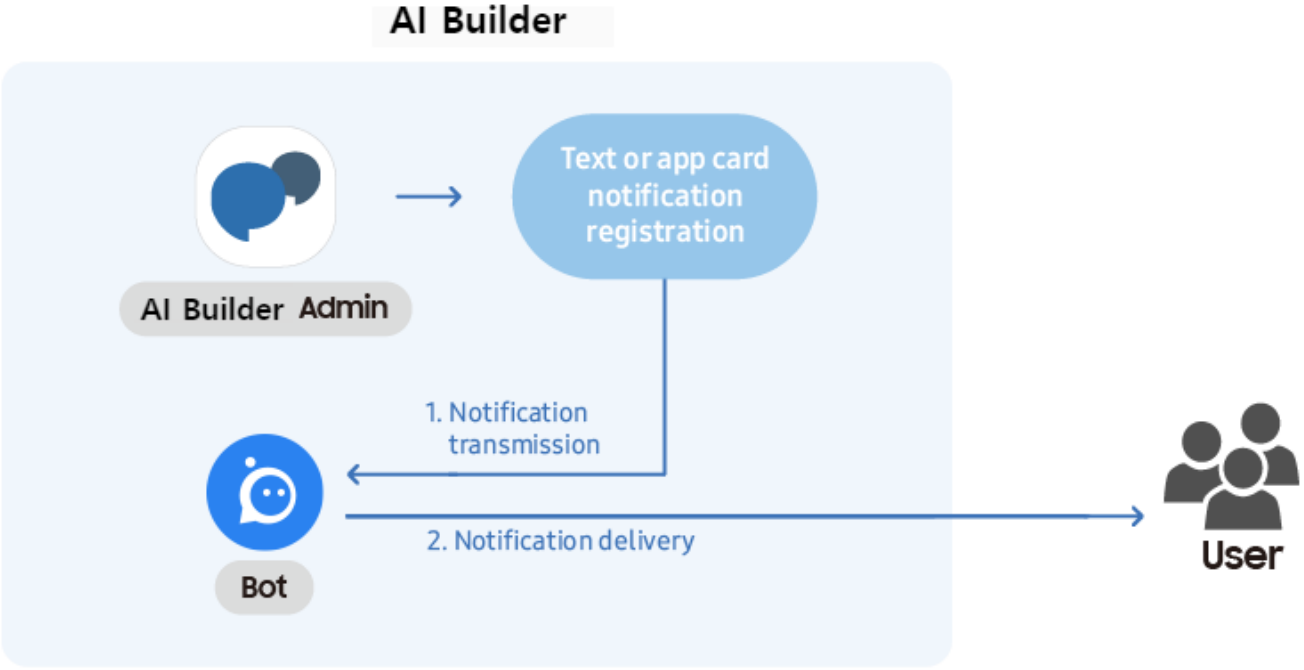
Note

It is recommended that you understand the information provided in the following tutorial before proceeding.

- Developing app card (Getting ready to update)

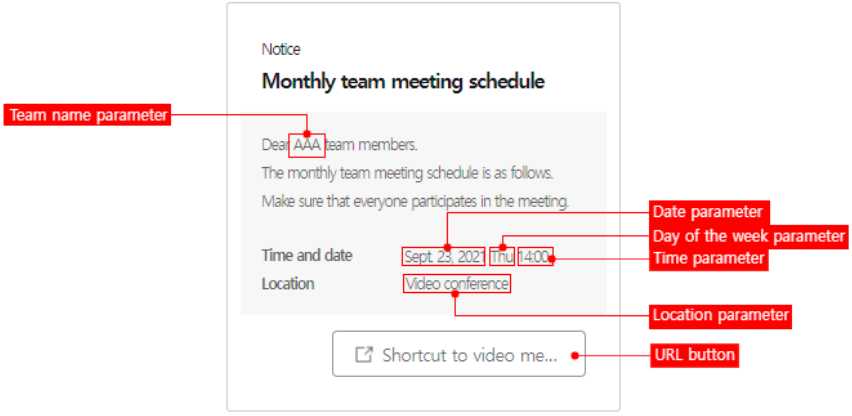
1.1 Notification delivery flow

The following explains the notification registration and delivery flow in AI Builder.



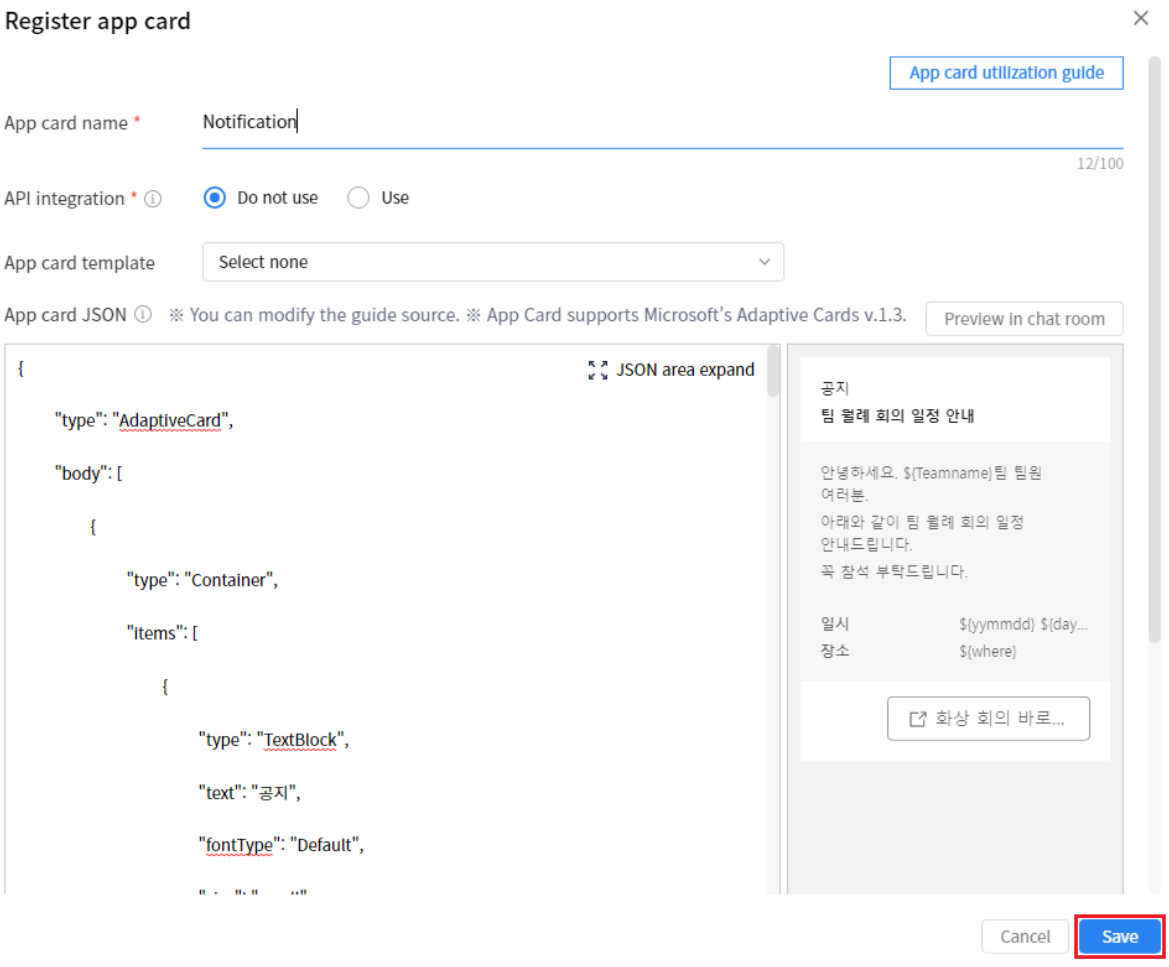
1.2 Registering and sending notifications

You can follow the instructions provided in the tutorial to register app cards and send app card message notifications



To send an app card message notification, complete the following steps:

1. At the top of the AI Builder home screen, click the **My bots** tab.
2. On the bot list of the "My bots" screen, click a bot to register the notification app card.
3. On the bot details screen, click **App card** > **App cards** > **Register**.
4. In the "Register app card" pop-up window, enter the registration information, fill in the **App card JSON** field referring to the following example, and then click **Save**.
   - You must fill in the mandatory fields indicated with asterisks (*).
   - You can modify the `"text":` item by entering the desired text content.
   - `"${VariableName}"` is a user-defined variable and you must upload the substitute phrase to replace it when registering the notification.




```
{
    "type": "AdaptiveCard",
    "body": [
        {
```

```
 5              "type": "Container",
 6              "items": [
 7                  {
 8                      "type": "TextBlock",
 9                      "text": "Notice",
10                      "fontType": "Default",
11                      "size": "small"
12                  },
13                  {
14                      "type": "TextBlock",
15                      "text": "Monthly team meeting schedule",
16                      "fontType": "Default",
17                      "size": "medium",
18                      "weight": "Bolder",
19                      "wrap": "true"
20                  }
21              ]
22          },
23          {
24              "type": "Container",
25              "items": [
26                  {
27                      "type": "TextBlock",
28                      "text": "Dear ${Teamname} team members.",
${Teamname} substitute variable
29                      "wrap": true,
30                      "fontType": "Default",
31                      "size": "Small",
32                      "isSubtle": true
33                  },
34                  {
35                      "type": "TextBlock",
36                      "text": "The monthly team meeting schedule is
as follows.",
37                      "wrap": true,
38                      "fontType": "Default",
39                      "size": "Small",
40                      "isSubtle": true
41                  },
42                  {
```

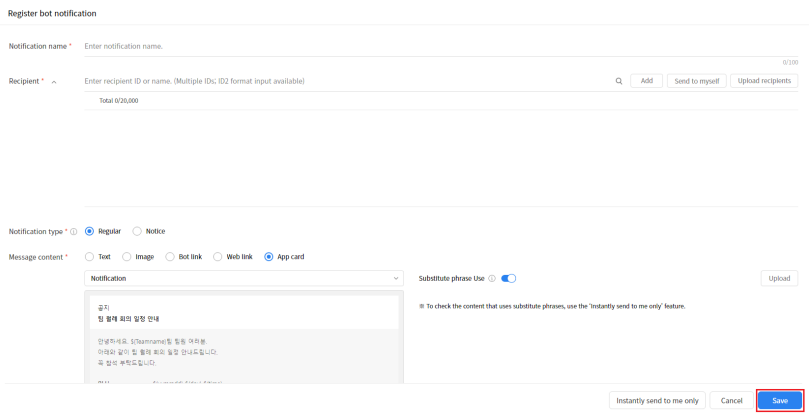
```
                    "type": "TextBlock",
                    "text": "Make sure that everyone participates
in the meeting.",
                    "wrap": true,
                    "fontType": "Default",
                    "size": "Small",
                    "isSubtle": true
                },
                {
                    "type": "Container"
                },
                {
                    "type": "Container"
                },
                {
                    "type": "Container"
                },
                {
                    "type": "Container"
                },
                {
                    "type": "Container"
                },
                {
                    "type": "Container"
                },
                {
                   "type": "ColumnSet",
                   "columns": [
                       {
                           "type": "Column",
                           "width": "112px",
                           "items": [
                               {
                                   "type": "TextBlock",
                                   "text": "Time and date",
                                   "fontType": "Default",
                                   "isSubtle": true,
                                   "size": "small",
                                   "weight": "Bolder"
```

```
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${yymmdd} ${day}
${time}",  // Substitute variable
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "size": "small",
                                    "isSubtle": true
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                },
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "112px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "Location",
                                    "fontType": "Default",
                                    "size": "small",
                                    "weight": "Bolder",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
```

```
                                "type": "Column",
                                "width": "stretch",
                                "items": [
                                    {
                                        "type": "TextBlock",
                                        "text": "${where}",  // Substitute variable
                                        "spacing": "None",
                                        "fontType": "Default",
                                        "wrap": true,
                                        "size": "small",
                                        "isSubtle": true
                                    }
                                ],
                                "spacing": "None"
                            }
                        ]
                    }
                ],
                "spacing": "Large",
                "style": "emphasis",
                "bleed": true
            },
            {
                "type": "ColumnSet",
                "columns": [
                    {
                        "type": "Column",
                        "width": "stretch"
                    },
                    {
                        "type": "Column",
                        "width": "auto",
                        "items": [
                            {
                                "type": "ActionSet",
                                "actions": [
                                    {
                                        "type": "Action.OpenUrl",
                                        "title": "Shortcut to video
```

```
meeting",  // The URL button name
160                                     "url": "http://aaa.com",  //
Enter the URL to access when clicked.
161                                     "iconUrl":
"https://aibuilder.britymail.com/images/webhook/ic_window.png"
162                                 }
163                             ]
164                         }
165                     ],
166                     "spacing": "Medium"
167                 }
168             ],
169             "spacing": "Large"
170         }
171     ],
172     "$schema":
"http://adaptivecards.io/schemas/adaptive-card.json",
173     "version": "1.3"
174 }
```

5. On the bot details screen, click **Notification** > **Bot notifications** > **Register**.
6. In the "Register bot notification" pop-up window, enter the notification information and click **Save**.
   - You must fill in the mandatory fields indicated with asterisks (*).
   - Example of app card notification settings
     - **Notification type**: Select **Regular** or **Notice**.
     - **Message type**: Select **App card** and link it with a registered app card.
     - **Substitute phrase**: Select **Use** and upload the substitute phrase file for the variable that was used in the registration. Click "Upload" button to download the file form.

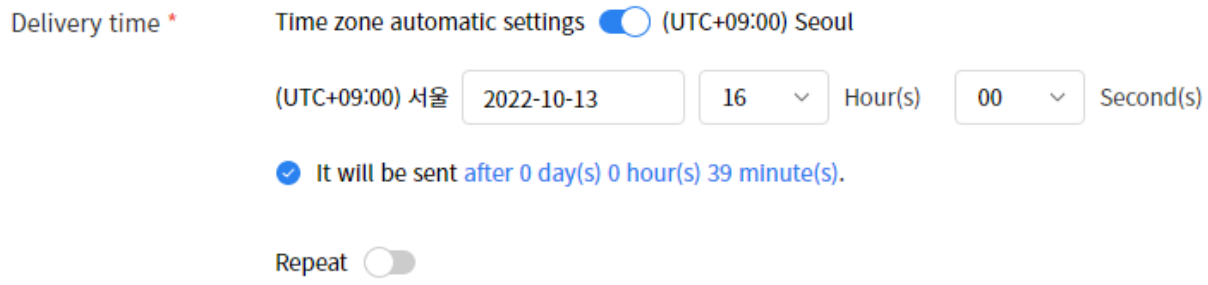


Note

- **Sending text message notifications**

You can select **Text** as the **Message type** and enter the notification message to send a text message notification without registering an app card.

After the notification registration is complete, notifications will be sent based on the date and time and the repeat conditions configured during the registration.

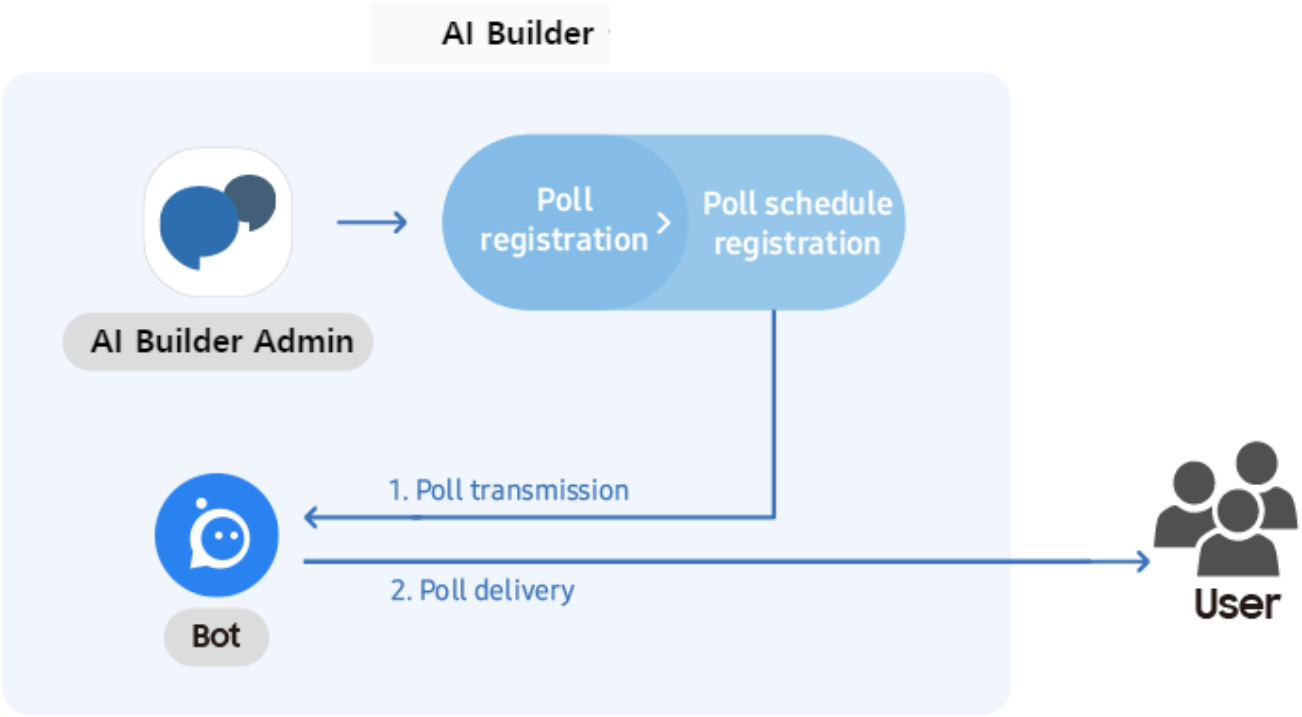


On the **Notification** > **Notification delivery result** screen, you can search for and view the notification delivery results and download them as a file.



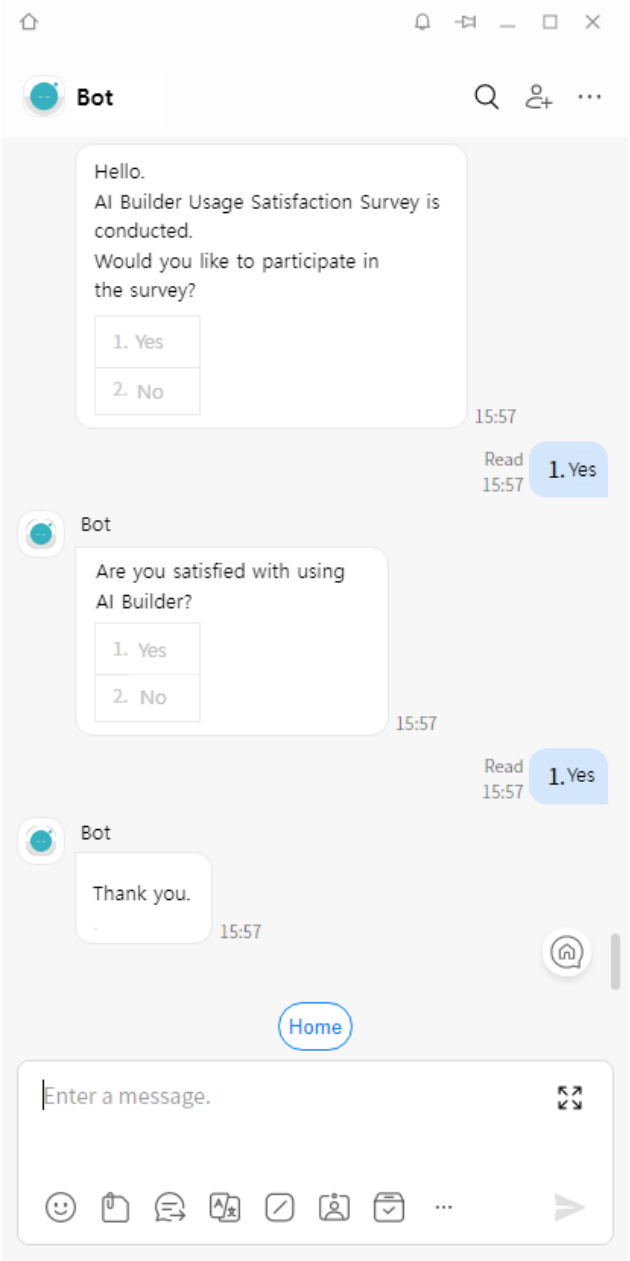
2.
Poll delivery flow

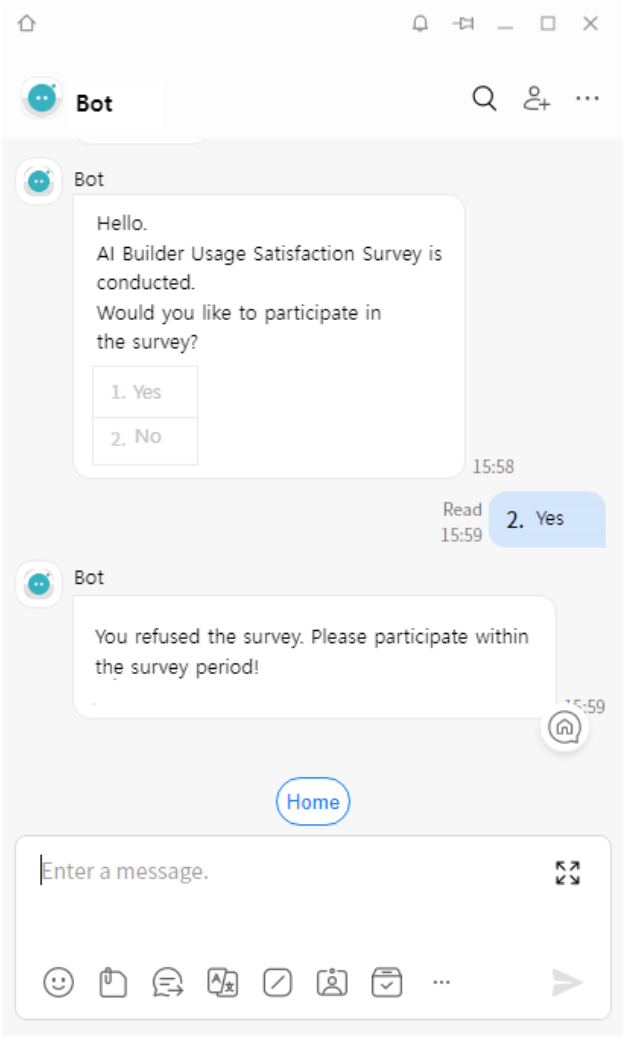
The following explains the poll registration and delivery flow in AI Builder.



3.
Registering polls

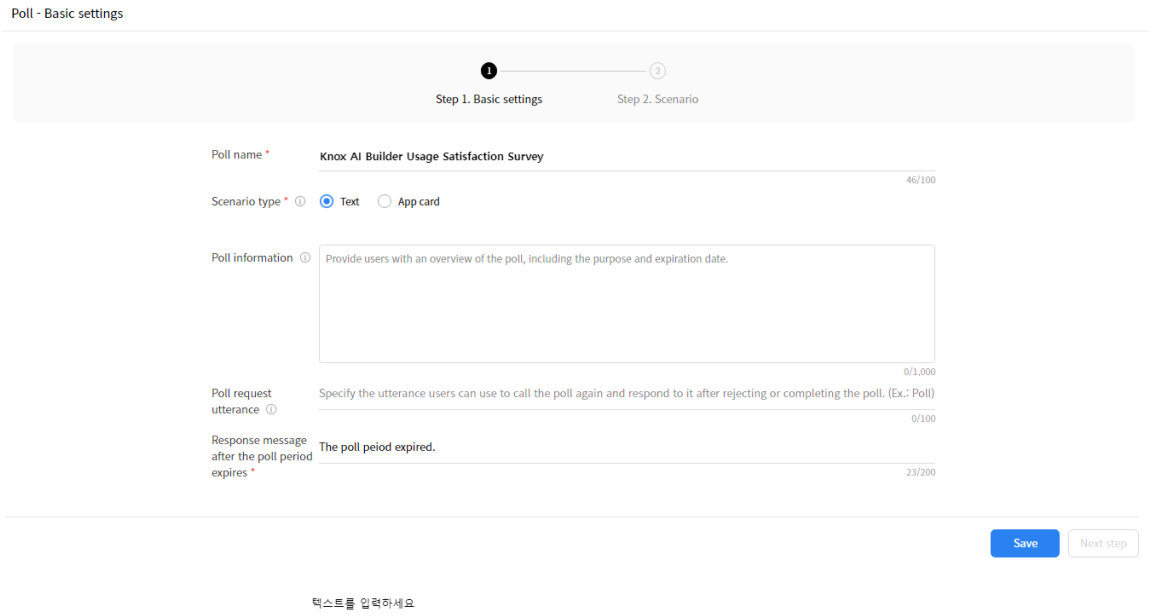
You can register a poll and send it via the Knox Teams in the following ways.



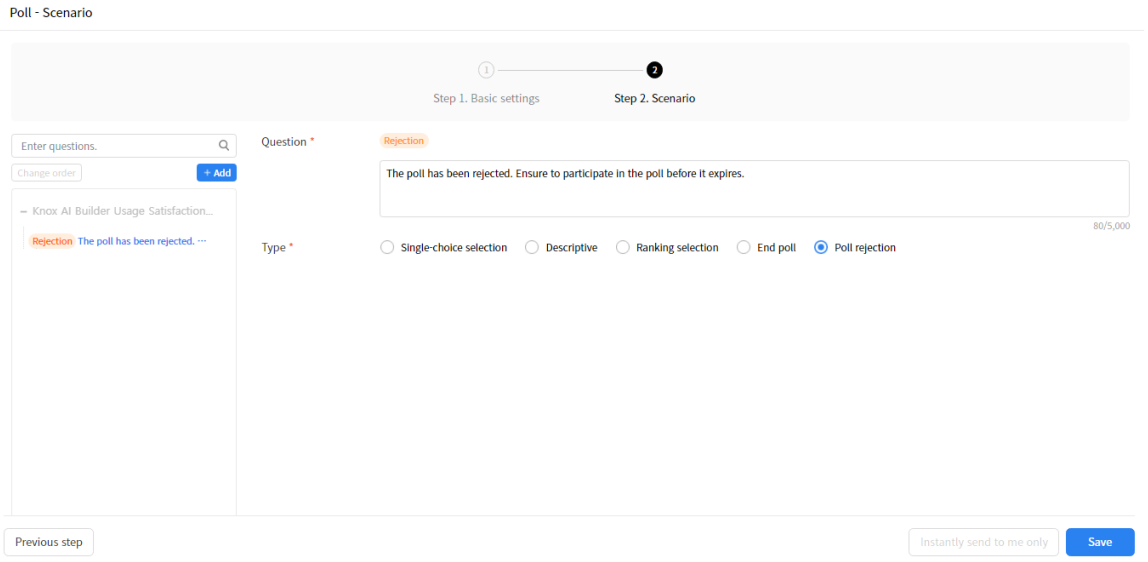


To register a poll, complete the following steps:

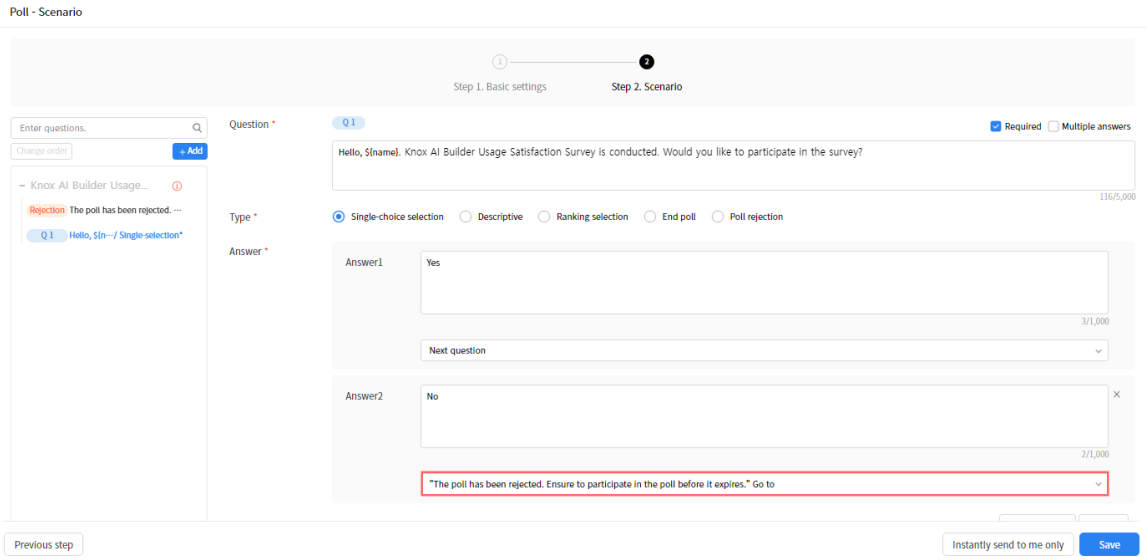
1. At the top of the AI Builder home screen, click the **My bots** tab.
2. On the bot list of the "My bots" screen, click a bot to register the poll.
3. On the bot details screen, click **Poll** > **Poll scenarios** > + **Register poll.**
4. Enter the basic information in the “Step 1. Basic settings” on the “Poll Registration” screen and click **Save**.
   - Example of a poll configuration (Knox AI Builder Satisfaction Survey)
     - **Poll name**: Enter “Knox AI Builder Satisfaction Survey.”
     - Scenario **Type**: Choose the "Text".
     - **Response message after the poll period expires**: Enter “The poll period expired.”



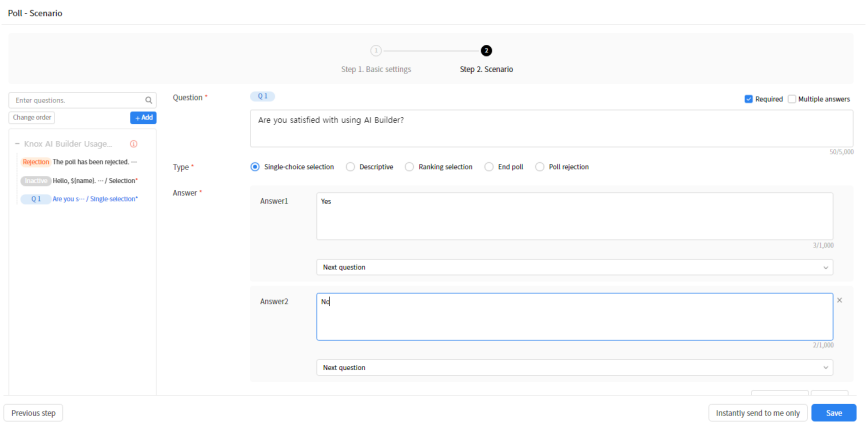
5. Create a poll scenario In “Step 2. Writing a Poll.” Click **Add**, enter the poll content, and then click **Save**.
   - Example of a poll configuration (Knox AI Builder Satisfaction Survey)
     - **Question**: Enter “The poll has been rejected. Ensure to participate in the poll before it expires.”
     - **Type**: Select **Poll rejection**.


Poll - Scenario
1
Step 1. Basic settings
2
Step 2. Scenario
Enter questions.
Change order
+ Add
Knox AI Builder Usage Satisfaction...
Rejection The poll has been rejected. ···
Question *
Rejection
The poll has been rejected. Ensure to participate in the poll before it expires.
80/5,000
Type *
Single-choice selection
Descriptive
Ranking selection
End poll
Poll rejection
Previous step
Instantly send to me only
Save

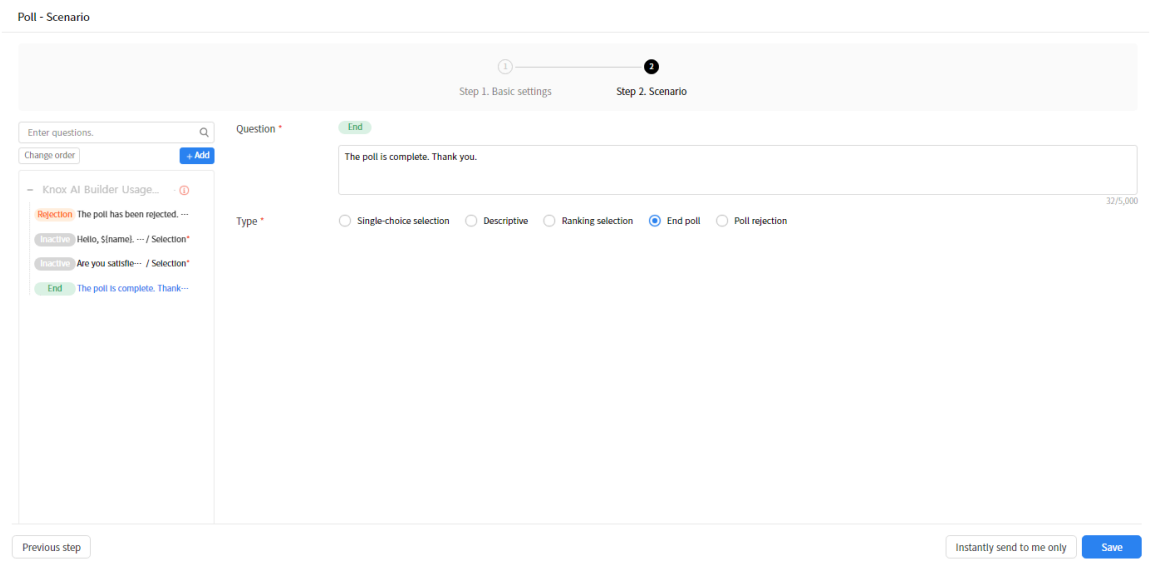
6. Click **Add**, enter the poll content, and then click **Save**.
   - Example of a poll configuration (Multiple choices - single choice)
     - **Question**: Enter “Hello, ${name}. The Knox AI Builder Satisfaction Survey is in progress. Will you participate in the poll?” (“${name}” is a user-defined variable. You need to upload the substitute phrase file for “${name}” when registering the poll schedule.)
     - **Type**: Choose the **Multiple choices**.
     - **Answer**: Enter “Yes” in the **Answer1** field, “No” in the **Answer2** field, and then, for the **Answer2 Next** item, select to move to “The poll has been rejected. Ensure to participate in the poll before it expires.”



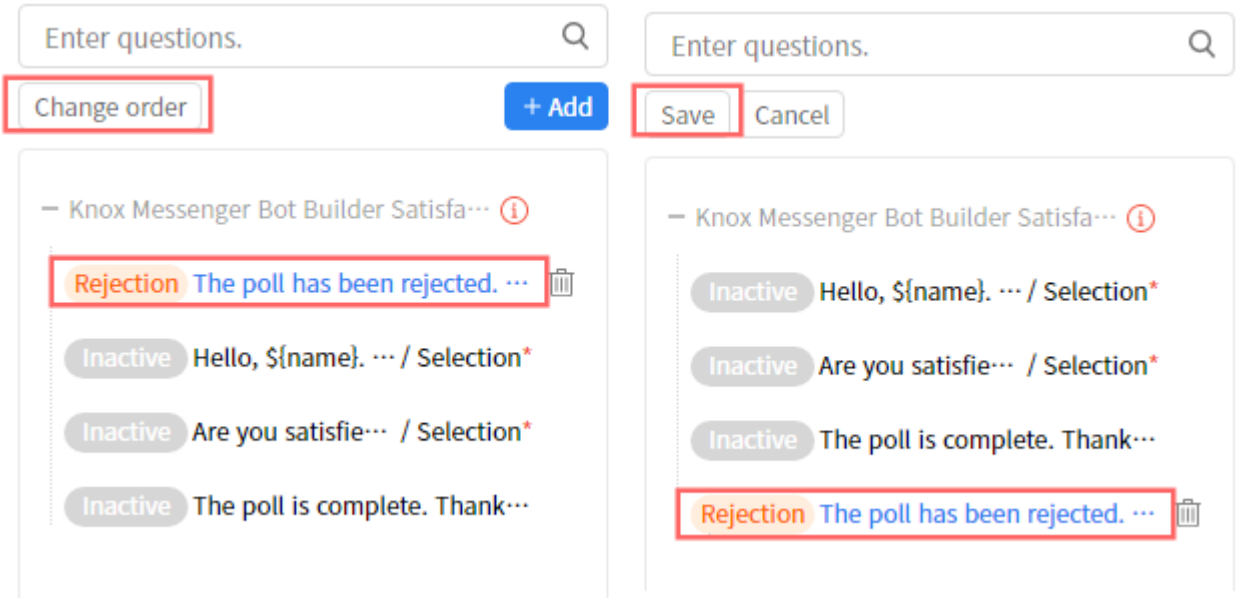
7. Click **Add**, enter the poll content, and then click **Save**.
   - Example of a poll configuration (Knox AI Builder Satisfaction Survey)
     - **Question**: Enter “Are you satisfied with Knox AI Builder?”
     - **Type**: Select **Single-selection**.
     - **Answer**: Enter “Yes” in the **Answer1** field and “No” in the **Answer2** field.



8. Click **Add**, fill in the closing statement, and then click **Save**.
    - Example of a poll configuration (Knox AI Builder Satisfaction Survey)
        - **Question**: Enter “The poll is complete. Thank you.”
        - **Type**: Select **End poll**.



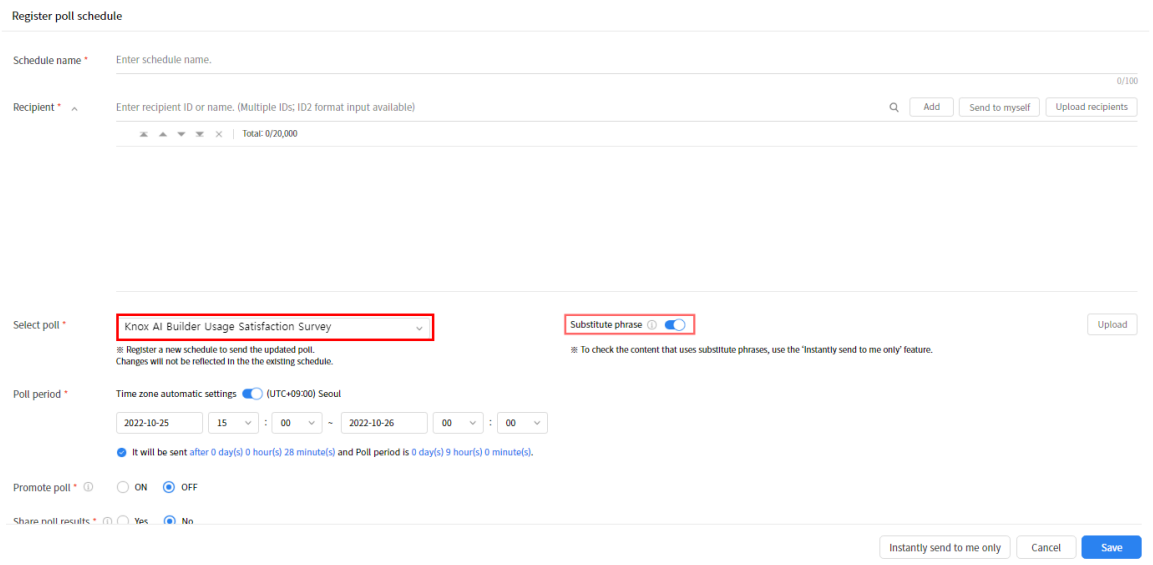
9. Click **Change order**, move the poll rejection item to the last, and then click **Save**.



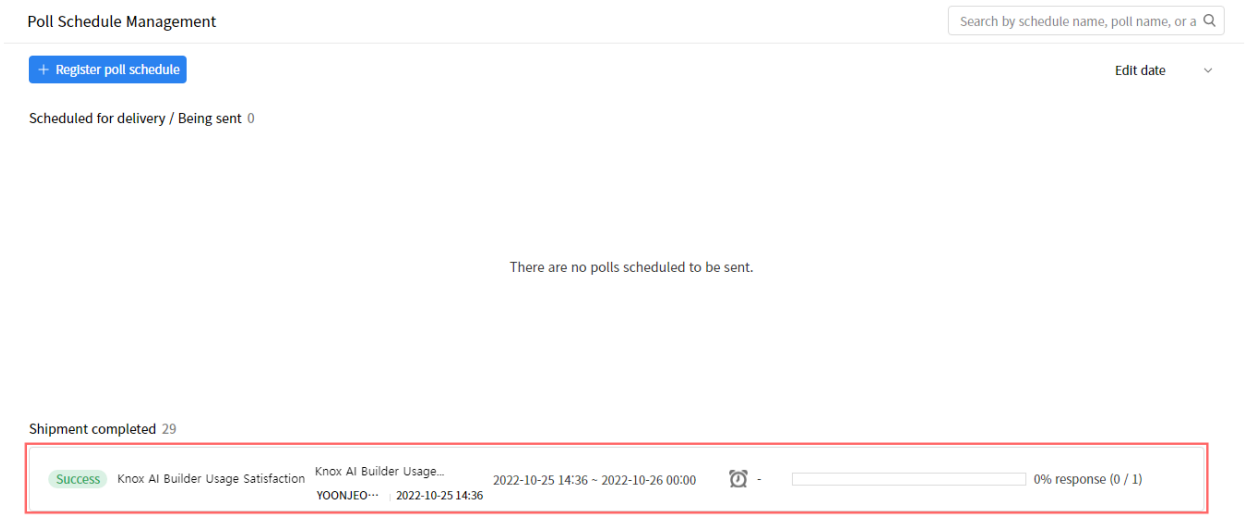
4. Registering poll schedules

To register poll schedules, complete the following steps:

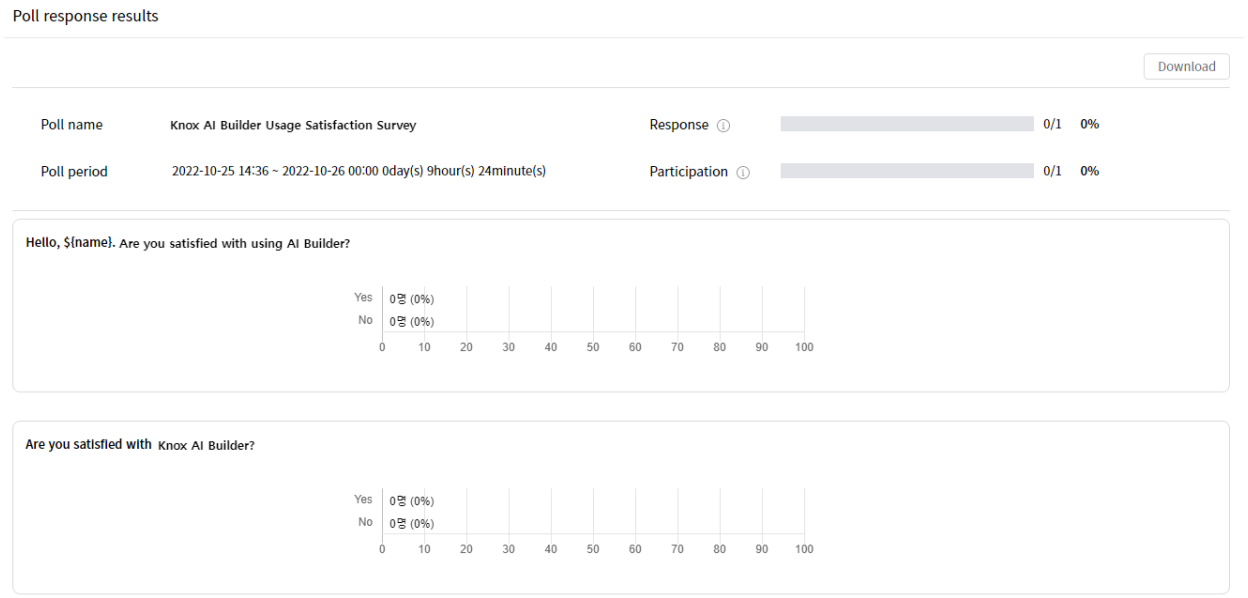
1. On the bot details screen, click **Poll** > **Poll delivery** > **Poll schedule** > **Register**.
2. In the "Register poll schedule" pop-up window, enter the poll schedule information and click **Save**.
   - You must fill in the mandatory fields indicated with asterisks (*).
   - Example of poll schedule registration
     - **Select poll**: Select the poll registered in 3. Registering polls.
     - **Substitute phrase**:Change the toggle to **ON** and upload the substitute phrase file for ${name}, which was used in the registration. For more information about registering the substitute phrase, refer to the user manual.



3. Click the list in the “Shipment completed” area of **Poll** > **Poll Schedule Management** to view detailed poll results.



You can click a poll to view the detailed poll results.

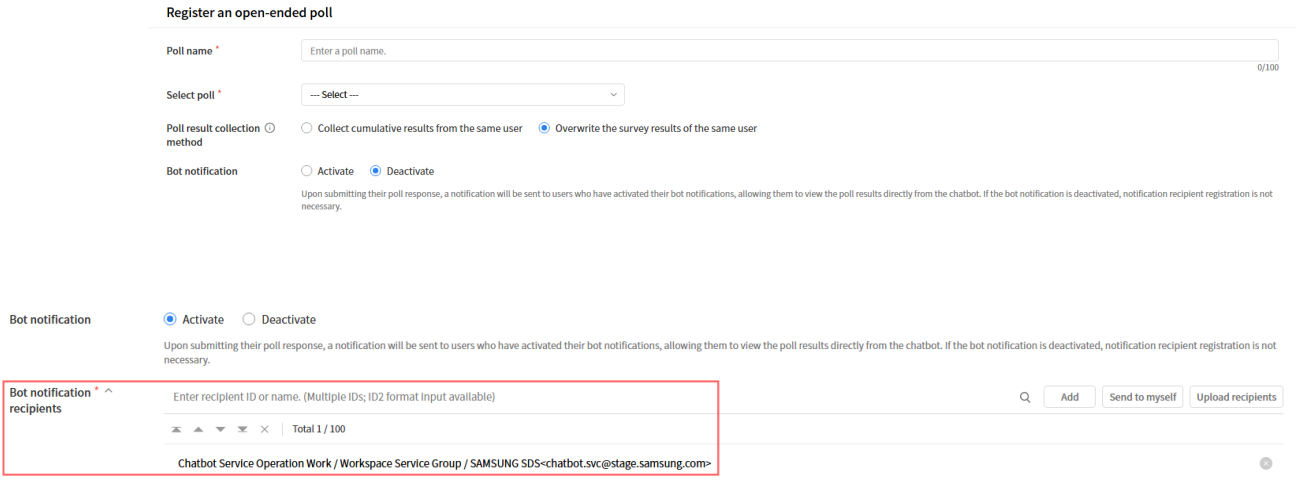


5.
Registering open-ended polls

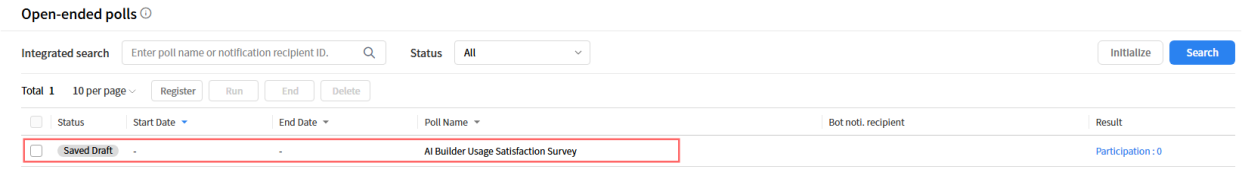
Open-ended poll have no restrictions on the duration or the recipients, and bot users can search for and participate in them at any time when needed.

To register an open-ended poll, follow the procedure below.

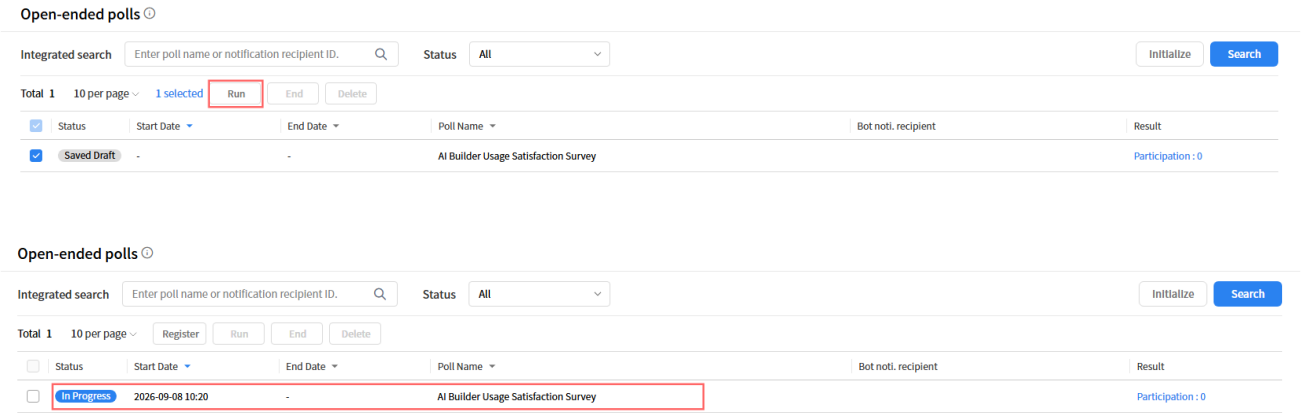
1. On the bot details screen, click **Poll** > **Open-ended polls** > **Register**.
2. In the **"Register an open-ended poll"** screen, enter the poll information and click Save.
   - You must fill in the mandatory fields indicated with asterisks (*).
   - Example of poll schedule registration
     - **Poll name**: Enter open-ended poll name
     - **Select poll**: Select the poll registered in Registering polls
     - **Poll result collection method**: Choose how to handle multiple submissions from the same user — either (1) all submitted answers are stored, or (2) the most recent answer overwrites the previous one
     - **Bot notification**: When set to "Active," a notification will be sent via chatbot to registered recipients whenever a poll response is submitted. Recipients can view poll results directly from the chatbot.



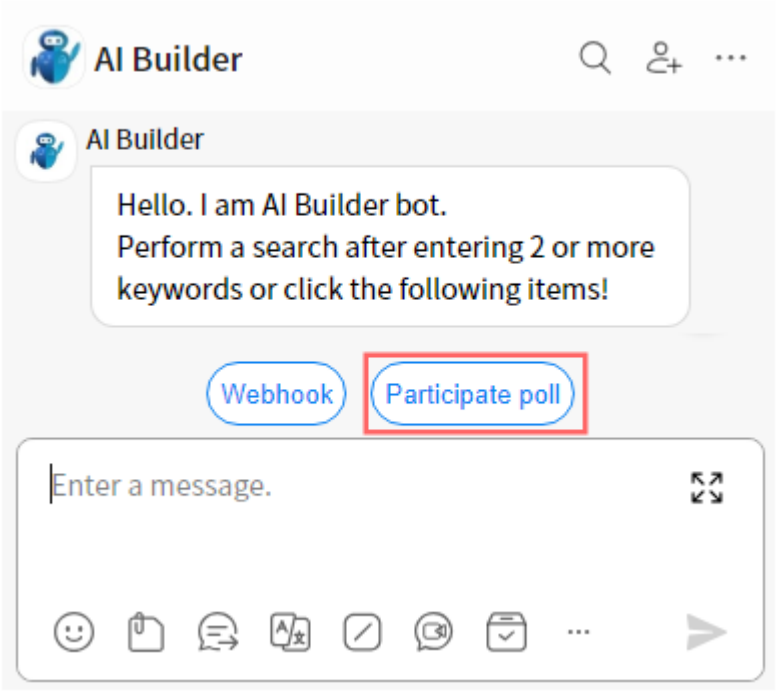
3. After saving, the registered open-ended poll's status will be displayed as "Saved Draft".



4. When you click "Run" on a poll with **"Saved Draft"** status, the status changes to **"In Progress"** and the poll begins.

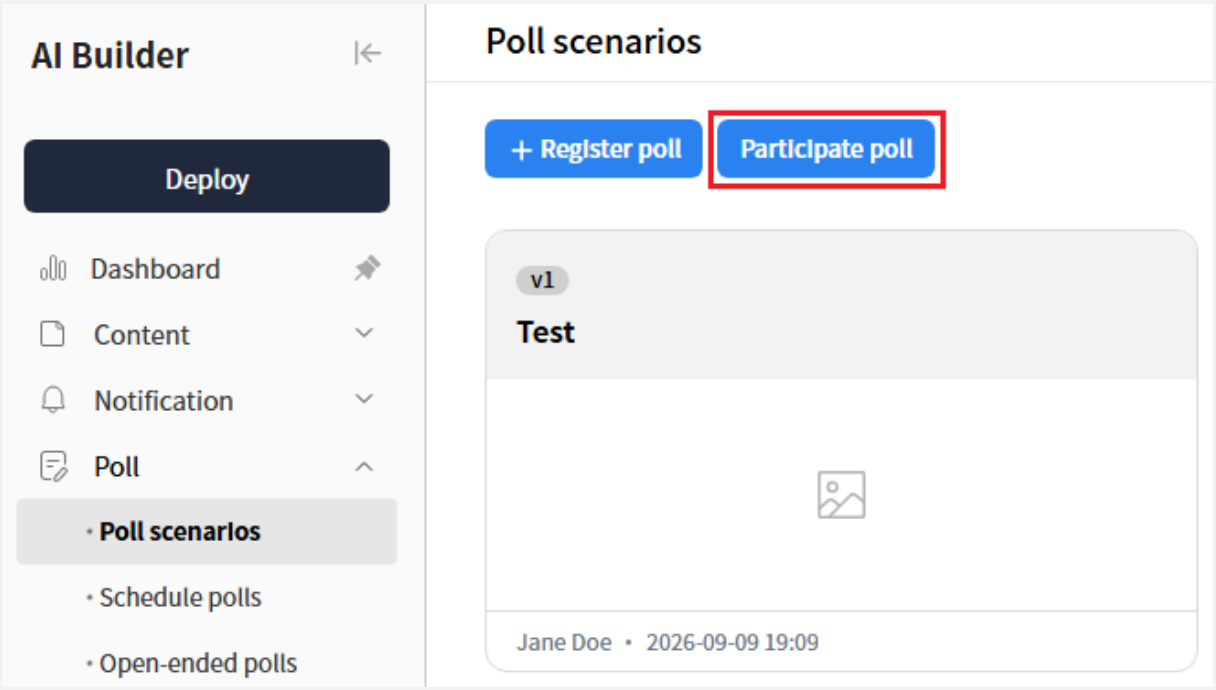


5. For an in-progress open-ended poll, users can view and participate immediately by clicking the **"Participate poll"** button at the bottom of the messenger bot chat window without additional deployment is needed.

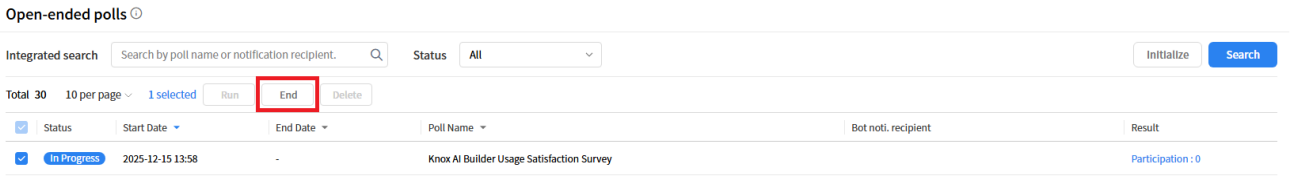


Note

The button name “Participate poll” that appears in the messenger can be changed via the “Poll button configuration” button located at the top-left of the “Poll scenarios” menu.



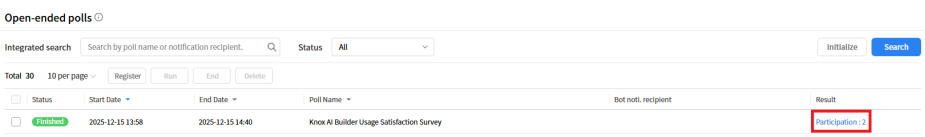
6. To end an in-progress open-ended poll, select it and click the **"End"** button.

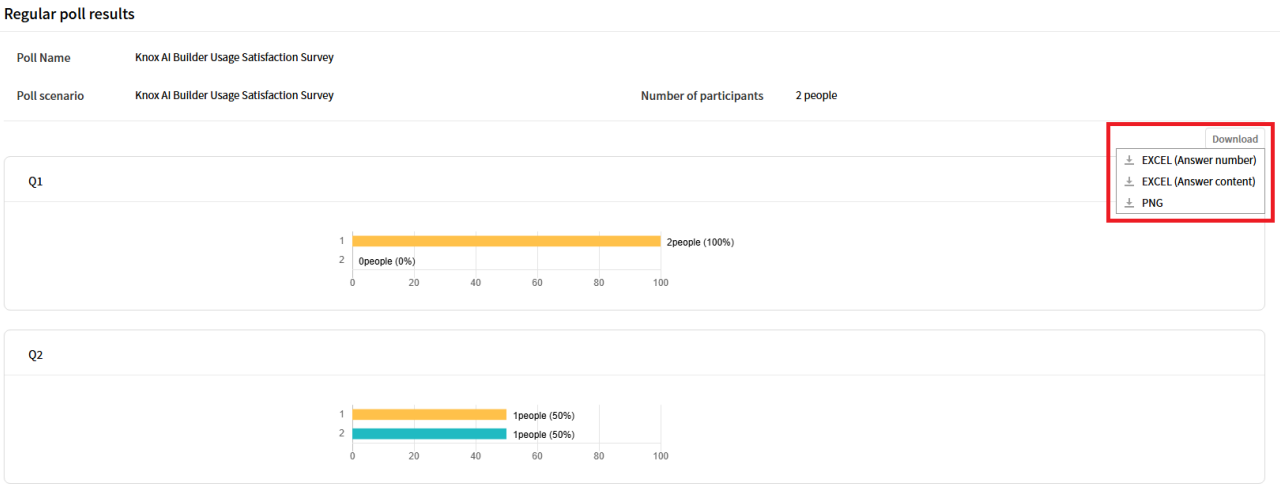


Note

Even if the poll has ended, you can re-run it by clicking the “Run” button.

7. Click on **"Participation"** to check and download the poll results.




Regular poll results
Poll Name
Knox AI Builder Usage Satisfaction Survey
Poll scenario
Knox AI Builder Usage Satisfaction Survey
Number of participants
2 people
Download
EXCEL (Answer number)
EXCEL (Answer content)
PNG
Q1
1
2
2people (100%)
0people (0%)
0
20
40
60
80
100
Q2
1
2
1people (50%)
1people (50%)
0
20
40
60
80
100

Part VI.
Bot Management

1. Dashboard

You can check scheduled notification/poll schedule and registered feedback inquiries, monitor the non-response status of the bot, and check statistical data, such as the number of monthly bot users and response status. Manage more easily with the dashboard.

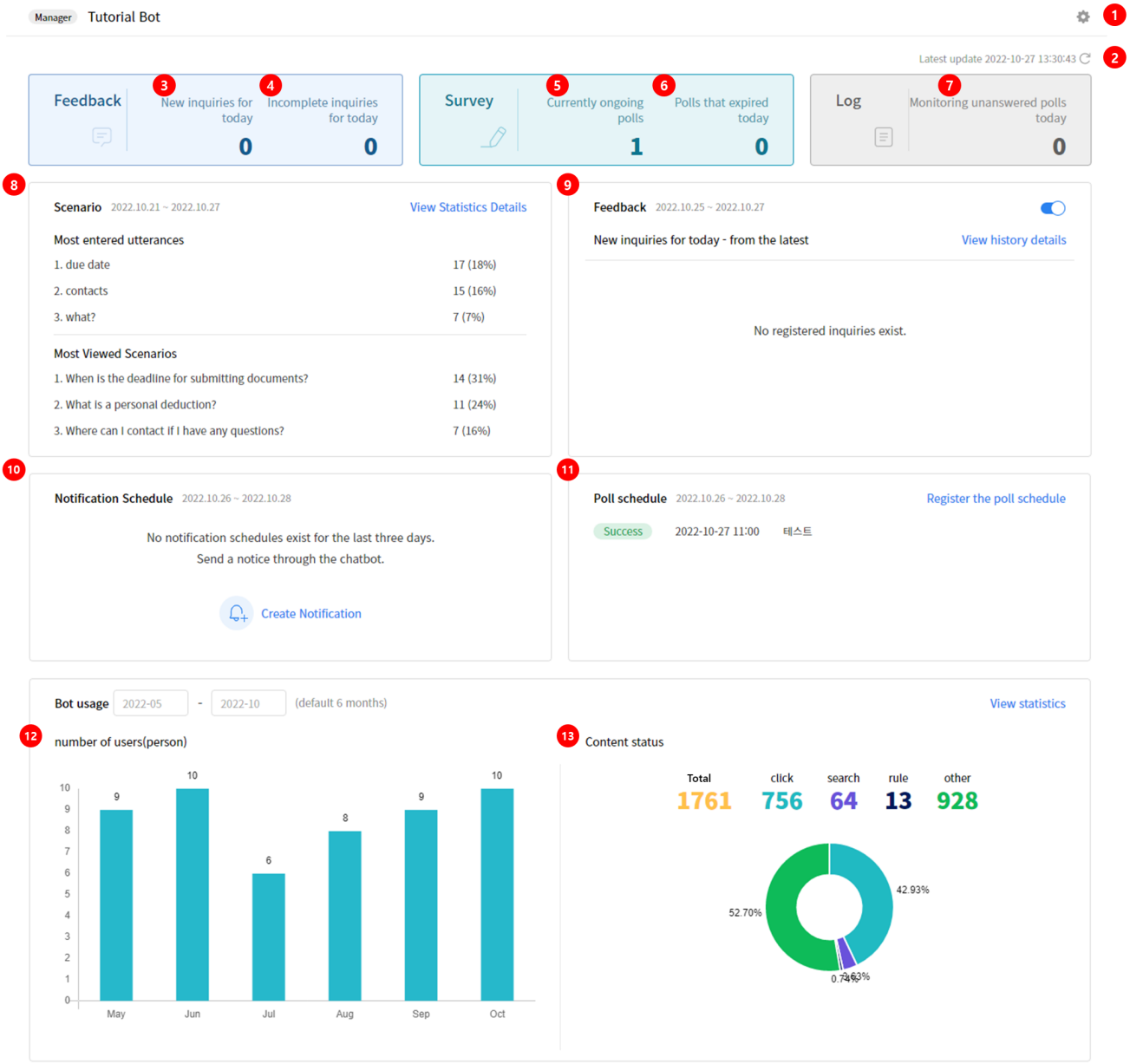
Complete the following steps to access the dashboard.

1. Click the **Bot** tab on top of the AI Builder home screen.

2. Select a bot from the bot list on the "Bot" screen.

3. Click **Dashboard** in the left menu of bot details screen.

> Note
>
> If the pin in the right of **Dashboard** menu is **ON** 📌 the dashboard will be displayed on the first screen when accessing the bot.

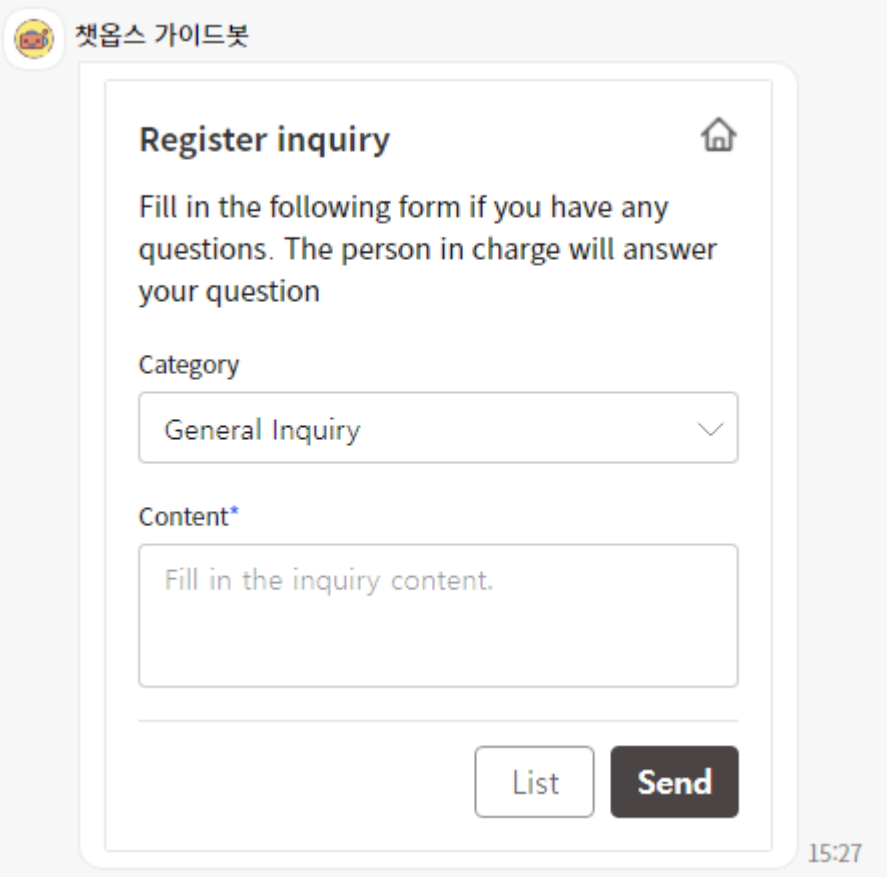


번호	항목	설명
❶	**Settings**	Click to move to the bot settings.
❷	**Refresh**	Click to reload my information in the dashboard.
❸	**Today's New Inquiries**	Shows the number of new feedback for the day. Click to move to **Feedback** > **Feedback hisotry**. Today's new inquiries are not displayed if the feedback feature is OFF.
❹	**Today's Incomplete Inquiries**	Shows the number of feedback inquiries for the day that have not been responded to. Click to move to **Feedback** > **Feedback history**. Today's incomplete inquiries are not displayed if the feedback feature is OFF.
❺	**Currently Ongoing Poll**	Shows the number of ongoing polls for the day. Click to move to

번호	항목	설명
		Poll > **Poll Schedule**.
❻	**Polls that expired today**	Shows the number of polls expired for the day. Click to move to **Poll** > **Poll Schedule**.
❼	**Monitoring unanswered polls today**	Shows the number of unanswered inquiries for the day from the bot. Click to move to **Statistics** > **Log**.
❽	**Scenario**	Shows the figures of the top three utterances entered the most. Shows the figures of the top three scenarios searched the most.
❾	**Feedback**	Feedback inquiries over the last three days are displayed from the latest. However, if there are seven items of feedback or more, the latest seven will be displayed. You can turn the feedback feature ON/OFF with the toggle button.
❿	**Notification Schedule**	Shows the notification schedule for the last three days.
⓫	**Poll Schedule**	Shows the poll schedule for the last three days.
⓬	**Bot Usage** **: Number of users**	Shows the number of monthly bot users during the set period.
⓭	**Bot Usage** **: Content status**	Shows the number of response modes(%) of the bot during the set period. Chat is excluded in the pie chart as it combines click/search/rules/others.

2.
Managing Feedback

The Feedback feature allows users to register inquires and managers to leave responses.



2.1 Using feedback

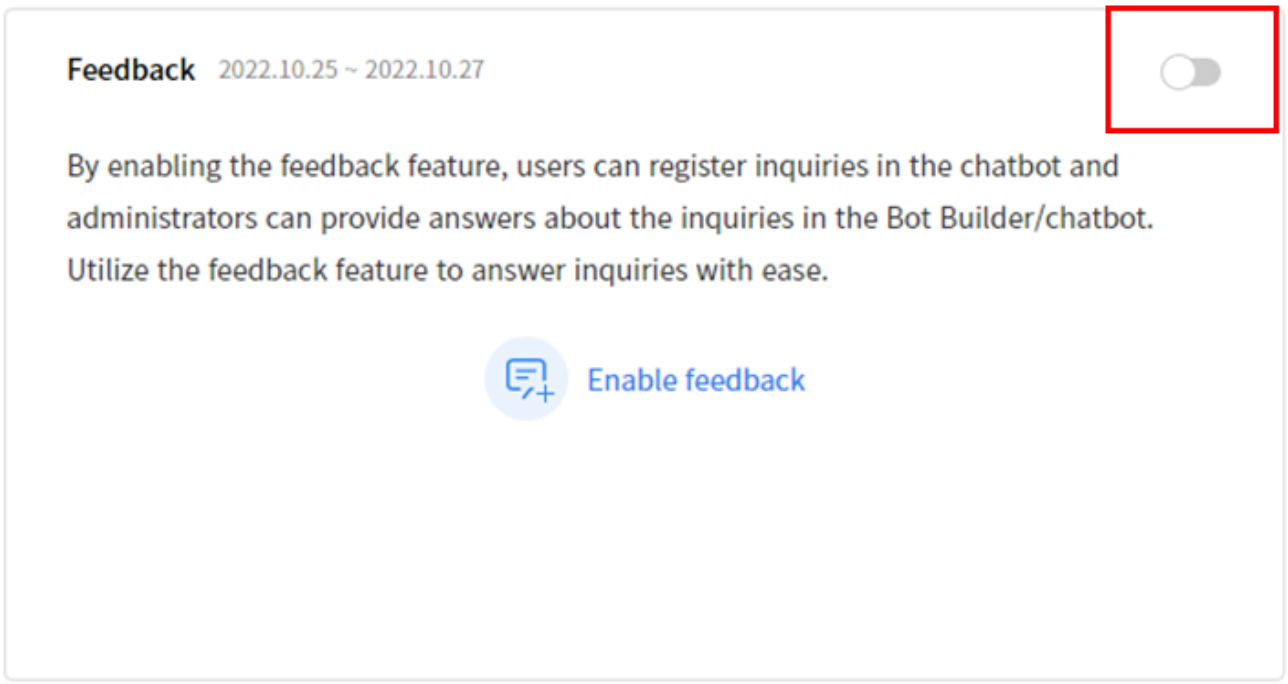
To use the feedback feature, complete the following steps:

1. Click the **Bot** tab on top of the AI Builder home screen.

2. Select a bot from the bot list on the **Bot** screen.

3. Click **Dashboard** on the bot details screen in the left.

4. Click the toggle button on the right of the feedback card in the “Dashboard” to the **ON** position.



5. You can see that the **Feedback** menu has been added on the LNB menu.

2.2 Setting a Feedback App Card

You can customize feedback-related app cards where bot users register inquiries and check responses from the managers via messenger in AI Builder. For feedback app card settings, complete the following steps:

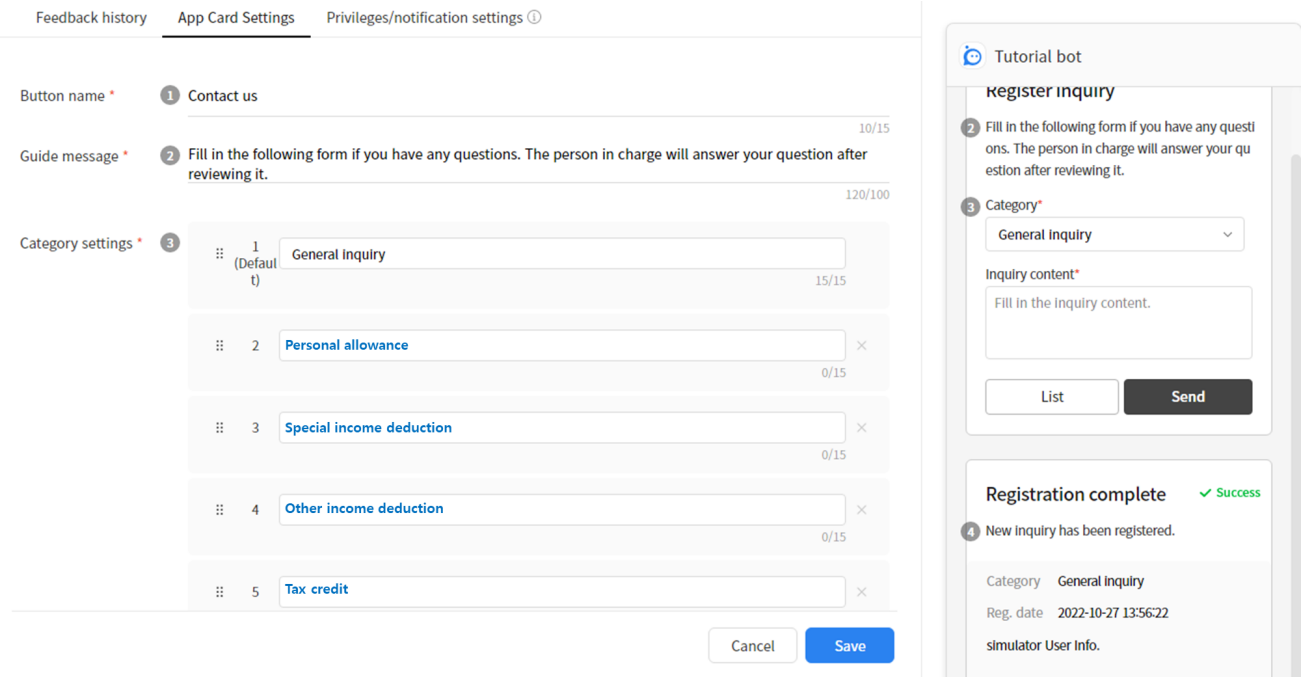
1. Click **Feedback** > **Feedback Management** > **App Card Settings** tab on the bot detail screen.

2. **App Card Settings** provides default values of the button name, guide message, registration success message, etc. You can change the default settings if needed.

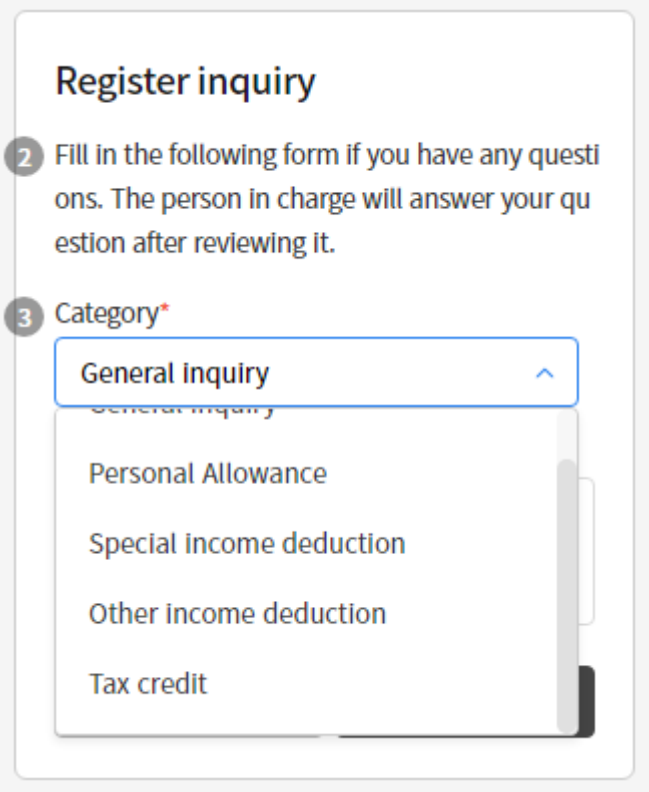
> **Note**
> It would be convenient for you to check the simulator on the right while changing the settings to see how the information would be displayed on the messenger.

E.g., Configure the basic information for the year-end tax adjustment as follows.

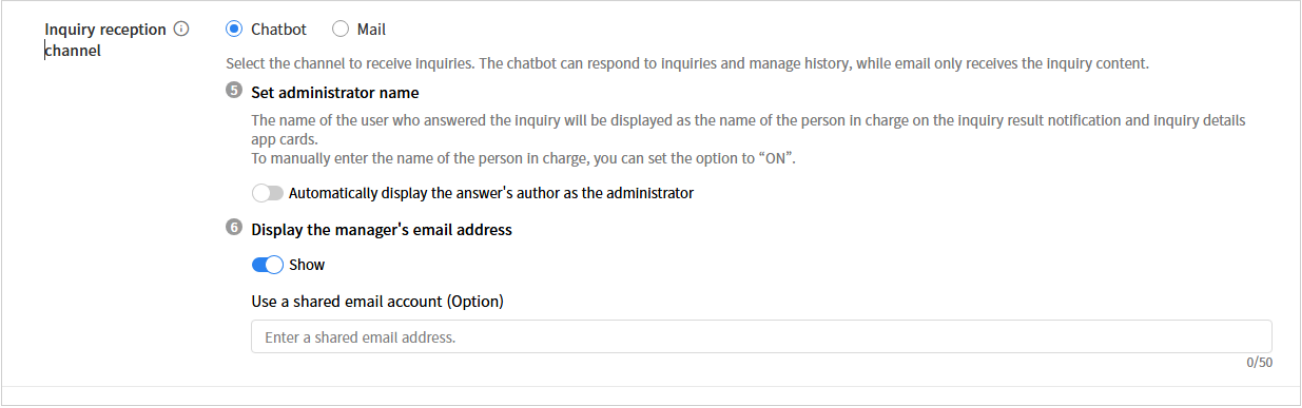
- **Category Settings** : General inquiry, personal allowance, special income deduction, other income deduction, tax credit



If you check the simulator on the right, you can see that you can choose from the categories you've set when registering an inquiry on the messenger chatbot as below.



3. On **App Card Settings** screen, select the **Inquiry reception channel**.



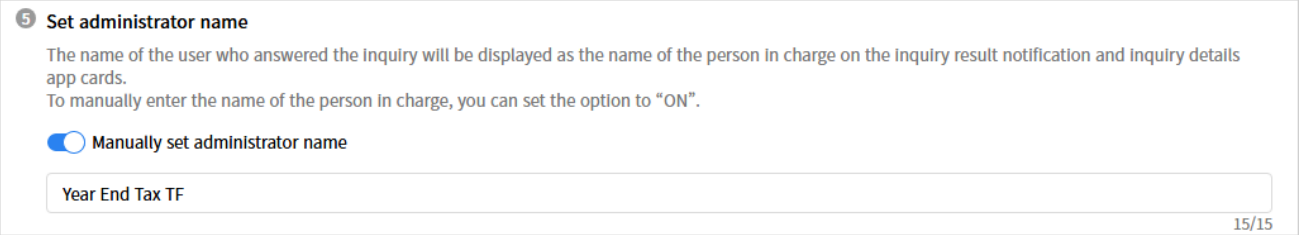
You can choose to send inquiries via chatbot or email.

- Receiving via chatbot: Administrators can respond to inquiries using AI Builder (or chatbot) and continuously manage follow-up questions.
- Receiving via email: Administrators receive inquiries through the registered email addresses. In this case, AI Builder does not manage responses or follow-up questions. It is recommended to reply to inquiries through a registered email accounts, such as a personal email.

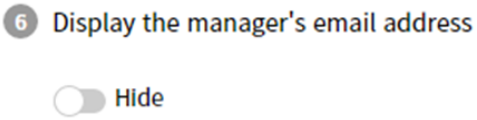
The method for configuring the app card is as follows, depending on the selected receiving channel.

3.1. Configure receiving via the chatbot channel

- Set administrator information: Toggle **ON** (manually set administrator name: Year End Tax TF)



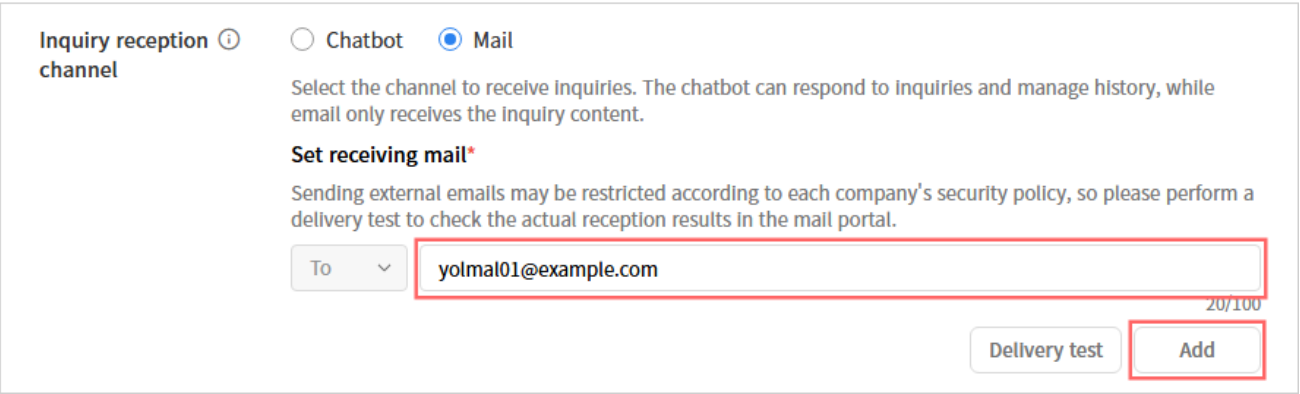
- **Display the manager's email address**: Set the toggle to **OFF** to not expose the administrator's email address.



Click **Save** after entering all information on the "App Card Settings" screen. The information will be instantly reflected on the messenger without separate distribution when you save it.

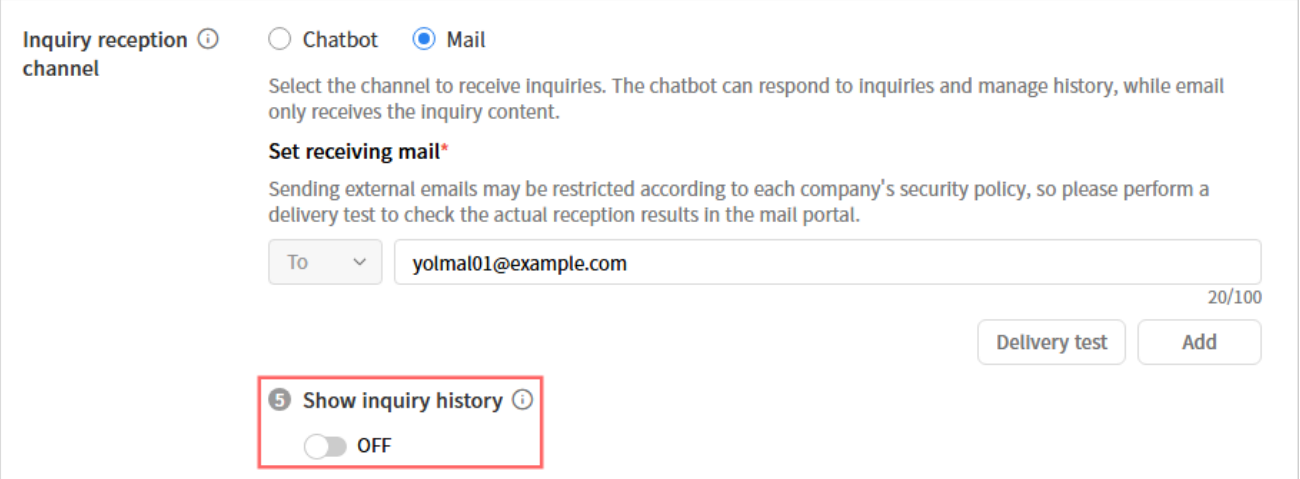
3.2. Configure receiving via the mail channel

- **Receiving mail settings:** The **"To"** (recipient) field must contain at least one address. Depending on the operational situation, additional recipients can be added using **"Cc"** or **"Bcc"** types besides "To".



> Tip
> You can perform a **"Delivery test"** to verify in advance that the inquiry is received correctly in the mail portal.

- **Show inquiry history**
  - **Toggle OFF:** Inquiry history is not shown in the messenger. This history can only be viewed in the **AI Builder** > **Feedback** > **Feedback History** tab.
  - **Toggle ON:** Inquiry history is visible both in AI Builder and Messenger.



Click **Save** after entering all information on the “App Card Settings” screen. The information will be instantly reflected on the messenger without separate distribution when you save it.

2.3 Setting Feedback Privileges/notification

You can set managers by feedback category. Only the authorized manager can view inquiries or send responses to the specified category.

> Note
> **Privileges/notification settings** is available only when the **Inquiry reception channel** is set to **Chatbot**. If inquiries are received via "Mail", this menu is disabled.

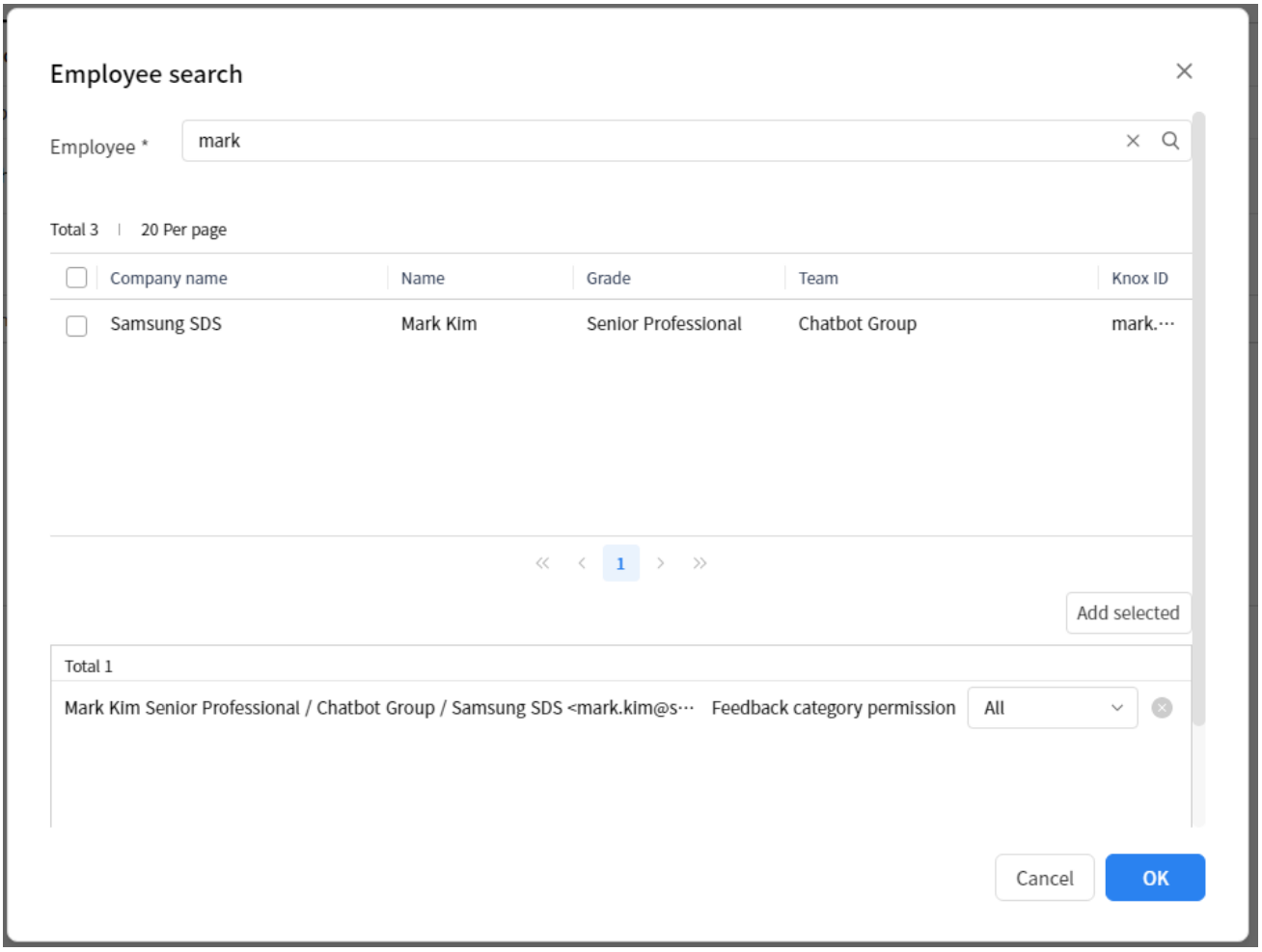
To register a manager, complete the following steps:

1. Click **Feedback** > **Feedback Management** > **Privileges/notification settings** on the bot details

screen.

2. Click **Add** button on **Privileges/notification settings** screen.

3. On **Employee search** popup, search employees, select feedback categories to give authorization, then click **OK**.

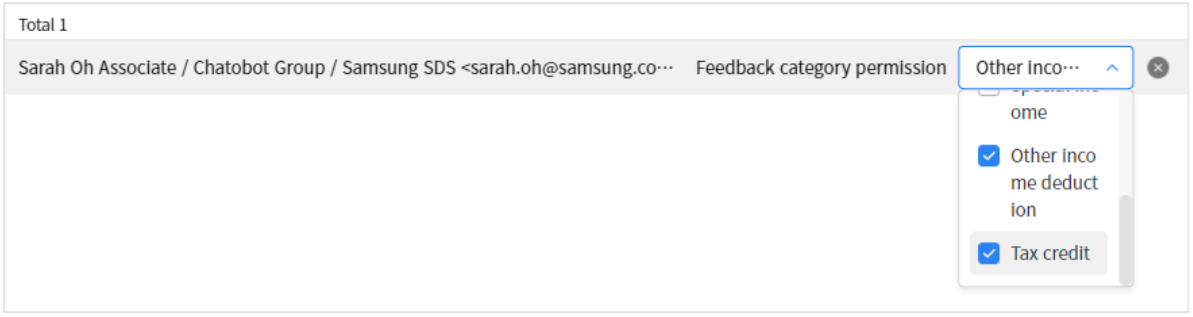


E.g., Configure feedback authority for the year-end tax adjustment as follows.

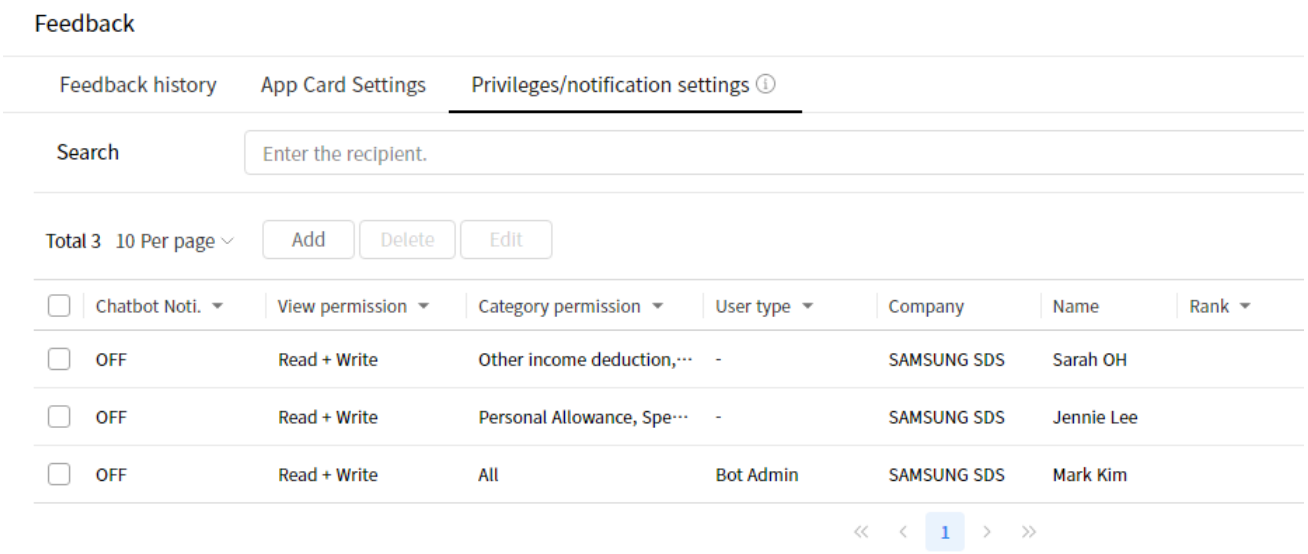
- Mark Kim: All

- Jennie Lee: Personal allowance, special income deduction

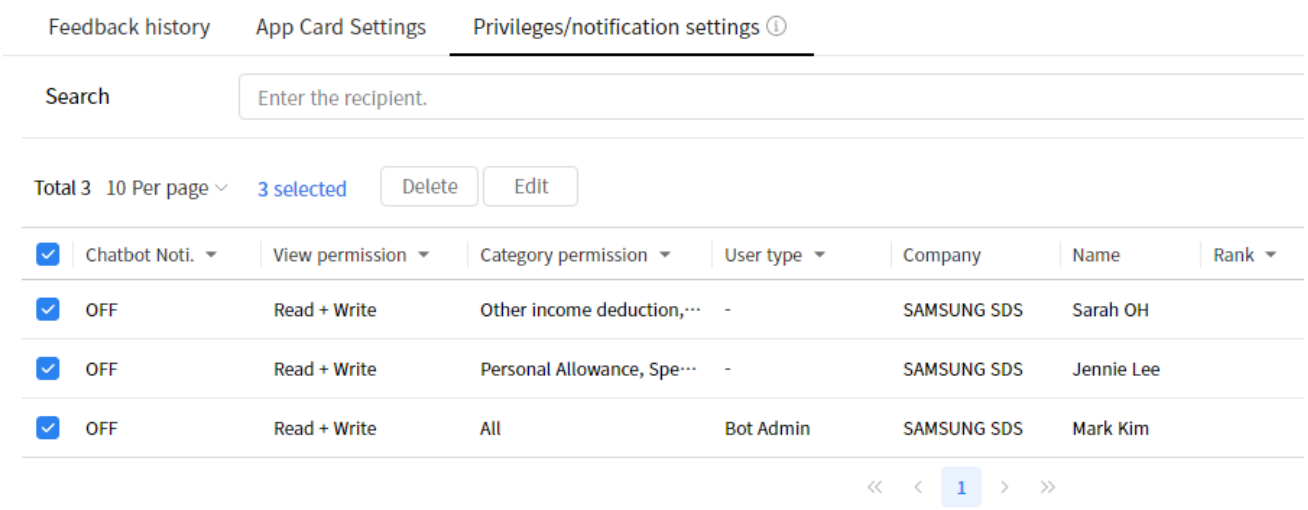
- Sarah Oh: Other income deduction, tax credit



You can check that feedback managers are registered in the list on **Privileges/notification setting** screen as below.

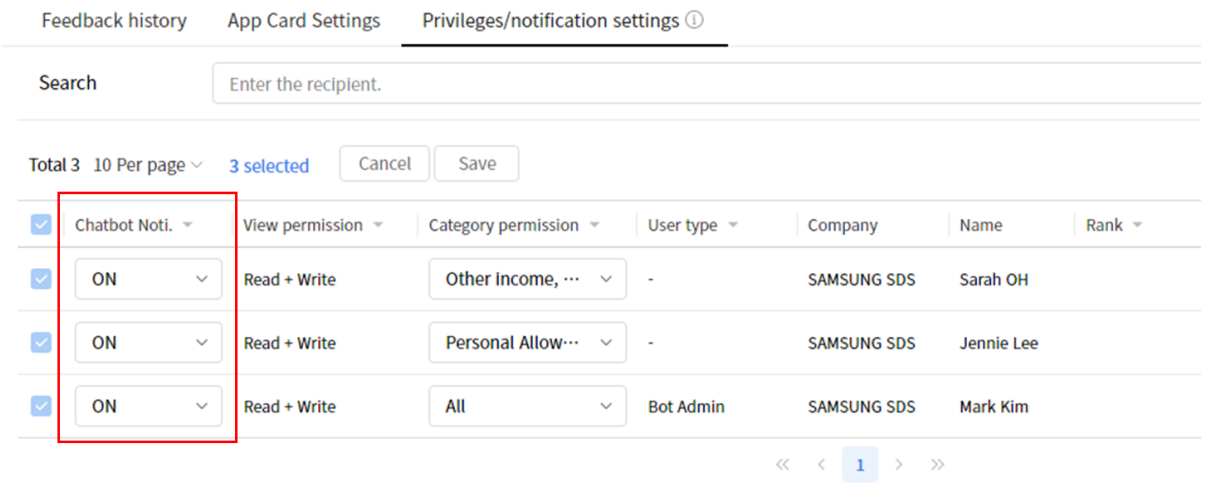


4. Select the managers to modify from the **Privileges/notification setting** list then click **Edit** button.



5. Change the **chatbot notification** settings to **ON**, then click **Save**.

If it is set to ON, inquiries about authorized categories will be sent to the manager's chatbot once registered, allowing the manager to enter and send a response via messenger.



Note

. **Chatbot notification**: To get an inquiry notification via chatbot when an user inquiry is registered

. **View permission**: To set authorization to read or read + write user inquiries

. **Category permission**: To receive chatbot notifications, review the history, or write responses for user inquiries in authorized categories

E.g., If you don't have authorization for "General inquiry," you cannot view user inquiries that come as "General inquiry".

Note

Employees registered as a **Guest** in the bot only have "read" authorization. They cannot write or send responses for user inquiries.

Feedback managers can receive notifications on user inquiries, write and send responses on the messenger as below.

2.4 Checking the Feedback History in AI Builder

The use of feedback history varies depending on whether the inquiry reception channel is set as chatbot or mail.

4.1. When the inquiry reception channel is set as chatbot

You can check the user inquiry (feedback) history, write, and send responses in AI Builder.
You can only view the authorized feedback categories.

E.g., Year-end Tax Adjustment

- Mark Kim: Allowed to view all user inquiries (general inquiry, personal allowance, special income deduction, other income deduction, and tax credit)

- Jennie Lee: Allowed to view user inquiries on personal allowance and special income deduction

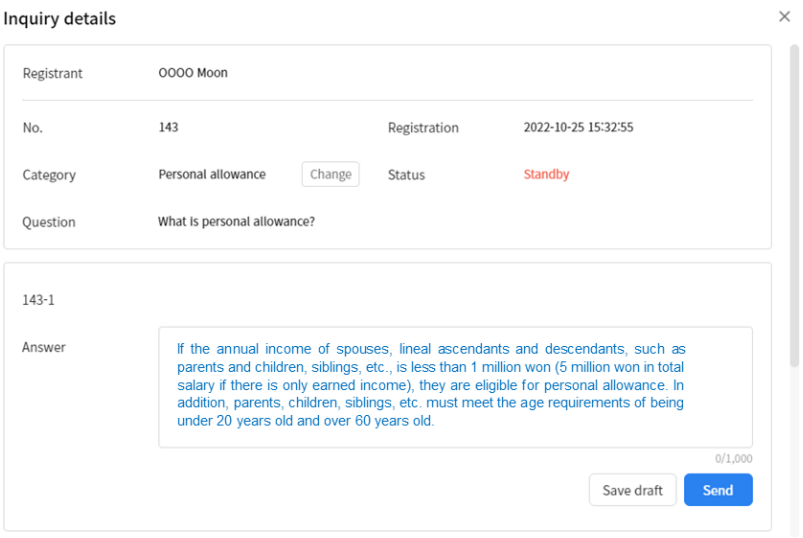
- Sarah Oh: Allowed to view user inquiries on other income deduction and tax credit

Complete the following steps to check the feedback history and send responses.

1. Click **Feedback** > **Feedback Management** > **Feedback history** tab on the bot details screen.

2. Click a feedback in **Standby** status. The Standby status means there is no responses.

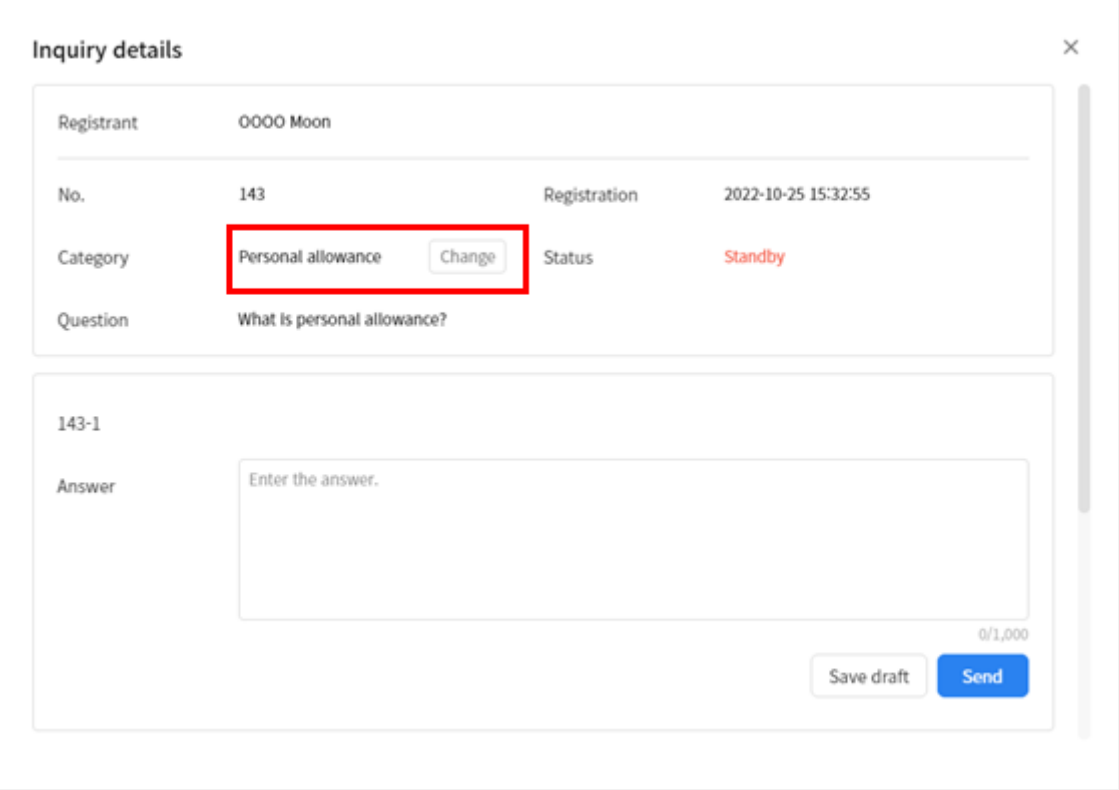
3. Check the inquiry in the **Inquiry details**, write an **Answer**, then click **Send**.



Note

Q : What should I do when an inquiry is sent in incorrect category? For example, an user sent an inquiry on “other income deduction” in the “personal allowance” category. The manager for each category is different and cannot respond correctly, how can I resolve that?

A: In such cases, you can change the **Category** in the **Inquiry details** pop-up window. If you change the category and **Save**, the personal allowance manager will receive a notification on the messenger chatbot and can view the history in AI Builder.



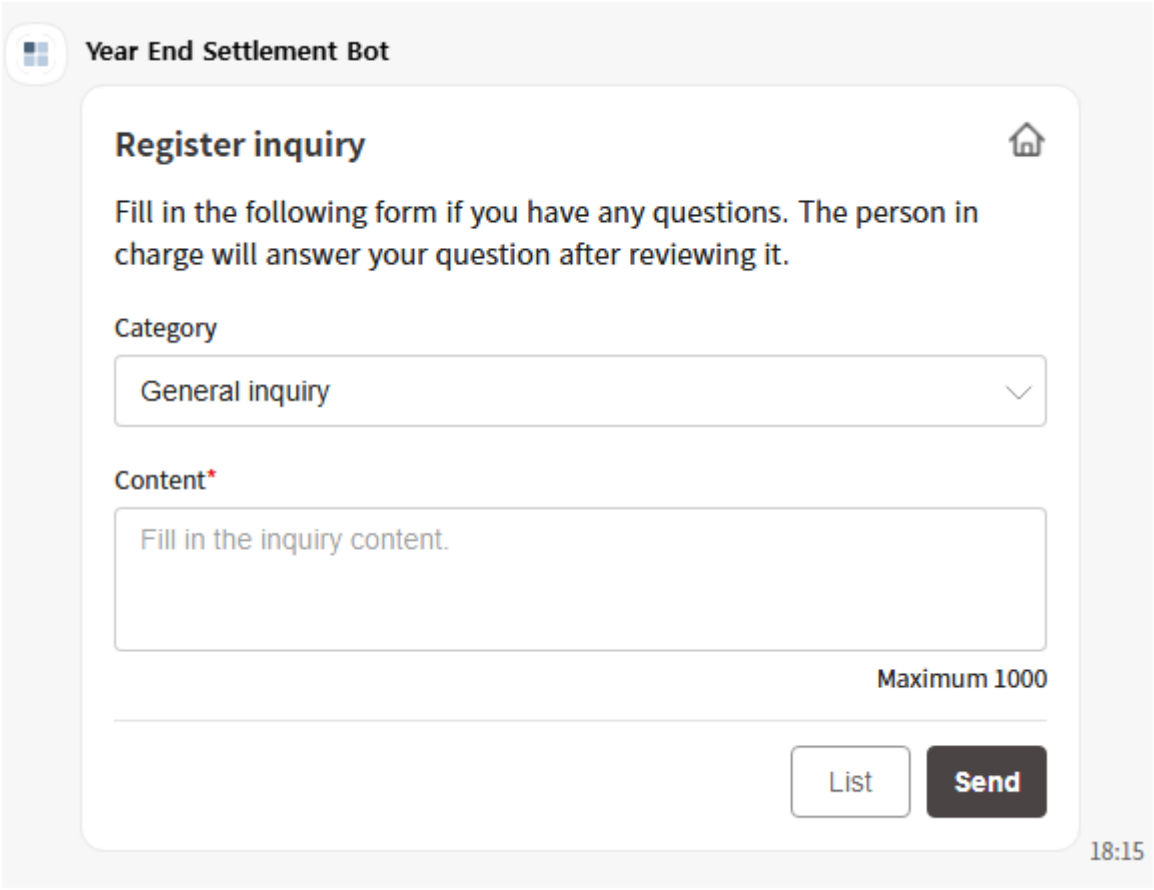
4.2. When the inquiry reception channel is set as mail

In AI Builder, bot admins can only view user inquiry (feedback) history. **Responses to inquiries or related follow-up questions** are not managed.

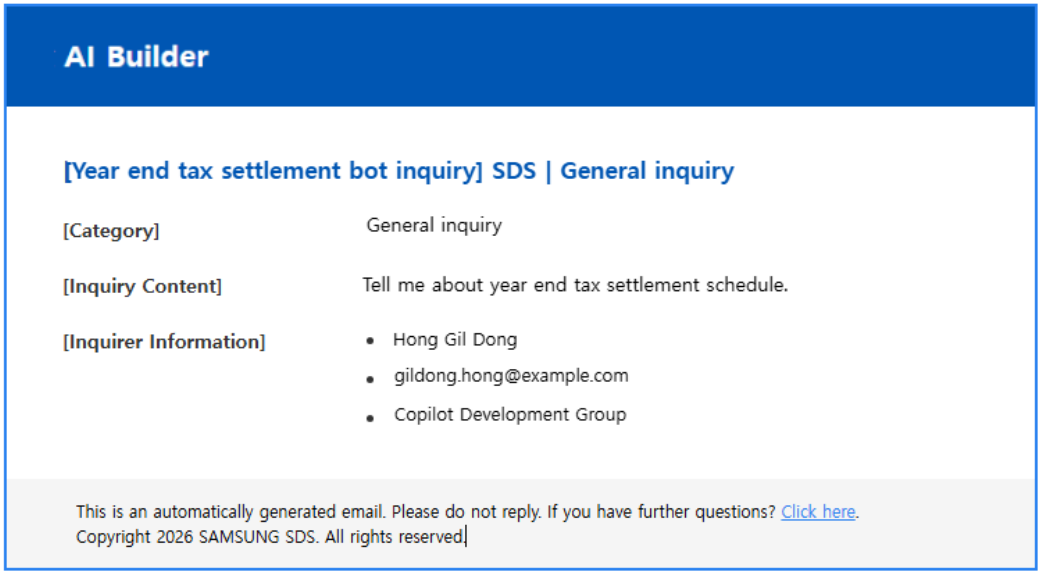
Note

"Mail sent" or **"Mail failed"** status indicates the result of AI Builder's email sending request, which may differ from the actual delivery status. Check the actual delivery result in the **Mail**

portal.

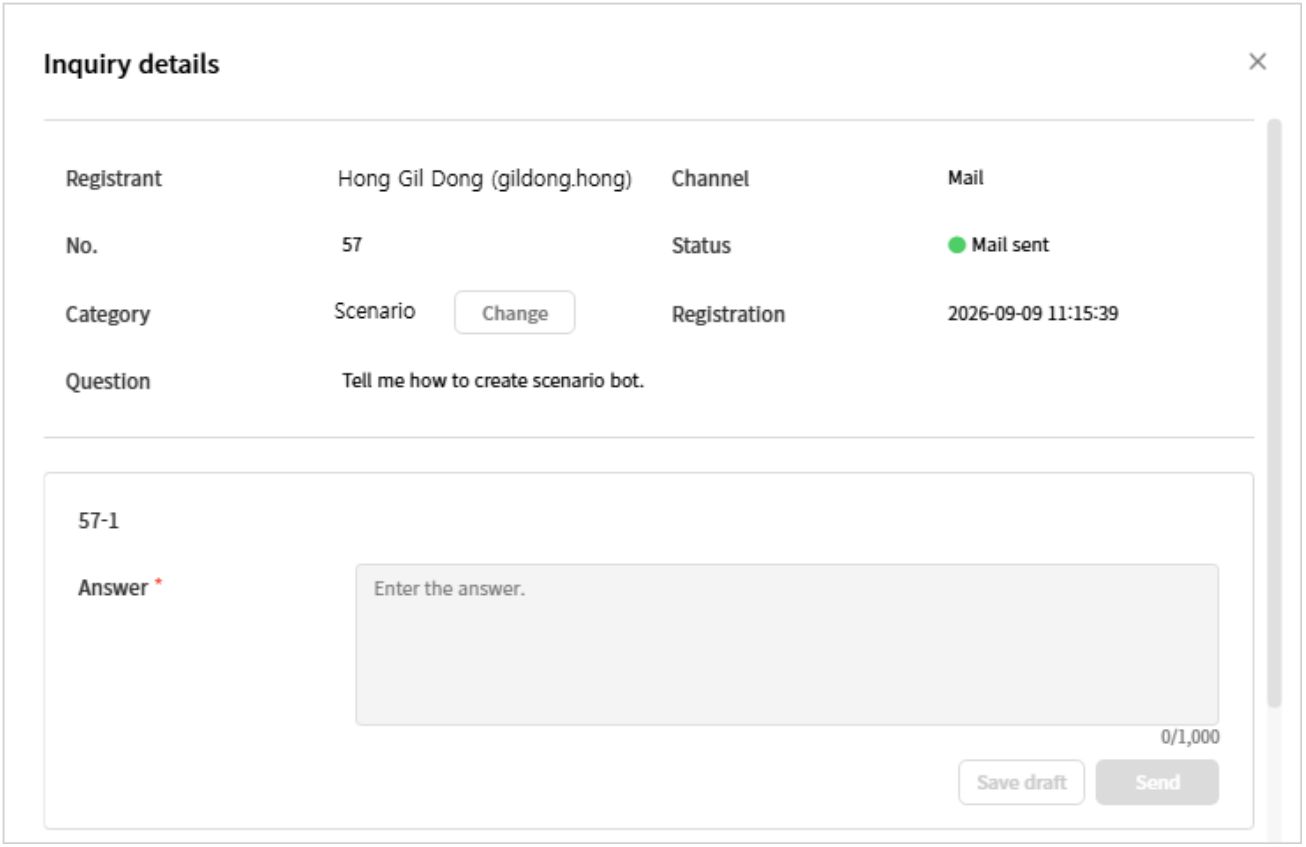


When a user registers an inquiry in the messenger, an email will be sent to the configured recipient address as follows.



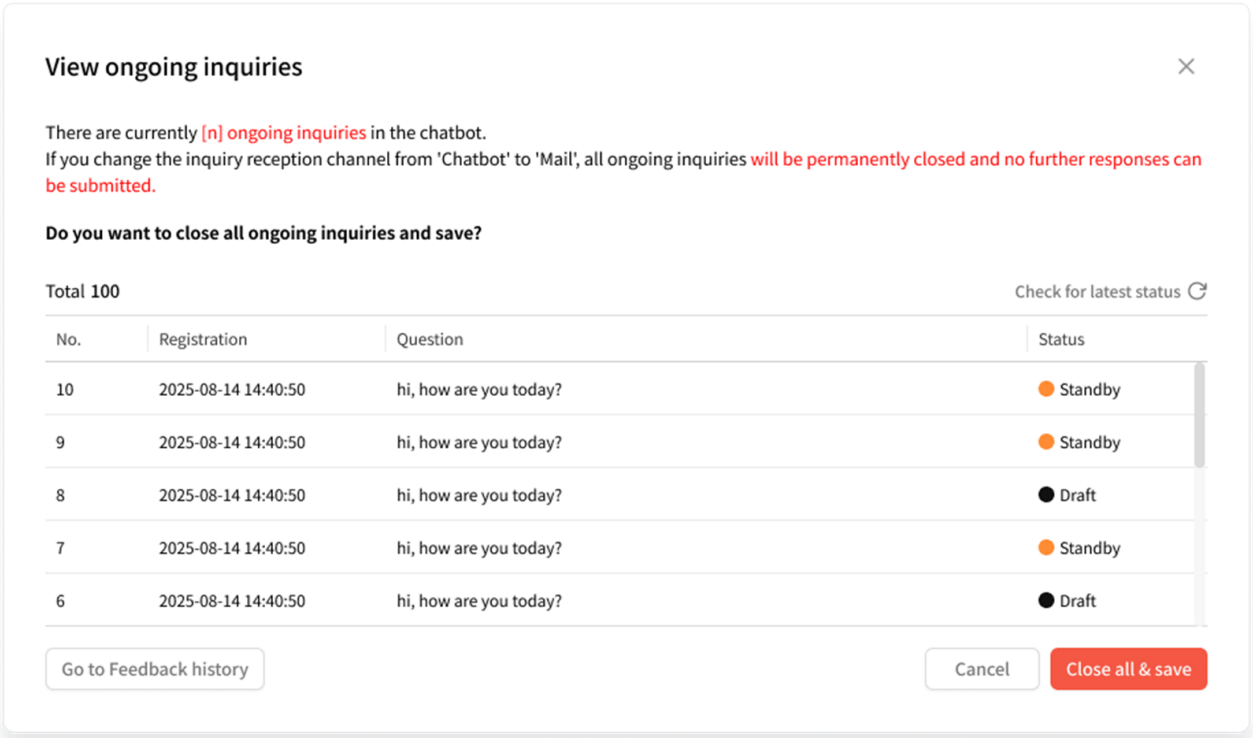
Note

In **Inquiry details** on AI Builder, you can only view inquiries.



Note

- When the inquiry reception channel is changed to email, it is no longer possible to add responses to inquiries previously received via chatbot. Therefore, it is recommended to **review all pending inquiry history in advance and complete them before process the change**.
- If processing is unnecessary, you can choose to **close all pending inquiries** when saving the **inquiry reception channel** change in the **App Card Settings** menu.


View ongoing inquiries
There are currently [n] ongoing inquiries in the chatbot.
If you change the inquiry reception channel from 'Chatbot' to 'Mail', all ongoing inquiries will be permanently closed and no further responses can be submitted.
Do you want to close all ongoing inquiries and save?
Total 100
Check for latest status
No.
Registration
Question
Status
10
2025-08-14 14:40:50
hi, how are you today?
Standby
9
2025-08-14 14:40:50
hi, how are you today?
Standby
8
2025-08-14 14:40:50
hi, how are you today?
Draft
7
2025-08-14 14:40:50
hi, how are you today?
Standby
6
2025-08-14 14:40:50
hi, how are you today?
Draft
Go to Feedback history
Cancel
Close all & save

3. Monitoring usage statistics

You can view the statistics data by chatbot.
On the "User status" screen, you can view the chat status for the period you have selected from the cumulated chat data since the creation of your chatbots.
On the "Logs" screen, you can view the usage logs by period, which include account information of users who used chatbots, user intent, speech, and chatbot responses.
On the "Content status" screen, you can check the top ten chart and figures of the most searched scenarios and utterances.

This chapter explains the following topics:

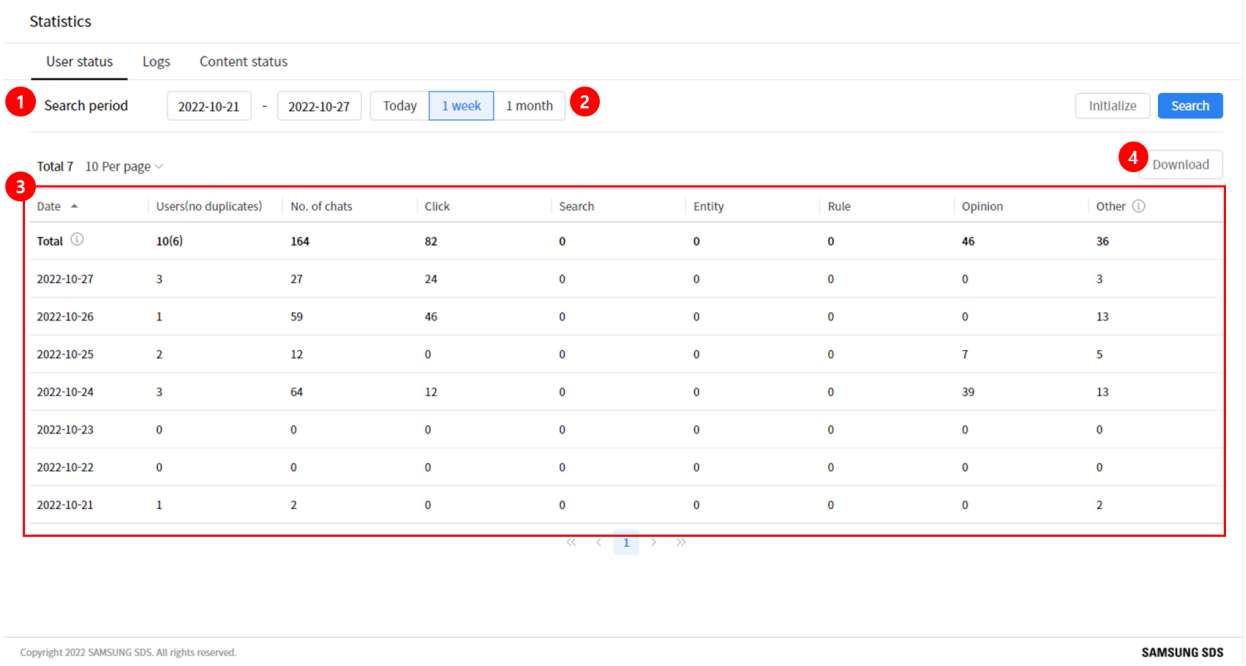
- Viewing user status
- Viewing usage logs
- Viewing content status

To use the usage status monitoring feature, complete the following steps:

1. At the top of the screen, click the **Bot** tab.
2. On the bot list of the "Bot" screen, select a bot.
3. In the menu on the left side of the bot details screen, click **Statistics**.

3.1 Viewing user status

1. At the top of the "User status" screen, set the search period and click **Search**. You can view and analyze the user status, such as the number of users and the number of chats in chatbot.



No.	Item	Description
❶	Search period (calendar)	You can set a search period on the calendar. You can set a search period up to 3 months.
❷	Search period (range)	You can select today, 1 week, or 1 month as the search period instead of setting a period on the calendar.
❸	View user status	View the usage statistics by number of users, chats, clicks, searches, opinions, and other conditions.
❹	**Download**	Download the searched data as a csv file.

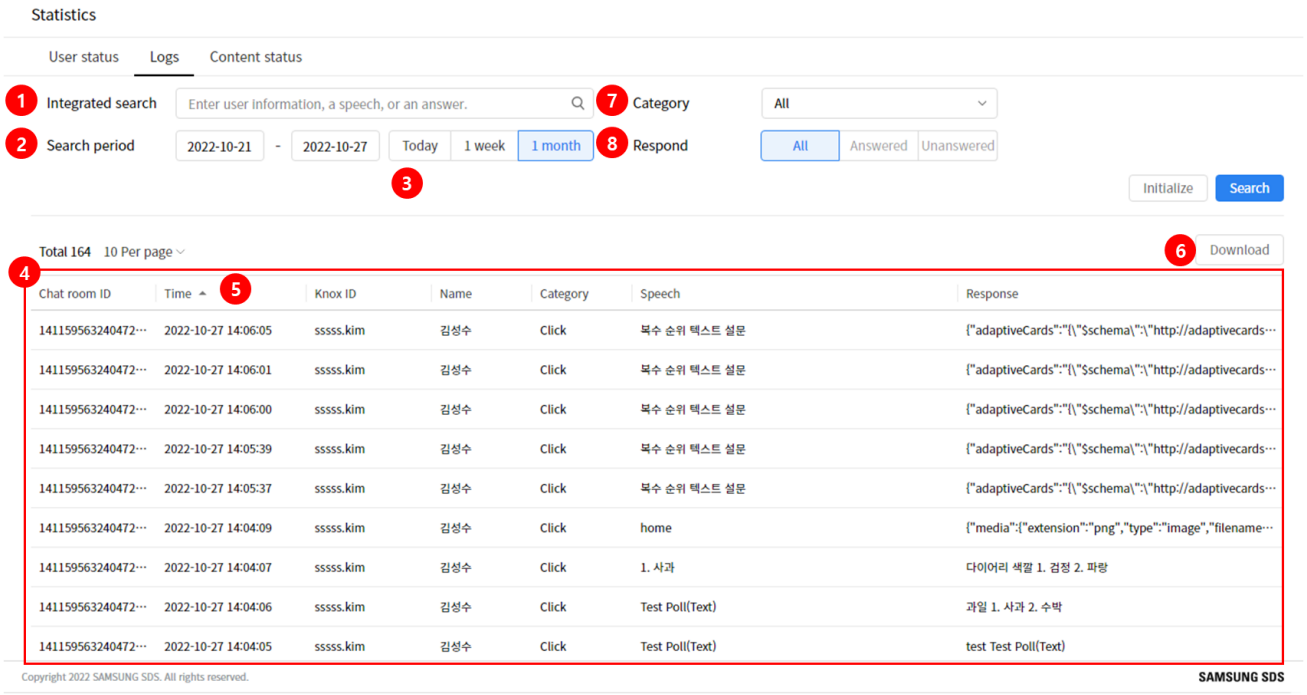
3.2 Viewing usage logs

View the usage logs by chatbot, for the period you have specified. You can view detailed information about the use date, user account, user speech, user intent, and the execution result.

To view the usage logs, complete the following steps:

1. Click the **Log** tab on the top.
2. Set the search conditions (keyword, category, search period, response status) in the search area at the top of the "Log" screen, then click **Search**.

- You can click **Initialize** to initialize previously set search conditions.



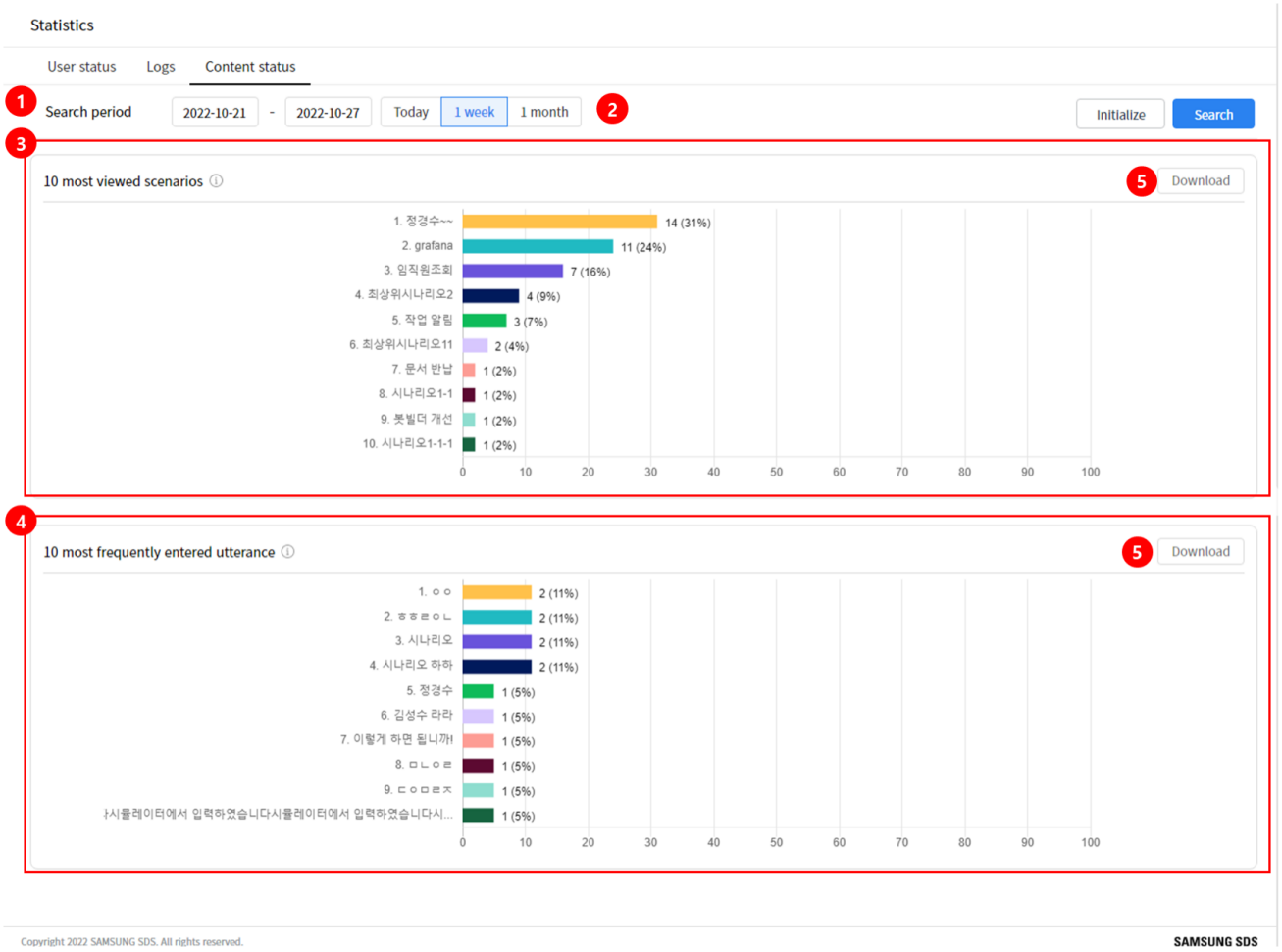
No.	Item	Description
❶	**Integrated search**	Search logs by entering user information, user speech, or the response.
❷	Search period (calendar)	You can set a search period on the calendar. You can set a search period up to 3 months.
❸	Search period (range)	You can select today, 1 week, or 1 month as the search period instead of setting a period on the calendar.
❹	View logs	View logs analyzed by the **Date, Knox ID, Name, Category, Speech, and Response** items.
❺	Sort	Sort the logs on the list in ascending or descending order.
❻	**Download**	Download the searched logs as a csv file.
❼	Category	Select intent category, such as, click, search, rule, and other intent, and perform a search.
❽	Respond	You can search by bot response by choosing All/Answered/Unanswered.

3.3 Viewing content status

On content status, you can check the figures of the top 10 most viewed scenarios viewed and the top 10 most frequently entered utterances. You can download the Excel file by clicking the Download button.

To view the content status, complete the following steps:

1. In the menu on the left side of the bot details screen, click **Statistics**. You can check and analyze the current status and analysis of the top 10 most viewed scenarios and top 10 most entered utterances .



번호	항목	설명
❶	**Search period (calendar)**	You can set a search period on the calendar. You can set a search period up to 3 months.
❷	**Search period**	You can select today, 1 week, or 1 month as the search period instead

번호	항목	설명
	(range)	of setting a period on the calendar.
❸	**Top 10 most viewed scenarios**	Show the chart and figures for the most viewed scenario.
❹	**Top 10 frequently entered utterances**	Show the chart and figures for the most frequently entered utterances.
❺	**Download**	Download the figures of the searched chart as a Excel file.

4. Chatbot authority management

Users with chatbot authority can use the chatbot in Knox Teams.
You can manage these chatbot authority by adding or deleting them in AI Builder.

Go to the Authority management menu to manage authority.
1. Click the **Bot** tab at the top the screen.
2. Select the bot from the list of bots from the "**Bot**".
3. Go to the **Setting** menu of the bot details screen.
4. Click the **Chatbot authority** tab in Setting.

Note
Chatbot authority can be managed in AI Builder as below :

- Individual users (for all users)
- Department-level authority within the bot manager's company

Company/Regional office's authority can be managed through the **Service desk**.

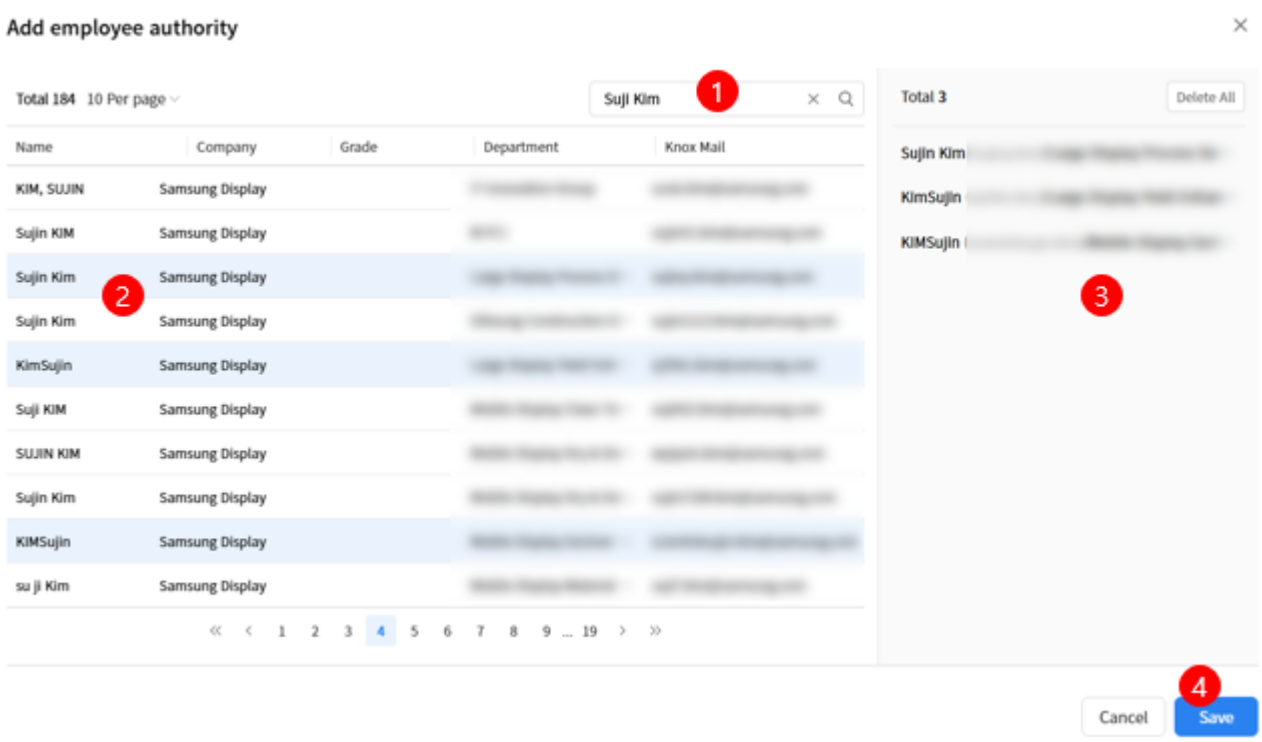
4.1 Managing user authority

Add user authority

To add user authority, follow these steps on the Chatbot authority :

1. Authority : Select **User** .

2. Click + Add authority .

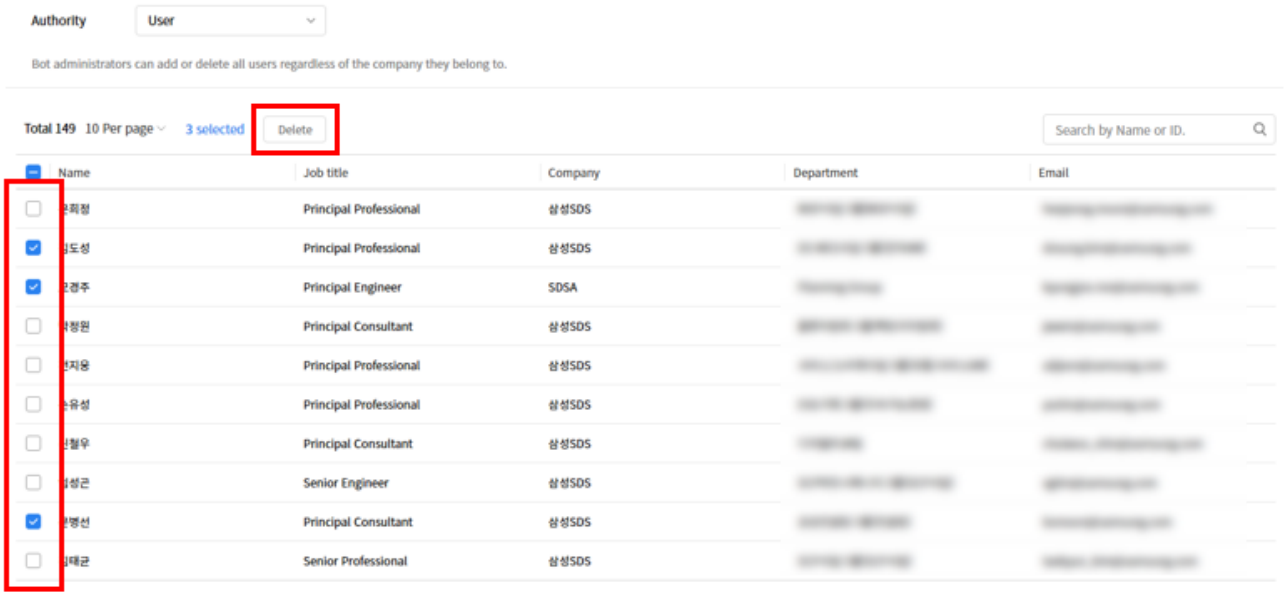


No	Item	Description
❶	**Employee search input box**	Search by entering the name or Knox ID of the employee.
❷	**Employee search results**	The search results will be displayed, and you can click on each row to select the employee you want to add authorities to.
❸	**Selected list**	The selected employees in step ② are displayed.
❹	**Save**	Add chatbot authorities to selected employees.

Delete user authority

To delete user authority, follow these steps on the Chatbot authority :

1. Authority : Select **User** .
2. If necessary, search by name or Knox ID.
3. Select the **checkbox** you want to delete and click the **Delete** button.



> Tip
> You can add or delete of all users' authorities regardless of your company.

4.2 Managing department authority

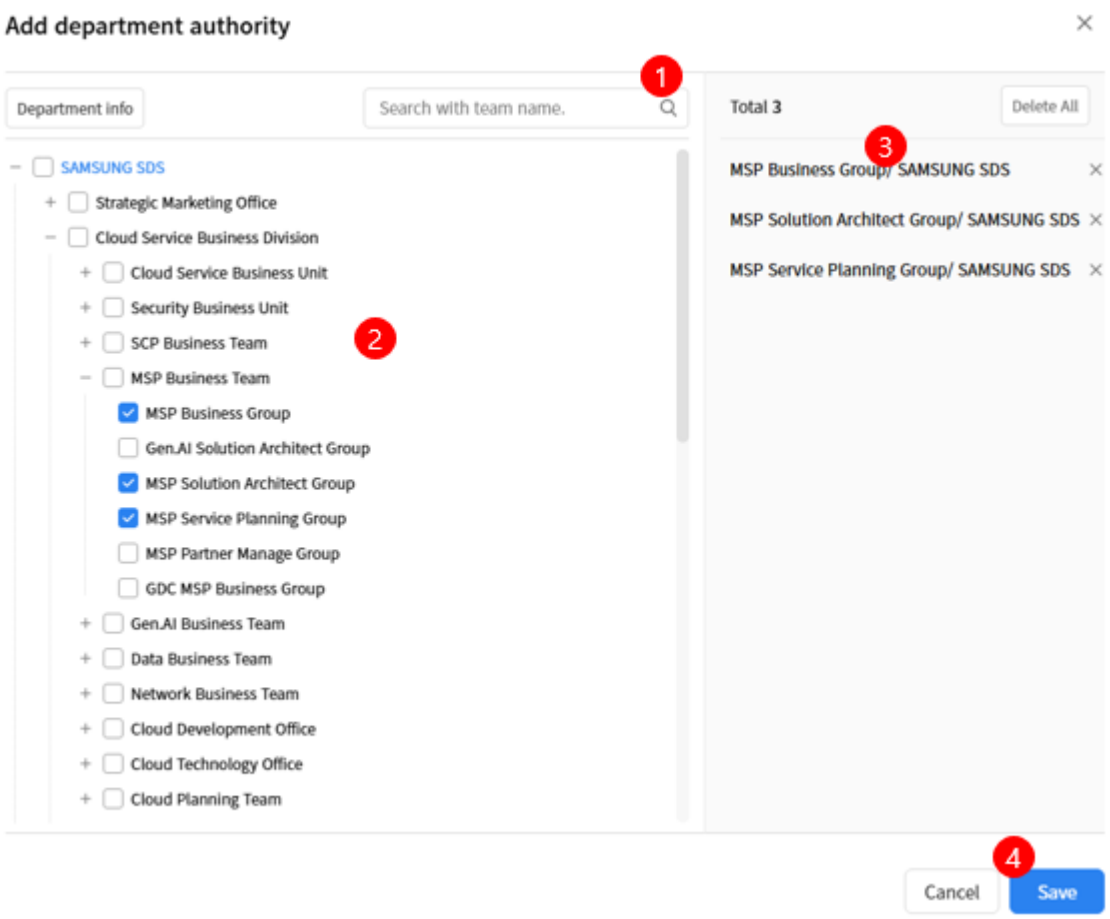
Add department authority

To add department authority, follow these steps on the Chatbot authority :

1. Authority : Select **Company** .
2. Select your company from the **company list**.

3. Click the Departments button to view the department authority list.

4. Click ＋ Add authority .

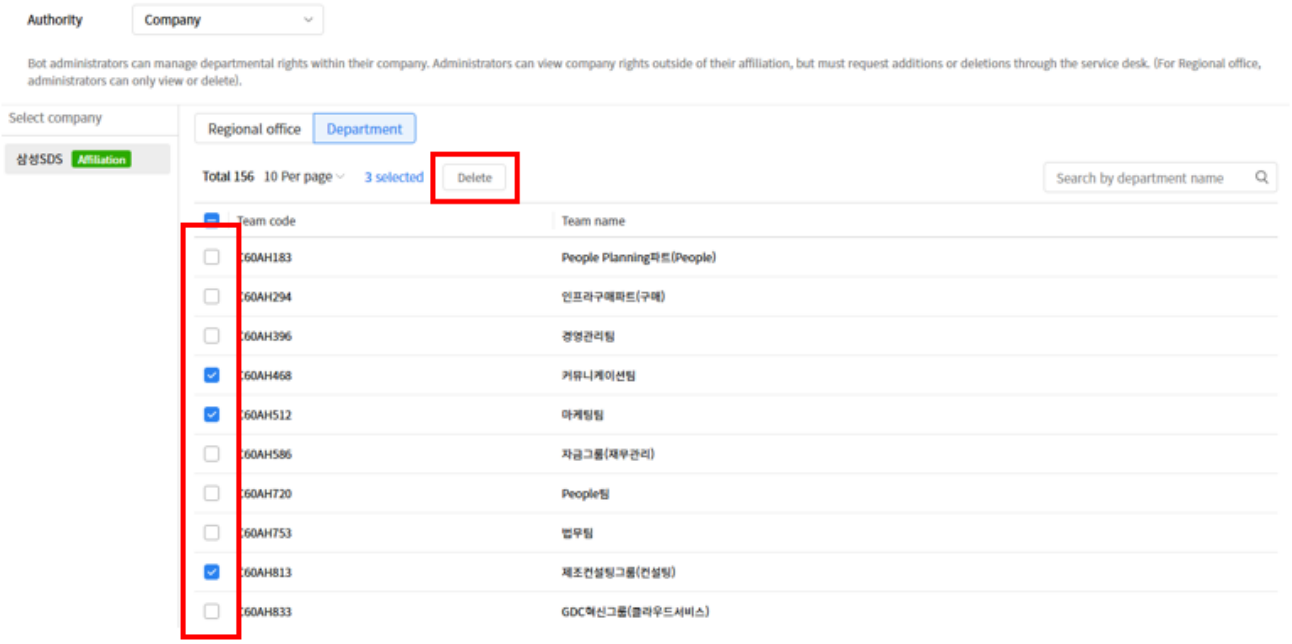


번호	항목	설명
❶	**Department search input box**	Search by entering department's name you want to search for.
❷	**Organization Chart**	Your company's organizational chart will be displayed, and search results will be displayed when you search by name. Select the departments you want to add authorities to by checking.
❸	**Selected list**	The selected departments in step ② are displayed.
❹	**Save**	Add chatbot authorities to selected departments.

Delete department authority

To delete department authority, follow these steps on the Chatbot authority :

1. Authority : Select **Company** .
2. If necessary, search by department's name.
3. Select the **checkbox** you want to delete and click the **Delete** button.



> Tip
>
> You can add or delete department's authorities only for your own company.
> You can only view other companies' authorities, and please contact the Service desk if you need to add or delete the others.

Part VII.
ChatOps bot

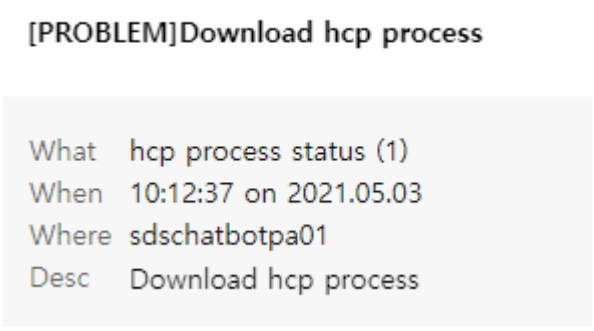
You can create ChatOps bot that include versatile business operation features, such as the FAQ, event notification, system status monitoring, and remote operation features, to enhance business efficiency.

This chapter explains the following topics:

- Receiving Zabbix event notifications
- Configuring Grafana monitoring features
- Remotely operating a Jenkins system

1. Receiving Zabbix event notifications

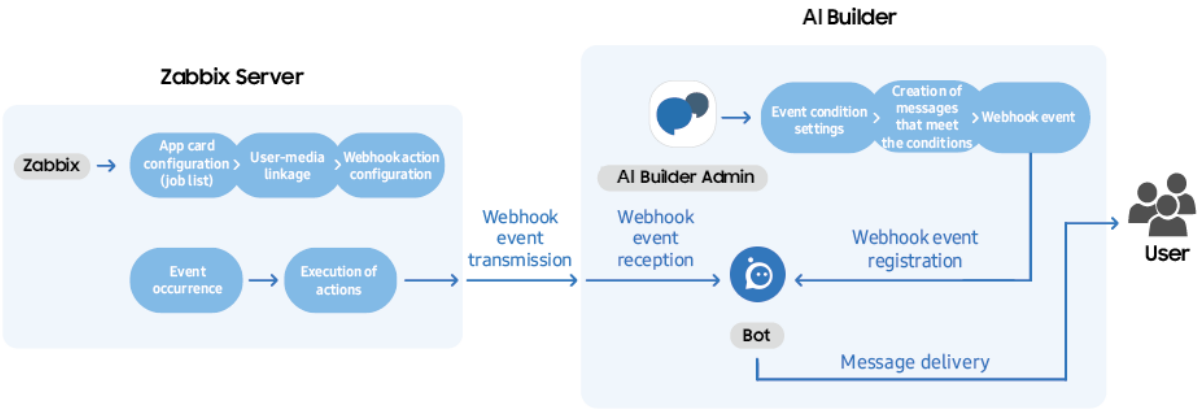
You can use the Zabbix webhook feature to configure the ChatOps bot to receive notifications when events occur.
This tutorial explains how to integrate Knox Teams and a Zabbix system to allow users to view the event details on app cards in the Knox Teams.



> Note
> The Zabbix webhook feature is available only in Zabbix 4.4 or later.

1.1 Zabbix event reception flow

The following explains how events occurred in Zabbix are delivered to a ChatOps bot.



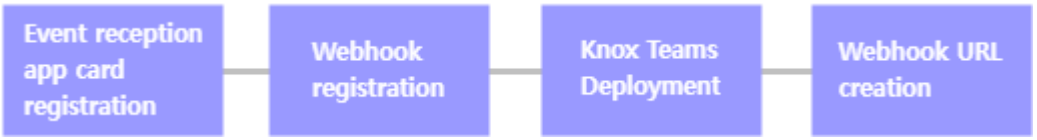
1.2 Disabling the firewall before operation

To utilize the webhook feature in AI Builder, you must first disable the firewall of the system that will be intergrated.
Disable the firewall for the following range.

Category	Source	Destination		
		URL	IP	Port
Webhook integration	External	proxy-aibuilder.samsung.net	112.107.147.103	80, 443
	Internal	aibuilder.samsung.net	182.195.83.147	80, 443

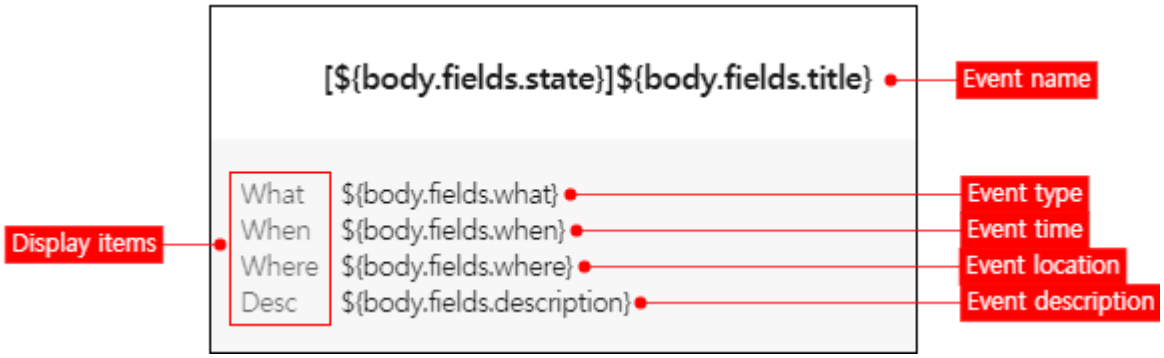
1.3 Configuring in AI Builder

Check the information provided in the tutorial according to the following stages, and configure in AI Builder to recognize Zabbix events and deliver relevant messages to a bot.



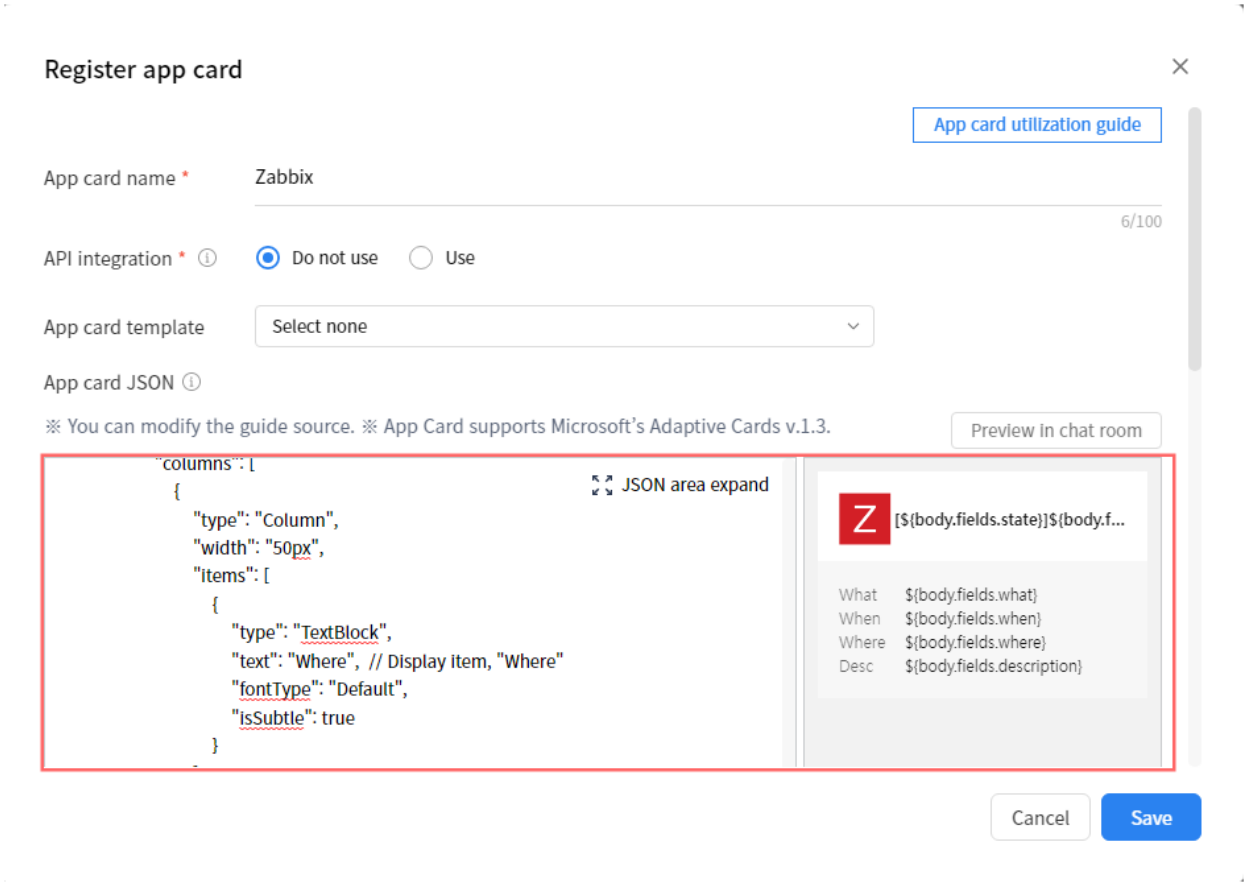
Registering an event reception app card

You can create and register an app card according to the following tutorial to create an app card that can receive events generated in Zabbix system.



To create an event reception app card, complete the following steps:

1. At the top of the AI Builder home screen, click the **Bot** tab.
2. On the bot list of the "Bot" screen, click a bot that the app card will be created for.
3. On the bot details screen, click **App card** > **App cards** > **Register**.
4. In the "Register app card" pop-up window, enter the app card name.
5. Refer to the following JSON to fill in the **App card JSON** field and click **Save**.




```
{
    "type": "AdaptiveCard",
    "body": [
        {
            "type": "ColumnSet",
            "columns": [
                {
                    "type": "Column",
                    "items": [
                        {
                            "type": "Image",
                            "url":
"https://aibuilder.britymail.com/images/chatops/zabbix-icon.png",   //
System image
                            "size": "Medium",
                            "spacing": "None",
                            "horizontalAlignment": "Center"
                        }
                    ],
```

```
                    "width": "auto"
                },
                {
                    "type": "Column",
                    "items": [
                        {
                            "type": "TextBlock",
                            "text":
"[${body.fields.state}]${body.fields.title}",  // Event name
                            "fontType": "Default",
                            "size": "Medium",
                            "weight": "Bolder"
                        }
                    ],
                    "width": "stretch",
                    "spacing": "Small",
                    "height": "stretch",
                    "verticalContentAlignment": "Center",
                    "style": "default"
                }
            ]
        },
        {
            "type": "Container",
            "items": [
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "50px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "What",  // Display item,
"What"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
```

```
                            },
                            {
                                "type": "Column",
                                "width": "stretch",
                                "items": [
                                    {
                                        "type": "TextBlock",
                                        "text": "${body.fields.what}",  // Event type
                                        "spacing": "None",
                                        "fontType": "Default",
                                        "wrap": true,
                                        "maxLines": 3
                                    }
                                ],
                                "spacing": "None"
                            }
                        ],
                        "spacing": "None"
                    },
                    {
                        "type": "ColumnSet",
                        "columns": [
                            {
                                "type": "Column",
                                "width": "50px",
                                "items": [
                                    {
                                        "type": "TextBlock",
                                        "text": "When",  // Display item, "When"
                                        "fontType": "Default",
                                        "isSubtle": true
                                    }
                                ]
                            },
                            {
                                "type": "Column",
                                "width": "stretch",
                                "items": [
```

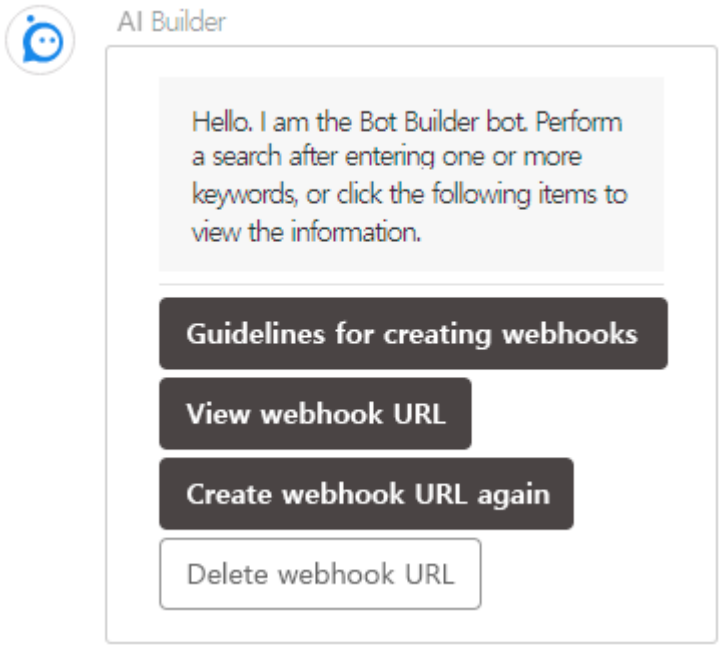
```
                                    {
                                        "type": "TextBlock",
                                        "text": "${body.fields.when}",  // Time of the event
                                        "spacing": "None",
                                        "fontType": "Default",
                                        "wrap": true,
                                        "maxLines": 3
                                    }
                                ],
                                "spacing": "None"
                            }
                        ],
                        "spacing": "None"
                    },
                    {
                        "type": "ColumnSet",
                        "columns": [
                            {
                                "type": "Column",
                                "width": "50px",
                                "items": [
                                    {
                                        "type": "TextBlock",
                                        "text": "Where",  // Display item, "Where"
                                        "fontType": "Default",
                                        "isSubtle": true
                                    }
                                ]
                            },
                            {
                                "type": "Column",
                                "width": "stretch",
                                "items": [
                                    {
                                        "type": "TextBlock",
                                        "text": "${body.fields.where}",  // Event location
                                        "spacing": "None",
```

```
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                },
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "50px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "Desc",  // Display item,
"Desc"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text":
"${body.fields.description}",  // Event description
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
```

```
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                }
            ],
            "spacing": "Large",
            "style": "emphasis",
            "bleed": true
        }
    ],
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3"
}
```

Registering webhooks

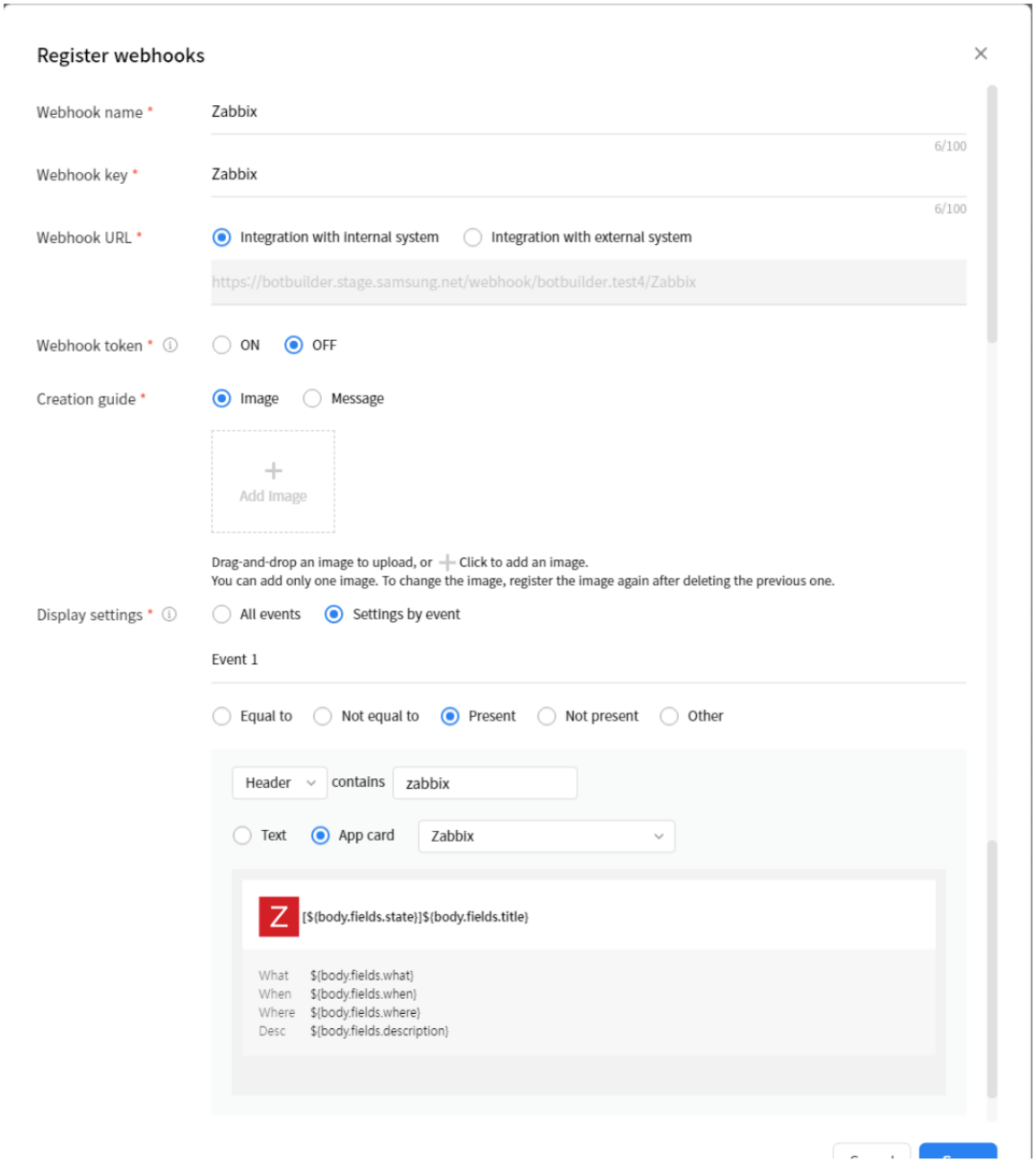
Register webhooks to display and manage webhooks in the Knox Teams as follows.



To register webhooks, complete the following steps:

1. On the bot details screen, click **Webhook** > **Webhooks** > **Register**.

2. In the "Register webhooks" pop-up window, enter the information and click **Save**.
   - You must fill in the mandatory fields indicated with asterisks (*).
   - Example of an event alarm setting
     - In the **Display settings** item, select **Settings by event** > **Present**.
     - Set the condition as “Header,” and “zabbix” is present.
     - Select **App card** and select and link with the app card you registered in Registering an event reception app card.



Deploying to the Knox Teams

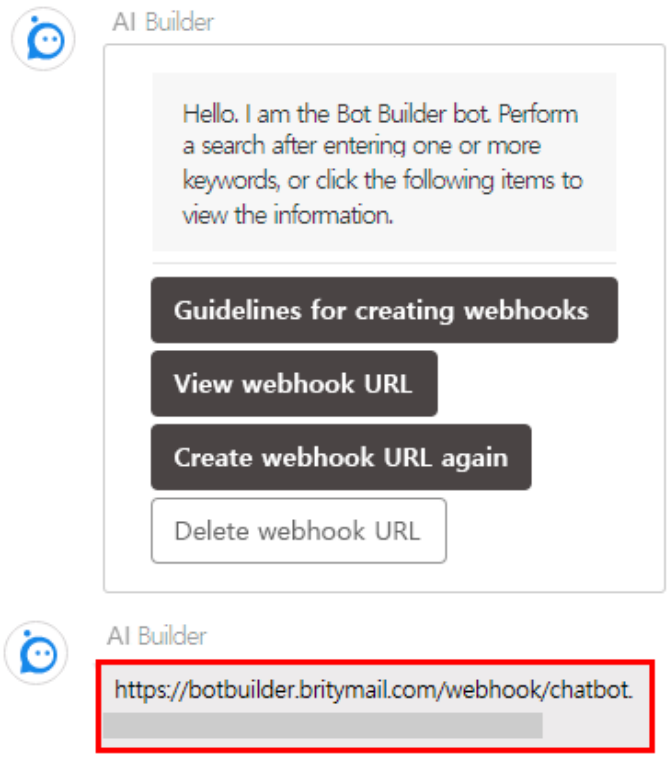
To reflect the webhooks and app cards that have been deployed and registered to the Knox Teams, complete the following steps:

1. In the top-left corner of the bot details screen, click **Deploy**.
2. In the "Deployment details" pop-up window, fill in the **Key revision history** field and click **Deploy**.

> Note
> To test the bot before deploying it to the Knox Teams, click "Test" in the bottom-right area of the "Scenario Management" screen.

Creating webhook URLs

You can create and view webhook URLs for integrating with a Zabbix system.



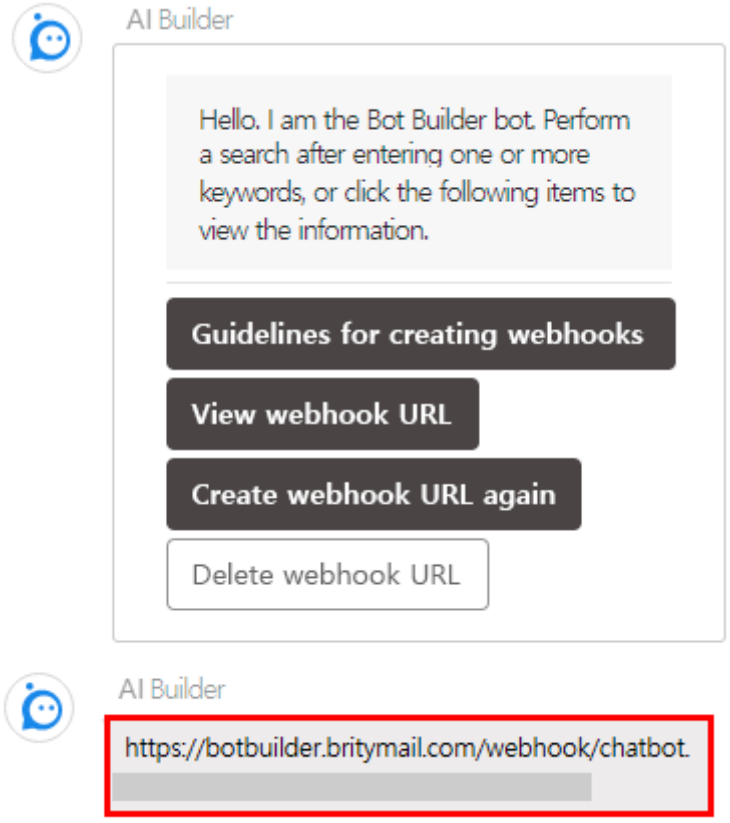
> **Note**
> Because the webhook URL will be linked with the currently open chat room, it is recommended that you create webhooks in a chat room with the administrator participants who are required to receive the event notifications.

To create and view a webhook URL, complete the following steps:

1. Access Knox T, search for the Knox bot ID of the bot where the webhook has been registered and deployed, and then double-click the Knox ID.
2. On the webhook that is displayed in the chat window after messaging ChatOps bot, click **View webhook URL**.

   A webhook URL for the current chat room will be created.

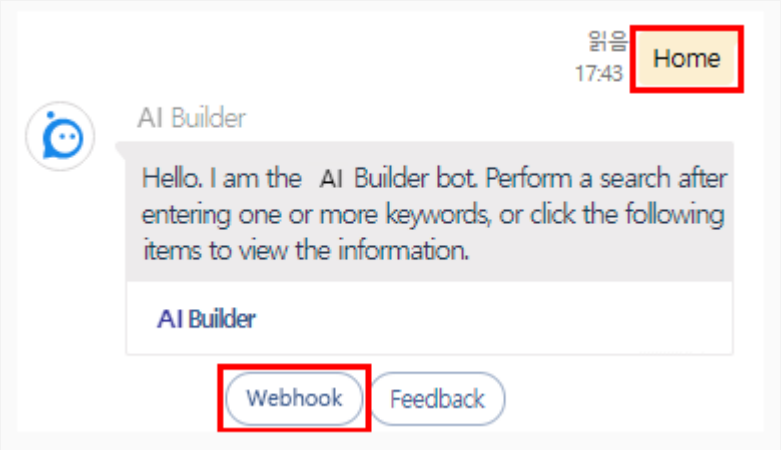
   - On the bot details screen, click **Webhook** > **Webhook URL Search**. Then, on the "Webhook URL Search" screen, you can view and manage the created webhook URL.



Note

If the bot has the “scenario + webhook” structure, **Webhook** will be displayed at the bottom of the chat window if you enter “Home” in the chat window.

You can click **Webhook** > **View webhook URL** to view the webhook URL.



1.4 Configuring the webhook in Zabbix

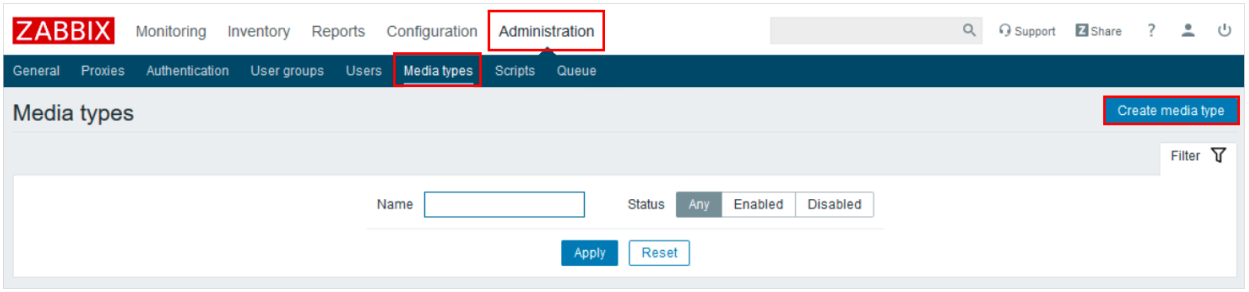
Check the tutorial about configuring a webhook to deliver Zabbix event notifications to the ChatOps bot according to the following stages and configure a webhook in Zabbix.



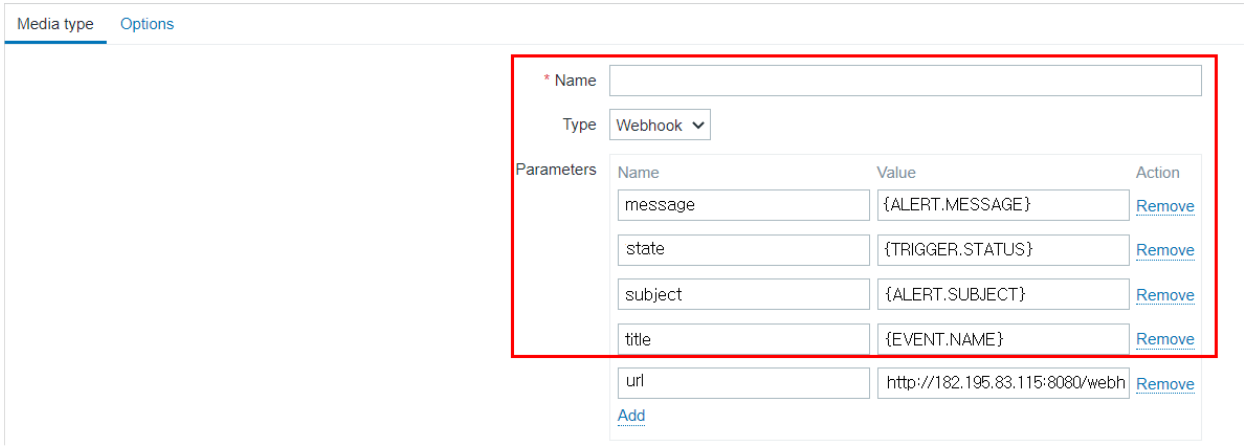
Creating media

To create media, complete the following steps:

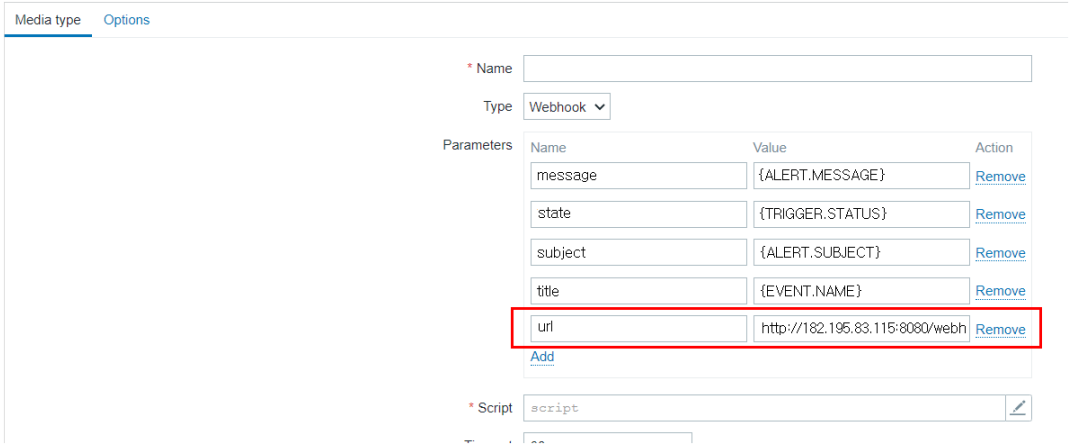
1. In the Zabbix system, click **Administration** > **Media types** > **Create media type**.



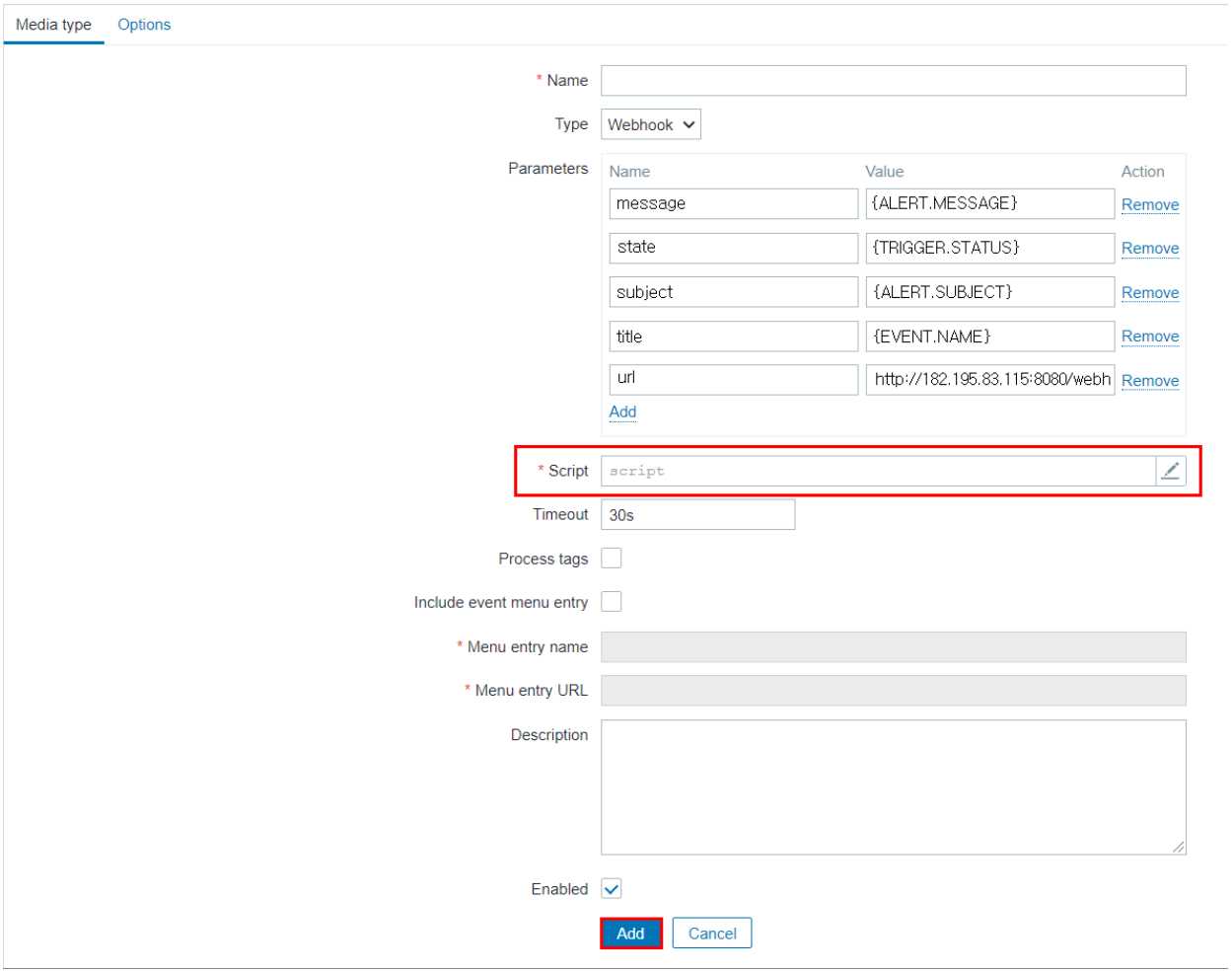
2. On the “Media type” input screen, enter the media name, set the type as **Webhook**, and then fill in the **Parameters** items referring to the following figure.



3. In the URL field, enter the unique webhook URL you created in Creating webhook URLs.



4. In the **Script** field, enter code referring to the following code block and click **Add**.



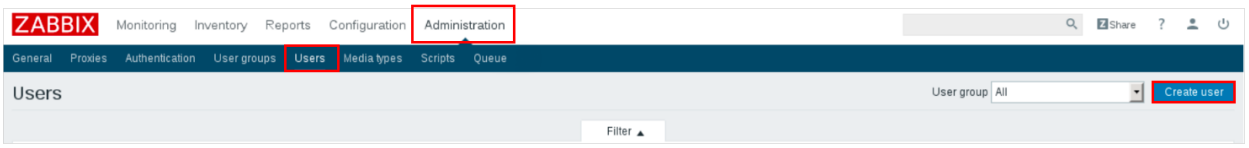

```
try {
  Zabbix.Log(4, &apos;Chatops webhook script value=&apos;+value);
  params = JSON.parse(value);
  req = new CurlHttpRequest();
  fields = {};
  req.AddHeader(&apos;Content-Type: application/json&apos;);
  req.AddHeader(&apos;zabbix: zabbix&apos;);
  fields.title = params.title;
  fields.subject = params.subject;
  fields.state = params.state;
  fields.message = JSON.parse(params.message);
  fields.what = fields.message.What
  fields.when = fields.message.When
  fields.where = fields.message.Where
  fields.severity = fields.message.Severity
```

```
    fields.description = fields.message.Description
    var resp = req.Post(params.url,
        JSON.stringify({"fields": fields})
    );
    if (req.Status() != 200) {
        throw JSON.parse(resp).errors[0];
    }
    resp = JSON.parse(resp);
    return "OK";
} catch (error) {
    Zabbix.Log(4, &apos;Chatops webhook failed : &apos;+error);
    throw &apos;Webhook failed : &apos;+error;
}
```

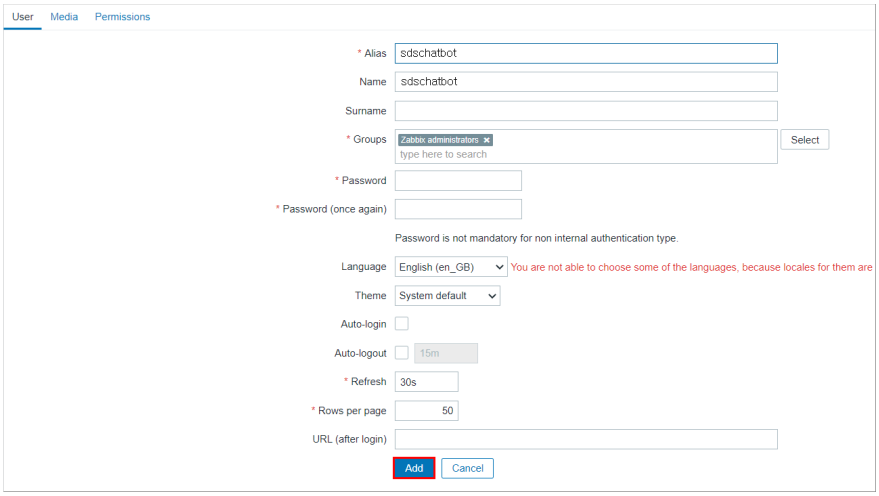
Creating users and linking with media

To create users that will access the webhook in the ChatOps bot and link the webhook with media, complete the following steps:

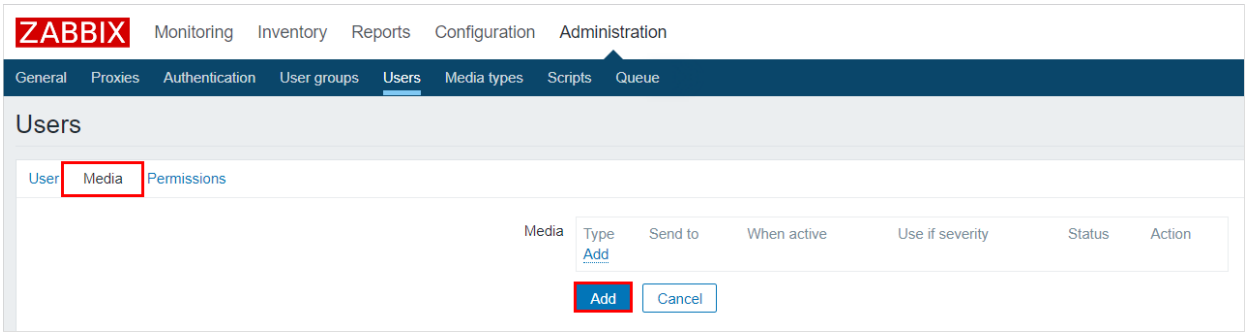
1. In the Zabbix system, click **Administration** > **Users** > **Create user**.



2. On the user registration screen, enter the registration information and click **Add**.



3. On the "Users" screen, click the **Media** tab. Then, click **Add** to add the media you created in Creating media.

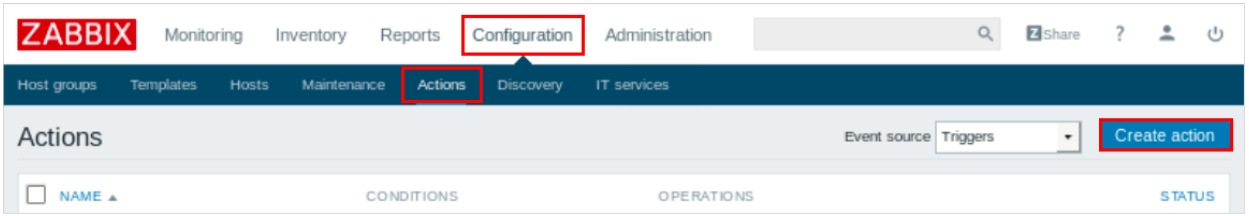


Creating actions

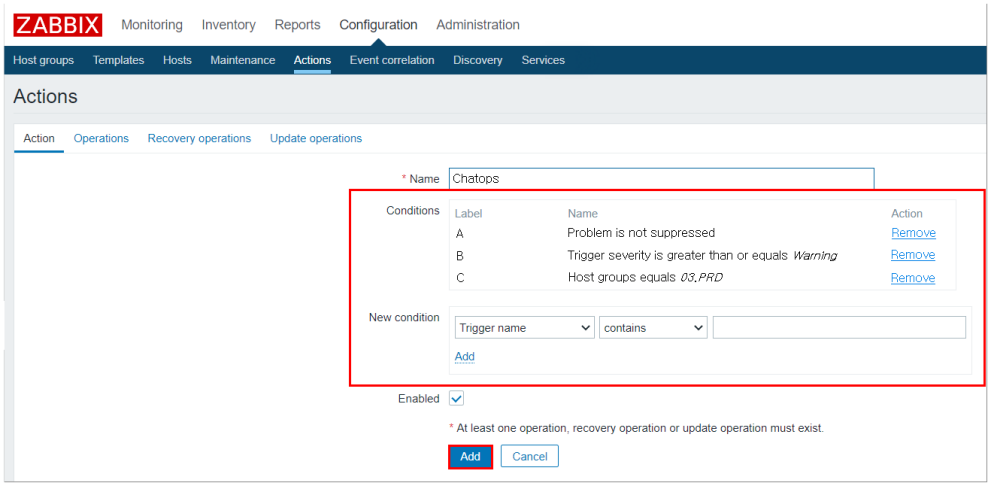
You can create actions to configure the conditions and messages for occurred events.

To create an action, complete the following steps:

1. In the Zabbix system, click **Configuration** > **Actions** > **Create action**.



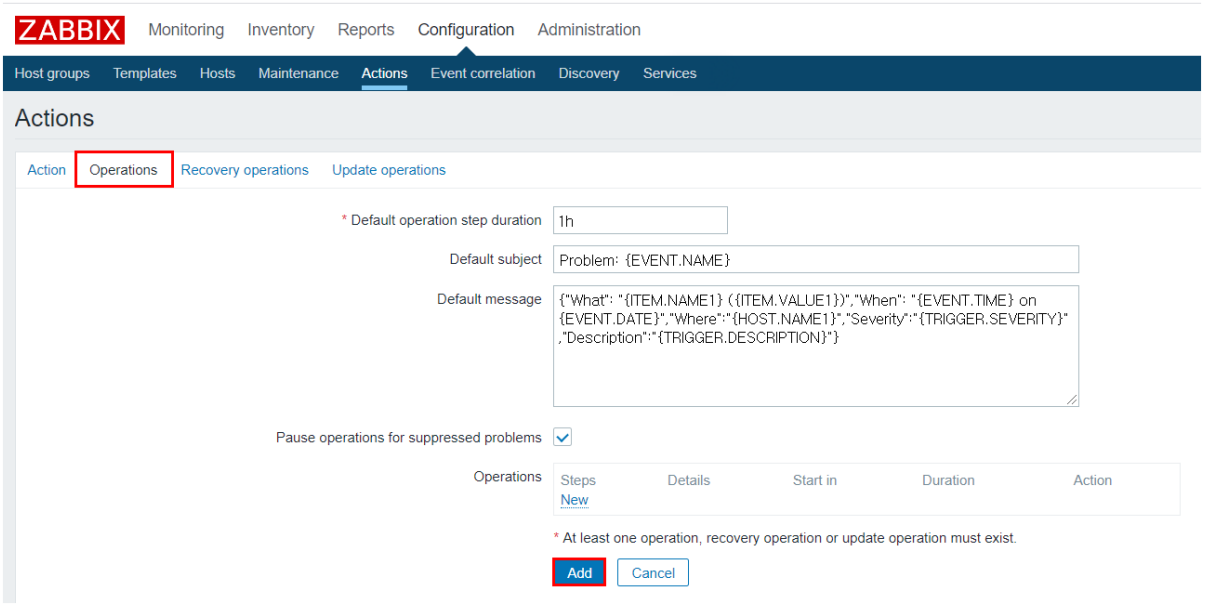
2. On the action registration screen, enter the action name, set the **New condition** item according to the site, and then click **Add** to add it to the condition list.



3. On the "Actions" screen, click the **Operations** tab, enter the information according to the site settings, and then click **Add**.
   - **Default subject**: Enter the information according to the site settings.
   - **Default message**: Enter the following values according to the site settings (JSON format is required).

```
{"What": "{ITEM.NAME1} ({ITEM.VALUE1})","When": "{EVENT.TIME} on {EVENT.DATE}","Where":"{HOST.NAME1}","Severity":"{TRIGGER.SEVERITY}","Description":"{TRIGGER.DESCRIPTION}"}
```

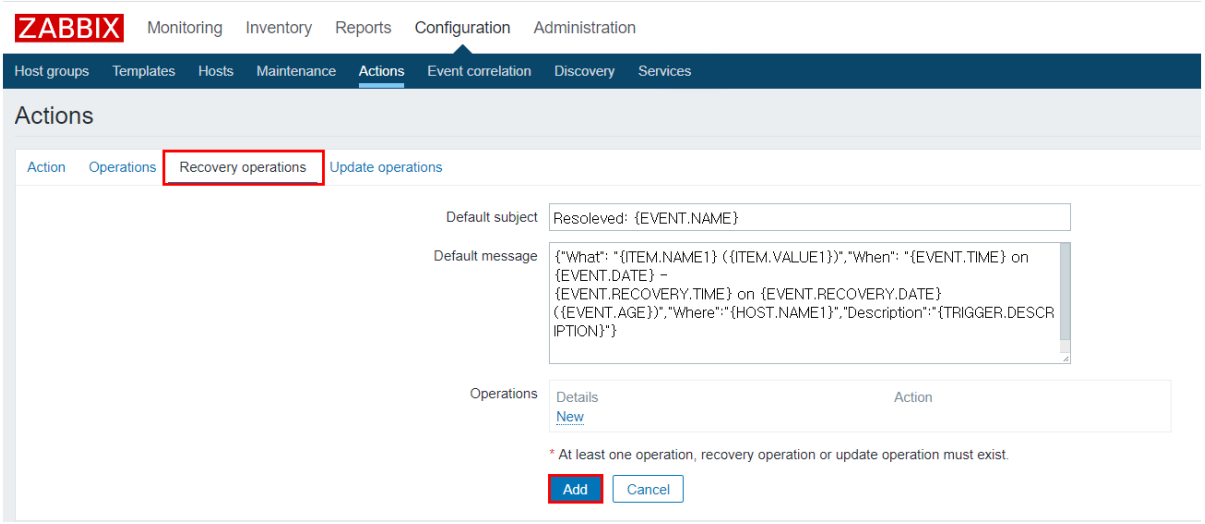
   - **Operations**: Link with the user you created in Creating users and linking with media.



4. On the "Actions" screen, click the **Recovery operations** tab, enter the information according to the site settings, and then click **Add**.
   - **Default subject**: Enter the information according to the site settings.
   - **Default message**: Enter the following values according to the site settings (JSON format is required).

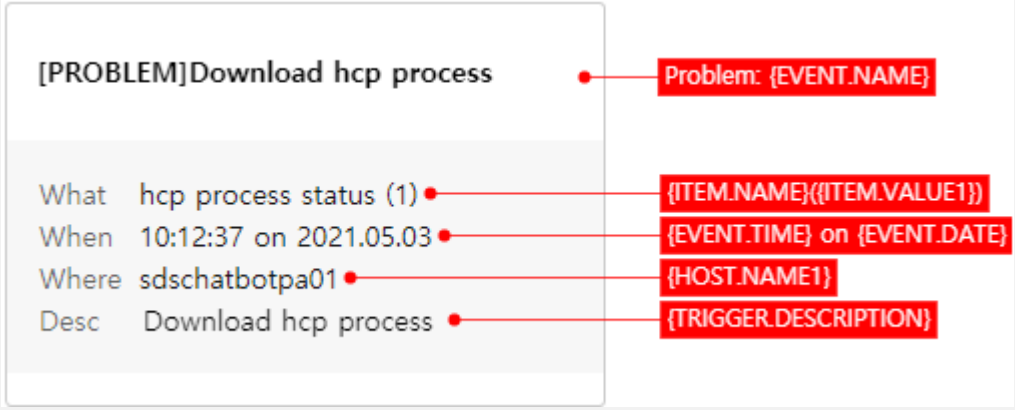
```
{"What": "{ITEM.NAME1} ({ITEM.VALUE1})","When": "{EVENT.TIME} on {EVENT.DATE} - {EVENT.RECOVERY.TIME} on {EVENT.RECOVERY.DATE} ({EVENT.AGE})","Where":"{HOST.NAME1}","Description":"{TRIGGER.DESCRIPTION}"}
```

   - **Operations**: Link with the user you created in Creating users and linking with media.



Note

The values provided in the default message item are identical to those in the following app card.

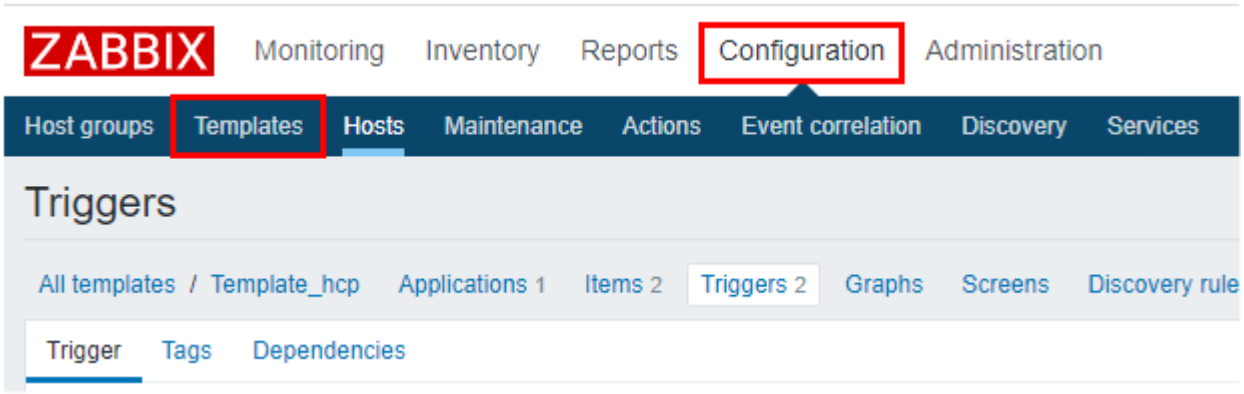


1.5 Testing

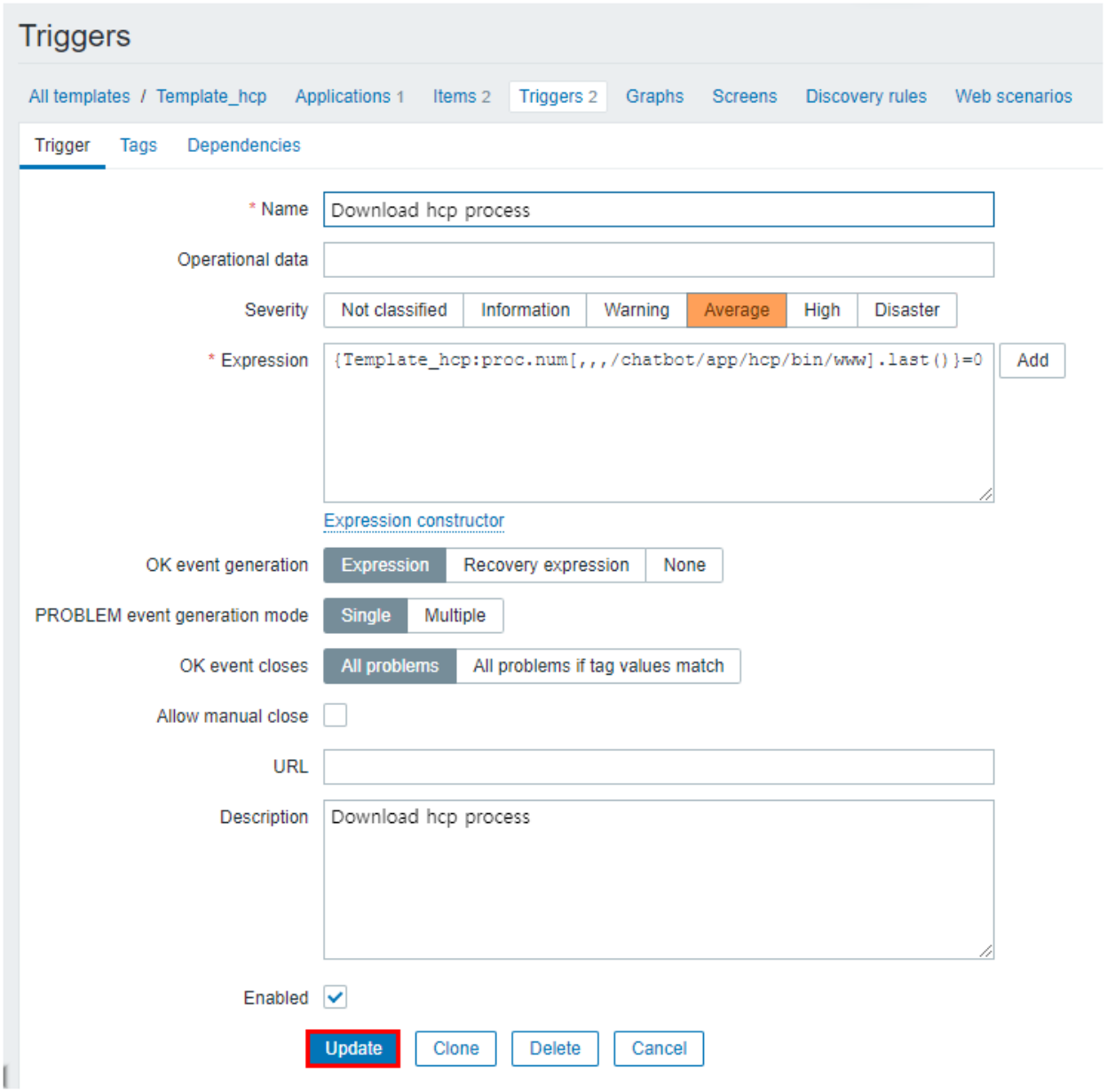
Through events generated by intentionally modified triggers, you can test the webhook to ensure that it has been correctly configured.

To perform a test and ensure that Zabbix event alarms are correctly received by the Knox Teams, complete the following steps:

1. Access Zabbix, click the template to modify in the **Configuration** > **Templates** menu, and click the trigger to modify.



2. On the trigger input screen, modify the existing **Expression** item and click **Update**.

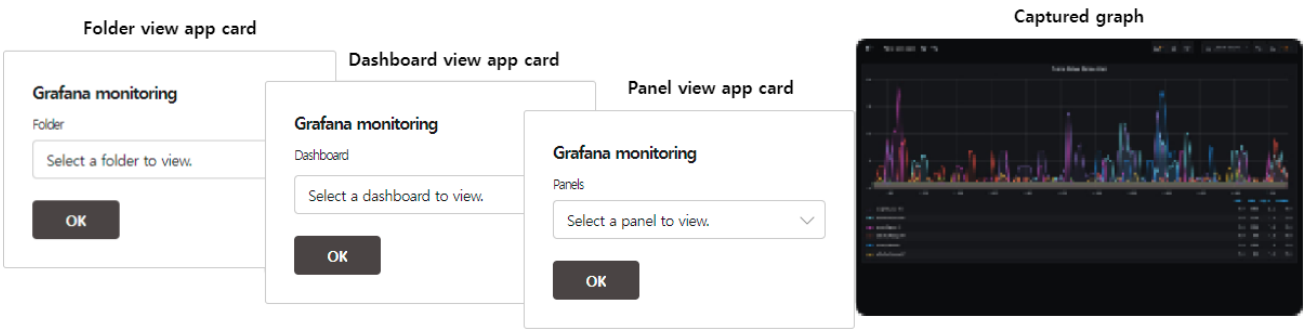


3. Access Knox Teams and check the app card delivered from the Zabbix system.

2.
Monitoring a Grafana system

You can integrate a Grafana system with the ChatOps bot to monitor the system through the graphs provided by Grafana.

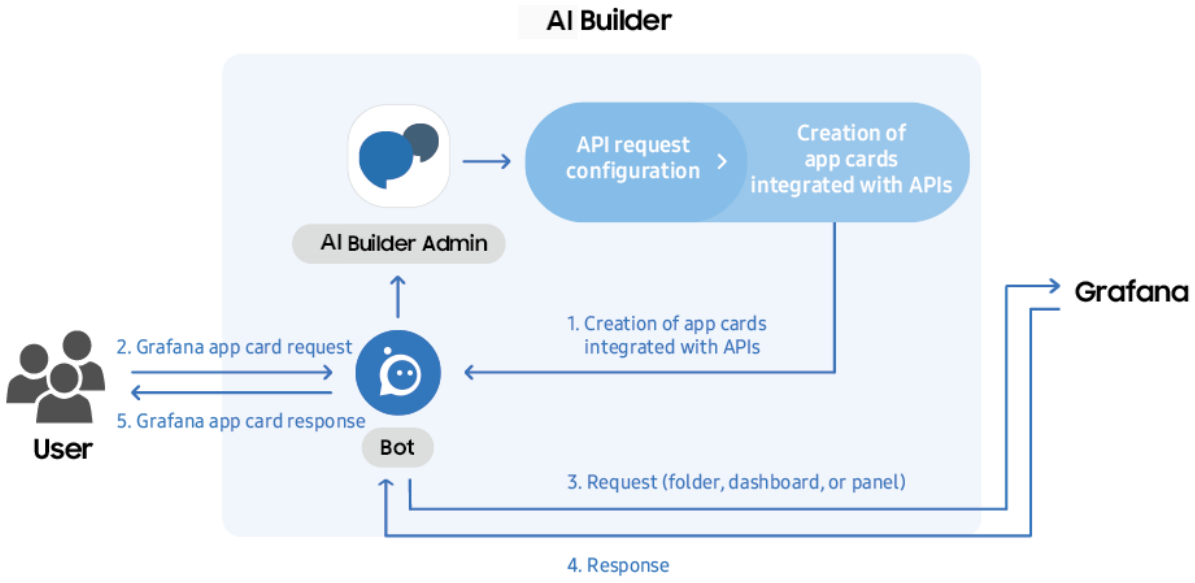
This tutorial explains how to integrate with a Grafana system to capture and view the system status expressed as graphs in the Knox Teams



> Note
> This tutorial has been created based on Grafana 7.4.3.

2.1 Grafana monitoring flow

The following process explains how the ChatOps bot views the Grafana system status.



2.2 Disabling the firewall before operation

To utilize the API integration feature in AI Builder, you must disable the firewall of the system that will be integrated.
Disable the firewall for the following range.

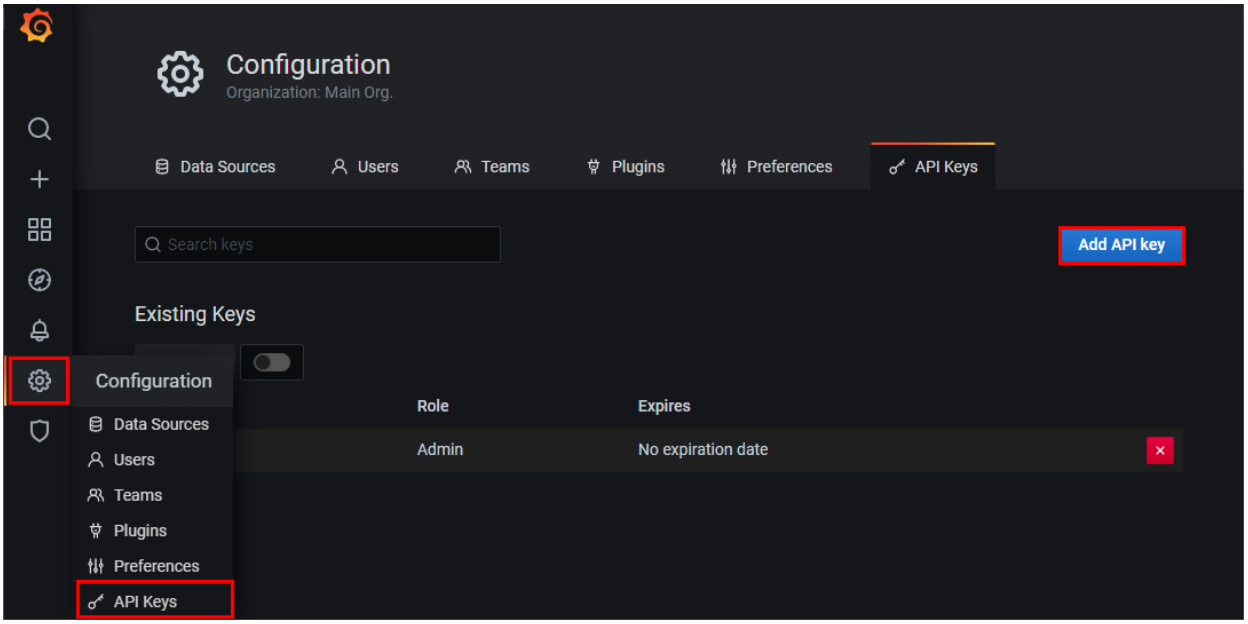
Category	Source	Destination		
		URL	IP	Port
API integration	182.195.131.87/182.195.131.89/182.195.131.105	Integrated system URL	Integrated system IP	Integrated system port

2.3 Configuring the integration in Grafana

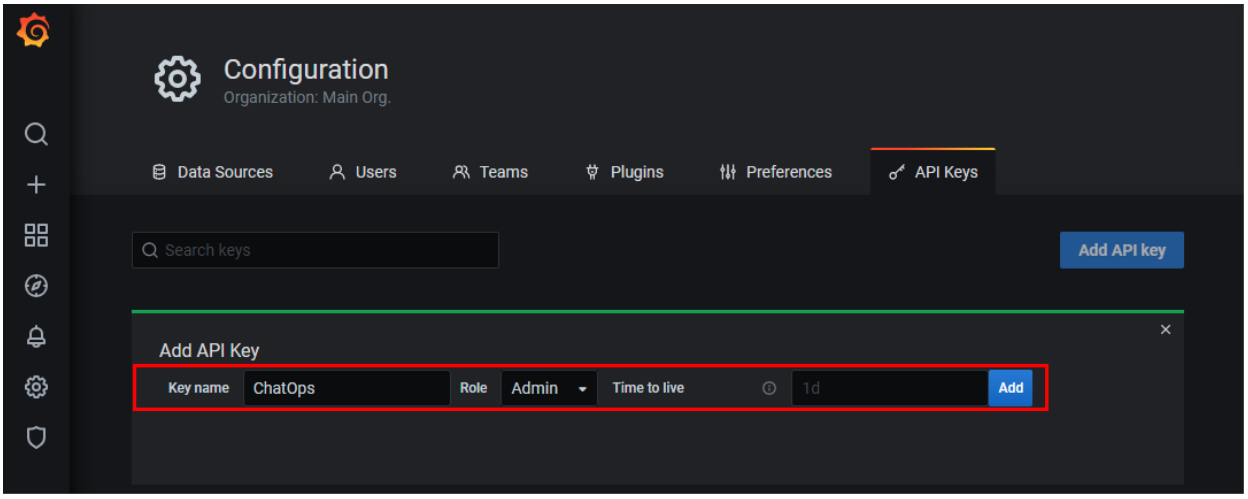
Issuing a Grafana token

To issue an authentication token for accessing the Grafana organization currently in use, complete the following steps:

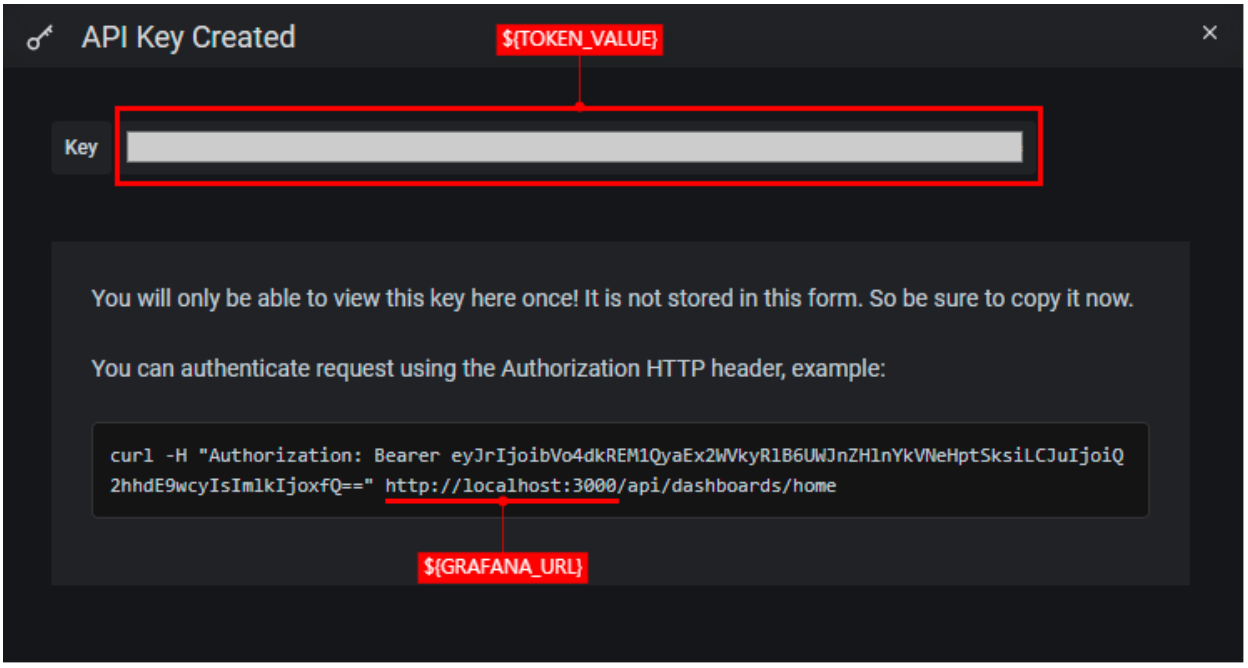
1. Access the Grafana system and click **(Configuration)** > **API Keys** > **Add API Key**.



2. Fill in the API key issuance window and click **Add**.

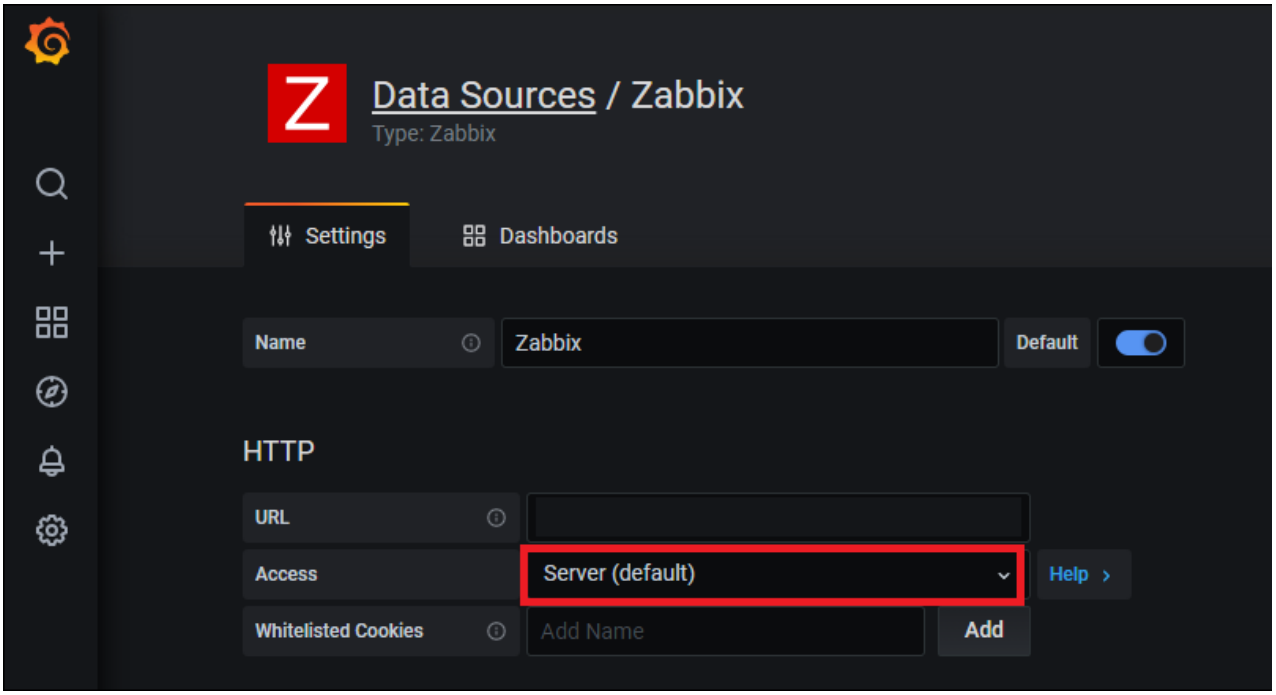


3. View the issued API key, make a copy, and store it so you can apply it to the app card.



Viewing the Grafana Data Source settings

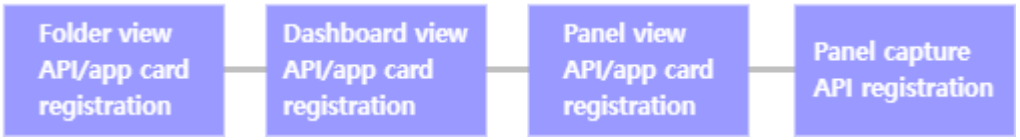
Access the Grafana system, click **(Configuration)** > **Data Sources** > **Zabbix**, and then ensure that the HTTP Access is set as “Server (default).”



> Note
> Applied only when integrating with Zabbix in Grafana.

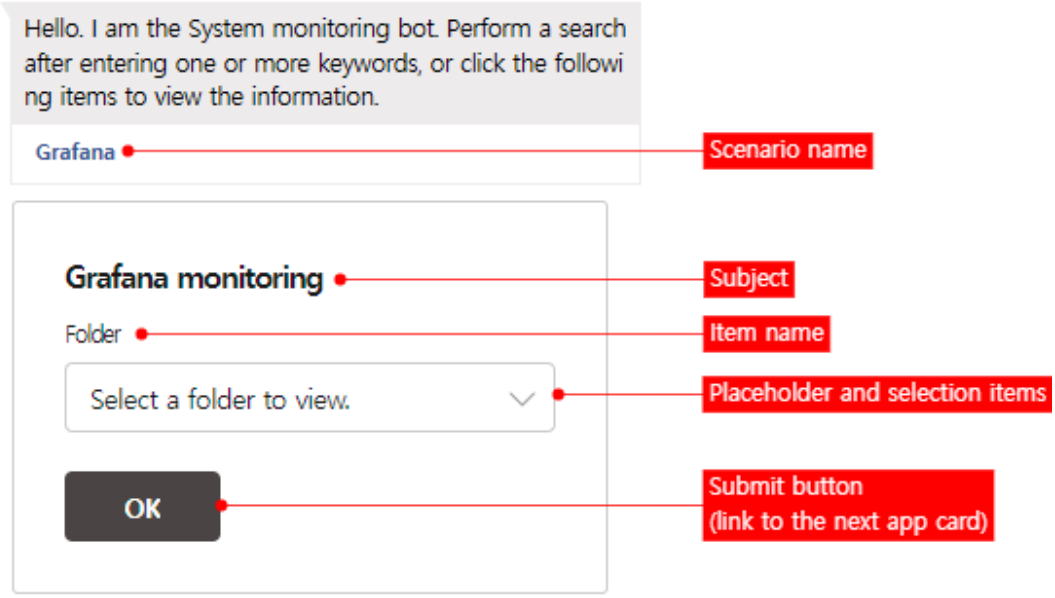
2.4 Configuring in AI Builder

Check the information provided in the tutorial according to the following stages and configure AI Builder to access the Grafana system.



Registering the folder view API and app card

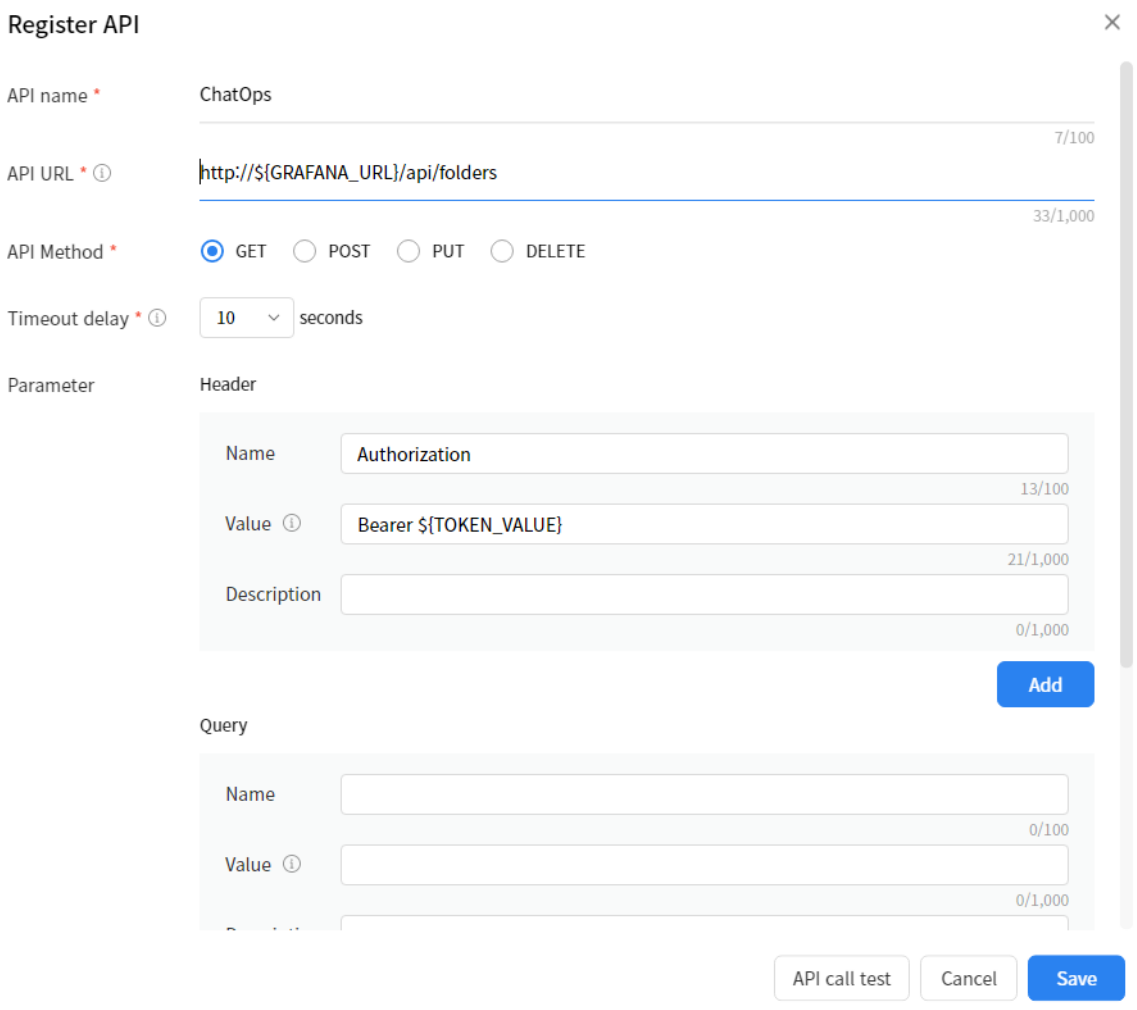
You can register an API and an app card and link them with a scenario according to the following tutorial to create an app card that can access the Grafana system folders.



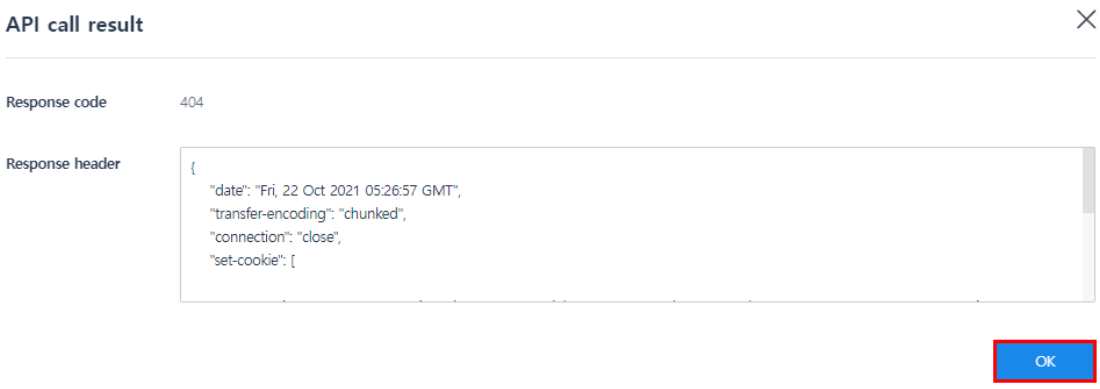
To register the folder view API and app card, complete the following steps:

1. At the top of the AI Builder home screen, click the **Bot** tab.
2. On the bot list of the "Bot" screen, click a bot to register the API.
3. On the bot details screen, click **API** > **API** > **Register**.
4. In the "Register API" pop-up window, enter the API name and refer to the examples in the following API specification table to set the parameter values for the **API URL**, **API Method**, and header.

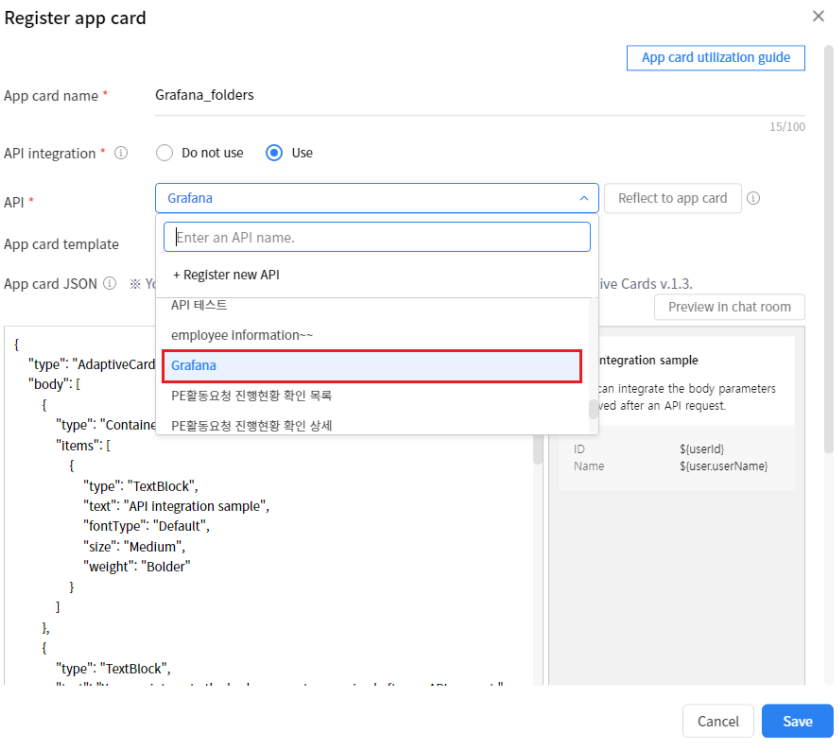
Category		**Key**	**Value**	**Reference**
API URL			http://${GRAFANA_URL}/api/folders	${GRAFANA_URL}: Enter the Grafana URL.
API Method			GET	
Parameter	**Header**	Authorization	Bearer ${TOKEN_VALUE}	${TOKEN_VALUE}: Enter the API key issued in Issuing a Grafana token.



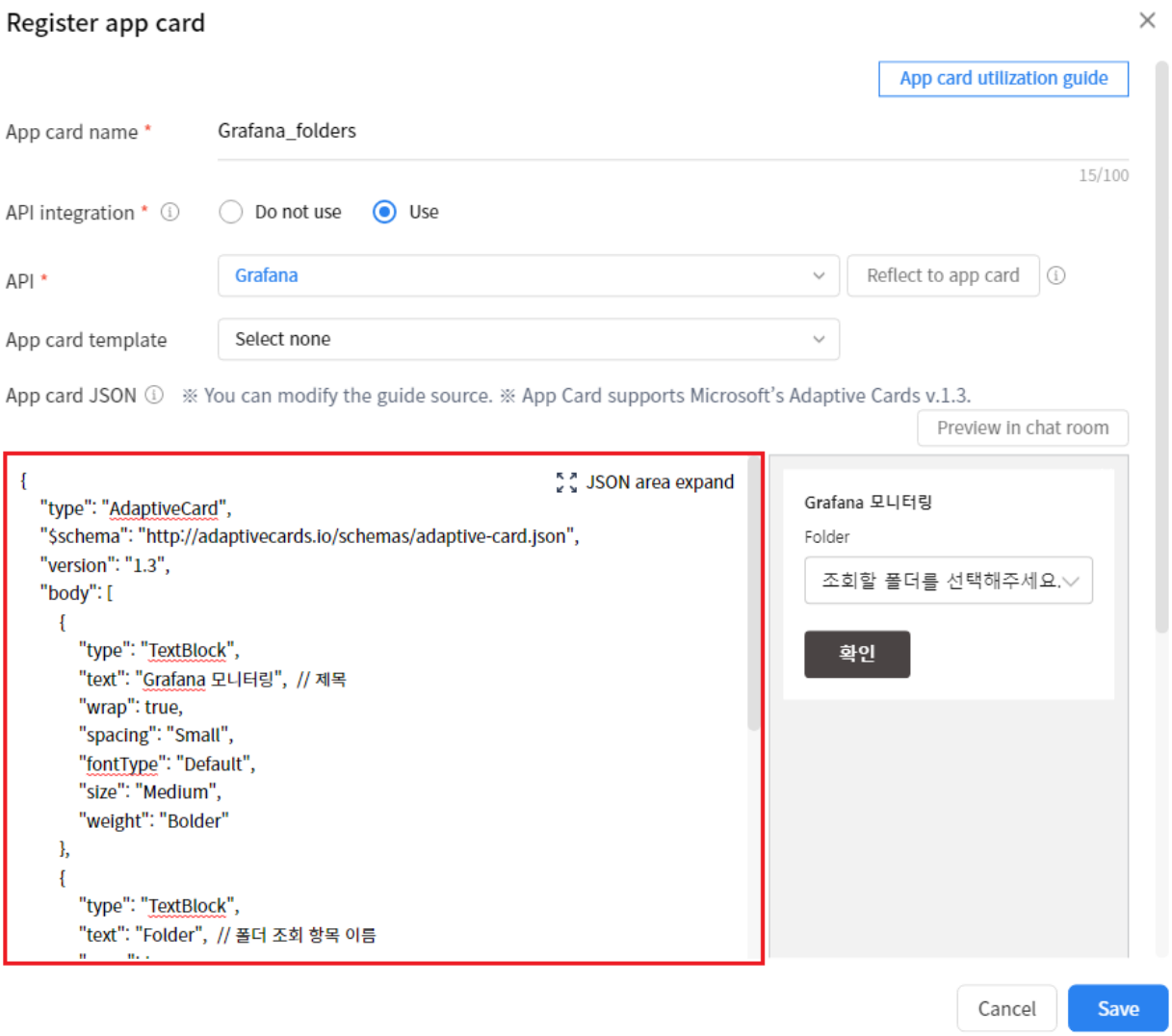
5. Click **API call test** to ensure that the API parameters work properly and click **OK**.
   - The received messages may vary depending on the response code of the API transmission.
     - 200 ~ 399: Sends specified app cards.
     - 400 ~ 500: Sends a message saying, "Proper messages cannot be provided due to an unstable connection with the external service."



6. In the "Register API" pop-up window, click **Save**.
7. On the bot details screen, click **App card** > **App cards** > **Register**.
8. In the "Register app card" pop-up window, enter the app card name and select **Use** for the **API integration** item.
9. In the **API** item, select a previously created API and link with it.



10. Refer to the following JSON to fill in the **App card JSON** field and click **Save**.

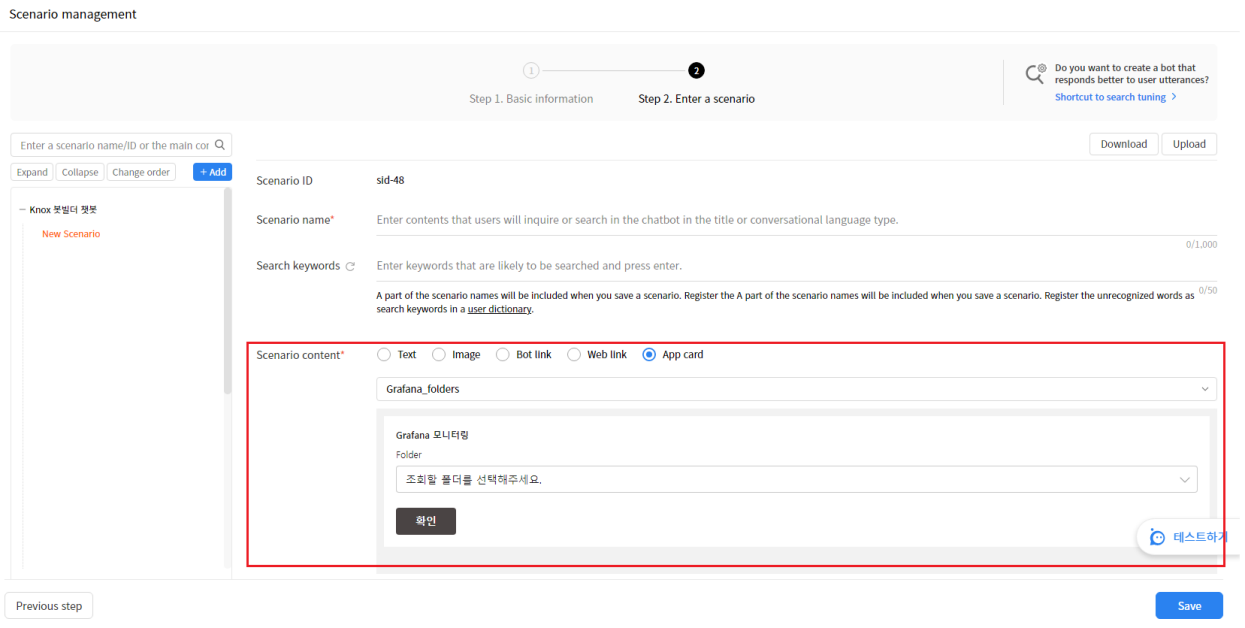



```
{
    "type": "AdaptiveCard",
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3",
    "body": [
        {
            "type": "TextBlock",
            "text": "Grafana monitoring",  // Title
            "wrap": true,
            "spacing": "Small",
            "fontType": "Default",
            "size": "Medium",
            "weight": "Bolder"
```

```
14          },
15          {
16              "type": "TextBlock",
17              "text": "Folder",  // Folder view item name
18              "wrap": true,
19              "spacing": "Medium",
20              "fontType": "Default",
21              "size": "Small"
22          },
23          {
24              "id": "selectedFolder",    // Parameter that delivers data to
the dashboard view app card
25
26              "placeholder": "Select a folder to view.",  // Placeholder
string
27              "spacing": "None",
28              "type": "Input.ChoiceSet",  // Item that will be selected
29              "choices": [
30                  {
31                      "$data": "${body}",
32                      "title": "${title}",
33                      "value": "${string(id)}"
34                  }
35              ]
36          },
37          {
38              "type": "ColumnSet",
39              "spacing": "Padding",
40              "columns": [
41                  {
42                      "type": "Column",
43                      "width": "auto",
44                      "items": [
45                          {
46                              "type": "ActionSet",
47                              "actions": [
48                                  {
49                                      "type": "Action.Submit",  //
Submission button
50                                      "title": "OK",  // Submission button
```

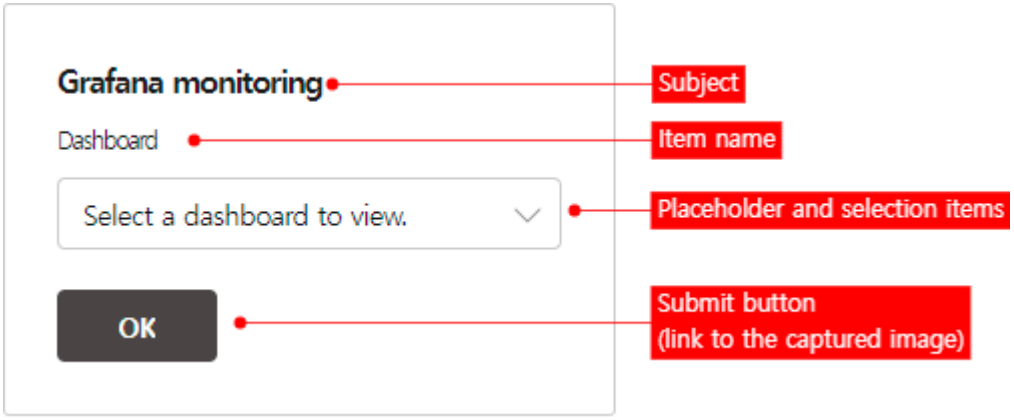
```
name
51                                      "style": "positive",
52                                      "data": {
53                                          "intent": "appCard",
54                                          "entity": {
55                                              "id":
"XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"    // Enter the dashboard view app card
ID (ID of the target app card).
56
57                                          }
58                                      }
59                                  }
60                              ]
61                          }
62                      ],
63                      "spacing": "Medium"
64                  }
65              ]
66          }
67      ]
68  }
```

11. On the bot details screen, click **Scenario** > **Scenario management** and link the created app card with a new or existing scenario.



Registering the dashboard view API and app card

You can register an API and an app card according to the following tutorial to create an app card that can access the Grafana system dashboard.
Also, by applying query parameter values when registering the API, you can transfer data from one app card to another.



Refer to the Registering the folder view API and app card tutorial and the following example to register the API and app card. Scenario registration is not required at this stage.

Category		Key	Value	Reference
API URL			http://${GRAFANA_URL}/api/search	${GRAFANA_URL}: Enter the Grafana URL.
API Method			GET	
Parameter	**Header**	Authorization	Bearer ${TOKEN_VALUE}	${TOKEN_VALUE}: The API key issued in Issuing a Grafana token
	Query	folderIds	${selectedFolder}	Parameter that is received from the folder view app card

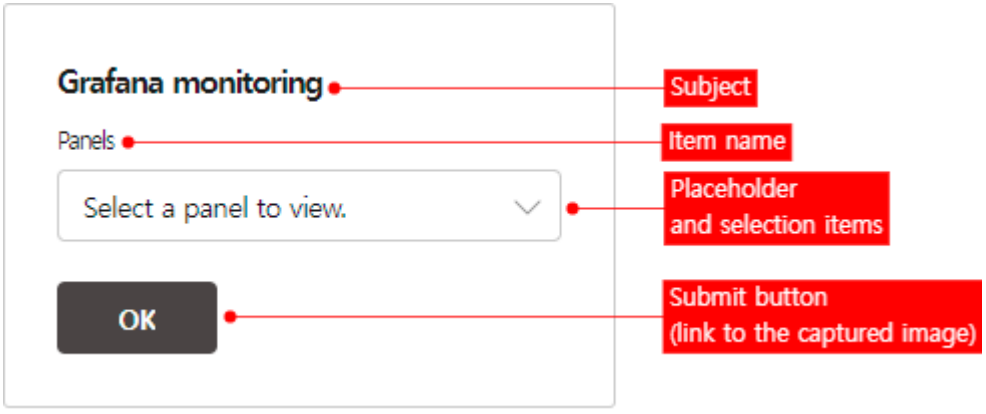
```
{
    "type": "AdaptiveCard",
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3",
    "body": [
        {
            "type": "TextBlock",
            "text": "Grafana monitoring",  // Title
            "wrap": true,
```

```
10              "spacing": "Small",
11              "fontType": "Default",
12              "size": "Medium",
13              "weight": "Bolder"
14          },
15          {
16              "type": "TextBlock",
17              "text": "Dashboard",  // Dashboard view item name
18              "wrap": true,
19              "spacing": "Medium",
20              "fontType": "Default",
21              "size": "Small"
22          },
23          {
24              "id": "selectedDashboard",  // Parameter that delivers data to the
panel view app card
25              "placeholder": "Select a dashboard to view.",  // Placeholder
string            "spacing": "None",
26              "type": "Input.ChoiceSet",  // Item that will be selected
27              "choices": [
28                  {
29                      "$data": "${body}",
30                      "title": "${title}",
31                      "value": "${uid}"
32                  }
33              ]
34          },
35          {
36              "type": "ColumnSet",
37              "spacing": "Padding",
38              "columns": [
39                  {
40                      "type": "Column",
41                      "width": "auto",
42                      "items": [
43                          {
44                              "type": "ActionSet",
45                              "actions": [
46                                  {
47                                      "type": "Action.Submit",  // Submission
```

```
button
48                                      "title": "OK",  // Submission button name
49                                      "style": "positive",
50                                      "data": {
51                                          "intent": "appCard",
52                                          "entity": {
53                                              "id":
"XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"    // Enter the panel view app card ID (ID
of the target app card).
54                                          }
55                                      }
56                                  }
57                              ]
58                          }
59                      ],
60                      "spacing": "Medium"
61                  }
62              ]
63          }
64      ]
65  }
```

Registering the panel view API and app card

You can register an API and an app card according to the following tutorial to create an app card that can access the Grafana system panel.
Also, you can create and register the API and app card to capture the viewed panel data (graphs).



Refer to the Registering the folder view API and app card tutorial and the following example to register the API and app card. Scenario registration is not required at this stage.

Category		Key	Value	Reference
API URL			http://${GRAFANA_URL}/api/dashboards/uid/${selectedDashboard}	• ${GRAFANA_URL}: Enter the Grafana URL • ${selectedDashboard}: Parameter that is received from the dashboard view app card
API Method			GET	
Parameter	**Header**	Authorization	Bearer ${TOKEN_VALUE}	${TOKEN_VALUE}: The API key issued in Issuing a Grafana token

```
{
    "type": "AdaptiveCard",
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3",
    "body": [
        {
            "type": "TextBlock",
            "text": "Grafana monitoring",  // Title
            "wrap": true,
            "spacing": "Small",
            "fontType": "Default",
            "size": "Medium",
            "weight": "Bolder"
        },
        {
            "type": "TextBlock",
            "text": "Panels",  // Panel view item name
            "wrap": true,
            "spacing": "Medium",
            "fontType": "Default",
            "size": "Small"
        },
        {
            "id": "selectedPanel",
            "placeholder": "Select a panel to view.",  // A placeholder string
            "spacing": "None",
```

```
27              "type": "Input.ChoiceSet",  // Item that will be selected
28              "choices": [
29                  {
30                      "$data": "${dashboard.panels}",
31                      "title": "${title}",
32                      "value": "${string(id)}"
33                  }
34              ]
35          },
36          {
37              "type": "ColumnSet",
38              "spacing": "Padding",
39              "columns": [
40                  {
41                      "type": "Column",
42                      "width": "auto",
43                      "items": [
44                          {
45                              "type": "ActionSet",
46                              "actions": [
47                                  {
48                                      "type": "Action.Submit",  // Submission
button
49                                      "title": "OK",  // Submission button name
50                                      "style": "positive",
51                                      "data": {
52                                          "intent": "capture",  // Screen
capture intent
53
54                                          "entity": {
55                                              "id":
"XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX",   // Enter the screen capture API ID
56                                              "vars": {
57                                                  "url": {
58                                                      "uid":
"${$root.dashboard.uid}",   // The panel uid parameter to be transferred to the
captured image
59
60                                                      "title":
"${$root.dashboard.title}"   // The panel title parameter to be transferred to the
```

```
captured image
61
62                                                  }
63                                              }
64                                          }
65                                      }
66                                  }
67                              ]
68                          }
69                      ],
70                      "spacing": "Medium"
71                  }
72              ]
73          }
74      ]
75  }
```

Note

- You can add the `entity.sleep` parameter to specify a delay for the capture if graphs in the captured image are not correctly displayed.
  - Default: 1 sec
  - Max. delay: 9 sec
- You can add the `entity.width` and `entity.height` parameters to adjust the image size.
  - Width: 1280
  - Height: 800

```
1  "data": {
2      "intent": "capture",
3      "entity": {
4          "id": "XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX",
5          "vars": {
6              "url": {
7                  "uid": "${$root.dashboard.uid}",
8                  "title": "${$root.dashboard.title}"
```

```
            }
        },
        "sleep": 5    // Delay time (sec)
        "width": 1280    // Image width (horizontal)
        "height": 800    // Image height (vertical)
    }
}
```

Registering a panel capture API

You can use the API specifications in the following example to create and register a panel capture API.

Refer to the Registering the folder view API and app card tutorial and the following example to register the API. App card and scenario registrations are not required at this stage.

Category		Key	Value	Reference
API URL			http://${GRAFANA_URL}/d/${uid}/${title}	• ${GRAFANA_URL}: Enter the Grafana URL • ${uid}/${title}: Parameter received from the dashboard panel view
API Method			GET	
Parameter	**Header**	Authorization	Bearer ${TOKEN_VALUE}	${TOKEN_VALUE}: The token issued in Issuing a Grafana token
	Query	viewPanel	${selectedPanel}	${selectedPanel}: Parameter received from the dashboard panel view

2.5 Deploying

Follow the instructions in the tutorial to reflect the app card and scenario that have been deployed and registered to the Knox Teams in Knox Teams.

1. In the top-left corner of the bot details screen, click **Deploy**.
2. In the "Deployment details" pop-up window, fill in the **Key revision history** field and click **Deploy**.

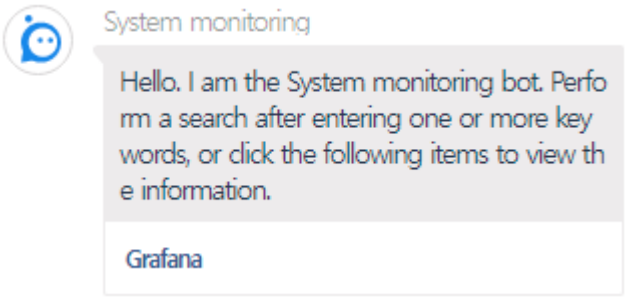
Note
To perform a test before deploying the bot to the Knox Teams, click "Test" in the bottom-right area of the "Scenario management" screen.

2.6 Testing

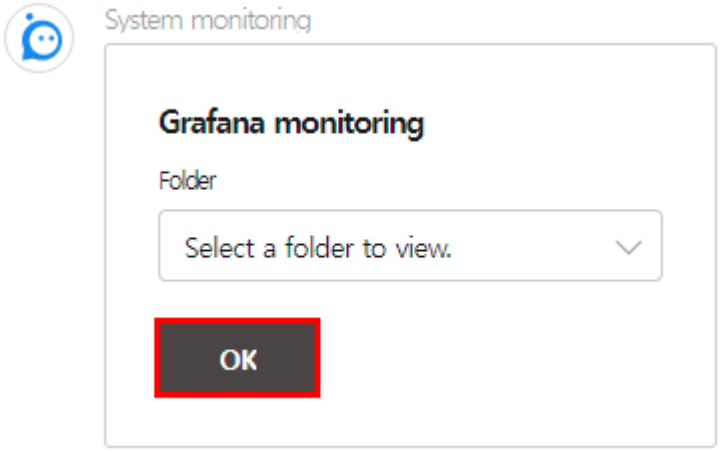
You can access Knox Teams and test the ChatOps bot to ensure that it operates properly according to the scenario.

To test the API and app card registered to monitor the Grafana system, complete the following steps:

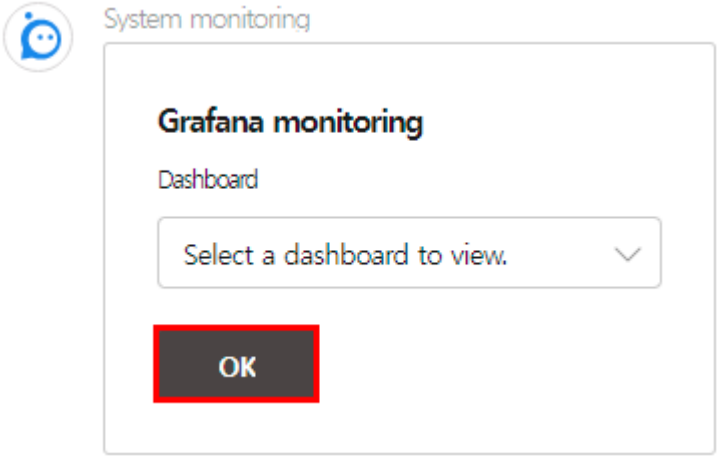
1. Access Knox Teams.
2. Click **Knox Teams** > **Apps** > **Knox Bot**, search for the bot's Knox ID you entered that registered the API and app card, and then double-click the bot.
3. Click the scenario registered in Registering the folder view API and app card.



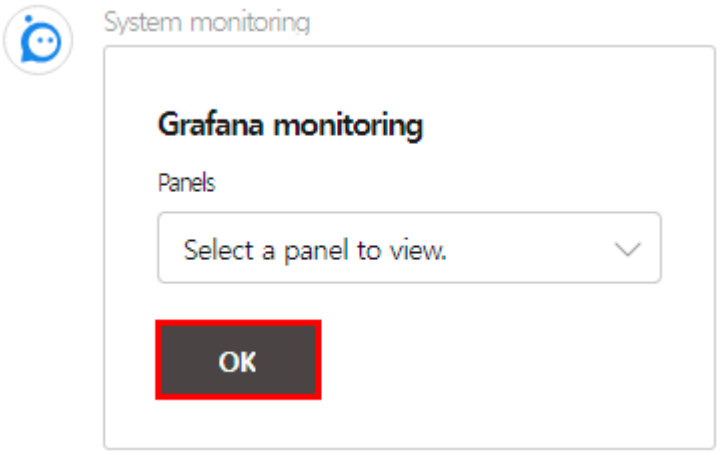
4. On the folder view app card, select a folder to view and click **OK**.



5. On the dashboard view app card, select a dashboard to view and click **OK**.



6. On the panel view app card, select a panel to capture and click **OK**.



The captured image of the selected panel will be displayed as follows.

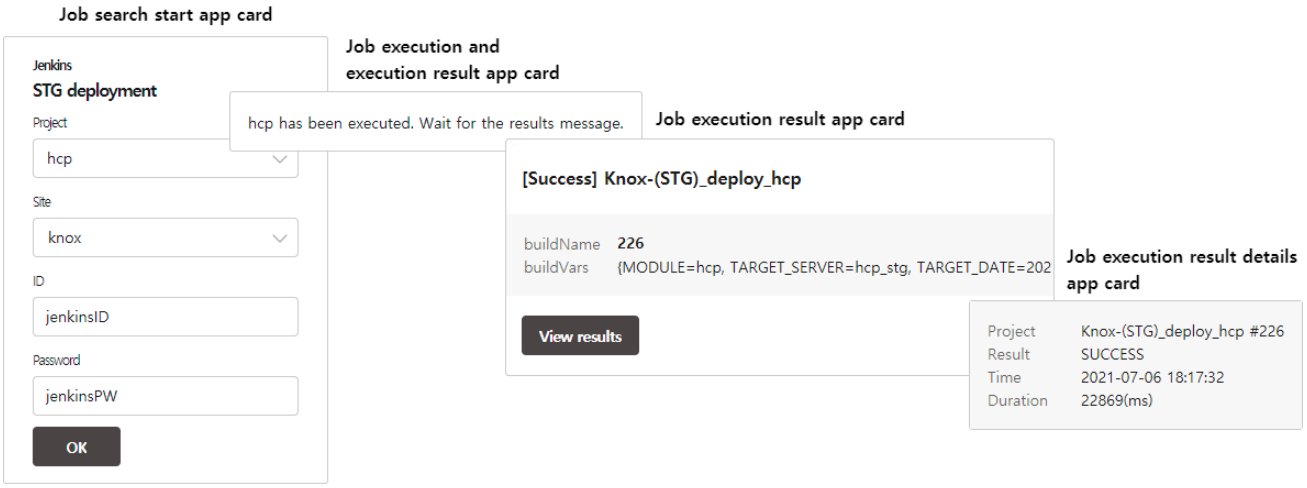


3.

Remotely operating a Jenkins system

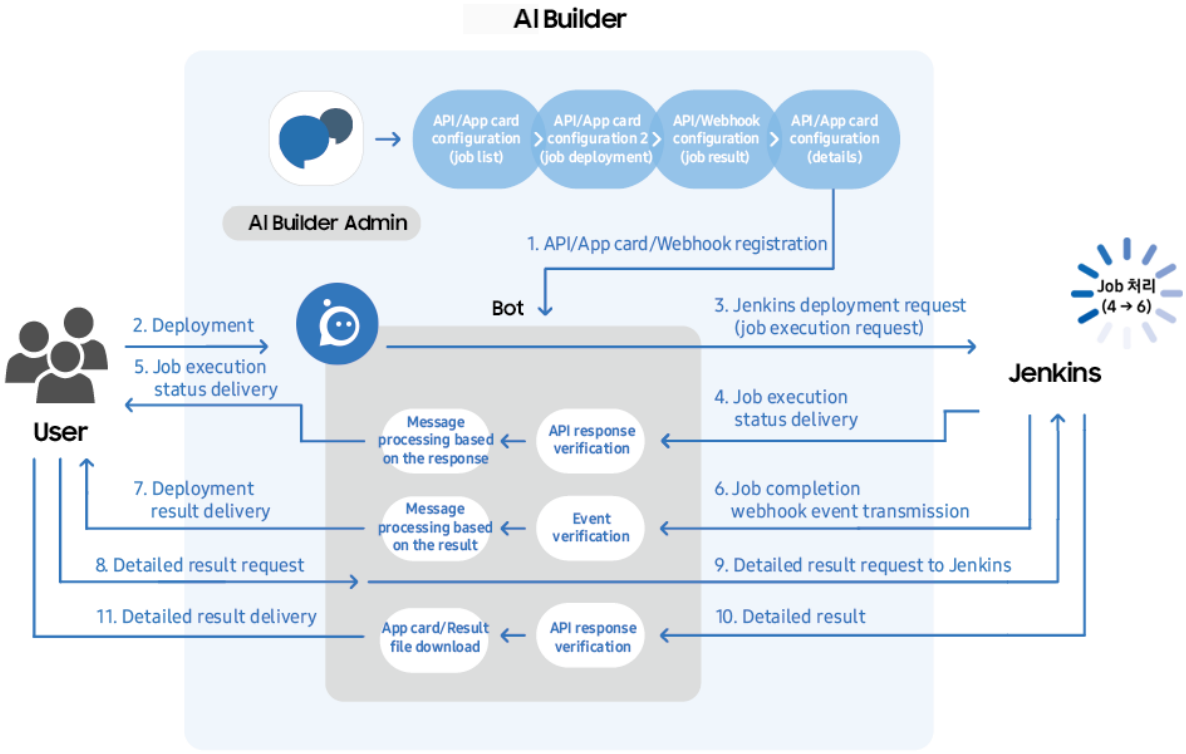
You can integrate the ChatOps bot with Jenkins to execute jobs specified in Jenkins and view the execution results.

This tutorial explains how to configure the Knox Teams to remotely operate a Jenkins system through an integration in "Jenkins remote operation".



3.1 Jenkins remote operation flow

The following explains the process for remotely operating a Jenkins system and receiving the execution results with the ChatOps bot.



3.2 Preliminary preparations

Disabling the firewall

To utilize the webhook and API integration features in AI Builder, you must disable of the firewall of the system that will be integrated.
Disable the firewall for the following range.

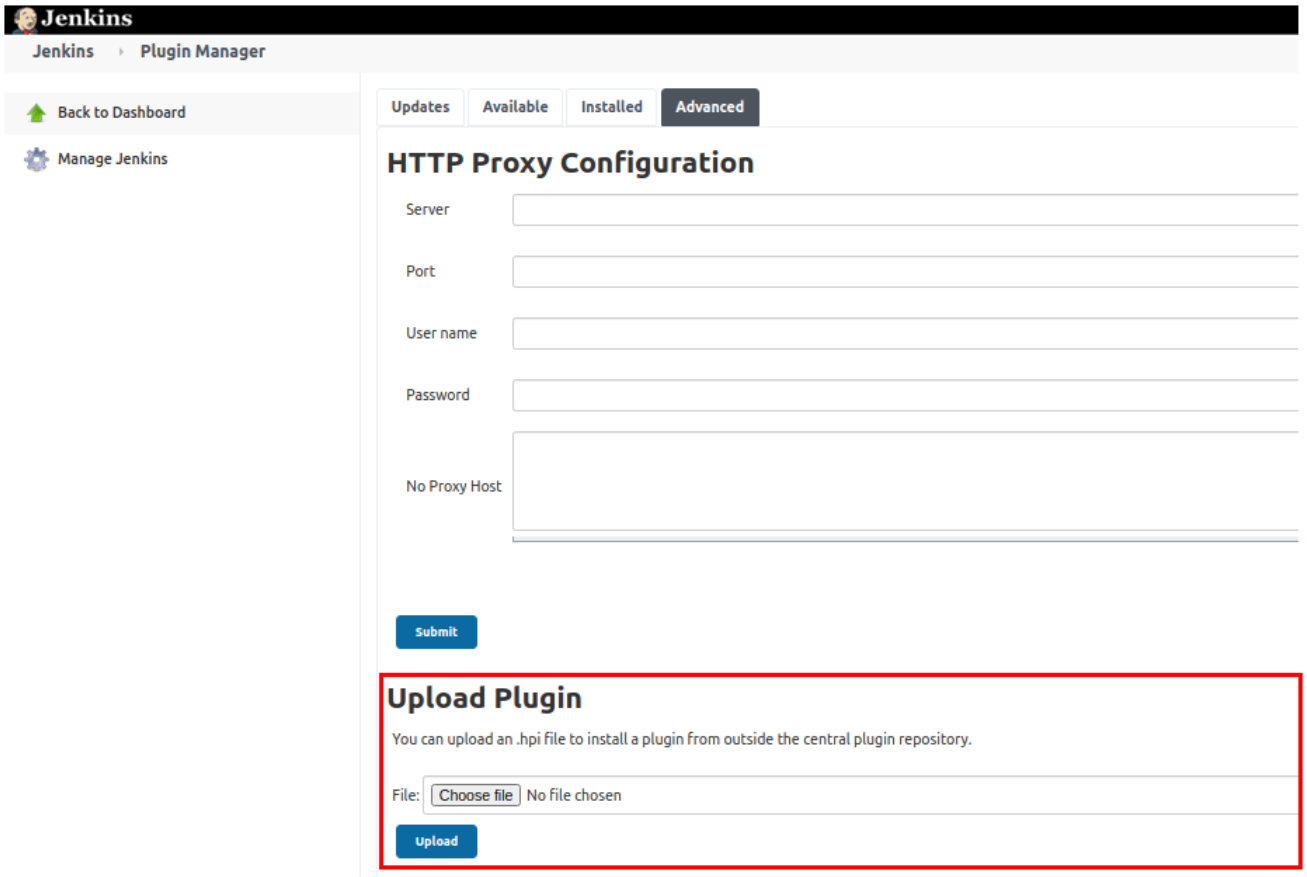
Category	Source	Destination		
		URL	IP	Port
Webhook integration	External	proxy-aibuilder.samsung.net	112.107.147.103	80, 443
	Internal	aibuilder.samsung.net	182.195.83.147	80, 443
API integration	182.195.131.87/182.195.131.89/182.195.131.105	Integrated system URL	Integrated system IP	Integrated system port

Installing the Outbound WebHook for build events plugin

Select one of the following ways to install the Outbound WebHook for build events plugin, which generates Jenkins events.

- In the **Manage Jenkins** > **Manage Plugins** > **Available** menu, search for and install “Outbound WebHook for build events.”
- Download the outbound-webhook.hpi plugin (for Java 8) on the Jenkins website and upload the file in the **Manage Jenkins** > **Plugin Manager** > **Advanced** > **Upload Plugin** menu.

> Note
> To install the plugin for Java versions other than Java 8, create a Gradle build in the server environment where the plugin will be installed after downloading a release from outbound-webhook-plugin.

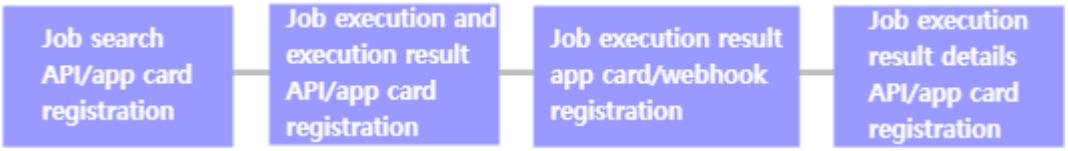


3.3 Remotely deploying in a Jenkins system

By integrating the ChatOps bot with a Jenkins system, you can configure the ChatOps bot to execute the deployment jobs specified in the Jenkins system and view the results in the ChatOps bot.

Configuring in AI Builder

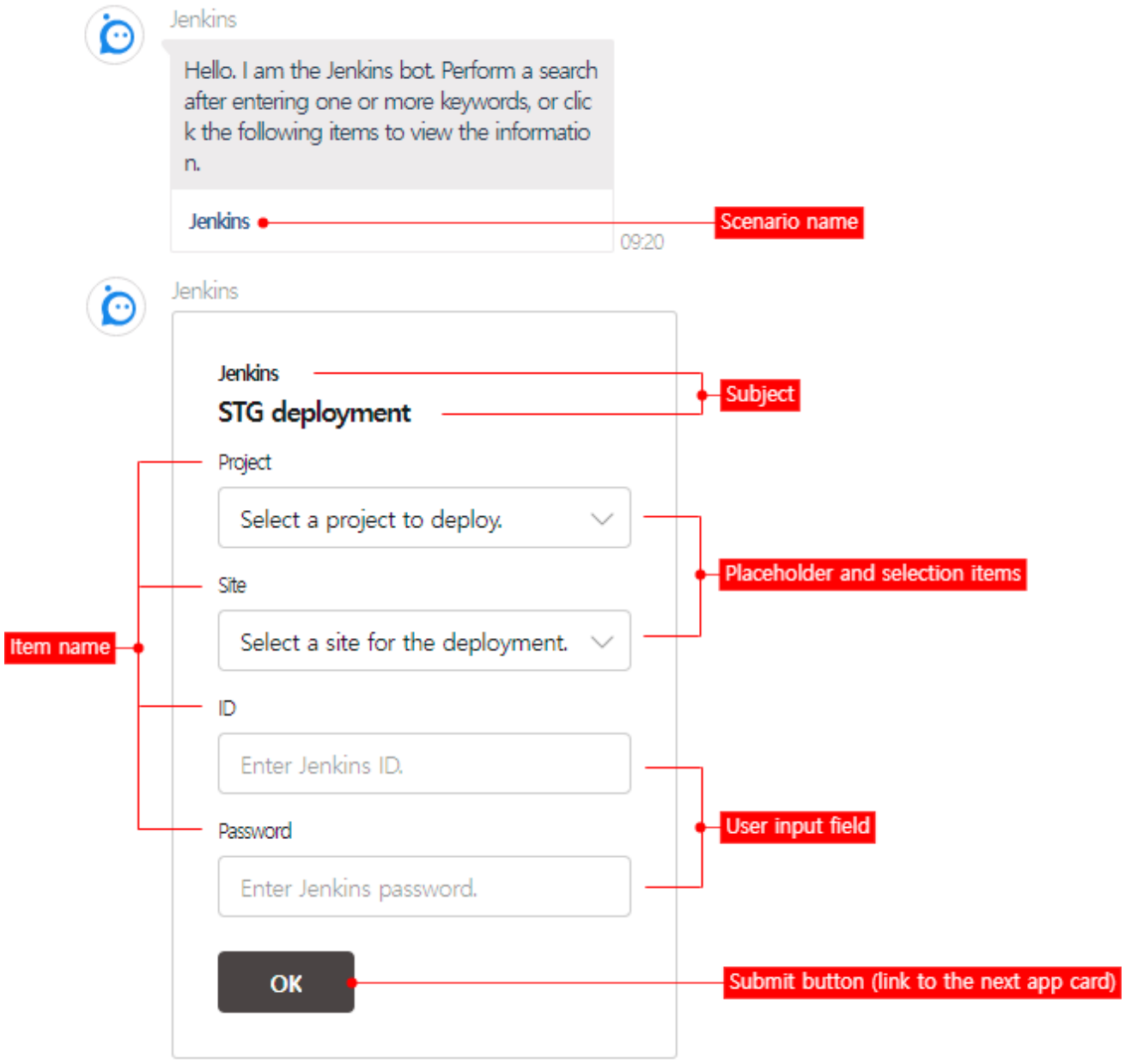
Check the information provided in the tutorial according to the following stages and configure AI Builder to allow the ChatOps bot to remotely execute jobs in a Jenkins system.



> Note
> For more information about the Jenkins API, see Jenkins Remote Access API.

Registering the job view start API and app card

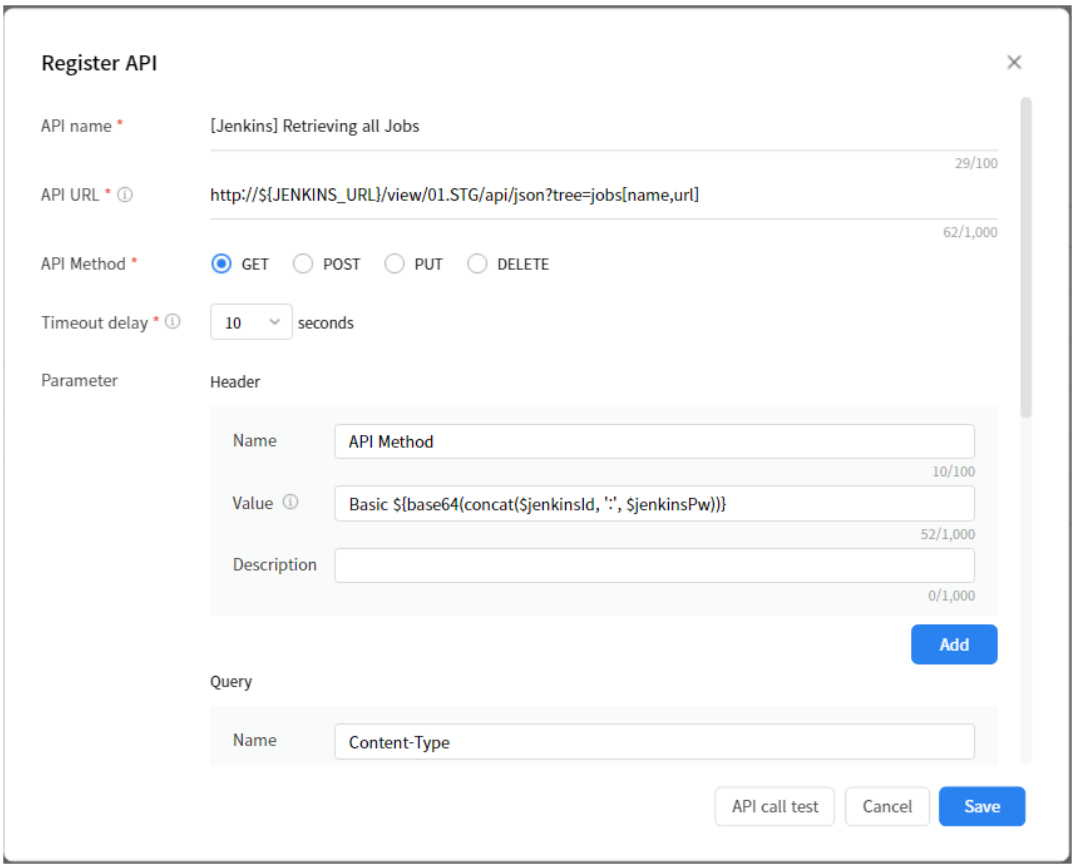
You can register an API and an app card and link them with a scenario according to the following tutorial to create an app card that can access the Jenkins system.



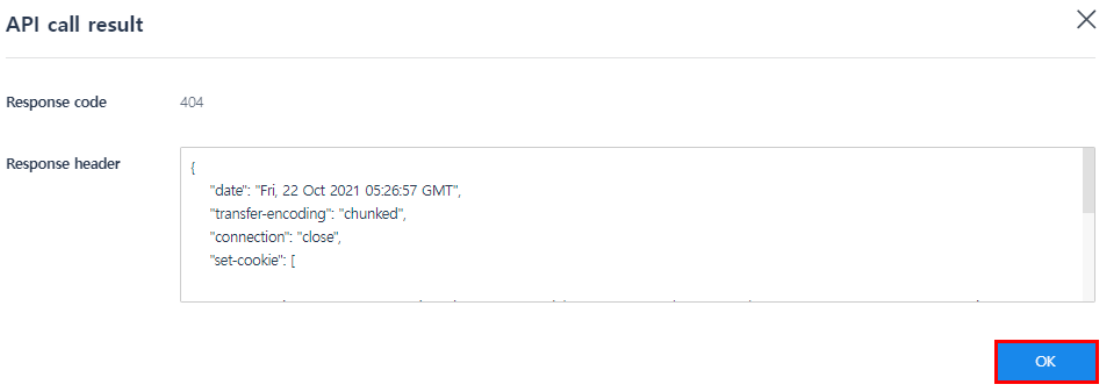
To register the view start API and app card, complete the following steps:

1. At the top of the AI Builder home screen, click the **Bot** tab.
2. On the bot list of the "Bot" screen, click a bot to register the API.
3. On the bot details screen, click **API** > **API** > **Register**.
4. In the "Register API" pop-up window, enter the API name and refer to the examples in the following API specification table to set the parameter values for the **API URL**, **API Method**, and header.
    - The following is an example configuration for fetching only jobs that are registered to the Jenkins Categorized View (01.STG).

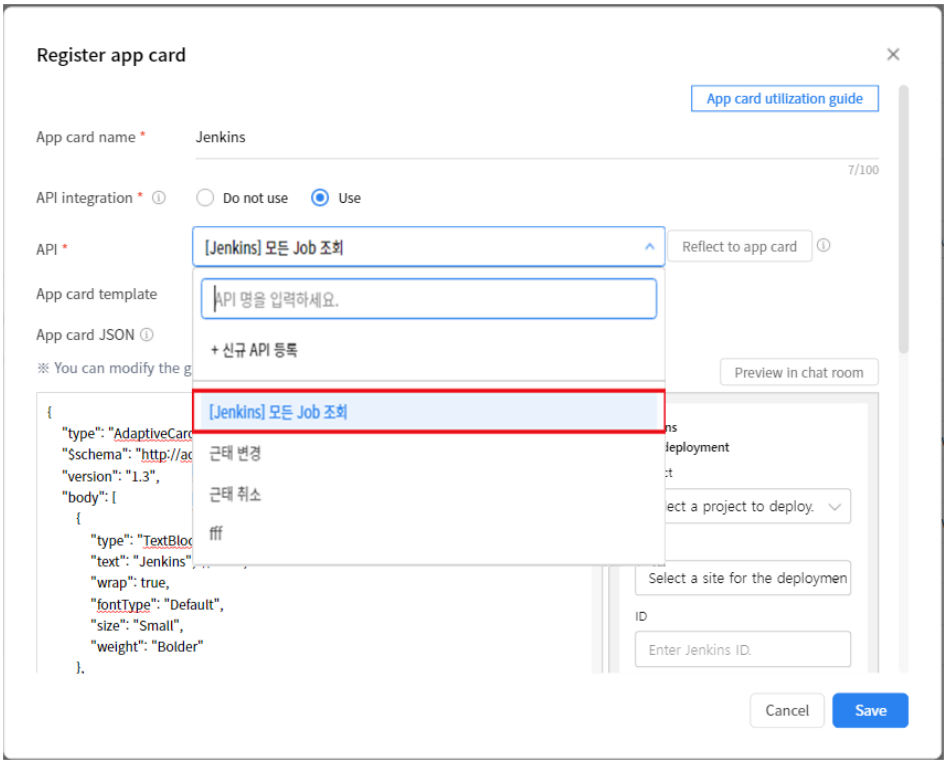
Category		**Key**	**Value**	**Reference**
API URL			http://${JENKINS_URL}/view/01.STG/api/json?tree=jobs[name,url]	${JENKINS_URL}: Enter the Jenkins URL.
API Method			GET	
Parameter	**Header**	Content-Type	application/json	
		Authorization	Basic ${base64(concat($jenkinsId, ':', $jenkinsPw))}	${jenkinsId}, ${jenkinsPw}: Enter the Jenkins account information.



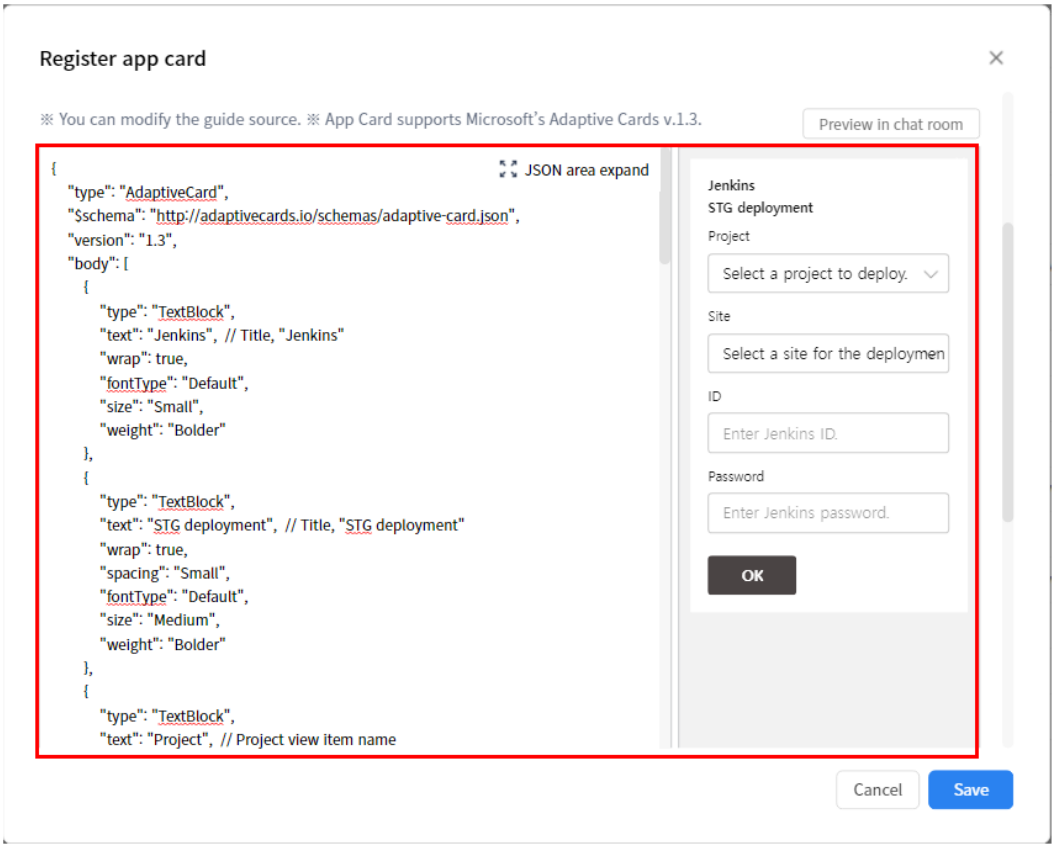
5. Click **API call test** to ensure that the parameters work properly and click **OK**.
   - The received messages may vary depending on the response code of the API transmission.
     - 200 ~ 399: Sends specified app cards.
     - 400 ~ 500: Sends a message saying, "Proper messages cannot be provided due to an unstable connection with the external service."



6. In the "Register API" pop-up window, click **Save**.
7. On the bot details screen, click **App card** > **App cards** > **Register**.
8. In the "Register app card" pop-up window, enter the app card name and select **Use** for the **API integration** item.
9. In the displayed **API** item, select a previously created API and link with it.



10. Refer to the following JSON to fill in the **App card JSON** field and click **Save**.
    - The following is an example configuration for fetching only the predefined jobs that begin with “hcp.”




```
{
    "type": "AdaptiveCard",
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3",
    "body": [
        {
            "type": "TextBlock",
            "text": "Jenkins",  // Title, "Jenkins"
            "wrap": true,
            "fontType": "Default",
            "size": "Small",
            "weight": "Bolder"
        },
```

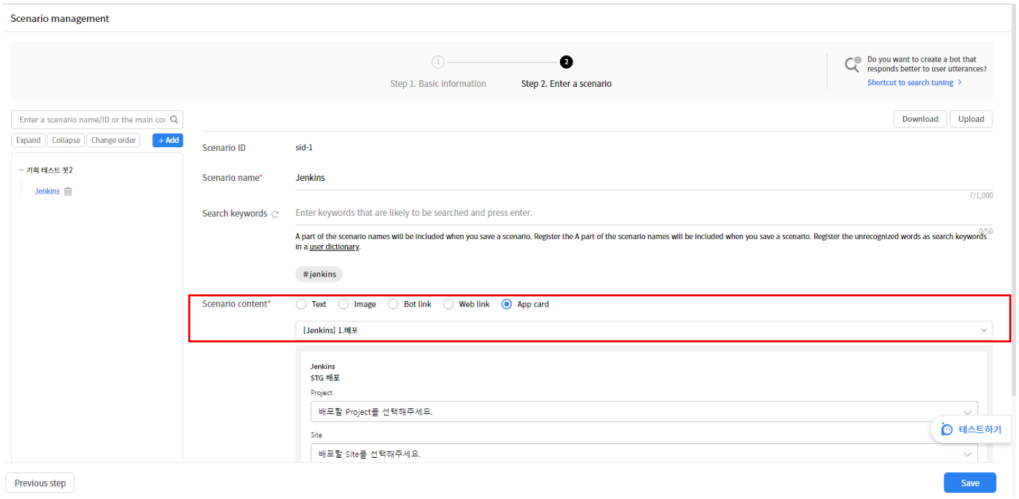
```
14          {
15              "type": "TextBlock",
16              "text": "STG deployment",  // Title, "STG deployment"
17              "wrap": true,
18              "spacing": "Small",
19              "fontType": "Default",
20              "size": "Medium",
21              "weight": "Bolder"
22          },
23          {
24              "type": "TextBlock",
25              "text": "Project",  // Project view item name
26              "wrap": true,
27              "spacing": "Medium",
28              "fontType": "Default",
29              "size": "Small"
30          },
31          {
32              "id": "selectedProject",    // Project to be deployed,
which will be delivered to Jenkins
33              "placeholder": "Select a project to deploy.",  //
Placeholder string 1
34              "spacing": "None",
35              "type": "Input.ChoiceSet",  // Selection item 1
36              "choices": [
37                  {
38                      "$data": "${jobs}",
39                      "title": "${name}",
40                      "value": "${name}",
41                      "$when": "${isMatch(name,&apos;^hcp*&apos;)}"
// Syntax for filtering the projects with “hcp” after calling the Jenkins API.
Modifications are required to suit each project.
42                  }
43              ]
44          },
45          {
46              "type": "TextBlock",
47              "text": "Site",  // Site view item name
48              "wrap": true,
49              "spacing": "Medium",
```

```
50              "fontType": "Default",
51              "size": "Small"
52          },
53          {
54              "id": "selectedSite",    // Deployment target that will be
delivered to Jenkins
55              "placeholder": "Select a site for the deployment.",  //
Placeholder 2
56              "spacing": "None",
57              "type": "Input.ChoiceSet",  // Selection item 2
58              "choices": [
59                  {
60                      "title": "all",    // Modification is required to
suit each project (rows 59 through 70).
61                      "value": "all=true"
62                      },
63                      {
64                      "title": "knox",
65                      "value": "knox=true&all=false"
66                      },
67                      {
68                      "title": "brity",
69                      "value": "brity=true&all=false"
70                      }
71                  ]
72              },
73              {
74                  "type": "TextBlock",
75                  "text": "ID",  // ID input item name
76                  "wrap": true,
77                  "spacing": "Medium",
78                  "fontType": "Default",
79                  "size": "Small"
80              },
81              {
82                 "type": "Input.Text",  // User input field 1
83                 "placeholder": "Enter Jenkins ID.",  // User input
phrase 1
84                 "id": "jenkinsId",    // ID that will be delivered to
Jenkins
```

```
                "spacing": "None",
                "maxLength": 50
            },
            {
                "type": "TextBlock",
                "text": "Password",  // Item name, "Password"
                "wrap": true,
                "spacing": "Medium",
                "fontType": "Default",
                "size": "Small"
            },
            {
                "type": "Input.Text",  // User input field 2
                "placeholder": "Enter Jenkins password.",  // User input phrase 2
                "id": "jenkinsPw",    //  Password that will be delivered to Jenkins
                "spacing": "None",
                "maxLength": 100
            },
            {
                "type": "ColumnSet",
                "spacing": "Padding",
                "columns": [
                    {
                        "type": "Column",
                        "width": "auto",
                        "items": [
                            {
                                "type": "ActionSet",
                                "actions": [
                                    {
                                        "type": "Action.Submit",  // Submission button
                                        "title": "OK",  // Submission button name
                                        "style": "positive",
                                        "data": {
                                        "intent": "appCard",
                                        "entity": {
```

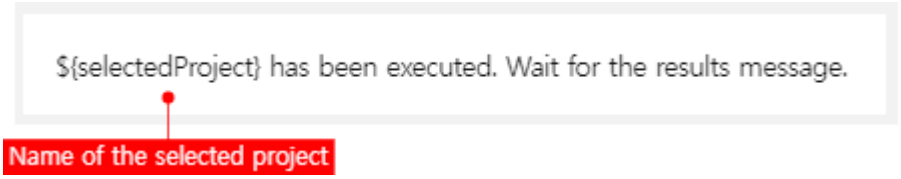
```
                                                    "id":
"XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"    // Enter the ID for the app card
to decide whether to execute Jenkins (ID of the target app card).
                                                }
                                            }
                                        }
                                    ]
                                }
                            ],
                            "spacing": "Medium"
                        }
                    ]
                }
            ]
        }
```

11. On the bot details screen, click **Scenario** > **Scenario management** and link the created app card with a new or existing scenario.



Registering the job execution/result APIs and app cards

You can create and register an API and an app card according to the following tutorial to execute the jobs received from a Jenkins system in the ChatOps bot and create an app card that can display the execution status and results in the Knox Teams.



Refer to the Registering the job view start API and app card tutorial and the following example to register the API and app card. Scenario registration is not required at this stage.

- The following example executes the jobs registered in Jenkins Pipeline.
- The received messages may vary depending on the response code of the API transmission.
  - 200 ~ 399: Sends specified app cards.
  - 400 ~ 500: Sends a message saying, "Proper messages cannot be provided due to an unstable connection with the external service."

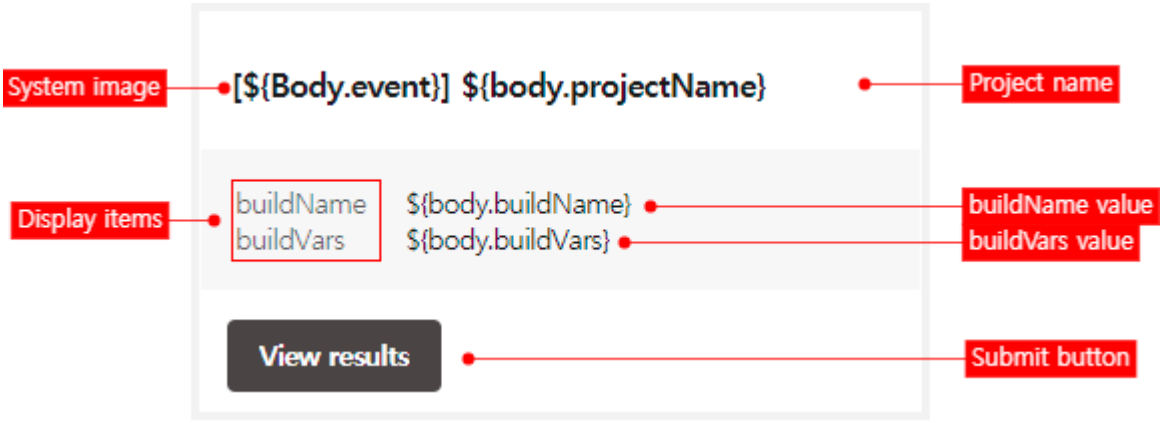
Category		Key	Value	Reference
API URL			http://${JENKINS_URL}/job/${selectedProject}/job/develop/buildWithParameters?${selectedSite}	• ${JENKINS_URL}: Enter the Jenkins URL. • ${selectedProject}, ${selectedSite}: Parameter received from the input of the job view start app card
API Method			POST	
Parameter	**Header**	Content-Type	application/json	
		Authorization	Basic ${base64(concat(jenkinsId,':',jenkinsPw))}	${jenkinsId}, ${jenkinsPw}: Parameter received from the input of the job view start app card

```
{
    "type": "AdaptiveCard",
    "body": [
        {
            "type": "TextBlock",
            "text": "${selectedProject} has been executed. Wait for the results message.",  // ${selectedProject}: Sends the project name that was selected earlier.
            "wrap": true
        }
    ],
```

```
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3"
}
```

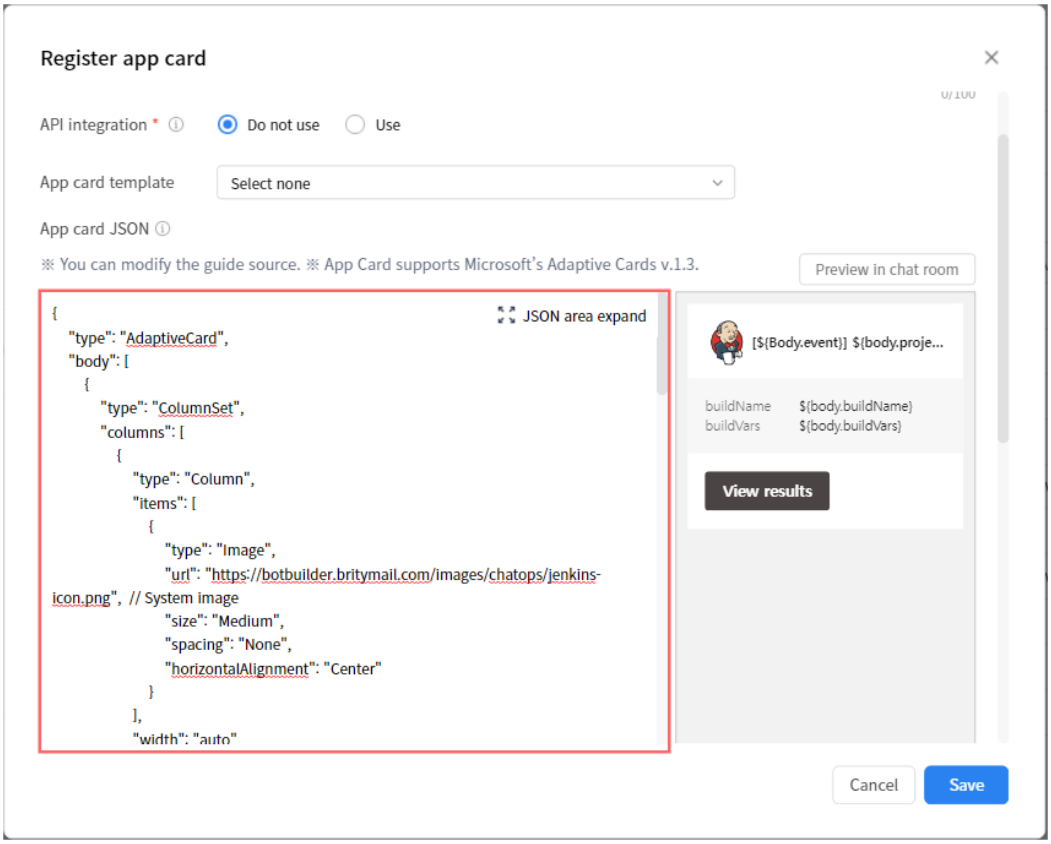
Registering the job execution results app card and webhook

You can register an app card and link it with a webhook according to the following tutorial to create an app card that can receive the events generated in the Jenkins system via the webhook URL.



To register an app card and link it with a webhook, complete the following steps:

1. On the bot details screen, click **App card** > **App cards** > **Register**.
2. In the "Register app card" pop-up window, enter the app card name.
3. Refer to the following JSON to fill in the **App card JSON** field and click **Save**.



```
{
    "type": "AdaptiveCard",
    "body": [
        {
            "type": "ColumnSet",
            "columns": [
                {
                    "type": "Column",
                    "items": [
                        {
                            "type": "Image",
                            "url":
"https://aibuilder.britymail.com/images/chatops/jenkins-icon.png",  //
System image
                            "size": "Medium",
                            "spacing": "None",
                            "horizontalAlignment": "Center"
                        }
                    ],
                    "width": "auto"
                },
                {
                    "type": "Column",
                    "items": [
                        {
                            "type": "TextBlock",
                            "text": "[${titleCase(body.event)}]
${body.projectName}",  // Project name
                            "fontType": "Default",
                            "size": "Medium",
                            "weight": "Bolder"
                        }
                    ],
                    "width": "stretch",
                    "spacing": "Small",
                    "height": "stretch",
                    "verticalContentAlignment": "Center",
                    "style": "default"
                }
            ]
```

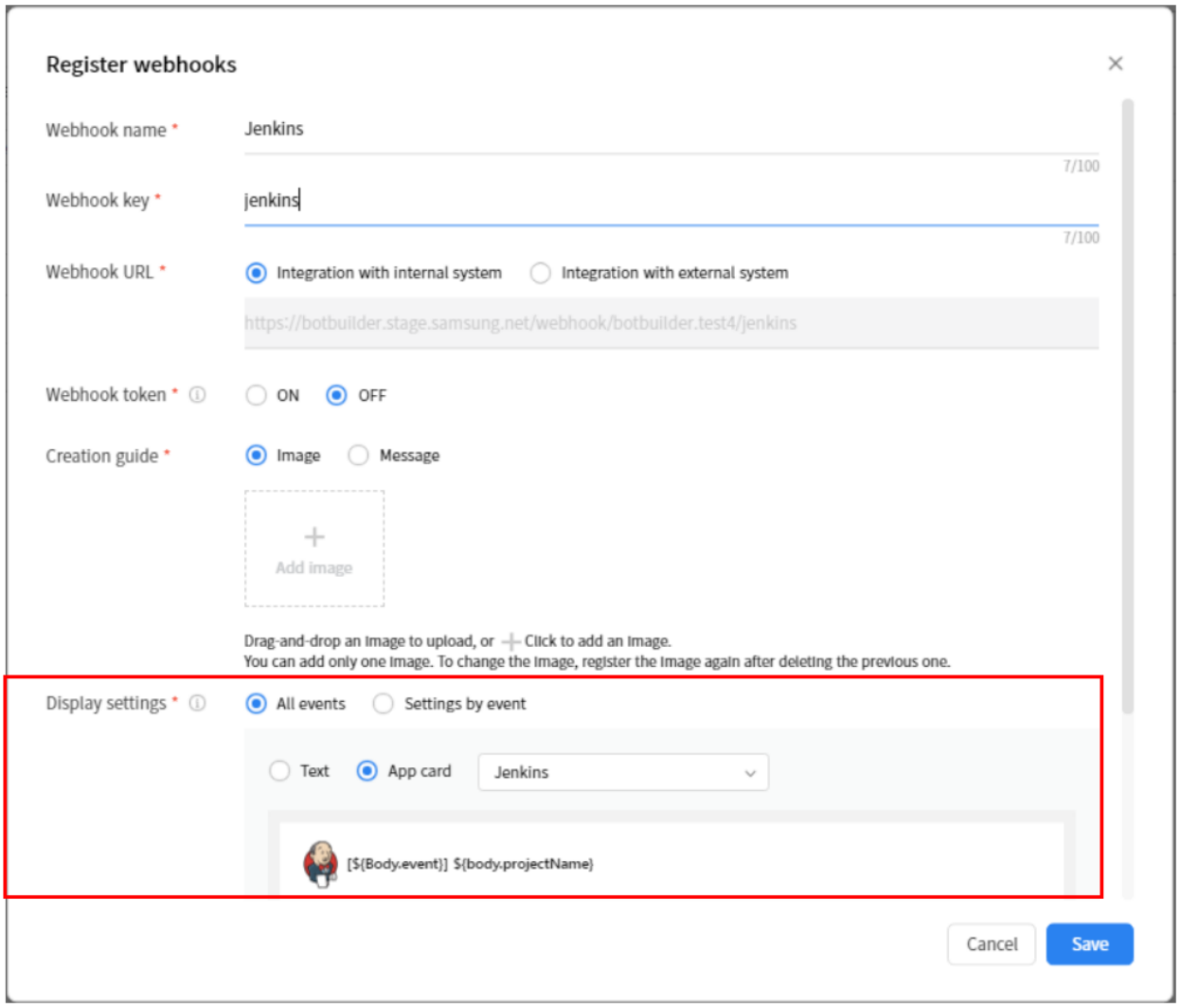
```
        },
        {
            "type": "Container",
            "items": [
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "buildName",  // Display
item, "buildName"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${body.buildName}", //
buildName value
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                },
                {
```

```
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "buildVars",  // Display item, "buildVars"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "auto",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${body.buildVars}",  // buildVars value
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None",
                            "wrap": true
                        }
                    ],
                    "spacing": "None"
                }
            ],
            "spacing": "Large",
            "style": "emphasis",
            "bleed": true
        },
```

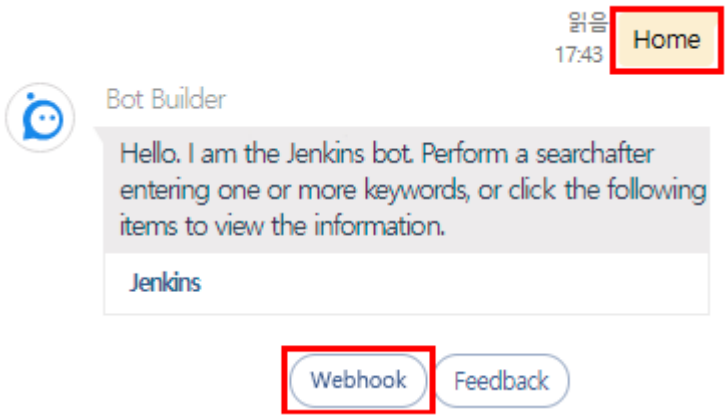
```
114          {
115              "type": "ColumnSet",
116              "spacing": "Padding",
117              "columns": [
118                  {
119                      "type": "Column",
120                      "width": "auto",
121                      "items": [
122                          {
123                              "type": "ActionSet",
124                              "actions": [
125                                  {
126                                      "type": "Action.Submit",  //
Submission button
127                                      "title": "View results",  //
Submission button name
128                                      "style": "positive",
129                                      "data": {
130                                          "intent": "appCard",
131                                          "entity": {
132                                              "id":
"XXXXXXXX-XXX-XXX-XXX-XXXXXXXXXXXX",  // Enter the detailed results app card ID
(ID of the target app card).
133                                              "vars": {
134                                                  "url": {
135                                                      "resultNum":
"${substring(body.buildName,1)}",
136                                                      "project":
"${body.projectName}"
137                                                  }
138                                              }
139                                          }
140                                      }
141                                  }
142                              ]
143                          }
144                      ],
145                      "spacing": "Medium"
146                  }
147              ]
```

```
148             }
149         ],
150         "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
151         "version": "1.3"
152     }
```

4. On the bot details screen, click **Webhook** > **Webhooks** and link the created app card with a new or existing webhook.

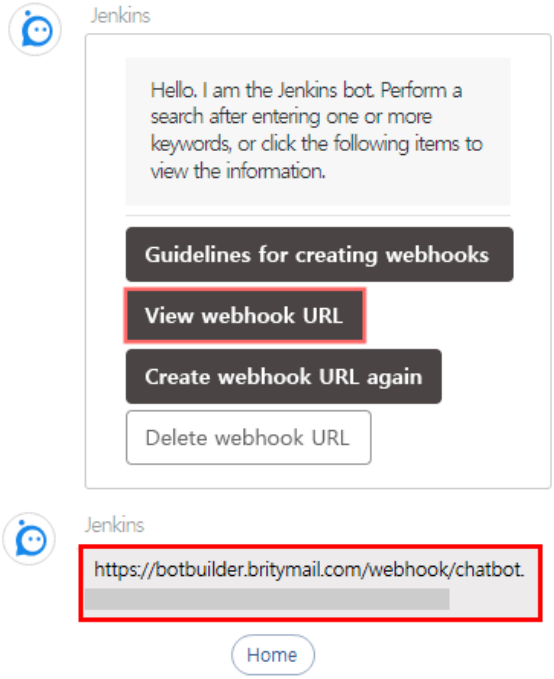


5. In the top-left corner of the menu, click **Deploy** to reflect the registered webhook in the Knox Teams.
6. In the "Deployment details" pop-up window, fill in the **Key revision history** field and click **Deploy**.
7. Access Knox Teams, click on the chatroom that you want to generate a webhook URL for.
8. In the chat window, enter “Home” and click **Webhook** that will be displayed at the bottom of the chat window.



> Note
> **Webhook** will be displayed at the bottom of the chat window only when the bot has a “scenario + webhook” structure.

9. In the webhook window that is displayed in the chat room, click **View webhook URL** and view the created webhook URL.
   - If multiple registered webhooks exist, click ⌄ next to the webhook and click **View webhook URL**.
   - On the bot details screen, click **Webhook** > **Webhook URL Search**. Then, on the "Webhook URL Search" screen, you can view and manage the created webhook URL.



Registering the job execution details API and app card

You can create and register an API and an app card according to the following tutorial to create an app card that can fetch detailed information about the received Jenkins job results.



Refer to the Registering the job view start API and app card tutorial and the following example to register the API and app card. Scenario registration is not required at this stage.

- The following example displays the detailed job results by utilizing the value received from the Jenkins webhook event.
- The received messages may vary depending on the response code of the API transmission.
  - 200 ~ 399: Sends specified app cards.
  - 400 ~ 500: Sends a message saying, "Proper messages cannot be provided due to an unstable connection with the external service."

Category		Key	Value	Reference
API URL			http://${JEKNINS_URL}/job/${project}/${resultNum}/api/json	• ${JENKINS_URL}: Enter the Jenkins URL. • ${project}, ${resultNum}: The value received from `entity.vars` of the job execution results app card
API Method			GET	
Parameter	**Header**	Content-Type	application/json	
		Authorization	Basic ${base64(concat(jenkinsId,':',jenkinsPw))}	${jenkinsId}, ${jenkinsPw}: The value received from the input of the job execution results app card

```
{
    "type": "AdaptiveCard",
    "body": [
        {
            "type": "Container",
```

```
            "items": [
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "Project",  // Display item,
"Project"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${fullDisplayName}",  //
Project name
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                },
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
```

```
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "Result",  // Display item,
"Result"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${result}",  // Result
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                },
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "Time",  // Display item, "Time"
```

```
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text":
"${addToTime(formatEpoch(timestamp/1000),9,&apos;Hour&apos;,&apos;YYYY-MM-DD
HH:mm:ss&apos;)}",    // Convert based on the KST.
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                },
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "Duration",  // Display item,
"Duration"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
```

```
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${duration}(ms)",  // Duration
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
                }

            ],
            "spacing": "Large",
            "style": "emphasis",
            "bleed": true
        }
    ],
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3"
}
```

Configuring webhook events in Jenkins

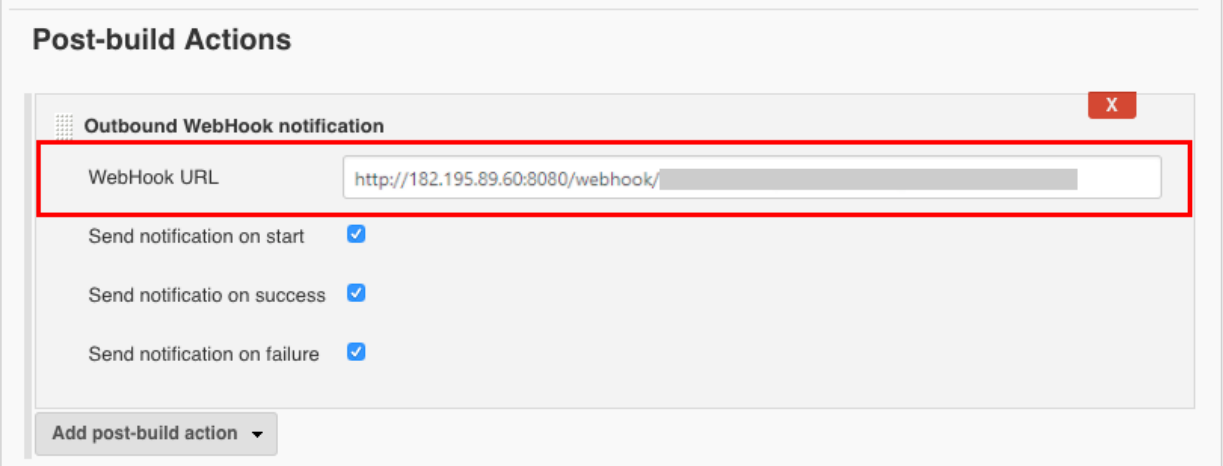
You can utilize a plugin installed to a specific Jenkins job to generate a webhook event in order to deliver the job execution results in the Jenkins system to the ChatOps bot.

To configure a webhook event, complete the following steps:

1. In the **Jenkins Job** > **Configure** > **Add post build action** menu, select “Outbound WebHook

notification.”

2. Enter the webhook URL you created in Registering the job execution results app card and webhook to configure the job execution results to create a webhook event.

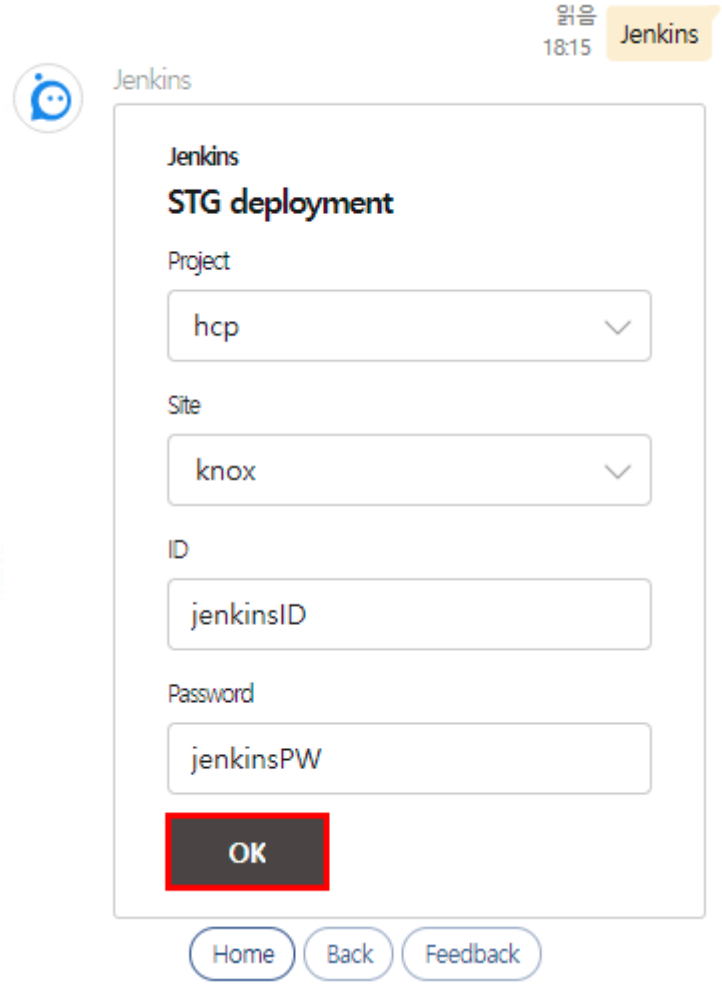


Testing

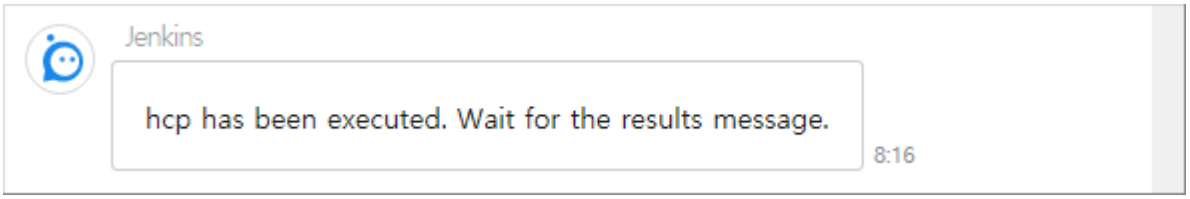
You can access Knox Teams and test the ChatOps bot to ensure that it operates properly according to the scenario.

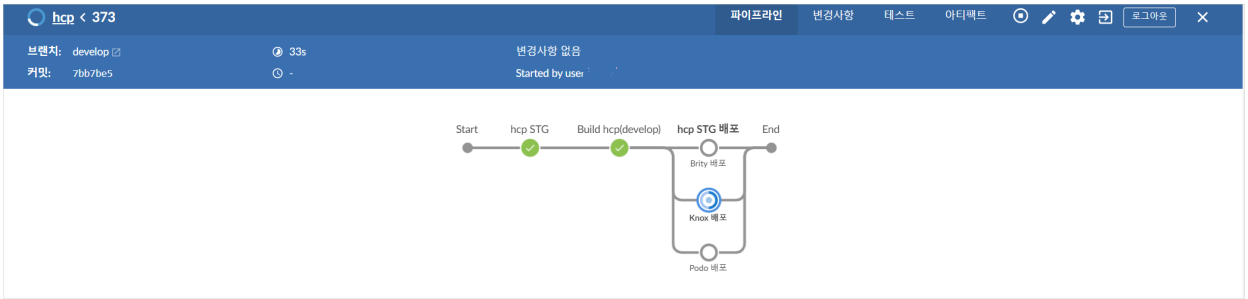
To test the API and app card registered for a remote deployment in the Jenkins system, complete the following steps:

1. Access Knox Teams.
2. Click **Knox Bot**, search for the bot’s Knox ID you entered when registering the API and app card, and then double-click the bot.
3. After the configured job has been properly executed in the Jenkins system and the deployment to the bot has been made, click **OK**.



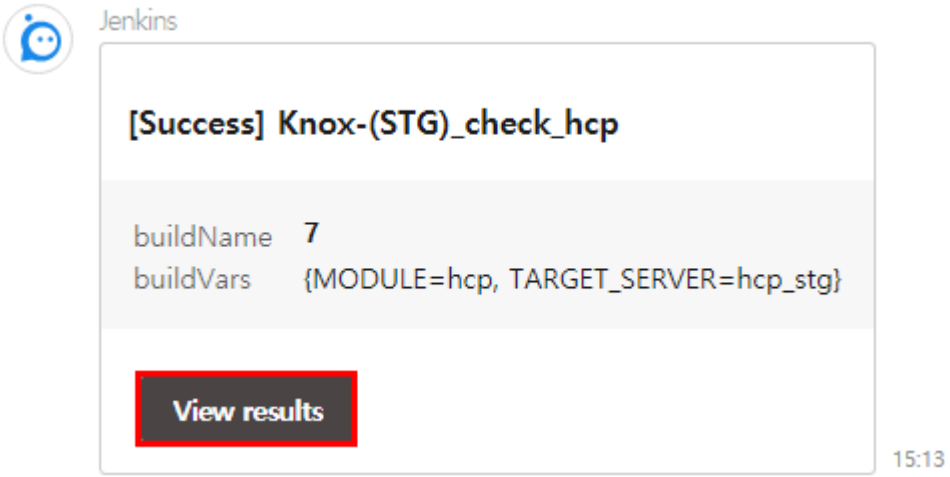
The deployment status will be displayed in the Knox Teams and web browser, and then the deployment execution results will be received.





On the app card received in the Knox Teams, you can click **View results** to view the detailed

deployment execution results via the app card.





3.4 Restarting Jenkins and viewing processes

Through the integration with the Jenkins system, you can configure the ChatOps bot to execute the restart and process jobs defined in the Jenkins system and view the execution results in the ChatOps bot.

Configuring in AI Builder

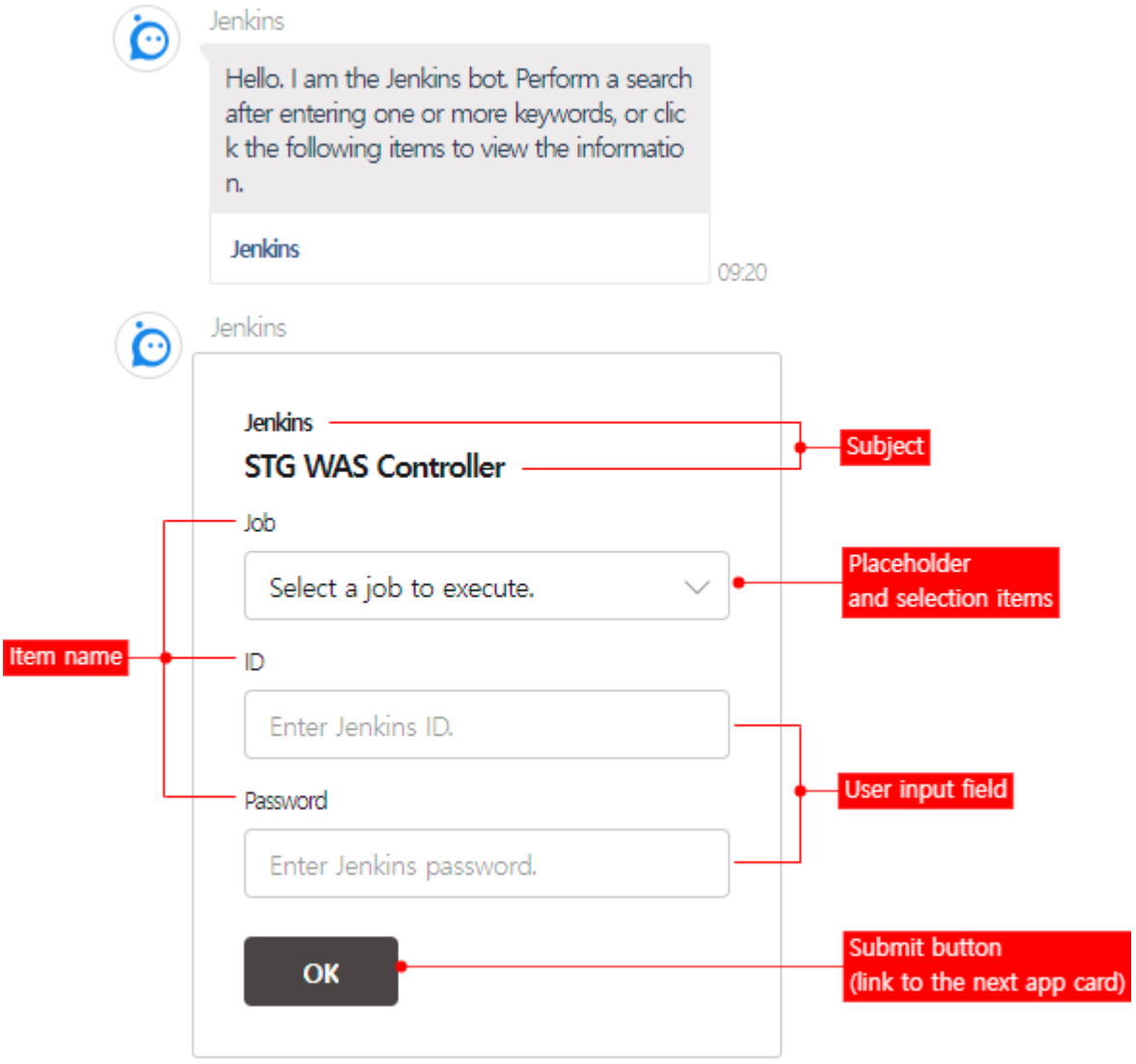
Check the information provided in the tutorial according to the following stages and configure AI Builder to allow the ChatOps bot to remotely execute jobs in a Jenkins system.



> Note
> For more information about the Jenkins API, see Jenkins Remote Access API.

Registering the job view start API and app card

You can register an API and an app card and link them with a scenario according to the following tutorial to create an app card that can access the Jenkins system.



Refer to the Registering the job view start API and app card tutorial and the following example to register the API and app card.

- The following is an example configuration for fetching only jobs that are registered to the Jenkins Categorized View (01.STG).
- The received messages may vary depending on the response code of the API transmission.
  - 200 ~ 399: Sends specified app cards.
  - 400 ~ 500: Sends a message saying, "Proper messages cannot be provided due to an unstable connection with the external service."

Category		Key	Value	Reference
API URL			https://${JENKINS_URL}/view/01.STG/api/json?tree=jobs[name,url]	${JENKINS_URL}: Enter the Jenkins URL.
API Method			GET	
Parameter	**Header**	Content-Type	application/json	
		Authorization	Basic ${base64(concat($jenkinsId, ':', $jenkinsPw))}	${jenkinsId}, ${jenkinsPw}: Enter the Jenkins account information.

```
{
    "type": "AdaptiveCard",
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3",
    "body": [
        {
            "type": "TextBlock",
            "text": "Jenkins",  // Title, "Jenkins"
            "wrap": true,
            "fontType": "Default",
            "size": "Small",
            "weight": "Bolder"
        },
        {
            "type": "TextBlock",
            "text": "STG WAS Controller",   // Title, "STG WAS Controller"
            "wrap": true,
            "spacing": "Small",
            "fontType": "Default",
            "size": "Medium",
```

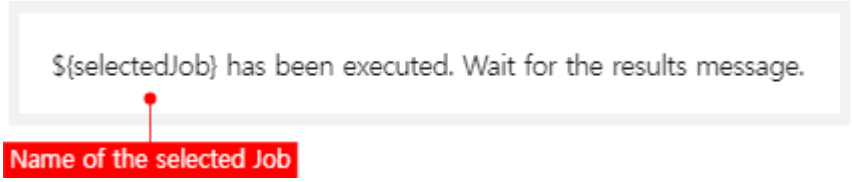
```
21              "weight": "Bolder"
22          },
23          {
24              "type": "TextBlock",
25              "text": "Job",  // Name of the job view item
26              "wrap": true,
27              "spacing": "Medium",
28              "fontType": "Default",
29              "size": "Small"
30          },
31          {
32              "id": "selectedJob",
33              "placeholder": "Select a job to execute.",  // The job to view,
which will be delivered to Jenkins
34              "spacing": "None",
35              "type": "Input.ChoiceSet",
36              "choices": [
37                  {
38                      "$data": "${jobs}",
39                      "title": "${name}",
40                      "value": "${name}",
41                      "$when": "${isMatch(name,&apos;restart|check&apos;)}"
// Syntax for filtering jobs to include only those with “restart” and “check” strings
after searching with the Jenkins API
42                  }
43              ]
44          },
45          {
46              "type": "TextBlock",
47              "text": "ID",  // ID input item name
48              "wrap": true,
49              "spacing": "Medium",
50              "fontType": "Default",
51              "size": "Small"
52          },
53          {
54             "type": "Input.Text",  // User input field 1
55             "placeholder": "Enter Jenkins ID.",  // User input phrase 1
56             "id": "jenkinsId",  // ID that will be delivered to Jenkins
57             "spacing": "None",
```

```
58              "maxLength": 50
59          },
60          {
61              "type": "TextBlock",
62              "text": "Password",  // Item name, "Password"
63              "wrap": true,
64              "spacing": "Medium",
65              "fontType": "Default",
66              "size": "Small"
67          },
68          {
69              "type": "Input.Text",  // User input field 2
70              "placeholder": "Enter Jenkins password.",  // User input phrase
2
71              "id": "jenkinsPw",  //  Password that will be delivered to Jenkins
72              "spacing": "None",
73              "maxLength": 100
74          },
75          {
76              "type": "ColumnSet",
77              "spacing": "Padding",
78              "columns": [
79                  {
80                      "type": "Column",
81                      "width": "auto",
82                      "items": [
83                          {
84                              "type": "ActionSet",
85                              "actions": [
86                                  {
87                                      "type": "Action.Submit",  // Submission
button
88                                      "title": "OK",  // Submission button
name
89                                      "style": "positive",
90                                      "data": {
91                                      "intent": "appCard",
92                                      "entity": {
93                                          "id":
"XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"  // Enter the ID for the app card to decide
```

```
whether to execute Jenkins (ID of the target app card).
                                            }
                                        }
                                    }
                                ]
                            }
                        ],
                        "spacing": "Medium"
                    }
                ]
            }
        ]
    }
```

Registering the job execution/result APIs and app cards

You can create and register an API and an app card according to the following tutorial to execute the jobs received from a Jenkins system in the ChatOps bot and create an app card that can display the execution status and results in the Knox Teams.



Refer to the Registering the job view start API and app card tutorial and the following example to register the API and app card. Scenario registration is not required at this stage.

- The following example executes the jobs registered in Jenkins Pipeline.
- The received messages may vary depending on the response code of the API transmission.
  - 200 ~ 399: Sends specified app cards.
  - 400 ~ 500: Sends a message saying, "Proper messages cannot be provided due to an unstable connection with the external service."

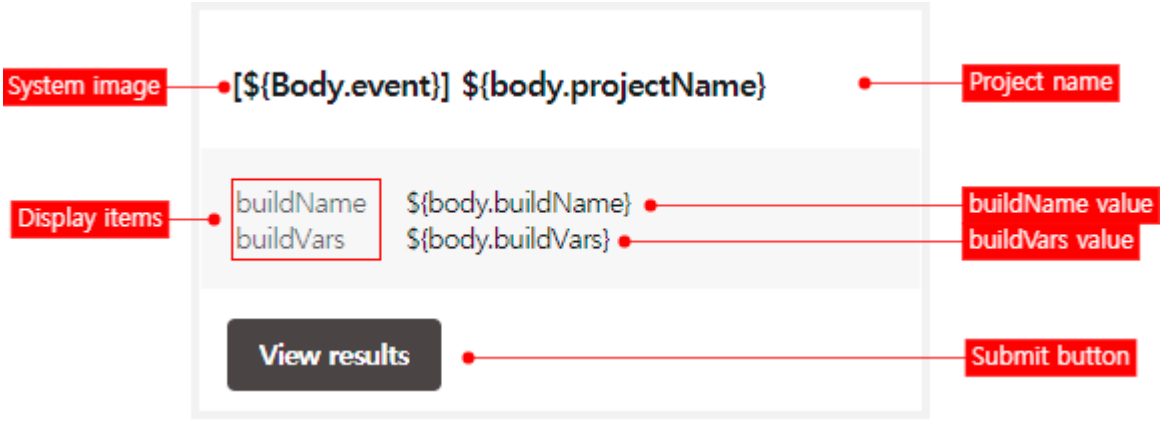
Category	Key	Value	Reference
API URL		https://${JENKINS_URL}/job/${selectedJob}/buildWithParameters	• ${JENKINS_URL}: Enter the Jenkins URL. • ${selectedJob}: Parameter received from the input of the job view start app card

Category		Key	Value	Reference
API Method			POST	
Parameter	**Header**	Content-Type	application/json	
		Authorization	Basic ${base64(concat(jenkinsId, ':', jenkinsPw))}	${jenkinsId}, ${jenkinsPw}: Parameter received from the input of the job view start app card

```
{
    "type": "AdaptiveCard",
    "body": [
        {
            "type": "TextBlock",
            "text": "${selectedJob} has been executed. Wait for the results message.",  // ${selectedJob}: Send the job name that was selected earlier.
            "wrap": true
        }
    ],
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3"
}
```

Registering the job execution results app card and webhook

You can register an app card and link it with a webhook according to the following tutorial to create an app card that can receive the events generated in the Jenkins system via the webhook URL.



Refer to the Registering the job execution results app card and webhook tutorial and the following

example to register the app card and link it with a webhook, and then create a webhook URL.

```
{
    "type": "AdaptiveCard",
    "body": [
        {
            "type": "ColumnSet",
            "columns": [
                {
                    "type": "Column",
                    "items": [
                        {
                            "type": "Image",
                            "url":
"https://aibuilder.britymail.com/images/chatops/jenkins-icon.png",  // System
image
                            "size": "Medium",
                            "spacing": "None",
                            "horizontalAlignment": "Center"
                        }
                    ],
                    "width": "auto"
                },
                {
                    "type": "Column",
                    "items": [
                        {
                            "type": "TextBlock",
                            "text": "[${titleCase(body.event)}]
${body.projectName}",  // Project name
                            "fontType": "Default",
                            "size": "Medium",
                            "weight": "Bolder"
                        }
                    ],
                    "width": "stretch",
                    "spacing": "Small",
                    "height": "stretch",
                    "verticalContentAlignment": "Center",
                    "style": "default"
```

```
                }
            ]
        },
        {
            "type": "Container",
            "items": [
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "buildName",  // Display item,
"buildName"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "stretch",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${body.buildName}", //
buildName value
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None"
                        }
                    ],
                    "spacing": "None"
```

```
                },
                {
                    "type": "ColumnSet",
                    "columns": [
                        {
                            "type": "Column",
                            "width": "85px",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "buildVars",  // Display item, "buildVars"
                                    "fontType": "Default",
                                    "isSubtle": true
                                }
                            ]
                        },
                        {
                            "type": "Column",
                            "width": "auto",
                            "items": [
                                {
                                    "type": "TextBlock",
                                    "text": "${body.buildVars}",  // buildVars value
                                    "spacing": "None",
                                    "fontType": "Default",
                                    "wrap": true,
                                    "maxLines": 3
                                }
                            ],
                            "spacing": "None",
                            "wrap": true
                        }
                    ],
                    "spacing": "None"
                }
            ],
            "spacing": "Large",
            "style": "emphasis",
```

```
112                 "bleed": true
113             },
114             {
115                 "type": "ColumnSet",
116                 "spacing": "Padding",
117                 "columns": [
118                     {
119                         "type": "Column",
120                         "width": "auto",
121                         "items": [
122                             {
123                                 "type": "ActionSet",
124                                 "actions": [
125                                     {
126                                         "type": "Action.Submit",  // Submission
button
127                                         "title": "View results",  // Submission
button name
128                                         "style": "positive",
129                                         "data": {
130                                             "intent": "appCard",
131                                             "entity": {
132                                                 "id":
"XXXXXXXX-XXX-XXX-XXX-XXXXXXXXXXXX",  // Enter the job execution result details app
card ID (ID of the target app card).
133                                                 "vars": {
134                                                     "url": {
135                                                         "resultNum":
"${substring(body.buildName,1)}",
136                                                         "project":
"${body.projectName}"
137                                                     }
138                                                 },
139                                                 "isDownload": "true"  //
Download the detailed job execution results.
140                                             }
141                                         }
142                                     }
143                                 ]
144                             }
```

```
                    ],
                    "spacing": "Medium"
                }
            ]
        }
    ],
    "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
    "version": "1.3"
}
```

> Note
> To download detailed job execution results, you must add the `"isDownload": "true"` parameter.

Registering a job execution details API

You can create and register an API to download a file containing the detailed Jenkins job execution results.

Refer to the Registering the job view start API and app card tutorial and the following example to register the API. Scenario registration is not required at this stage.

- The following example allows for downloading of the detailed job results by utilizing the values received from the Jenkins webhook event.
- The received messages may vary depending on the response code of the API transmission.
  - 200 ~ 399: Sends specified app cards.
  - 400 ~ 500: Sends a message saying, "Proper messages cannot be provided due to an unstable connection with the external service."

Category		Key	Value	Reference
API URL			https://${JEKNINS_URL}/job/${project}/${resultNum}/api/json	• ${JENKINS_URL}: Enter the Jenkins URL. • ${project}, ${resultNum}: The value received from `entity.vars` of the job execution results app card
API Method			GET	
Paramet	**Header**	Content-	application/json	

Category		Key	Value	Reference
er		Type		
		Authorization	Basic ${base64(concat(jenkinsId, ':', jenkinsPw))}	${jenkinsId}, ${jenkinsPw}: The value received from the input of the job execution results app card

Configuring webhook events in Jenkins

You can configure the server to utilize a plugin installed to a specific Jenkins job and generate a webhook event in order to deliver the job execution results in the Jenkins system to the ChatOps bot.

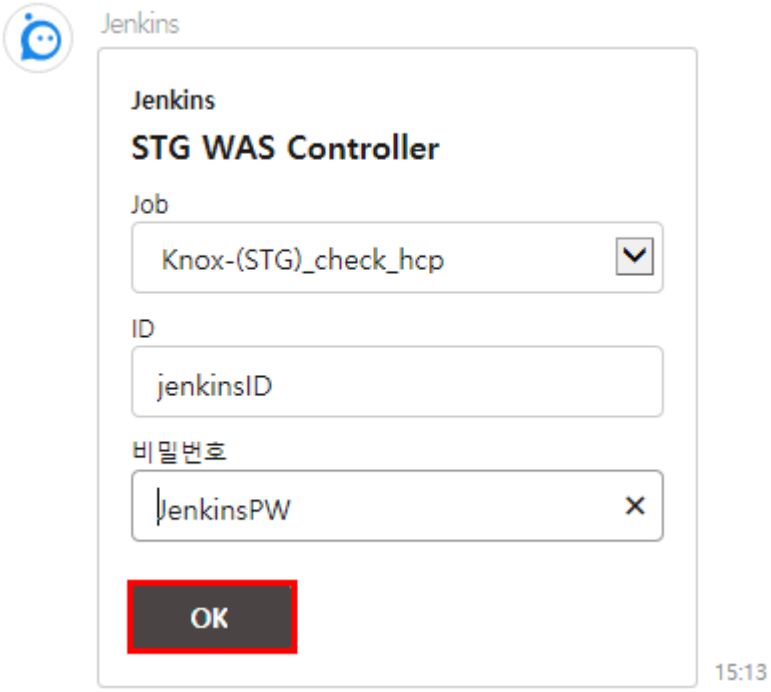
Refer to the Configuring webhook events in Jenkins tutorial and enter the URL you created in Registering the job execution results app card and webhook in the webhook URL field to allow job execution results to generate webhook events.

Testing

You can access Knox Teams and test the ChatOps bot to ensure that it operates properly according to the scenario.

To test the API and app card registered for the Jenkins restart and process view, complete the following steps:

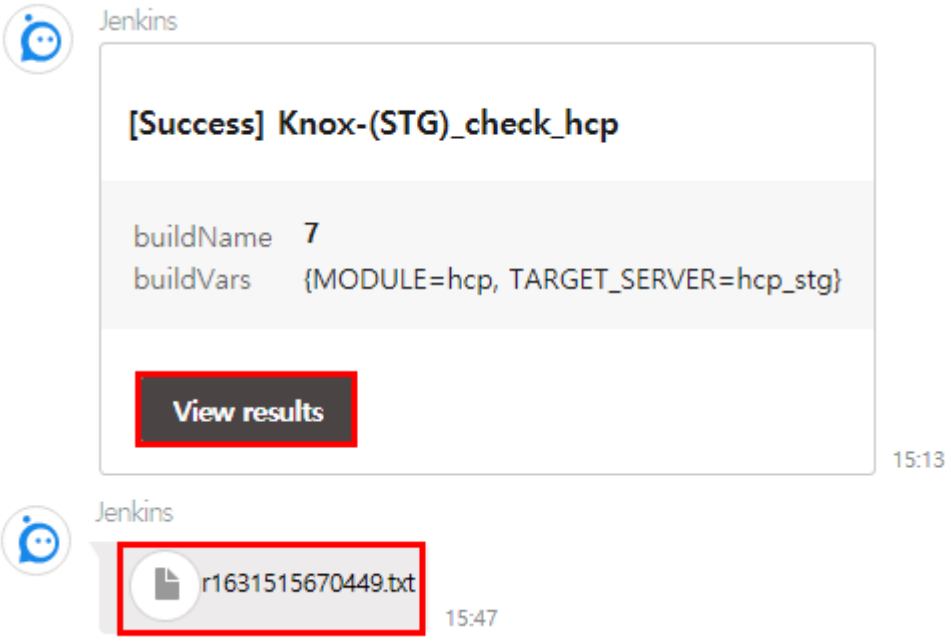
1. Access Knox Teams.
2. Click **Knox Bot**, search for the bot's Knox ID you entered when registering the API and app card, and then double-click the bot.
3. After the configured job has been properly executed in the Jenkins system and the deployment to the bot has been made, click **OK**.

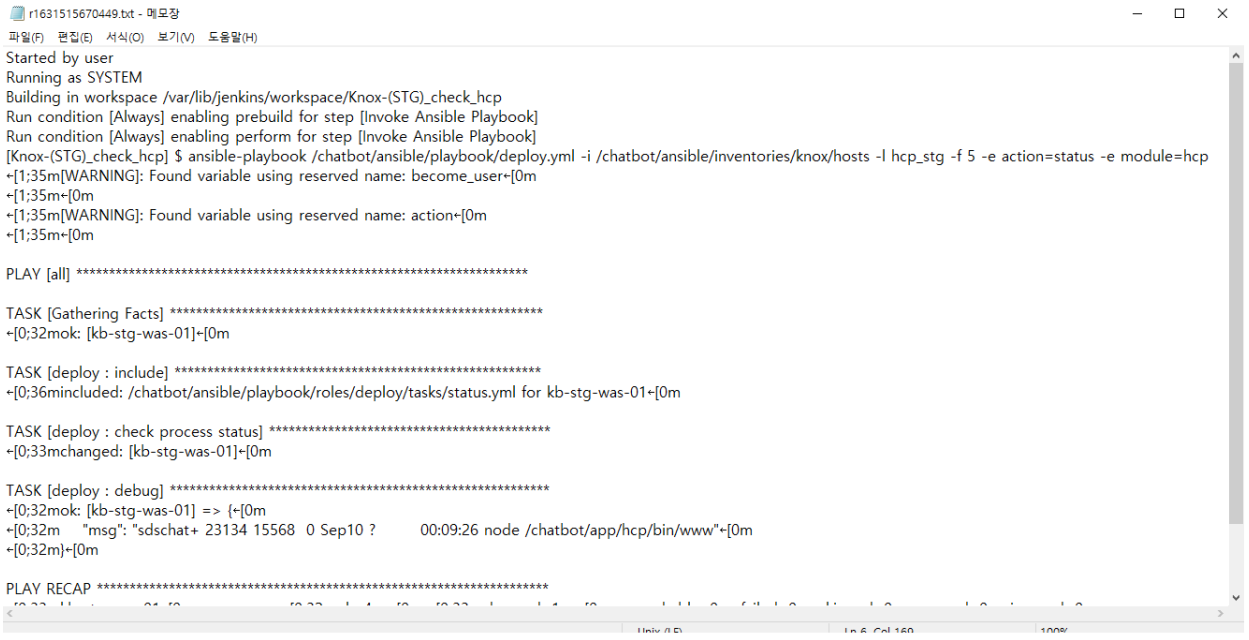


The restart and process view job status will be displayed in the Knox Teams and web browser, and then the deployment execution results will be received.



On the app card received in the Knox Teams, you can click **View results** to view the execution results file.



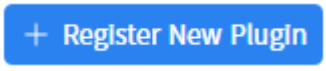


Part VIII.
Plugin

1.
Creating a plugin

1.1 What is plugin?

Plugin is a function that **interfaces with the API of external system**, calls the API through Knox Copilot and retrieves the results.
Users can **create, register, and manage plugins** in AI Builder.

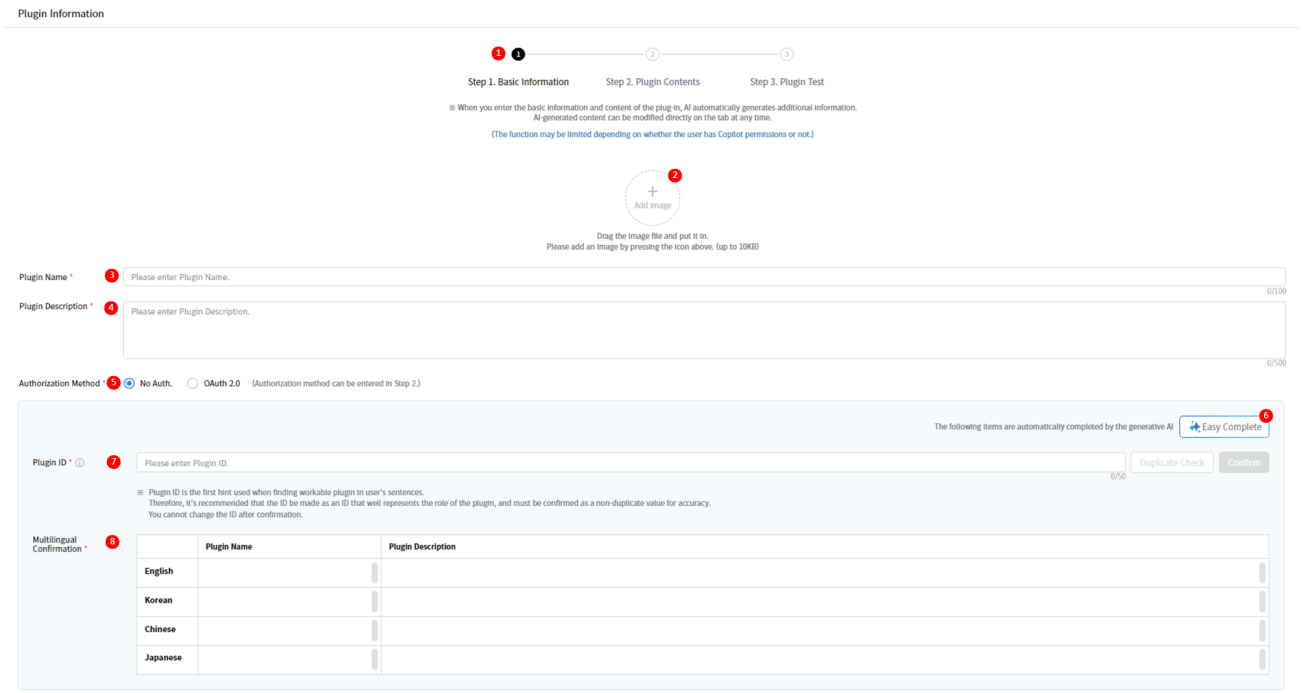
The functions related plugin are provided in the menu below :
Knox AI Builder > Plugin
You can create a new plugin by clicking 

button in the plugin menu, and saving the information in the order below.

※ *The Plugin menu may not be visible depending on the Copilot permissions for each company or individual.*
Please contact the administrator of each company for permission to use Copilot.

1.2 Step 1 : Basic Information

This is the step to save the basic information of plugin.



No	Feature	Description
❶	**Indicator**	Indicates which step of the plugin information you are, and click to go to that step. ※ *You need to save step by step from 1 to 3 the first time, after that you can move directly to the step you want.*
❷	**Plugin image**	Upload a plugin profile image to be displayed in Copilot.
❸	**Plugin name**	The plugin name managed by AI Builder.
❹	**Plugin Description**	The plugin description managed by AI Builder. We also recommend that it entered accurately, as **it may affect how accurately Copilot runs this plugin.**
❺	**Auth Type**	Select authorization type of the plugin. If you select 'OAuth' type, you should enter the required ingormation on Step2. ※ 'No Auth' is type for plugins that do not need any authorization.
❻	**Easy Complete**	By clicking, **the plugin ID and multiple languages are automatically entered** based on the plugin name and description above.
❼	**Plugin ID**	It's an ID that identifies a plugin. It should be indicated about plugin's function and role, because it's one of the hints for finding plugin in Copilot.

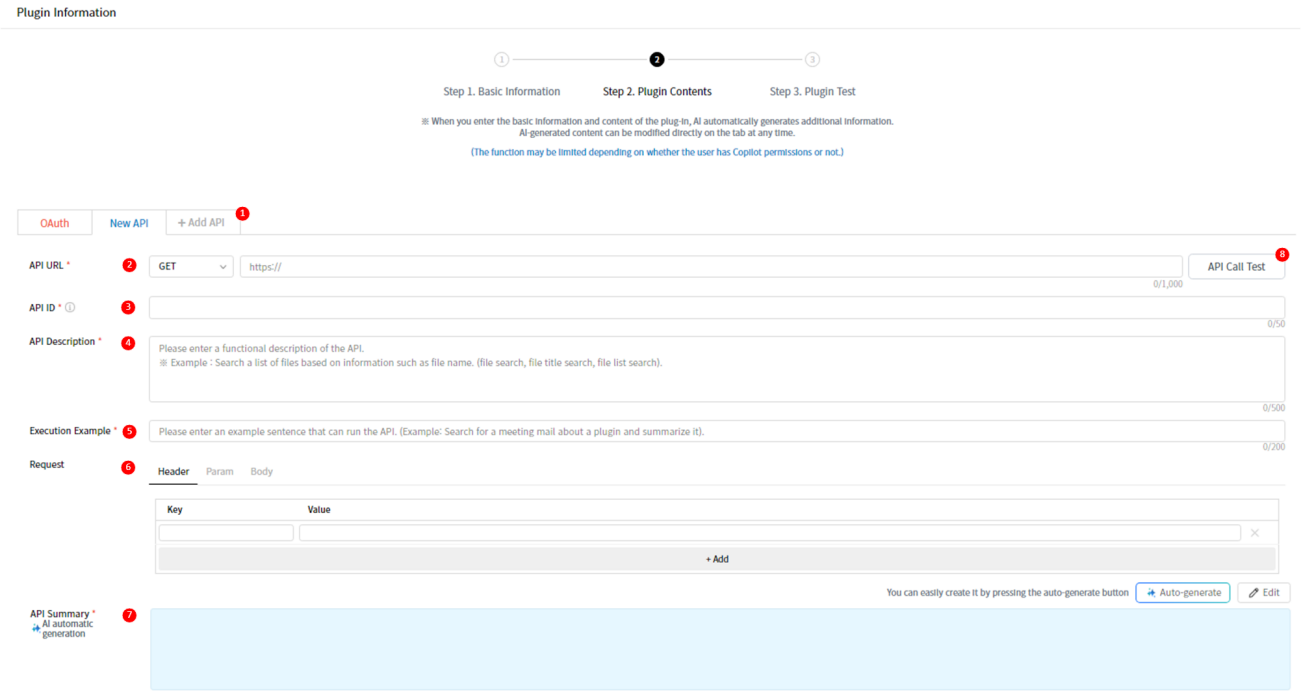
No	Feature	Description
		※ *Rules : Less than 50char., UpperCamelCase* ※ *ex) Keyword Search Plugin → SearchKeyword* It must be a unique value, so a **"Confirm" is required after checking for duplicates.** You can enter through "Easy Complete".
❽	**Multilingual Confirmation**	Plugin name and description that will be displayed based on the language settings of Knox Messenger/Teams. You can enter through "Easy Complete".

Tip

"Easy Complete" is a function that LLM generates and recommends the plugin ID and multilingual fields which are needed to search plugin in Copilot, based on plugin name and description entered by users.
After generated automatically, you can check and edit them.

1.3 Step 2 : Plugin Contents (API)

This is the step to API information to be executed through the plugin.



No	Feature	Description
❶	**API list**	You can manage the list of APIs included in the plugin with a tab. ※ *Up to 5 APIs can included in a plugin.*
❷	**API URL**	The call method type and URL of the API.
❸	**API ID**	The ID of the API to call.
❹	**API Description**	The description of the API. We recommend that it should be entered accurately, as **it may affect the accuracy with which Copilot executes the corresponding API**.
❺	**Execution Example**	An example Sentence that can call corresponding API Through Copilot. It's used as a hint for Copilot to find the API, so it is recommended that should be entered accurately.
❻	**Request**	The request fields of the API. You can manage Authorization / Header / Param / Body by each section, and enter them according to the API structure and information you want to enter. Please check each item above for details.
❼	**API Summary**	It's a hint for execution that API in Copilot. It can be automatically generated through "Auto-generate", and you can edit directly by clicking "Edit".

No	Feature	Description
❽	**API Call Test**	Test whether the API is called successfully. Please refer to 2.1 **API Call Test** for details.

Tip

"API summary auto-generate" is a function in which LLM automatically generates API summary using the saved API information.
LLM generates it in a structure that Copilot can understand well when generating, so it's recommended to edit only the content without structure when you want to edit it.

Note

For the API to be called successfully from Copilot, **the firewall between the Copilot and the system must be disabled** using the information below in Knox Portal > DC Manager.

- Source : 182.195.40.15~16
- Target : target system

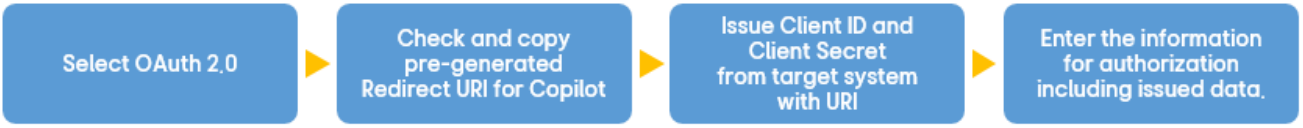
※ Depending on the business site, it may be necessary to additionally disable the inbound firewall of the target system. Please contact the network manger at your business site.

Authorization Information

Enter authorization information according to the selected value in Step1
※ *The OAuth tab is displayed only user select 'OAuth 2.0' on Step1.*

OAuth 2.0

It's needed if the API uses OAuth authorization.
You can enter it as follows.



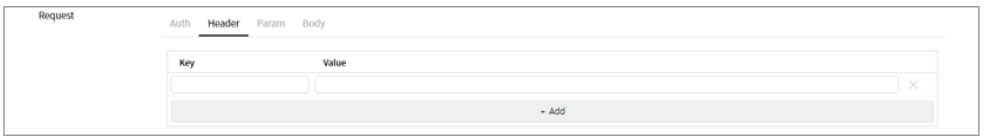
> Tip
> Pre-entered recommended values for authorization other than Client ID and Client Secret.



번호	항목	설명
❶	**Basic Data**	Client ID, Client Secret, error code for authorization, etc.
❷	**Rule**	Grant type, Login URL, etc.
❸	**Token**	Token issue URL, required fields for issue
❹	**Refresh**	Token re-issue URL, required fields for issue
❺	**Headers**	Header information

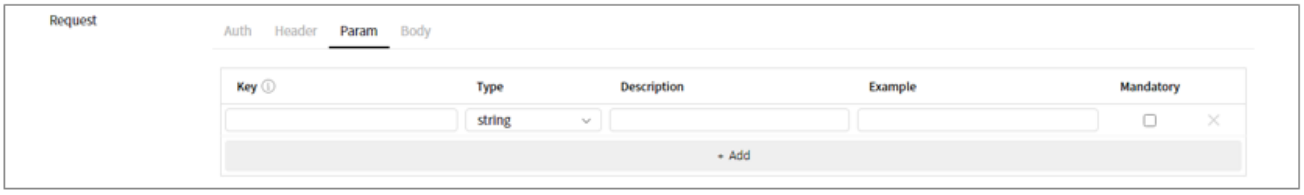
Request : Header

Header values of the API.
It's entered **key-value structure**, and you can **add row as much as needed by "+ Add"** button.



Request : Param

Parameter values of the API.
Enter the Key(name), Type, Description, Example, and **check "Mandatory" if it is mandatory**.
You can **add row as much as needed by "+ Add"** button.

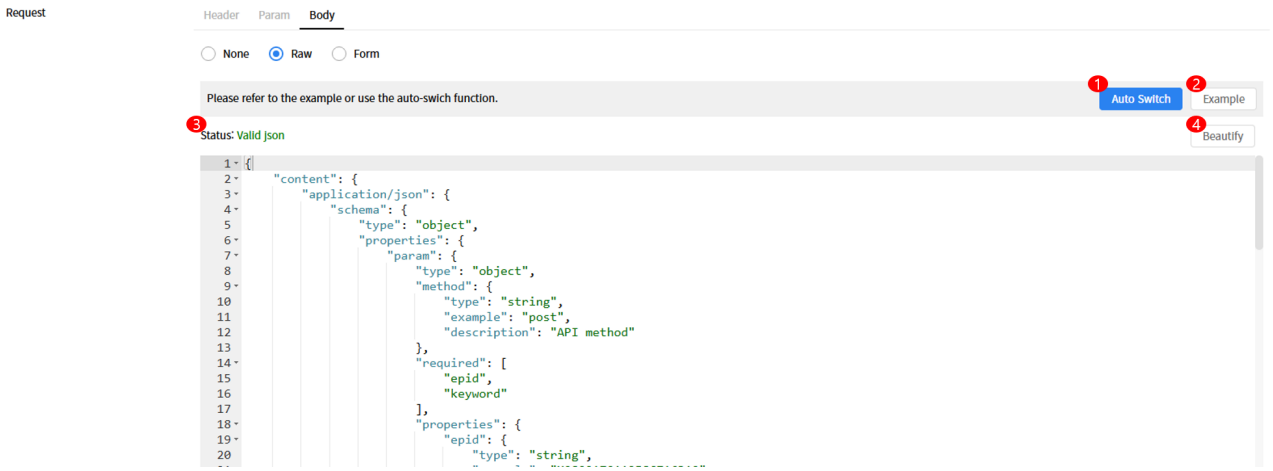


> Tip
> Knox ID, email, epid, companyCode, subOrgCode, and deptCode are predefined user information in Copilot that can be selected and used.
> Enter other items directly.

Request : Body

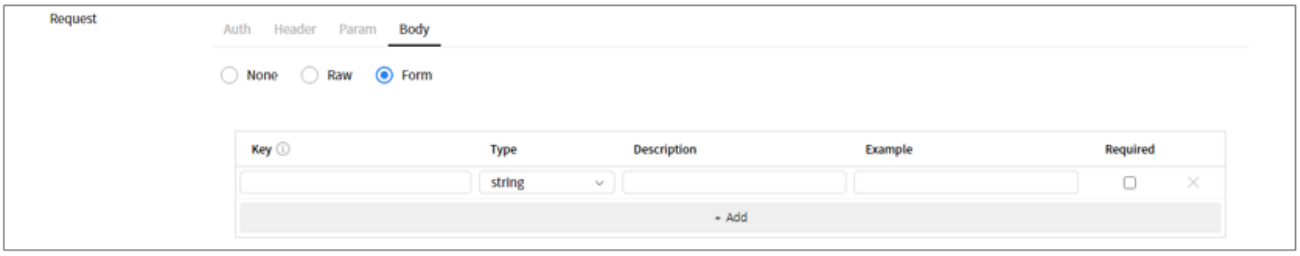
Body values of the API.
You can **choose the input type between raw and form**.

- **Raw type**



번호	항목	설명
❶	**Auto Switch**	Automatically converts the input code to fit the open api spec.
❷	**Example**	You can check the Raw input example. It is used as a reference.
❸	**Status**	You can check the validation results of the currently entered raw code.
❹	**Beautify**	Used when beautifying codes.

- **Form type**

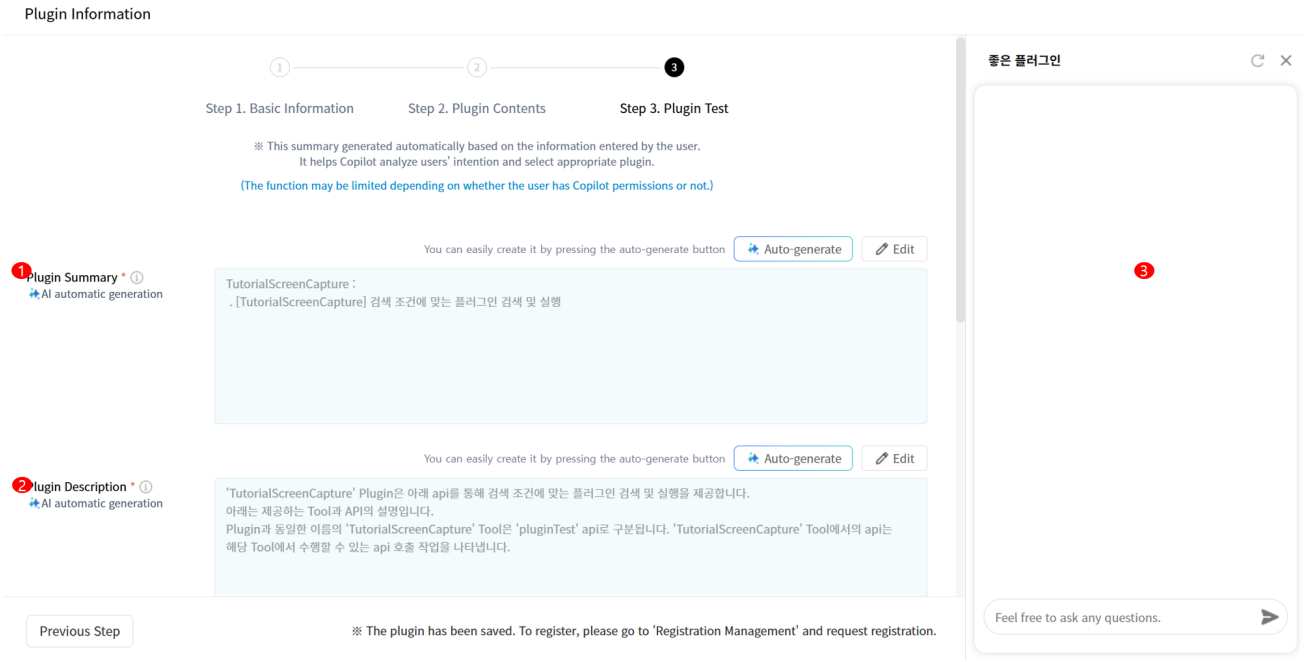


1.4 Step 3 : Check summary and test

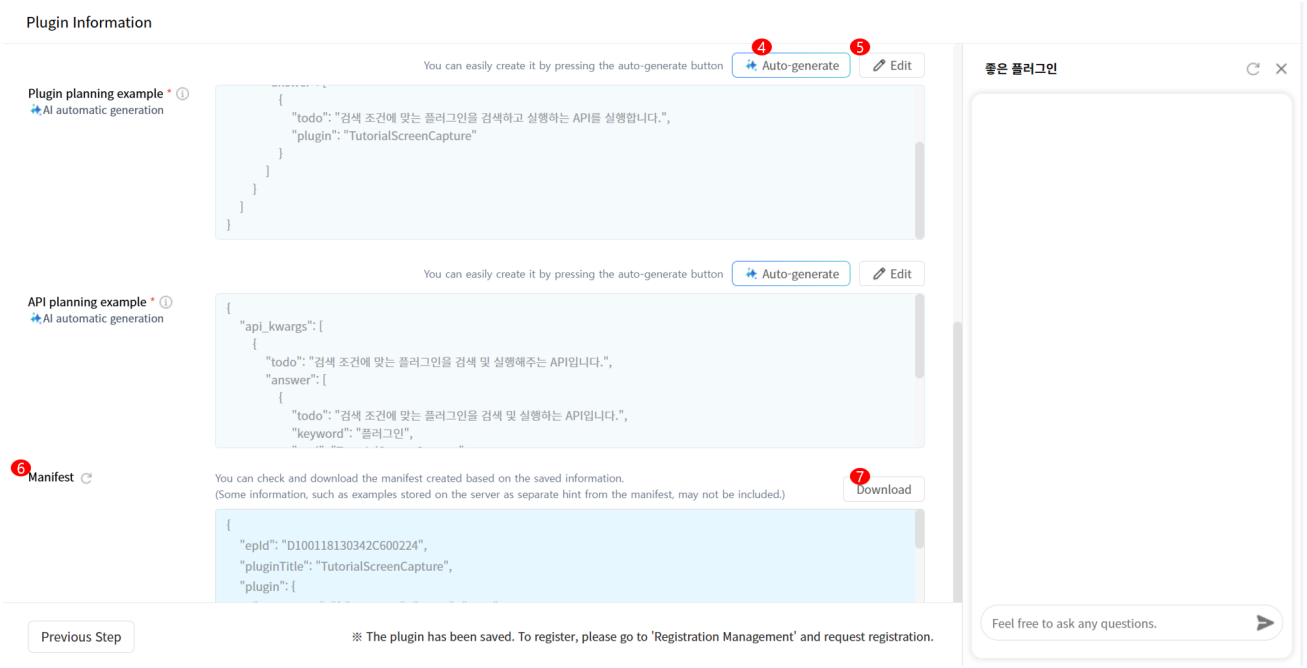
This is the step to check and test the summary and description of the plugin.
"Plugin Summary" and "Plugin Description" are important hints for selecting and running a plugin in Copilot.
They're generated based on the information entered by user in Step 1 and 2.

1) In case of entering the information at the first time
When you save the information from Step 1 to Step 2 for the first time, **"Plugin Summary" and "Plugin Description" are automatically generated by LLM**.
User checks them and **clicks "Edit" button if necessary**.
2) In case of editing information
If you have modified the information in Step 1 and 2, you can update "Plugin Summary" and "Plugin Description" through **"Auto-generate" button** each.
User checks them and **clicks "Edit" button if necessary**.

> Tip
> **"Auto-generate"** is a function in which LLM automatically generates Plugin's summary and description.
> **LLM generates it in a structure that Copilot can understand well when generating**, so it's recommended to edit only the content without structure when you want to edit it.



No	Feature	Description
❶	**Plugin Summary**	The summary of the plugin and is one of the important hints for selecting and running the plugin in Copilot. Please refer above for input method.
❷	**Plugin Description**	The description of the plugin and is one of the important hints for selecting and running the plugin in Copilot. Please refer above for input method.
❸	**Simulator**	Similar to Copilot, you can test the plugin. For testing through the simulator, please refer to 2.2 **Using Simulator**.



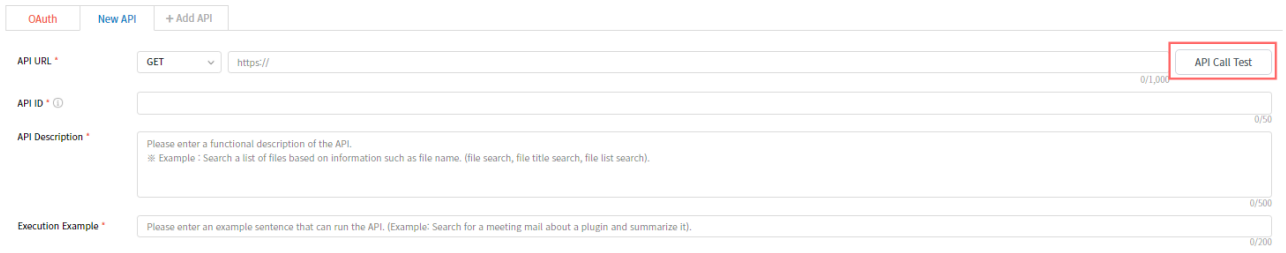
No	Feature	Description
❹	**Auto-Generate**	AI Builder provides auto-generate function for each item. When there are modifications in Steps 1 and 2, click the button to recreate the planning information with the changes.
❺	**Edit**	You can modify the planning information directly.
❻	**Manifest**	You can check the full plug-in information, such as the plug-in name, description, and authentication method.
❼	**Download**	You can download plugin manifest as a .txt file.

To register the saved plugin in Knox Copilot, c**heck and approval from each company's administrator is required**.
Please refer to 3. Plugin registration management for details.

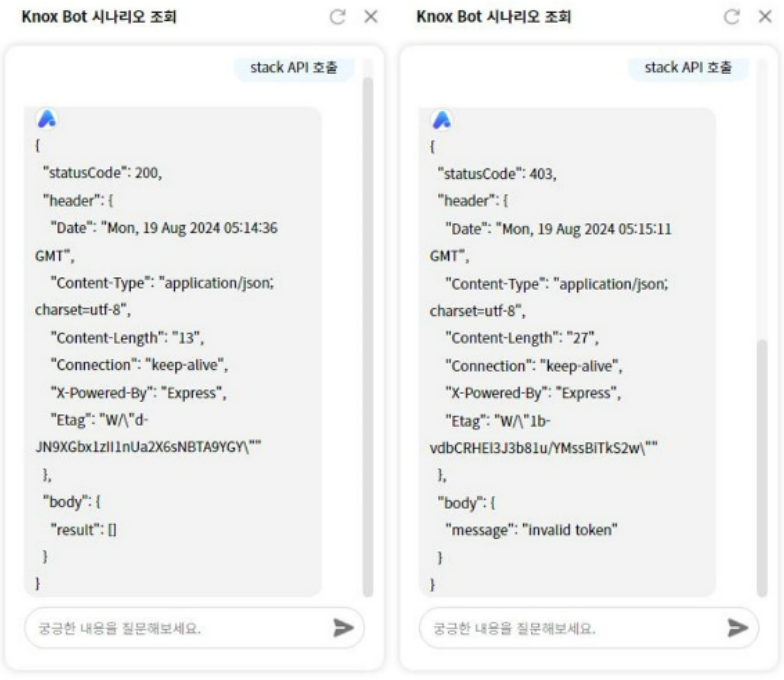
2.
Test with simulator

2.1 API Call Test

After registering the plugin basic information and API information, you can proceed test for the registered API by clicking the "API Call Test" button.



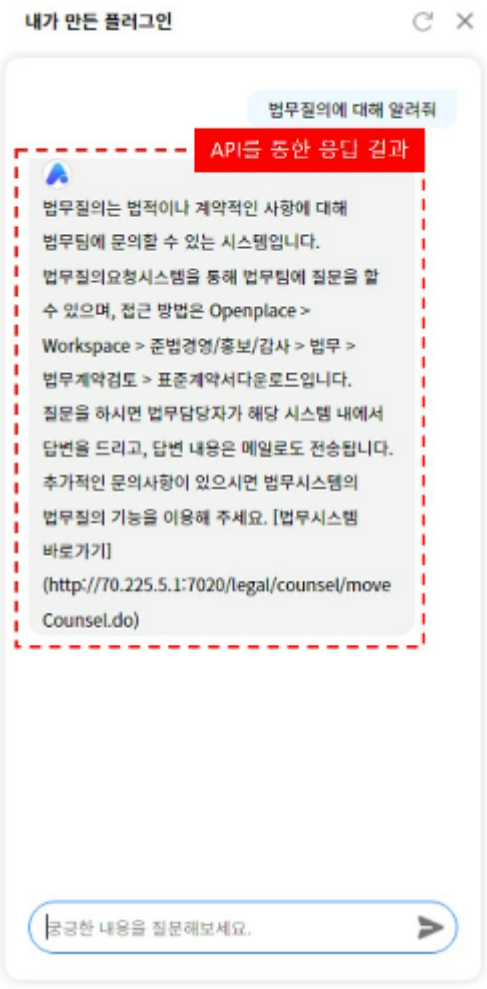
The API call results can be found in the right simulator area.



2.2 Using Simulator

You can ask query through the simulator and check response that created by plugin.
※ The test through the simulator is not supported in case of OAuth 2.0 authorization is included.

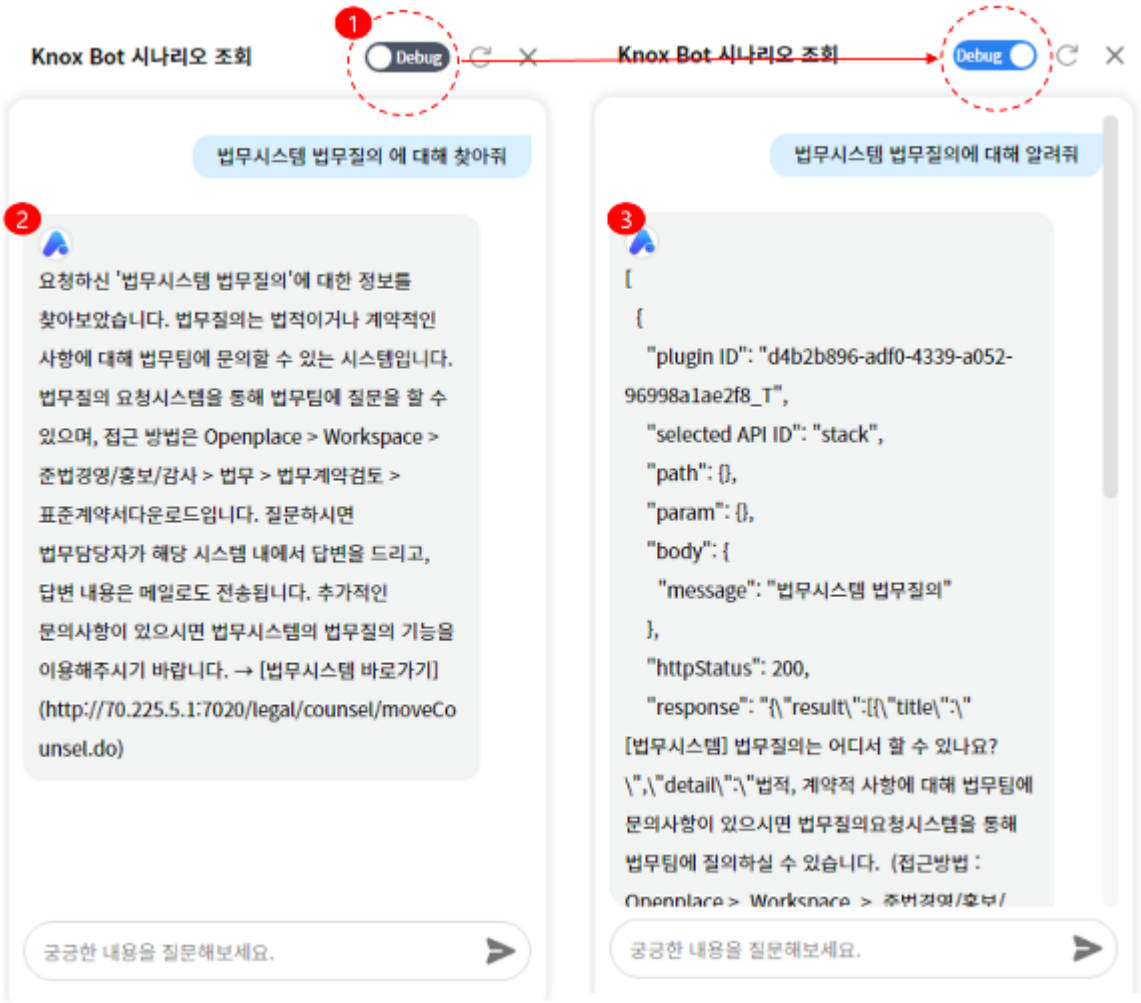
> Tip
> In order to use simulator, the firewall between the Copilot and API server should be opened.



> Note
> If error occured, LLM returns a general response not a specific error message.

Using Debug Mode

You can set 'Debug mode' by clicking the toggle button in the upper right corner of the simulator for additional planning information such as parameter mapping results.



No	Feature	Description
❶	**Debug On/Off**	You can set the 'Debug Mode' by clicking the toggle button.
❷	**Answer**	This is the LLM response result for the question. It is always displayed regardless of whether 'Debug Mode' is set or

No	Feature	Description
		not.
❸	**Plannig Information**	When 'Debug Mode' is setting, it shows the additional plannig information of LLM. - Plugin ID : selected plugin ID - Selected API ID : selected API ID - param : parameter mapping result - Response : Full response from API

3. Plugin registration management

3.1 Register plugin

For using the created plugin in Copilot, the procedures are required as following.

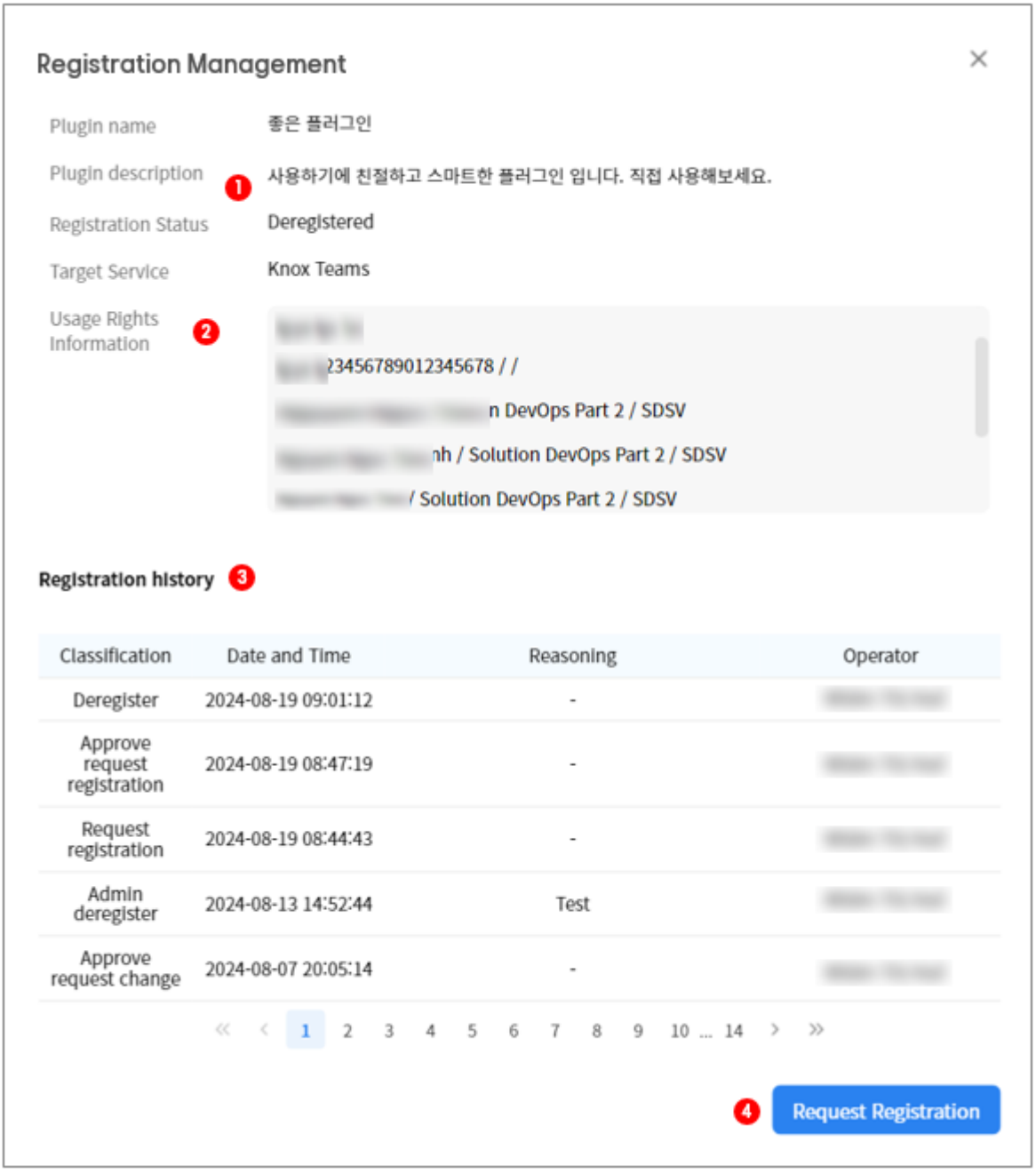


Create plugin

Create a plugin by referring to 1. Creating a plugin.

Request registration

After you create a plugin, you can request registration by clicking **"Registration Management"** at the top of the left menu of the plugin details page.



No	Feature	Description
❶	**Plugin information**	The basic information of the created plugin. (Name, description, status, etc.)
❷	**Permission list**	List of added plugin permission. To add plugin permissions, refer to 4. Plugin permission management.
❸	**Registration history**	List of plugin registration history.
❹	**Request Registration**	Click to request registration to administrator with saved information currently.

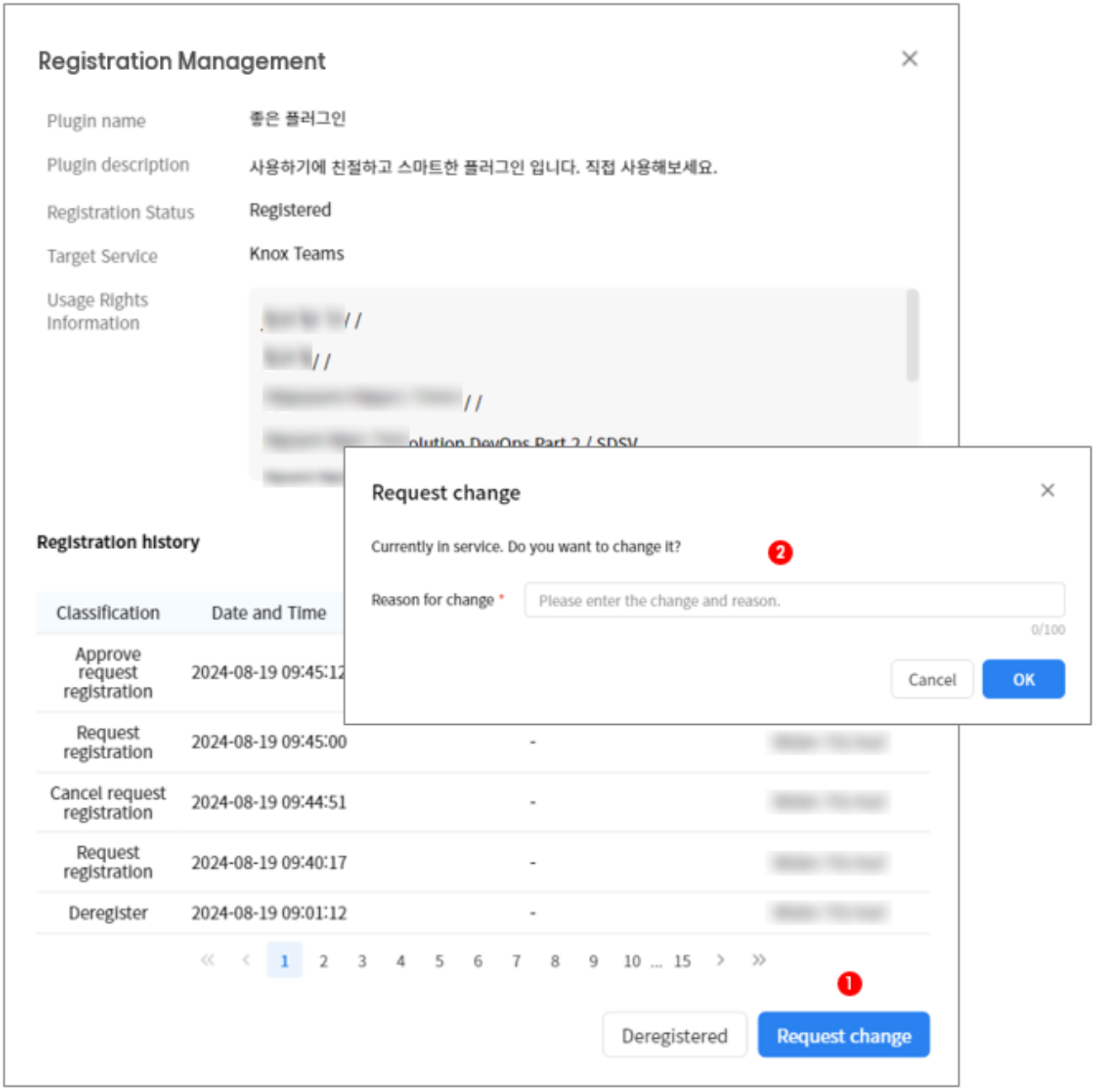
No	Feature	Description
		※ *Your plugin will be not registered only request,* *and it may be refused if it's not suitable upon review by administrator.*

Tip

If you need to cancel your registration request, you can do as follows.
"Registration Management" at the top of the left menu of the plugin details page > Click "Cancel Request" at the bottom of the popup

3.2 Request change

If you modify the plugin that has already been registered, you can request registration with changes. After modifying on the plugin details page, click **"Registration Management"** at the top of the left menu of the plugin details page.



No	Feature	Description
❶	**Request change**	Click to request to change as modified contents to administrator. ※ *Until approved, Copilot has the content as before changes.*
❷	**Reason for change**	Enter the reason for the change so that the administrator can refer to it when reviewing.

3.3 Deregister

If you need to deregister a plugin that has already been registered, you can deregister it yourself.
Click **"Registration Management"** at the top of the left menu of the plugin details page, click **"Deregister"** on the popup to deregister.
※ *If you need to register again after deregistration, you should request registration following* *Request registration*

> Caution
> In addition, if administrator determines that the registered plugin should be deregistered temporarily or permanently,
> **they can deregister plugin registered for each company.**

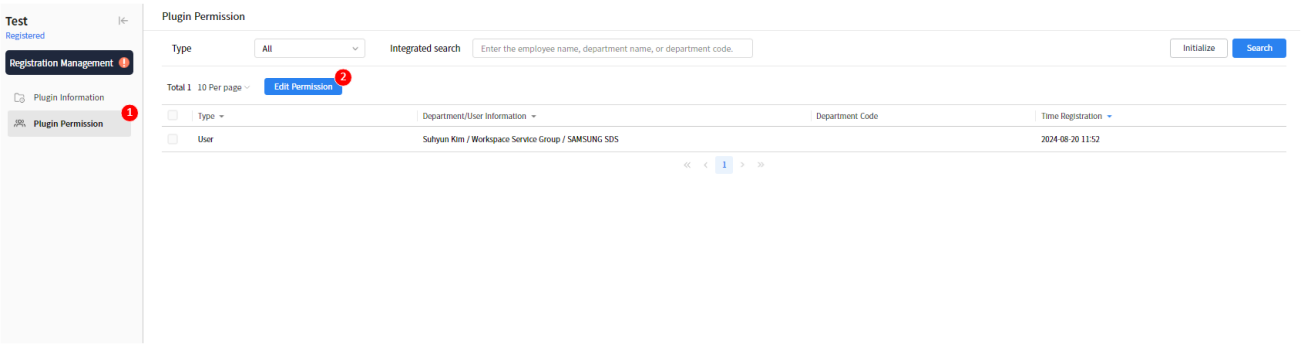
4.
Plugin permission management

4.1 Add Permission

Create Plugin

Create a plugin refer to 1. Creating a plugin

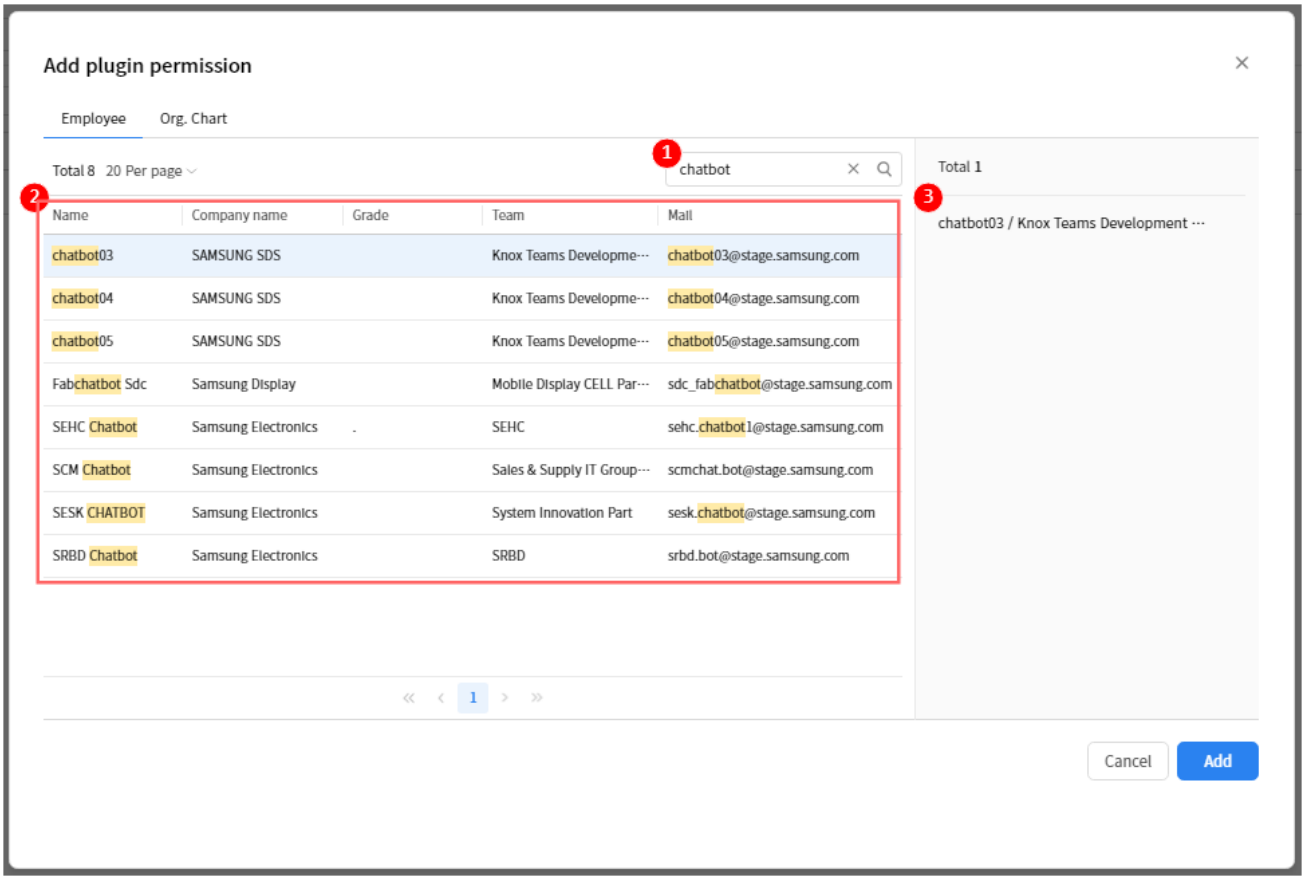
Edit permission



No	Feature	Description
❶	**Plugin Permission**	Move to Plugin's Permission List. The currently registered permission list is shown. Plugin's creator is automatically entered with default permissions and cannot be deleted.
❷	**Edit Permission**	Move to edit mode. User can add or delete permission.

> Tip
> You can edit permissions after api test is completed. If the "**Edit Permission**" button is disabled, please return to the "**Plugin Information**" menu and complete the information registration and testing.

Plugin permission can be added on a personal, departmental, or company basis. You can also search and add employees directly on the Employees tab, or set permissions on a company or department basis on the Organization Chart tab.



No	Feature	Description
❶	**Search Box**	Search by entering the employee name, department name, or email
❷	**Search Result**	Search results matched to the keywords.
❸	**Add User**	Click any data in the list(2), it added to right side area(3). Click "Add" button to complete the addition to permission list.

Request Registration

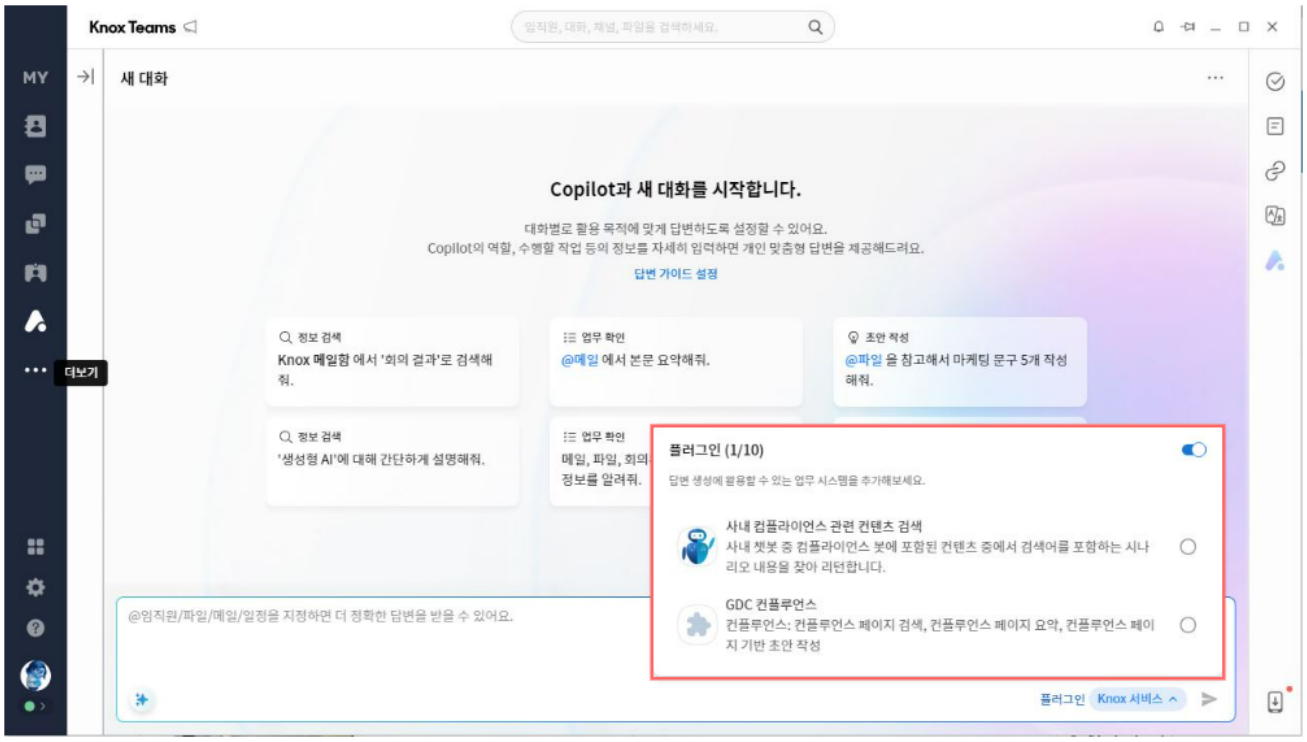
After finishing editing plugin permission, request registration to each company's administratror refer to 3. Plugin registration management

5.
Using plugin in messenger

Log in to http://www.samsung.net/ with your Knox account and run Knox Messenger/Teams.

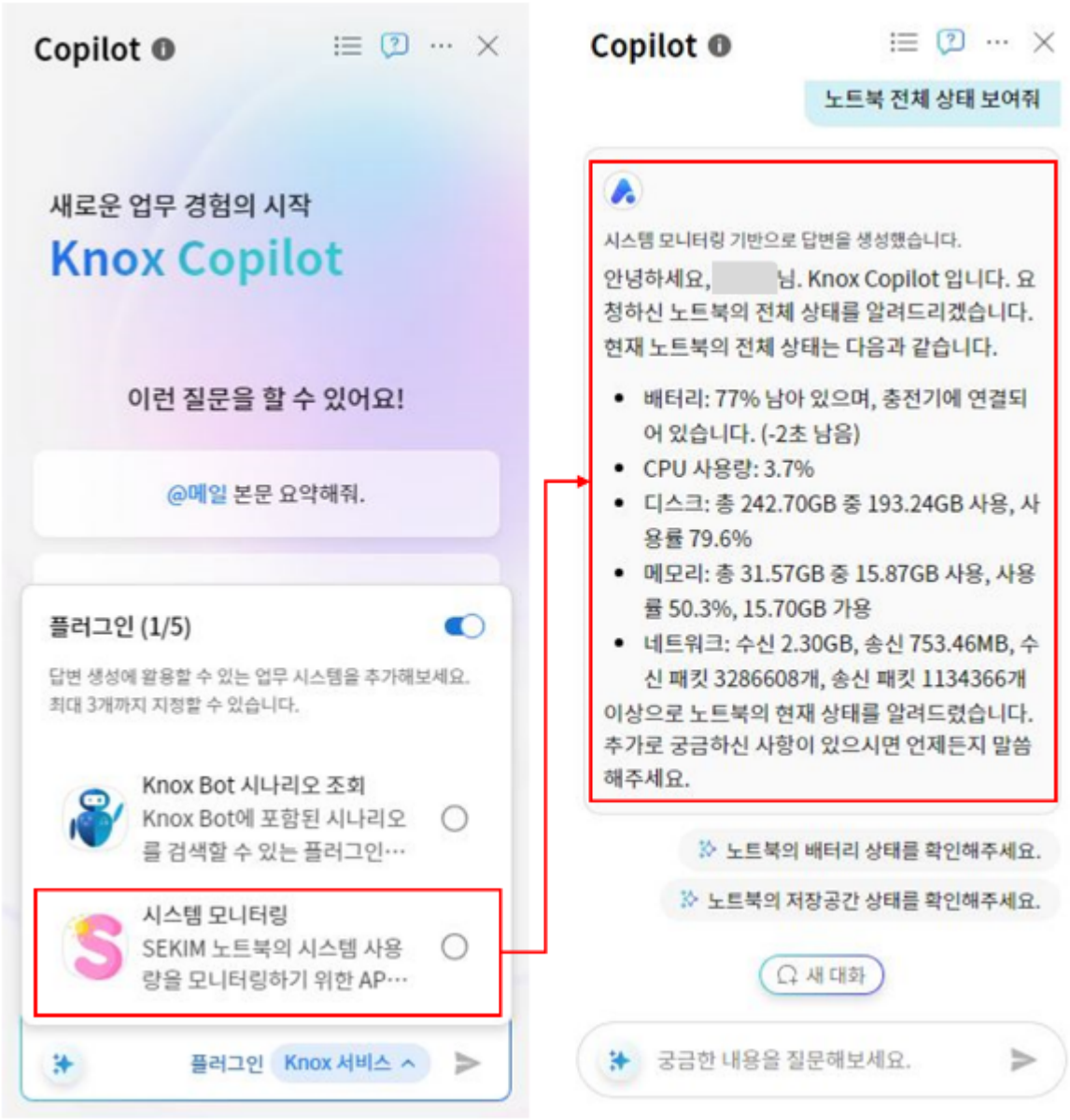
> Tip
> Only plugins in "Registered" status are shon Knox Copilot's plugin list.
> If the plugin is not visible on the list, Please check ① Plugin's status ② Authorization information

Click the Copilot icon to satart a chat with Knox Copilot. When you focus on input area, you can see list of authorized plugins.



If you checked plugin and ask some query, then Knox Copilot gives you an answer that generated

through that plugin.



Appendix A.
Integrating notifications/polls with an open API

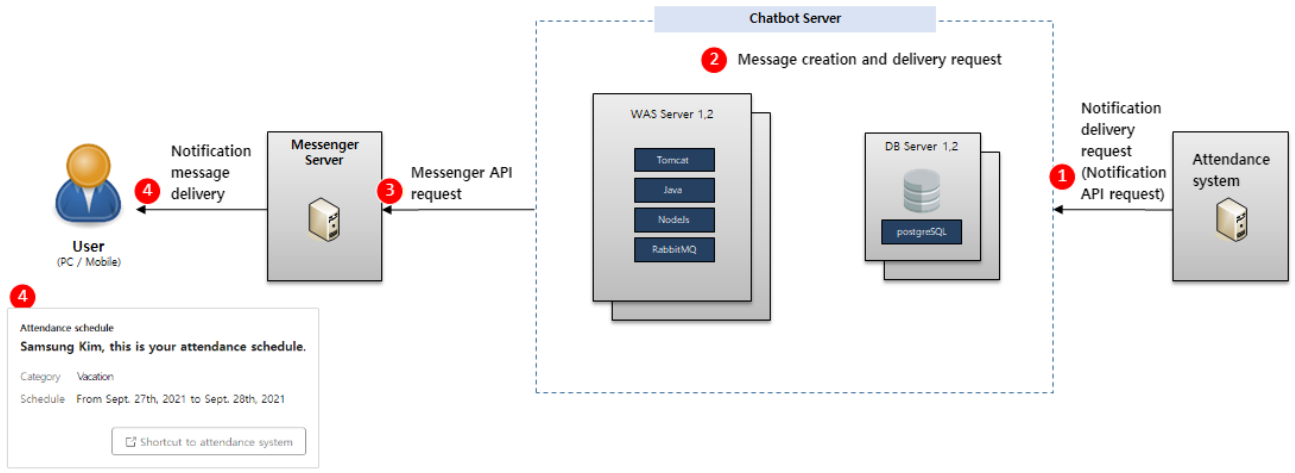
This chapter explains the following topics:

- Utilizing a notification delivery API
- Utilizing a poll delivery API

A.1 Utilizing a notification delivery API

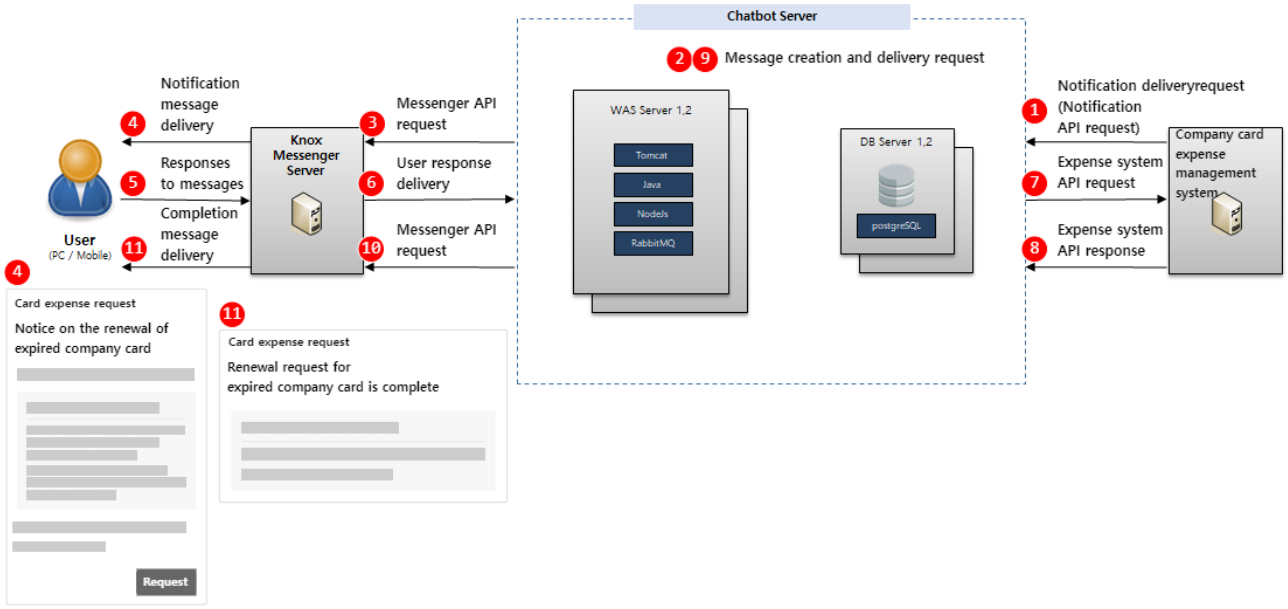
An open API integration allows you to send chatbot notifications directly on a legacy system. Chatbots can also send responses via the legacy system.
On the legacy system, you can send notifications in the app card or text message format to specific users when they are required.

Attendance schedule notifications



No.	Item	Description
❶	Notification API request	The attendance system calls an API to deliver notifications.
❷	Message creation and delivery request	The chatbot server creates messages and requests the delivery.
❸	Messenger API request	The chatbot server calls the messenger API.
❹	Notification message delivery	Notification messages are delivered to users.

Company card expiration notification and renewal request



No.	Item	Description
❶	Notification API request	The company card expense management system calls an API to deliver notifications.
❷	Message creation and delivery request	The chatbot server creates messages and requests the delivery.
❸	Messenger API request	The chatbot server calls the messenger API.
❹	Notification message delivery	Notification messages are delivered to users.
❺	Responses to messages	Users can respond to messages.
❻	User response delivery	User responses are delivered to the chatbot server.
❼	Expense system API request	The chatbot server calls the company card expense management system API.
❽	Expense system API	The chatbot server responds to the company card expense

No.	Item	Description
	response	system.
❾	Message creation and delivery	The chatbot server creates messages and requests the delivery.
❿	Messenger API request	The chatbot server calls the messenger API.
⓫	Completion message delivery	Completion messages are delivered to users.

> Note
> The notification delivery API delivers app cards or custom messages to specific users.

Refer to the following notification delivery API specifications for the legacy system integration.

Request		
URL	https://aibuilder.samsung.net/v1/alarm/send	
Method	POST	
Header	Authorization	bearer Token value • It refers to a token for API authentication, rather than the token provided by the Knox system. You can issue this API token on the AI Builder (My Bot > Notif/Poll API > API token mgmt). Only Bot admins can access the API token management menu.
	Content-Type	application/json
	botEnv	Bot Environment E.g., Operation - "botEnv": "prod" E.g., Stage - "botEnv": "stage"
	contentsType	Notification Format E.g., App cards - "contentsType": "appCard" E.g., Texts - "contentsType": "text" E.g. Excel - "contentsTpye" : "excel"
	contentsId	App Card ID (required for app cards) (App Card ID registered in the bot you want to call and can be checked in AI Builder)
	botId	Bot ID (The Knox Bot ID issued by the Knox Dev Center at your request)

Request		
Body	targetUserId	Knox ID of the Recipient User E.g., Deliver the ID in a string if sending to one - "targerUserId": "user1" E.g., Deliver the ID in an array if sending to many - "targerUserId": ["user1", "user2"]
	subject	Notification Subject (*required for texts) If a subject is not specified for an app card, the app card title will be used as the subject
	contents	Notification Content (*required for texts) - Notification content when sending notifications in texts - For excel, paste the html text that fits the excel format
	contentsParams	Data for App Card Substitution Variables (${variable name}) E.g., When substituting ${name} phrases in app cards, write {"name": "Samsung Kim"} Deliver in an object if there is one recipient Deliver in an array if sending different messages for multiple recipients Deliver in an object if sending the same message to multiple recipients
	chatType	(Optional) Messenger Chat Room Type (the default value is "general") The parameter to set one of the two types (general chat room and notice chat room) for messenger chat rooms Use "general” for general chat rooms and "notice" for notice chat rooms ※ Notice chat room: A chat room in Chat Bot that only supports one-way sending and blocks user chats
	target	(Optional) Designating Recipients (the default value is "user") The parameter to designate recipients for messages to users or chat rooms Use "user” to send messages to users and "chatRoomId" to send them to chat rooms
	targetChatRoomId	ChatRoom ID of Recipient Chat Room (required to send messages to chat rooms) Deliver the ID in a string when sending it to one chat room E.g., "targetChatRoomId": "123456789012345678" Deliver the ID in an array when sending it to multiple chat rooms E.g., "targetChatRoomId": ["123456789012345678", "234567890123456789"] ※ ChatRoom ID: Unique value for each messenger chat room

Body example

```
Case 1) Inserting data values into a specific bot's app card and sending it to a general chat room for a specific user
{
    "contentsType": "appCard",
    "contentsId": "1b73fc820-9b33-11eb-ab6d-bfb35eb9f7c5", // AppCard ID
    "botId": "microappcard", // Bot ID
    "targetUserId": "samsung.kim" // Knox Portal ID
    "contentsParams": {
        "name": "Kim",
        "title": "Pro"
    }
}

Case 2) Send the app card in Case 1 to several people with different data values
{
    "contentsType": "appCard",
    "contentsId": "1b73fc820-9b33-11eb-ab6d-bfb35eb9f7c5", // AppCard ID
    "botId": "microappcard", // Bot ID
    "targetUserId": ["samsung.kim", "samsung.lee"] // Knox Portal ID
    "contentsParams": [
        {
            "name": "Kim",
            "title": "Pro"
        },
        {
            "name": "Lee",
            "title": "Pro"
        }
    ]
}

Case 3) Insert data value into the app card in Case 1 and send it to a group chat room with a ChatRoom ID of 141159563240472576
{
    "contentsType": "appCard",
    "contentsId": "1b73fc820-9b33-11eb-ab6d-bfb35eb9f7c5", // AppCard ID
    "botId": "microappcard", // Bot ID
```

```
36      "target": "chatRoom", // The recipient of the message is specified as
the chat room.
37      "targetChatRoomId": "141159563240472576" // ChatRoom ID
38      "contentsParams": {
39          "name": "OOO Scrumm Chatroom",
40          "title": "Pros"
41      }
42  }
```

Response		
Status Code	200	Normal
	401	Invalid token
	404	Failed to locate the bot ID and/or app card ID
	500	Server error

Response example

```
1  &apos;- Normal
2  {
3    "resultMessage": "Success",
4    "schedulerId": "477289a0-c811-11eb-8e0d-47c464655299"
5  }
6
7  - Error
8  {
9    "errorMessage": "The bot could not be found. Please check botId."
10 }
```

400 error details		
Parameter name	**Error message**	**Description**
contentsType	contentsType should not be empty	The contentsType value is missing.
	contentsType should be equal to appCard or text	The contentsType value is not "appCard” or "text."
contentsId	contentsId should not be empty when contentsType is appCard	The contentsId value is missing when the contentsType is "appCard."

400 error details		
Parameter name	**Error message**	**Description**
contents	contents should not be empty when contentsType is text	The contents value is missing when the contentsType is "text."
subject	subject should not be empty when contentsType is text	The subject value is missing when the contentsType is "text.”
contentsParams	If contentsParams is array, targetUserId should be array	The targetUserId value is not an array when the contentsParams value is an array.
	Length of contentsParams is different from length of targetUserId	When the array sizes differ for the contentsParams and targetUserId values.
Other	The company that use approval could not be supported. Please use bot builder	For a bot in a company that utilizes the approval feature

Note
For more information, see “API utilization details” provided in the AI Builder Tutorial.

A.2 Utilizing a poll delivery API

You can deliver polls on a legacy system via an open API integration and collect the real-time responses using a chatbot.
Select poll scenarios in the text message or app card format to deliver polls to specific users.
On a legacy system, you cannot schedule the delivery because polls are delivered when the API is called.

Note
A poll delivery API is used to deliver polls to specific users.

Refer to the following poll delivery API specifications for the legacy system integration.

Request		
URL	https://aibuilder.samsung.net/v1/survey/send	
Method	POST	
Header	Authorization	bearer Token value • It refers to a token for API authentication, rather than the token provided by the Knox system. You can issue this API token on the AI Builder(My Bot > Notif/Poll API > API token mgmt). Only Bot admins can access the API token management menu.
	Content-Type	application/json
Body	botId	Bot ID (The Knox Bot ID issued by the Knox Dev Center at your request)
	surveyParams[].receiver	Knox ID of the recipient user (write in an array)
	surveyParams[].subject	Poll schedule title
	surveyParams[].startDate	Poll delivery time
	surveyParams[].endDate	Poll closing time
	surveyParams[].creator	User who created the poll schedule
	surveyParams[].singleViewMode	(Optional) Poll exposure format (display in list view when sending multiple polls) E.g., List - "singleViewMode": "list
	surveyParams[].responseStatus	(Optional) Poll response status display (default value is "false")
	surveyParams[].anonymous	(Optional) Anonymous poll (default value is "false")
	surveyParams[].survey[].templateId	Poll Template ID (The poll template ID registered in the bot you want to call and can be checked in AI Builder)
	surveyParams[].survey[].params	Data for poll template substitution variables (${variable name}) Eg., When substituting ${name} phrase in poll templates, write {"name": "Samsung Kim"}
	surveyParams[].survey[].promoti	(Optional) Poll promotion delivery time

Request		
	on[].promoteDate	
	surveyParams[].survey[].promotion[].promoteMessage	(Optional) Poll promotion message

Body example

```
{
  "botId": "{Bot’s Knox ID}",
  "surveyParams" : [
    {
      "receiver" : ["user1"],
      "subject” : "Subject",
      "startDate" : "2020-08-08 19:00",
      "endDate" : "2020-08-08 20:00",
      "creator”: "Poll sender’s ID",
      "singleViewMode": "list (In list view mode, single item polls are also displayed as lists)",
      "survey" : [
        {
          "templateId": "Poll template ID",
          "params" : {
            "User-defined parameter key": "User-defined parameter value",
            "User-defined parameter key": "User-defined parameter value",
          }
        },
        {
          "templateId": "bbbb",
          "params" : {
            "User-defined parameter key": "User-defined parameter value",
            "User-defined parameter key": "User-defined parameter value",
          }
        },
      ]
    },
    {
      ... A JSON object identical to the one above
```

```
    }
  ]
}
```

Response		
Status Code	200	Normal
	400	Body information error
	401	Invalid token
	404	Poll templates do not exist.
	500	Other server errors

Response example

```
{
  "result": "OK",
  "data" : [
    {
      "schedulerId": "eb51e988-d9f5-403a-b2fa-fc2f980e4847",
      "surveyId": [
        "6e8fd710-a0b4-11eb-b6f3-8fbc2556eaa0",
        "1e8fd710-a0b4-11eb-b6f3-8fbc2556eaa4",
        "228fd710-a0b4-11eb-b6f3-8fbc2556eaa4"
      ]
    },
    {
      "schedulerId": "6e8fd710-a0b4-11eb-b6f3-8fbc2556eaa4",
      "surveyId": ["77dec391-a0b3-11eb-818a-93e0c305a203"]
    }
  ]
}
```

400 error details		
Parameter name	**Error message**	**Description**
receiver	Receiver must exist and be array.	The value does not exist or it is not an array.
	Receiver must be at least 1 character and cannot be exceed 200 characters.	The userID of the relevant recipient has not been input or it is longer than 200 characters.

400 error details		
Parameter name	**Error message**	**Description**
subject	Subject must exist and be string.	The value does not exist or it is not a string.
	Subject must be at least 1 character and cannot be exceed 100 characters.	The subject has not been input or it is longer than 100 characters.
startDate	StartDate must exist and be date string. ex) YYYY-MM-DD HH:mm	The value does not exist or it is not in the defined date format.
	StartDate cannot be earlier than the today.	The start or end date of the poll has been set as a past date.
endDate	EndDate must exist and be date string. ex) YYYY-MM-DD HH:mm	The value does not exist or it is not in the defined date format.
	EndDate cannot be earlier than the startDate.	The poll end date is earlier than the poll start date.
creator	Creator must exist and be string.	The value does not exist or it is not a string.
	Creator must be at least 1 character and cannot be exceed 200 characters.	The creator has not been input or it is longer than 200 characters.
	Please check the creator. (User ID)	The creator has not been registered to the user table.
templateId	TemplateId must exist and be string.	The value does not exist or it is not a string.

Note

For more information, see “API utilization details” provided in the AI Builder Tutorial.

Modify a poll

Request		
URL	https://aibuilder.samsung.net/v1/survey/send	
Method	Put	
Header	Authorization	bearer Token
	Content-Type	application/json
Body	botId	Bot ID
	surveyId	Poll ID
	surveyParams.receiver	(Optional) Poll receiver

Request		
	surveyParams.startDate	(Optional) Poll send date
	surveyParams.endDate	(Optional) Poll end date
	surveyParams.singleViewMode	(Optional) Survey exposure format (exposed in list form when multiple surveys are sent)
	surveyParams.responseStatus	(Optional) Survey response status exposure (Default value is "false")
	surveyParams.anonymous	(Optional) Anonymous poll (Default value is "false")
	surveyParams[].promotionTargetUser	(Optional) Promotion target (Default is "all") ex) In progress - "in_progress" ex) No response - "no_response" ex) All - "all"
	surveyParams.params	Data to be entered into the survey template substitution variable (${variable name}) ex) When replacing ${name} phrases in the survey template, write {"name": "Samsung Kim"}
	surveyParams.promotion[].promoteDate	(Optional) Promotion time
	surveyParams.promotion[].promoteMessage	(Optional) Promotion message

Body sample

```
Case 1) Modify end date before sending
{
  "botId": "Bot ID",
  "surveyId": "Poll ID",
  "surveyParams" : {
    "endDate" : "2023-01-03 00:00"
  }
}

Case 2) Modify substitution variable before sending
{
```

```
  "botId": "Bot ID",
  "surveyId": "Poll ID",
  "surveyParams" : {
    "params": {
      "name": "Kim",
      "title": "pro"
    }
  }
}

Case 3) Modify to use promotion survey before sending
{
  "botId": "Bot ID",
  "surveyId": "Poll ID",
  "surveyParams" : {
    "promotion": [
      {
        "promoteDate": "2022-01-01 16:00",
        "promoteMessage": "Message"
      },
      {
        "promoteDate": "2022-01-01 20:00",
        "promoteMessage": "Message"
      }
    ]
  }
}
```

400 error details		
botId	Please check botId.	There is no botId
surveyId	Please check surveyId.	There is no surveyId
	Survey cannot be modified after sending.	The survey to be revised has already been sent out
surveyParams	SurveyParams must not be array.	surveyParams is array
startDate	StartDate must exist and be date string. ex) YYYY-MM-DD HH:mm	There is no value or it's not in a defined date format
	StartDate cannot be earlier than the today.	The survey start time and send time are before the current time
endDate	EndDate must exist and be date	There is no value or it's not in a

400 error details		
	string. ex) YYYY-MM-DD HH:mm	defined date format
	EndDate cannot be earlier than the startDate.	The survey end time is earlier than the survey start time
responseStatus	ResponseStatus must be boolean.	The survey response status display option is not true or false
anonymous	Anonymous must be boolean.	The anonymous survey option is not true or false
promotionTargetUser	PromotionTargetUser must be no_response or in_progress or all	The encouragement target option is not set to no_response, in_progress, or all
promotion	Promotion can be up to 5.	The number of encouragement notifications exceeds five
promoteDate	PromoteDate cannot be earlier than the current time.	The time for sending encouragement notification is earlier than the current time
	PromoteDate cannot be earlier than startDate.	The time for sending encouragement notification is earlier than the start time of the survey
	PromoteDate must be eralier than endDate.	The time for sending encouragement notification is later than the end time of the survey

Delete a poll

Request		
URL	https://aibuilder.samsung.net/v1/survey/${botId}/${surveyId}	
Method	Delete	
Header	Authorization	bearer Token

400 error details		
botId	Please check botId.	There is no botId
surveyId	Please check surveyId.	There is no surveyId
	Survey has already been sent and cannot be deleted.	The survey to be revised has already been sent out

View a poll

Request		
URL	https://aibuilder.samsung.net/v1/survey/${botId}/${surveyId}	
Method	Get	
Header	Authorization	bearer Token

400 error details		
botId	Please check botId.	There is no botId
surveyId	Please check surveyId.	There is no surveyId

View a survey result

Request		
URL	https://aibuilder.samsung.net/v1/survey/result	
Method	Post	
Header	Authorization	bearer Token
Body	botId	Bot ID
	surveyId	Survey ID (array)
	schedulerId	(Optional) Scheduler ID
	startDate	(Optional) Poll send date

Body sample

```
{
  "result": "OK",
  "data":[
    {
      "userId": "userId1",
      "params" : {
        "Key" : "Value",
        "Key" : "Value"
      }
      "result": [
        {
          "index" : 1,
```

```
            "question" : "Q1",
            "type" : "Multiple-choice selection",
            "answer" : "Answer",
            "answerIndex": 1
          },
          {
            "index" : 2,
            "question" : "Q2",
            "type" : "Descriptive type",
            "answer" : "Answer"
            "answerIndex": 0
          }
        ]
      },
      {
        "userId": "userId2",
        "result": [
          {
            "index" : 1,
            "question" : "Question",
            "type" : "Multiple-choice selection",
            "answer" : "Answer",
            "answerIndex": 2
          },
          {
            "index" : 2,
            "question" : "Question,
            "type" : "Descriptive type",
            "answer" : "Answer",
            "answerIndex": 0
          }
        ]
      }
    ]
}
```

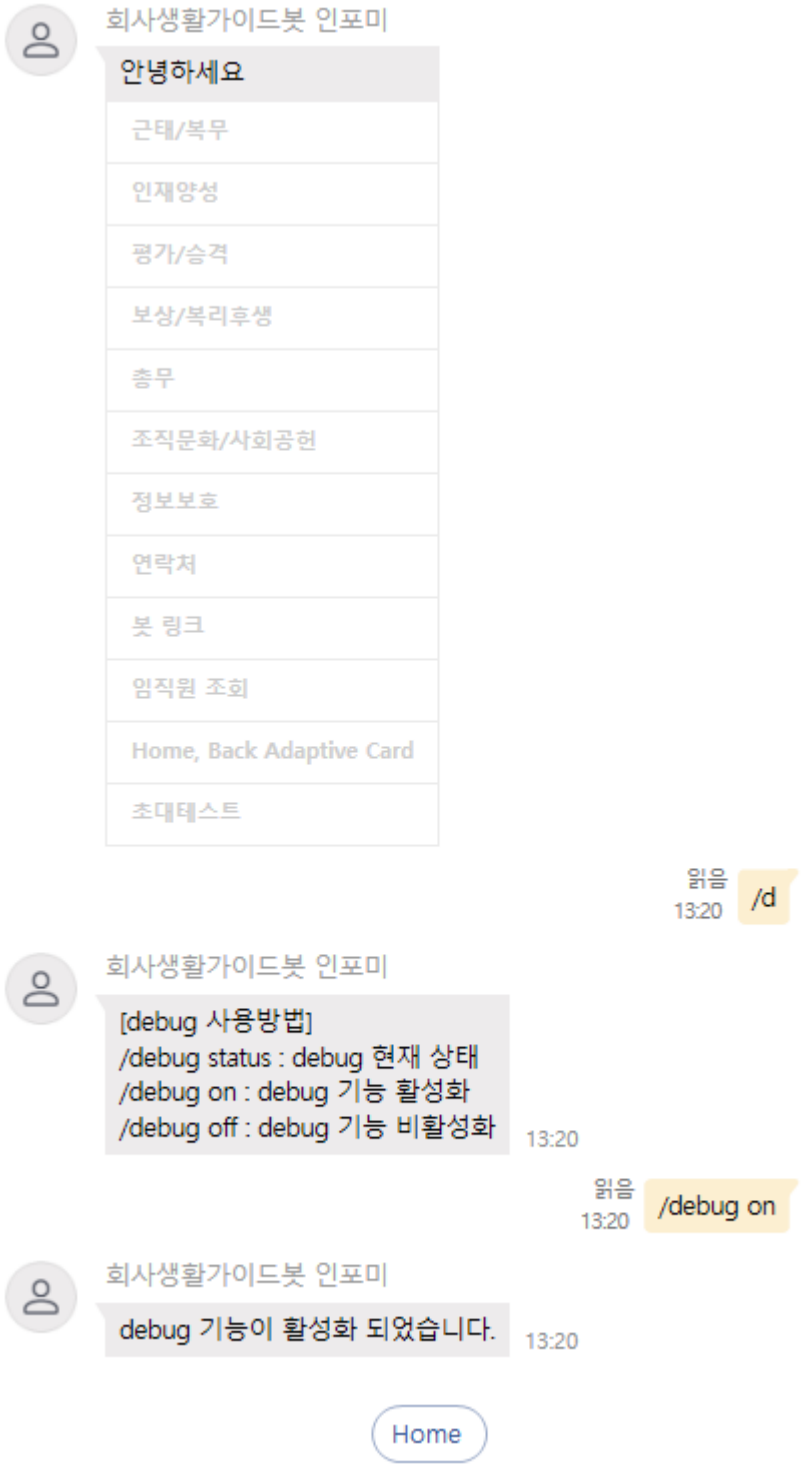
Appendix B.
Checking bot operation status with Slash Command

Bot administrators can utilize slash commands to analyze speech sentences and view the cumulated data.

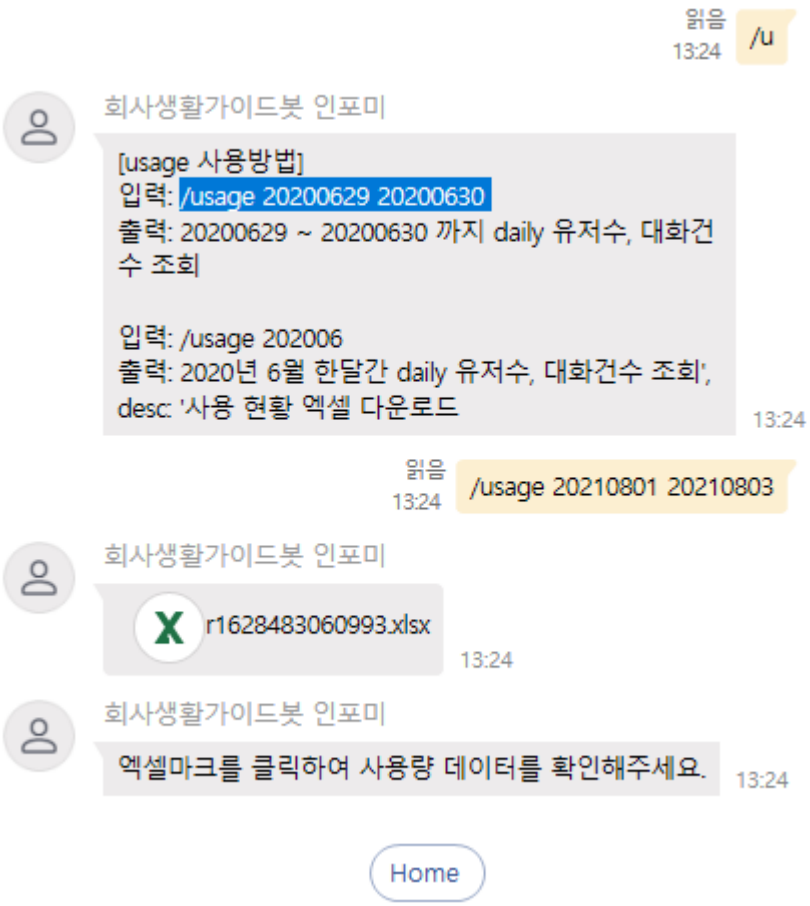
> Note
> Bot administrators and company administrators can utilize slash commands for the bots that belong to their groups.

Acronym	Slash Command	Description
/d	/debug status	Show the current debug status.
	/debug on	Enable the debug feature.
	/debug off	Disable the debug feature.
/u	/usage yyyymmdd YYYYMMDD	View the daily number of users and chats from "yyyymmdd" to "YYYYMMDD."
	/usage yyyymm	View the daily number of users and chats for one month ("yyyymm").
/c	/count	Enable or disable the minimum number of keywords. Administrators can change the minimum number of keywords for test purposes, whereas the set restriction will be applied to bot users.
/so	/so	View the list of ongoing surveys.
/se	/se	View the list of completed surveys.
/s	/survey	Check the survey response status based on the current time.

View and change debug status (/d)

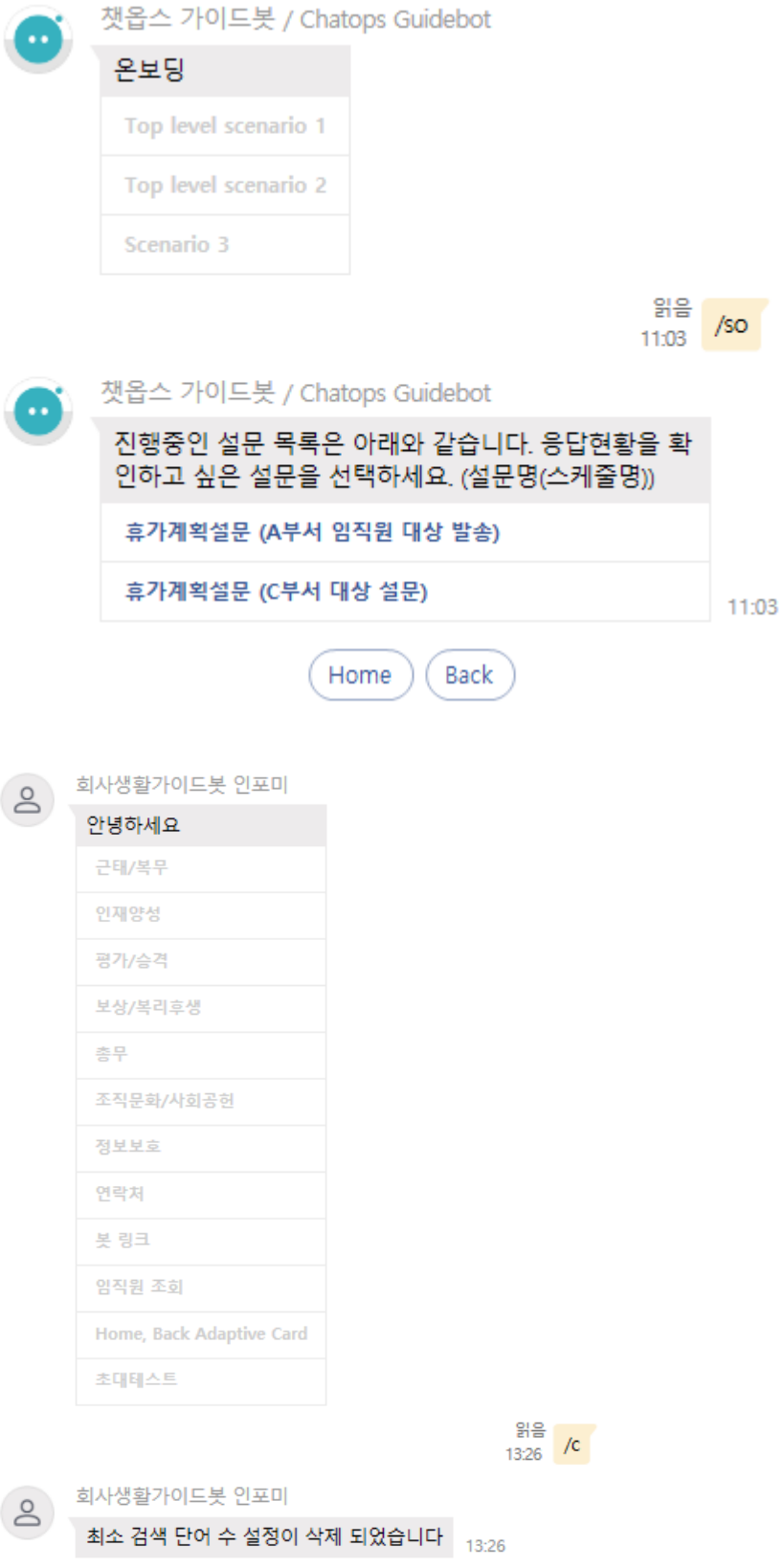


View usage status (/u)

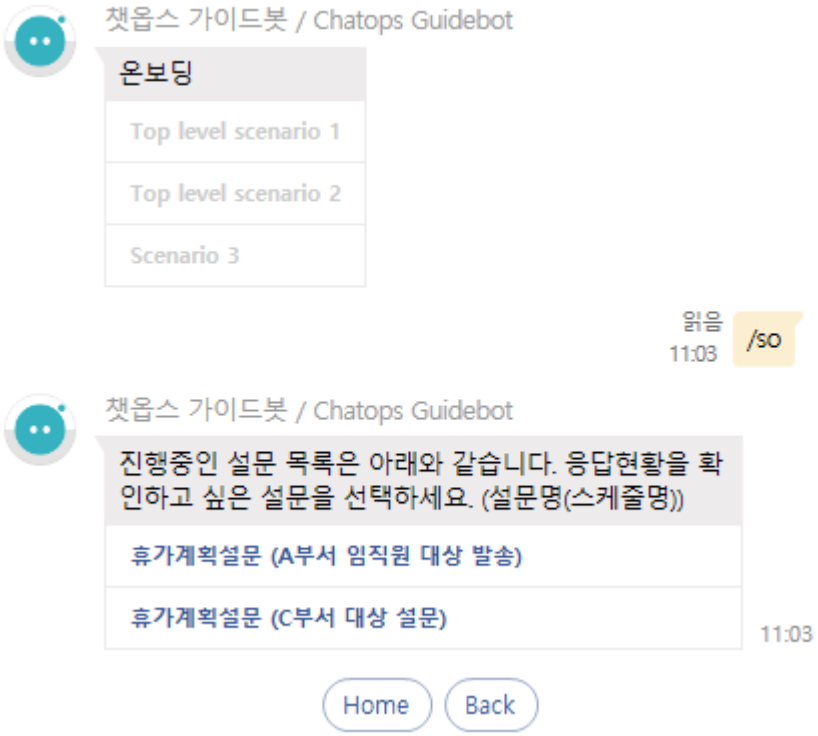


> Note
> Excel files cannot be downloaded in the simulator. Try downloading the file again in Knox Messenger.

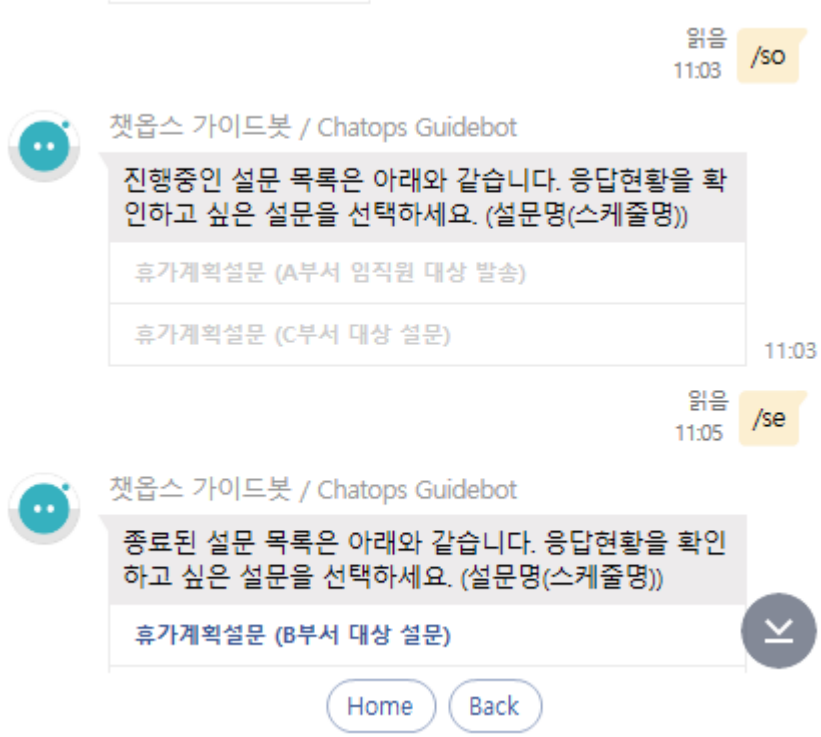
Enable or disable the minimum number of keyword (/c)



View the list of ongoing polls (/so)



View the list of completed polls (/se)



Appendix C.
Developing a Workflow

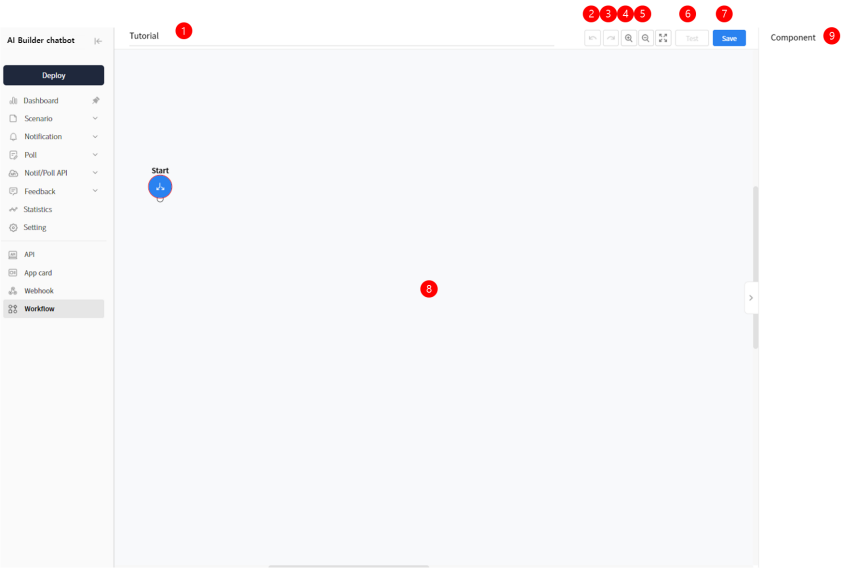
C.1 About Workflow

In Workflow, users without development knowledge can easily create business workflows by selecting and arranging the components of their choice (text, app cards, APIs, and condition comparison) in a canvas and linking each component with lines.

In Bot Builder, you can use previously created app cards as templates or create layouts by using the drag-and-drop feature of the Adaptive Card Designer and integrate app card ↔ app card/API in Workflow to design your business workflow so that is integrated with the internal system.

Complete the following steps to access Workflow.

1. Click the **My Bot** tab on the top of the Bot Builder home screen.
2. Select a bot from the bot list on the "My Bot" screen.
3. Click **Workflow** on the bot details screen.
4. Click the **Register** button.



No.	Item	Description
❶	**Workflow scenario name**	Shows the scenario name entered when creating a workflow.
❷	**Undo**	Cancels the content.
❸	**Redo**	Re-executes the content.
❹	**Magnify**	Magnifies the canvas area (flow)
❺	**Reduce**	Reduces the canvas area (flow)
❻	**Full-screen flow**	Adjusts the screen to view the entire flow created on the canvas.
❼	**Test**	Tests the workflow using the simulator.
❽	**Save**	Saves the workflow.
❾	**Canvas**	Drags and moves the workflow canvas.
❿	**Component settings**	Shows the settings for each component when you click on each one of them in the canvas.

Note

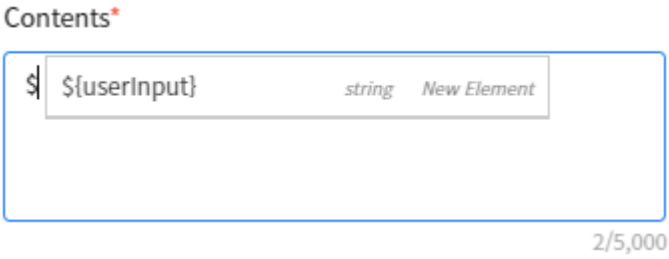
Click the Test button to test the **saved workflow** in the simulator. Please save and test your workflow because the workflow being edited on the canvas may differ from the saved one.

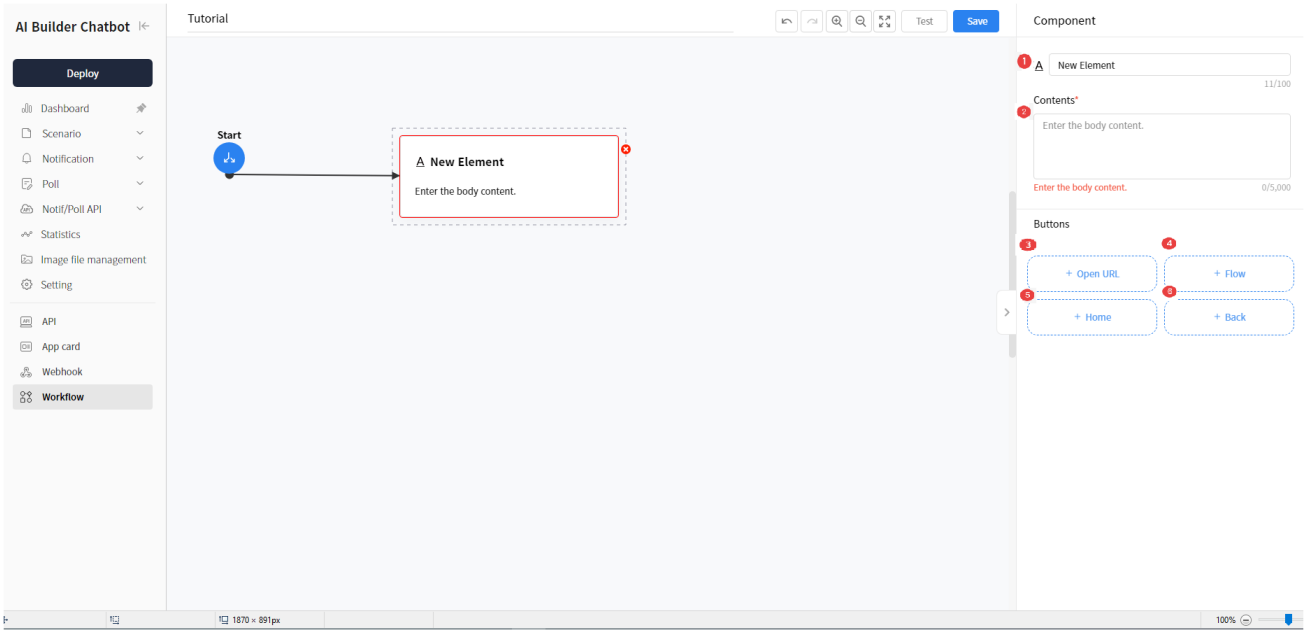
C.2 Component type of Workflow

Let's learn about each component type provided by the workflow.

Text

The text component allows you to enter body text and add buttons. **When writing the body text, an autocomplete feature is provided when you enter "$", helping you to you check the list of variables declared in the previous workflow.**





No.	Item	Description
❶	**Name**	Enter the name of the text components.
❷	**Body text**	Enter the body text.
❸	**OpenURL**	A button that opens the preset URL link when clicked.
❹	**Flow**	A button that allows you to move to the next step.
❺	**Home**	A button that takes you to the home screen.
❻	**Back**	A button that allows you to move to the previous step.

App cards

The app card component allows you to search for and use app cards created in Bot Builder as templates or create app cards in Adaptive Card Designer through its link. Let's take a look at how to use existing app cards to complete your flow.

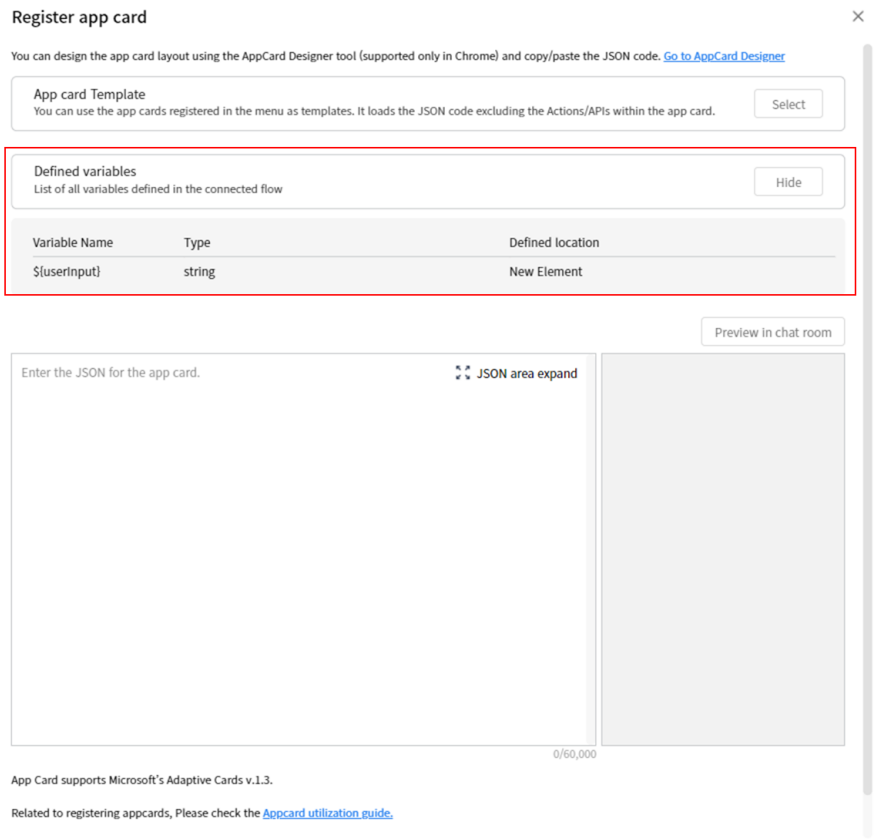
1. Click **Make appcard**
2. In the **App card registration pop-up window**, click **Select app card template** to select an app card as a template.
3. Enter the JSON code to complete your app card.

When writing an app card, if you declare an ID as shown below, you define a variable, and it becomes a global variable that can be used as ${variable name} anywhere in the subsequent flow. In the example below, 'name' is stored as a global variable, and the value input by the user can be used as ${name} in the flow that follows.

```
{
   "id":"name", //global variable
   "type":"Input.Text",
   "placeholder":"Please enter name."
   }
```

If there are variables declared in the previous step linked to app cards, you can check the variable names, data types, location, etc., in the “Check saved variables” area.

Refer to variables to complete your app card.

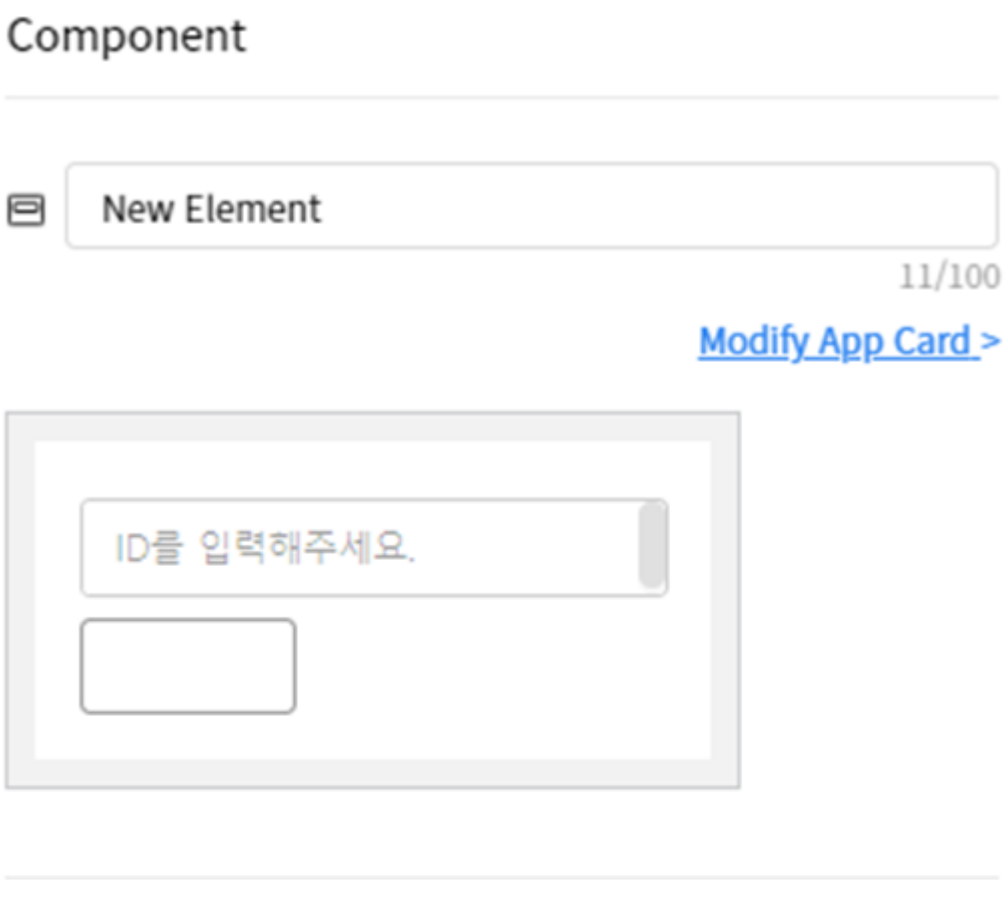


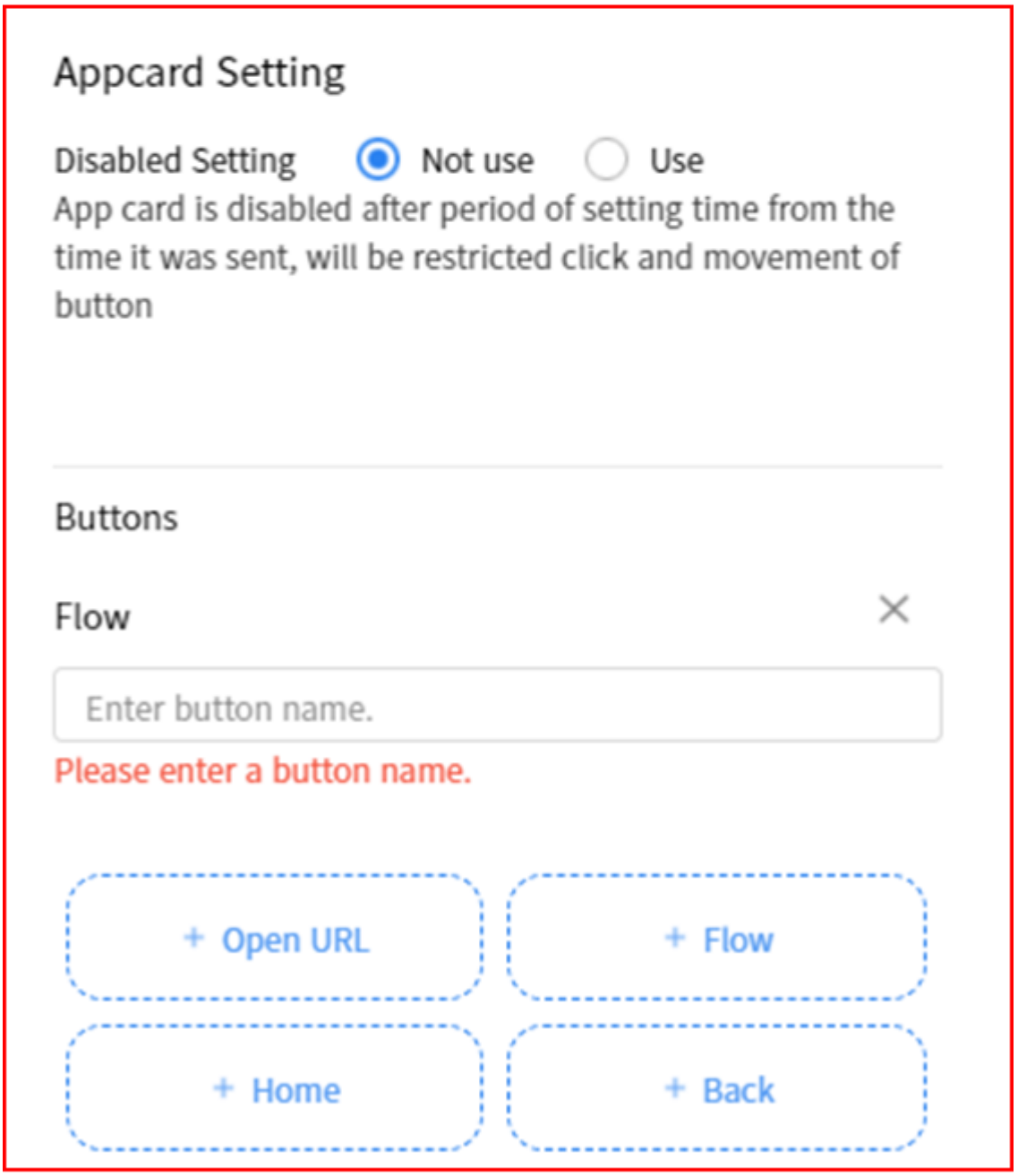
> Note
> App cards created and edited in workflow are only available in that workflow scenario and are not created in the **My Bot** > **App card** menu

If app cards are integrated with APIs, you can click the “Reflect to app card” button to make sure that the API results are properly reflected in the app cards.

Basic settings

- **Disabled Setting** : A feature that deactivates the app card and restricts button clicks and movements after a set period from the time the app card was sent.
- **Flow** : Takes you to the next step. Use this button by integrating it with the next step.
- **Home** : Takes you to the home screen.
- **Back** : Takes you to the previous step. Creating the Back button automatically integrates with the previous step.





API

1. Choose the API to call.
2. Check the requested header/body/query values and click the **Edit API** button if you need to modify them.
3. Click **API call test** and check the response value.

4. You can create user values. Enter the name and values of variables. You can use the variables you created in the integrated workflow.

> Note
> Integration between API ↔ API is also available in Workflow.

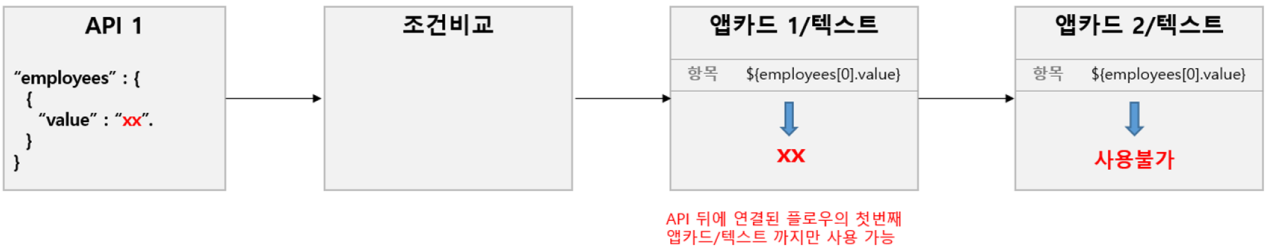
> Note
> APIs modified in Workflow are only applied in that workflow scenario and don't affect other menus using the same API.

There are two ways to utilize API response values within a workflow.

1. If you input API response values in the format **${variable name}**, you can use the API response body. If the **body is in JSON format, you can use it in the format ${variable name}, and if it's an array, you can use it in the format ${body[number].variable name}**.



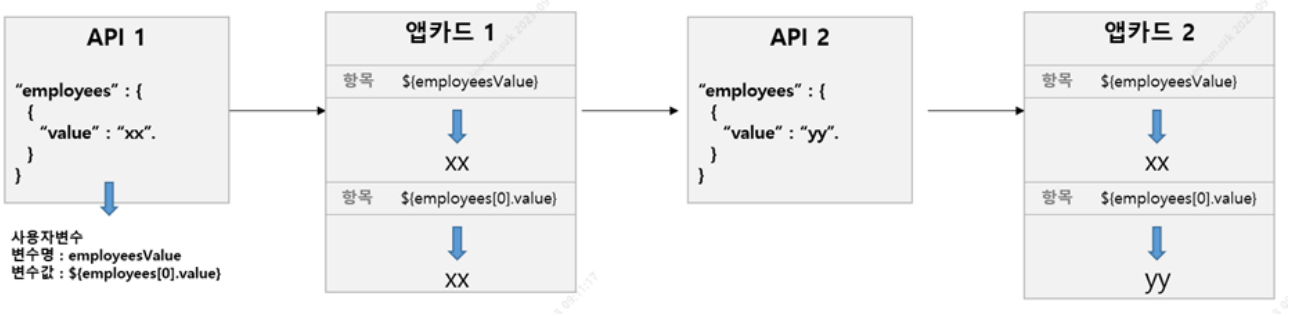
This usage method is only available within the flow up to the first app card connected after the API component, and it cannot be used in subsequent flows. For example, in the scenario below, you can use the response values of api1 in app card 1/text as ${variable name}, but it cannot be used in app card 2/text.



If you need to use it across multiple flows, please register it as a user variable (global variable) and use it.

2. You can save API response values as user variables (global variables) so that they can be used from anywhere in the subsequent flow.

Now, let's delve into the concept of user variables (global variables). When used as global variables, the value of the variable does not get overwritten. Let me illustrate with an example. Suppose you have the following flow.



save the response value of API1 in the user variable "employeeValue."

```
"employees": {

    {
          "value":"xx"
 }
```

In the following app card (app card 1) connected afterwards, the value of ${employeeValue} becomes 'xx,' and the value of ${employees[0].value} becomes 'xx' as well. Assuming API 2 has the same 'value'

parameter as API 1 but with a different value,

```
"employees": {

    {
          "value":"yy"
 }
```

In app card 2, the value of ${employeeValue} becomes 'xx,' and the value of ${employees[0].value} changes to 'yy.'

Condition comparison

In Workflow, you can branch flows based on conditions. Create flows based on conditions by integrating app cards or APIs with condition comparison. When entering variables for condition comparison, **an autocmplete feature is provided when you type “$,” allowing you to view the list of variables declared in the previous flows**.

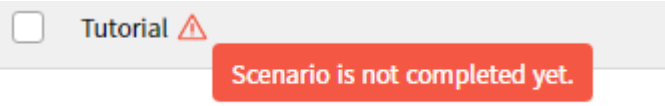
Workflow offers the following nine conditions.

Condition	Value
${} exists	-
${} does not exist	-
==	X is a number or a letter
!=	X is a number or a letter
>	X is a number
<	X is a number
>=	X is a number
<=	X is a number
Else	-

C.3 Using Workflow

You can use workflows by integrating them with scenarios.

However, only workflows with successfully completed scenarios can be connected. If you encounter the error message 'Scenario is not complete' as shown below in the workflow, it means that the workflow cannot be connected with the scenario. Please complete the workflow and try again.



(※ We'll be adding more options that can be integrated, such as notifications, webhooks, etc.)

To use the workflow, please follow the steps below.

3. **Click Scenario** > **Scenario management**.
4. Click workflow in the scenario content.
5. Click the workflow scenario you want to integrate with.
6. Click the Save button.
7. Lastly, click the **Deploy** button to apply it to the product bot.

> Note
> Before deployment, check if the workflow is working properly with the simulator